AFRILANG

Le langage a syntaxe naturelle
qui transpile vers C++17/C++20

Manuel officiel ultra-detaille
Guide d'utilisation - Specification du langage
Bibliotheque standard - Paquets - Outils - Architecture
Playground web - Extension VS Code - WASM

Auteur : Maxime Dzidula KELI
Telephone : 98 60 00 18
WhatsApp : +33 754830039
Email : MaximeKELI25@gmail.com
GitHub : https://github.com/MaximeKELI/AFRILANG

Edition 1.0 - Documentation complete de l'ecosysteme AFRILANG

A propos de l'auteur

Maxime Dzidula KELI est le createur et mainteneur principal du langage AFRILANG et de son ecosysteme
(compilateur, bibliotheque standard, gestionnaire de paquets, serveur Language Server, playground web Django, extension Visual Studio Code / Cursor, et application mobile Flutter).

AFRILANG poursuit un objectif clair : offrir une syntaxe naturelle, accessible en anglais (avec alias francais), tout en produisant des executables natifs performants via une transpilation vers C++17/C++20.

Ce manuel regroupe la specification du langage, les guides d'installation et d'utilisation, la reference de la bibliotheque standard, la documentation des outils, ainsi que des exemples commentes issus du depot officiel.

Coordonnees

- Nom : Maxime Dzidula KELI
- Telephone : 98 60 00 18
- WhatsApp : +33 754830039
- Email : MaximeKELI25@gmail.com
- Depot : https://github.com/MaximeKELI/AFRILANG

Table des matieres

L'index detaille avec numeros de pages se trouve en fin d'ouvrage. Plan general :

Partie I — Vision, philosophie et architecture

Qu'est-ce qu'AFRILANG ?

AFRILANG est un langage de programmation orienté objet à syntaxe naturelle. Il est conçu pour être lisible comme de l'anglais simple, tout en compilant vers du C++17/C++20 puis en produisant un exécutable natif via g++.

Le projet vise une expérience proche des écosystèmes modernes (inspiration Nim / Crystal) : langage haut niveau, compilateur, bibliothèque standard riche, gestionnaire de paquets, serveur LSP, playground web, et cibles natives ainsi que WASM.

Pipeline de compilation

```
fichier.afr
   → Compiler (imports)
   → Lexer
   → Parser (AST)
   → Sémantique
   → CodeGen C++
   → g++ / em++
   → exécutable
```

Principes de conception

- Syntaxe naturelle : say, ask … into, if … is greater than, repeat N times
- Bilingue : anglais + alias français (dire, creer, si, alors, fin…)
- Typage clair : number, text, bool, list, map, T?, T or error
- Performance : génération C++ et exécution native
- Écosystème : pkg, LSP, playground, stdlib multi-niveaux
- Erreurs utiles : codes stables, suggestions, locales en/fr

Organisation du dépôt

include/afrilang/, src/, runtime/, stdlib/, examples/, packages/, web/ (Django), site/, vscode-afrilang/, mobile/, docs/, scripts/.

Partie II — Installation et premiers pas

Installation rapide

```
curl -fsSL https://raw.githubusercontent.com/MaximeKELI/AFRILANG/main/install.sh | bash
```

Shell

```
export PATH="$HOME/.local/bin:$PATH"
export AFRILANG_HOME="$HOME/.local/afrilang"
afrilang version
```

Build source

```
mkdir build && cd build
cmake ..
make -j$(nproc)
```

Hello

```
say "Bonjour depuis AFRILANG!"
repeat 3 times
    say "Hello"
end
```

```
afrilang run examples/hello.afr
```

Programme PAHT

```
say "=== PAHT calculator ==="
say ""

ask "Last name: " into lastName
ask "First name: " into firstName
ask "Unit price (TTC): " into unitPrice as number
ask "VAT rate (%): " into vat as number

create paht = unitPrice / (1 + vat / 100)

say ""
say "Client: " + firstName + " " + lastName
say "Unit price TTC: " + unitPrice
say "VAT: " + vat + " %"
say "PAHT (purchase price excl. tax): " + paht
```

```
afrilang run examples/paht.afr
```

Partie III — Guide d'utilisation de la CLI

`afrilang version`
→ Version + AFRILANG_HOME

`afrilang build`
→ Compile le projet

`afrilang run f.afr`
→ Compile et exécute

`afrilang check f.afr`
→ Analyse sans link

`afrilang test`
→ Suite de tests

`afrilang init nom`
→ Nouveau projet

`afrilang init nom --lib`
→ Nouvelle bibliothèque

`afrilang lsp`
→ Language Server

`afrilang fmt f.afr [-w]`
→ Formateur

`afrilang repl`
→ REPL

`afrilang pkg add/install/update/list/search/publish`
→ Paquets

`afrilang serve`
→ Playground local :8080

`afrilang debug/lint/explain/doc`
→ Outils qualité

`afrilang compile-js / build-wasm-web`
→ Cibles web

`afrilang benchmark`
→ Mesures

Projet

```
[package]
name = "mon_app"
version = "0.1.0"
main = "src/main.afr"

[dependencies]
math = "^0.1.0"
```

Partie IV — Spécification complète du langage

AFRILANG Language Specification 1.0

Comment lire cette spécification

1. Types — primitives, collections, optionnels, Result 2. POO — classes, héritage, interfaces, records 3. Contrôle — conditions, boucles, match 4. Modules & paquets — import / use (voir aussi PKG.md) 5. Async, FFI, tests — sections dédiées plus bas 6. Guide web méthodique — documentation site /docs/ (parcours numéroté)

Convention : chaque bloc se ferme par end (ou fin). Comparaisons en mots (is equal to, ...). Optionnels : T?, nothing, is defined — pas none / is some.

Types

```
| Type | Syntax | C++ |
|------|--------|-----|
| number | `number` | `double` |
| text | `text` | `std::string` |
| bool | `bool` | `bool` |
| list | `list of T` | `std::vector<T>` (classes: `vector<unique_ptr<T>>`) |
| map | `map K to V` | `std::unordered_map<K,V>` |
| optional | `T?` | `std::optional<T>` |
| result | `T or error` | `AfrResult_T` |
```

Programmation orientée objet (POO)

Classes, héritage, encapsulation et interfaces :

```
class Animal
    function speak()
        say "..."
    end
end

class Dog extends Animal
    public field name text

    function init(aName text)
        set this.name = aName
    end

    function speak()
        say name + " says Woof!"
    end
end

create rex = new Dog("Rex")
rex.speak()
```

Fonctionnalités POO :

- **Classes** — `class Nom ... end` avec méthodes `function ... end`
- **Instanciation** — `new Classe` ou `new Classe(arg1, arg2)` (constructeur `init`)
- **Champs** — `public field` / `private field` / `protected field`, accès via `this.champ` ou `obj.champ`
- **Héritage** — `class Enfant extends Parent` avec redéfinition (`override` en C++)
- **super** — `super(arg)` dans `init`, `super.methode()` pour appeler le parent
- **static** — `static field` / `static function`, appel via `Classe.methode()`
- **abstract** — `abstract class` et `abstract function` (pas d'instanciation)

- **final** — `final class` (non extensible), `final function` (non redéfinissable)
- **property** — `property text name end` (getter/setter auto)
- **destroy** — bloc `destroy ... end` → destructeur virtuel
- **Polymorphisme** — `create pet Animal = new Dog("Rex")`, listes `list of Animal`
- **Classes génériques** — `class Box<T> ... end`, `new Box<number>(42)`
- **Interfaces** — `interface I ... end` et `class C implements I, J`
- **Records** — structs légers (`record Point ... end`)

Exemples : examples/oop.afr, examples/inheritance.afr, examples/poo_demo.afr, examples/oop_full.afr, examples/polymorphic_list.afr, examples/generic_class.afr, examples/poo_advanced.afr, examples/phase11_demo.afr

Maps

```
create m = map of "a" to 1, "b" to 2
create m map text to number = map of "x" to 10
create empty = empty map text to number
say m at "a"
set m at "c" = 3
for each key, value in m do
    say key
    say value
end
```

Exceptions

```
try
    raise "something failed"
catch error e
    say e
end
```

Enums et pattern matching

```
enum Status
    case Ok
    case Error with message text
end

create e = Status.Error with "failed"

match e
    case Error with msg then
        say msg
    end
    default
        say "other"
    end
end
```

Liaison de payload : case Error with msg then lie msg au champ message.

Union taguée — alias de enum pour les types somme :

```
union Shape
    case Circle with radius number
    case Rect with width number, height number
end

create s = Shape.Circle with 5

match s
    case Circle with r then
        say r
    end
    case Rect with w, h then
        say w
    end
end
```

Exhaustivité : tous les cas doivent être couverts ou un default doit être présent.

Match en expression

Retourne une valeur (utilisable dans create, say, return, etc.) :

```
create label = match s
    case Ok then "success"
    end
    case Error with msg then msg
    end
    default "unknown"
    end
end

say match Status.Ok then "ok" end default "?" end
```

Chaque bras : case Nom then <expression> end ou default <expression> end.

Interfaces (traits)

```
interface Speakable
    function speak()
end

class Dog implements Speakable
    function speak()
        say "Woof!"
    end
end

create pet Speakable = new Dog()
create pets list of Speakable = list of new Dog()
pet.speak()
```

- Vérification des **signatures** à l'implémentation (paramètres, retour)
- Variables et listes **typées par interface**
- Polymorphisme via `std::unique_ptr<Interface>` + virtual

Surcharge d'opérateurs

Dans une classe, déclarez des opérateurs binaires :

```
class Vector
    public field x number
    public field y number

    function init(ax number, ay number)
        set this.x = ax
        set this.y = ay
    end

    operator + (other Vector) returns Vector
        return new Vector(x + other.x, y + other.y)
    end
end

create c = a + b
```

Opérateurs supportés : +, -, *, /, ==, !=, <, >.

String interpolation

```
create name = "World"
say "Hello {name}!"
```

Literal braces: {{ and }}. JSON literals ("{\"key\": 1}") are supported — a { followed by " is not treated as interpolation.

Modules

Import with import "std/io" and use io.

```
module Math
    export function add(a number, b number) returns number
        return a + b
    end

    private function helper(x number) returns number
        return x * 2
    end
end

use Math
say add(3, 4)
say Math.add(5, 6)    -- accès qualifié sans ambiguïté
```

- **`export function`** — API publique du module (documentation explicite)
- **`private function` / `private class`** — invisible via `use` (accessible dans le module)
- **`Module.fn(...)`** — appel qualifié sans `use`

Available stdlib modules: io, json, fs, http, str, log, math, time, re, collections, args, path, async, ui, game2d, game3d, plus 2151 simple (std/nom), 102 medium (std/m/nom), 5710 complex (std/c/nom) — see docs/STDLIB_*.md.

Syntaxe bilingue (français / anglais)

Les mots-clés anglais acceptent des alias français (ex. dire = say, si/alors/sinon, fin = end, creer = create, fonction = function, classe = class, utiliser = use, importer = import, tantque = while, cas/defaut/avec pour match, types nombre/texte/booleen, etc.).

Exemple : examples/language_demo.afr

Generics

```
function identity<T>(x T) returns T
    return x
end

say identity(42)              -- inférence
say identity<number>(42)      -- paramètre de type explicite
```

Default parameters

```
function greet(name text, prefix text = "Hello") returns text
    return join(list of prefix, " ", name)
end

say greet("World")
say greet("Africa", "Bonjour")
```

Numeric for loops

```
for i from 1 to 10 do
    say i
end

for n from 0 to 10 step 2 do
    say n
end
```

Constants

```
create const MAX = 100
create const NAME text = "AFRILANG"
```

Lambdas (fonctions anonymes)

```
create doubleIt = function(x number) returns number
    return x * 2
end

say doubleIt(21)

function apply(fn function number to number, value number) returns number
    return fn(value)
end

say apply(doubleIt, 10)
```

Type fonction : function number to number, function number, text to text, etc.

map / filter / reduce (listes + lambdas)

Syntaxe naturelle (sans import) :

```
create doubled = map each x in nums do
    return x * 2
end

create big = filter each x in nums where x is greater than 3

create total = reduce nums from 0 with each acc, x do
    return acc + x
end

create flat = flatMap each n in nums do
    return list of n, n * 10
end
```

reduce fonctionne aussi sur list text (valeur initiale text).

Syntaxe explicite via std/collections :

```
import "std/collections"
use collections

create doubled = mapNumbers(nums, function(x number) returns number
    return x * 2
end)

create big = filterNumbers(nums, function(x number) returns bool
    if x is greater than 3 then
        return yes
    else
        return no
    end
end)

create total = reduceNumbers(nums, function(acc number, x number) returns number
    return acc + x
end, 0)
```

Async / await

Les fonctions async retournent implicitement un task T (coroutines C++20). Support complet : returns T or error, await dans les tests, I/O async.

```
import "std/async"
use async

async function wait(ms number) returns text or error
    if ms is less than 0 then
        return error "invalid delay"
    end
    await sleep(ms)
    return "ready"
end

async function main()
    create r = await wait(10)
    if r is error then
        say r.message
    else
        say r.value
    end
end

test "async in test block"
    create r = await wait(1)
    assert r.value is equal to "ready"
end

await main()
```

```
| Fonctionnalité | Détail |
|----------------|--------|
| `async function` | Retourne `task T` ou `task (T or error)` |
| `await` | Global, fonctions async, blocs `test` |
| `std/async` | `sleep(ms)` — scheduler avec file de timers |
| `std/http` | `httpGetAsync`, `httpPostAsync` — thread pool |
| `std/io` | `readFileAsync` — lecture fichier non bloquante |
| Compilation | C++20 coroutines (`-std=c++20 -fcoroutines -pthread`) |
```

Interfaces graphiques

Syntaxe naturelle pour créer des fenêtres SDL2 (backend runtime/ui.hpp).

```
open window titled "Demo" with width 640, height 480

while window is open do
    clear background color 28, 28, 36
    draw text "Bonjour!" at 80, 80 size 32
    if button "Quitter" at 220, 350 width 200 height 50 is clicked then
        close window
    end
    show frame
end
```

```
| Instruction / expression | Description |
|--------------------------|-------------|
| `open window titled ... with width ..., height ...` | Ouvre une fenêtre |
| `while window is open do` | Boucle principale (appelle `beginFrame` automatiquement) |
| `clear background color r, g, b` | Efface l'écran (RGB 0–255) |
| `draw text "..." at x, y size n` | Affiche du texte |
| `button "..." at x, y width w height h is clicked` | Bouton cliquable (booléen) |
| `show frame` | Présente l'image à l'écran |
| `close window` | Ferme la fenêtre |
```

Jeux 2D (module `std/ui`)

Primitives pour créer de vrais jeux (rectangles colorés, timing, clavier) :

```
| Fonction | Description |
|----------|-------------|
| `fillRect(x, y, w, h, r, g, b)` | Rectangle plein (RGB 0–255) |
| `drawRect(x, y, w, h, r, g, b)` | Contour de rectangle |
| `drawTextColor(text, x, y, size, r, g, b)` | Texte coloré |
| `deltaMs()` | Millisecondes depuis la dernière frame |
| `windowWidth()` / `windowHeight()` | Taille de la fenêtre |
| `isKeyDown(key)` / `wasKeyPressed(key)` | Clavier (WASD, flèches, Space, Escape) |
| `mouseX()` / `mouseY()` | Souris (position) |
| `isMouseDown()` / `wasMousePressed()` / `wasMouseClicked()` | Souris (bouton gauche) |
| `fps()` | Images par seconde (moyenne glissante) |
```

Moteur 2D (`std/game2d`)

Bibliothèque de jeu haut niveau (grille, timers, entrées, collisions) — bâtie sur std/ui :

```
| Fonction | Description |
|----------|-------------|
| `configureGrid(cols, rows, cellSize, padX, padY)` | Configure une grille logique |
| `gridWindowWidth()` / `gridWindowHeight()` | Taille fenêtre pour la grille |
| `fillCell(col, row, r, g, b)` | Dessine une case |
| `fillBoard` / `drawWalls` / `drawGridLines` | Arrière-plan et murs |
| `fillCircleSolid(x, y, radius, r, g, b)` | Cercle plein (nourriture, effets) |
| `everyMs(name, delta, interval)` | Timer nommé (retourne `yes` quand l’intervalle est écoulé) |
| `updateDirection(curDx, curDy)` + `inputDirX()` / `inputDirY()` | Entrées clavier sans demi-tour |
| `gridHas(xs, ys, count, gx, gy)` | Collision sur grille |
| `pulse01(period)` | Animation sinusoïdale 0–1 |
| `drawCenteredText` / `drawHud` | Interface de jeu |
| `moveIntervalForScore(score, base, min)` | Vitesse progressive |
```

Sprites PNG : loadSprite(name, path), drawSprite, drawSpriteScaled, drawSpriteCell, hasSprite

Spritesheet : loadSpriteSheet(name, path, frameW, frameH), drawSpriteFrame, drawSpriteFrameCell, sheetCols, sheetRows

Sons : loadSound(name, path), playSound, playSoundVolume(name, 0-128)

Musique : loadMusic(name, path), playMusic(name, loops), stopMusic(), setMusicVolume(0-128)

Caméra : setCamera, followCamera(x, y, smooth), configureViewport(cols, rows) pour cartes plus grandes que l’écran

Triggers : defineTrigger(name, x, y, w, h), pointInTrigger(name, wx, wy), mouseInTrigger(name)

Sauvegarde : saveValue(path, key, value), loadValue(path, key, default), loadHighScore(path), saveHighScore(path)

Debug : drawFps(x, y) affiche le FPS à l’écran

import "std/game2d" puis use game2d. Voir aussi std/game (score, niveau, XP) et std/gamestate (états menu/jeu/pause).

États de jeu (`std/gamestate`)

Machine à états simple pour menus, pause et game over :

```
| Fonction | Description |
|----------|-------------|
| `setState(name)` | Change l’état courant |
| `getState()` / `isState(name)` | Lecture de l’état |
| `stateTimeMs()` | Temps passé dans l’état courant |
| `wasStateChanged()` | `yes` une frame après un changement |
| `tickState(deltaMs)` | À appeler chaque frame dans la boucle |
```

Moteur 3D (`std/game3d`)

Jeux 3D avec OpenGL (SDL2 + GL) :

```
| Fonction | Description |
|----------|-------------|
| `openWindow` / `isOpen` / `beginFrame` / `showFrame` | Boucle 3D |
| `clearScreen(r, g, b)` | Efface l’écran et le tampon de profondeur |
| `setCamera(x, y, z, yaw, pitch)` + `applyCamera()` | Caméra perspective |
| `updateFlyCamera(moveSpeed, turnSpeed)` | Vol libre WASD + QE + flèches |
| `drawCube`, `drawSphere`, `drawPlane`, `drawGrid`, `drawAxis` | Primitives 3D |
| `setSceneRotation` / `applySceneRotation` | Rotation du monde |
| `pickBody(screenX, screenY)` | Picking simple (raycast) → index du corps touché, ou -1 |
| `pickBodyName(screenX, screenY)` | Nom du corps touché, ou texte vide |
| `mouseX()` / `mouseY()` / `isMouseDown()` | Souris en 3D |
| `drawSkyGradient(topR,G,B, botR,G,B)` | Ciel dégradé plein écran |
```

```
./build/afrilang examples/game3d_demo.afr -o game3d_demo --run
```

Contrôles : WASD déplacer, QE monter/descendre, flèches regarder, Échap quitter.

Textures 3D : loadTexture3d, drawPlaneTextured, drawCubeTextured

Modèles OBJ : loadModel, drawModel, drawModelColored

Physique simple : createBody, setBodyVelocity, stepPhysics(delta, gravity), bodyX/Y/Z, applyImpulse

```
./build/afrilang examples/game3d_physics.afr -o game3d_physics --run
```

Espace = lancer une bille supplémentaire.

3D avancé (éclairage, brouillard, particules)

```
| Fonction | Description |
|----------|-------------|
| `enableLighting`, `setAmbientLight`, `setSunLight`, `applyLighting` | Éclairage directionnel |
| `setFog`, `applyFog` | Brouillard atmosphérique |
| `drawModelLit` | Modèle OBJ avec normales et lumière |
| `createBoxBody`, `drawBox`, `drawBody` | Physique et rendu de boîtes |
| `stepPhysicsEx` | Gravité + vent + frottement |
| `setWind`, `setBodyFriction`, `setBodyRestitution` | Paramètres physiques |
| `emitBurst`, `updateParticles`, `drawParticles` | Système de particules |
| `followBody` | Caméra qui suit un corps |
```

```
./build/afrilang examples/game3d_complex.afr -o game3d_complex --run
```

C bascule caméra libre / suivi du héros.

3D expert (skybox, GLSL, glTF, physique box-box)

```
| Fonction | Description |
|----------|-------------|
| `loadSkyboxFace(face, path)` | Face du ciel : `px`, `nx`, `py`, `ny`, `pz`, `nz` |
| `hasSkybox()` / `drawSkybox(halfSize)` | Cube ciel texturé centré sur la caméra |
| `loadShader(name, vert, frag)` | Programme GLSL 1.20 |
| `useShader` / `stopShader` | Active / désactive le shader |
| `setShaderFloat` / `setShaderVec3` | Uniformes shader |
| `drawModelShader(shader, model, x,y,z, scale, rotY)` | Modèle OBJ avec shader |
| `loadGltf(name, path)` | Chargeur glTF 2.0 minimal (`.gltf` + buffer embarqué ou `.bin`) |
| `drawGltf` / `drawGltfLit` | Rendu modèle glTF |
| `stepPhysicsEx` | Collisions sphère-sphère, box-box et sphère-box |
```

```
./build/afrilang examples/game3d_advanced.afr -o game3d_advanced --run
```

Clic souris : pickBodyName sur les corps physiques.

glTF binaire, animations, éditeur, réseau

```
| Fonction | Description |
|----------|-------------|
| `loadGlb(name, path)` | Alias `.glb` (binaire) |
| `gltfAnimCount`, `playGltfAnim`, `updateGltfAnims`, `gltfAnimRotY` | Animations de rotation glTF |
| `setEditMode`, `addLevelGltf`, `addLevelModel`, `drawLevel` | Éditeur de niveau intégré |
| `saveLevel(path)`, `loadLevel(path)` | Format texte `type asset x y z scale rotY` |
| `pickGround(screenX, screenY)` + `pickGroundX/Z` | Clic sur le sol (y=0) |
```

Module std/gamenet (UDP) : hostGame(port), joinGame(host, port), sendPose, pollNet, remoteX/Y/Z/RotY

```
./build/afrilang examples/game3d_editor.afr -o editor --run   # E éditer, clic placer, S/L sauver/charger
./build/afrilang examples/game3d_net.afr -o net --run         # 2 instances : host + join
```

Exemple minimal :

```
import "std/ui"
use ui

openWindow("Jeu", 640, 480)
while isOpen() do
    beginFrame()
    clearBackground(18, 18, 24)
    fillRect(100, 100, 48, 48, 90, 200, 120)
    drawTextColor("Score: 0", 20, 20, 24, 240, 240, 245)
    showFrame()
end
```

Module stdlib équivalent : import "std/ui" puis use ui (openWindow, drawButton, etc.).

Compilation : -lSDL2 -lSDL2_ttf (SDL2 + SDL2_ttf requis).

Modules

Import with import "std/io" and use io.

Available stdlib modules: io, json, fs, http, str, log, math, time, re, collections, args, path, async, ui, game2d, game3d, plus 2151 simple (std/nom), 102 medium (std/m/nom), 5710 complex (std/c/nom) — see docs/STDLIB_*.md.

std/args

```
import "std/args"
use args

say count()
say at(0)
```

std/path

```
import "std/path"
use path

say join("home", "user")
say basename("/tmp/file.txt")
```

REPL

```
afrilang repl
```

Commandes : :help, :reset, :show, :run, :load fichier.afr, :type expr, :history, :paste / :end, :quit

Foreign Function Interface (FFI)

```
extern from "m" function sin(x number) returns number
```

Bibliothèques autorisées : m, pthread, dl, curl, c.

Types FFI : number → double, text → const char*, bool, pointer.

Identifiants et encodage

- Fichiers source : **UTF-8** obligatoire
- Identifiants : ASCII `[A-Za-z_][A-Za-z0-9_]*` ou **Unicode** (lettres UTF-8)
- Mots réservés : voir section « Syntaxe bilingue »

Gestion de paquets

```
[dependencies]
math = "0.1.0"
```

Commandes : afrilang pkg add math, pkg install, pkg list, pkg publish.

Le lockfile afrilang.lock fige les versions installées. Semver 2.0 appliqué à l'installation.

Grammaire (EBNF, extrait)

```
program     = { declaration | statement } ;
declaration = class_decl | function_decl | enum_decl | module_decl | extern_decl ;
statement   = "say" expression
              | "create" identifier [ type ] "=" expression
              | "if" expression "then" block [ "else" block ] "end"
              | "match" expression match_arm { match_arm } "end" ;
expression  = literal | identifier | call | binary_op | "match" expression ... ;
```

Politique de compatibilité

- **1.x** : compatibilité source ascendante pour les programmes valides 1.0
- Numéros d'erreur stables (`E3002`, etc.)
- Variable d'environnement `AFRILANG_LOCALE=en|fr` pour les messages

Complément README — syntaxe

Syntaxe du langage

Affichage

```
say "Bonjour le monde"
```

Variables

```
create x = 42
create name = "Alice"
create x number = 42
create flag bool = true
set x = 100
```

Booléens

```
create active bool = true
create ok = yes
create failed = no

if active is equal to true then
    say "Actif"
else
    say "Inactif"
end
```

Types booléens : true, false, yes, no, type bool.

Listes

```
create nums = list of 1, 2, 3
create bracket = [10, 20, 30]
create empty = empty list number

say nums at 0
say bracket[2]
say length of nums

set nums at 1 = 99
add 4 to nums
```

```
for each item in nums do
    say item
end
```

Entrée utilisateur

```
ask "What is your name?" into name
say name

ask "Age:" into age as number
say "Next year you will be " + (age + 1)

ask "Price:" into price as number
ask "VAT (%):" into vat as number
create ht = price / (1 + vat / 100)
say "Excl. tax: " + ht
```

ask déclare automatiquement la variable. Sans as, le type est text. Avec as number, la saisie est convertie en nombre.

Classes et méthodes

```
class Person
    function speak()
        say "Bonjour"
    end
end

create person = new Person
person.speak()
```

Champs et constructeurs

```
class Person
    public field name text
    private field age number

    function init(aName text, aAge number)
        set this.name = aName
        set this.age = aAge
    end

    function greet()
        say name
    end
end

create bob = new Person
set bob.name = "Bob"
```

Records (structs)

```
record Point
    field x number
    field y number
end
```

Modules

```
module Math
    function add(a number, b number) returns number
        return a + b
    end
end

use Math
say add(3, 4)
```

Imports multi-fichiers

```
import "helpers.afr"
```

Les fichiers importés apportent leurs déclarations (fonctions, classes, records, modules).

Héritage

```
class Animal
    function speak()
        say "..."
    end
end

class Dog extends Animal
    function speak()
        say "Woof!"
    end
end
```

Fonctions avec paramètres et retour

```
function add(a number, b number) returns number
    return a + b
end
```

Types disponibles : number, text, bool, list number, list text, ou nom de classe/record.

Conditions

```
if age is greater than 18 then
    say "Adulte"
else
    say "Mineur"
end

if score is equal to 100 then
    say "Parfait"
end

if x is not equal to 0 then
    say "Non nul"
end
```

Boucles

```
repeat 5 times
    say "Hello"
end

while count is less than 10 do
    say count
    set count = count + 1
end

for each item in nums do
    say item
end
```

Contrôle de boucle

```
while true do
    if done is equal to true then
        stop
    end
    if skipNext is equal to true then
        skip
    end
end
```

- `stop` → `break`
- `skip` → `continue`

Opérateurs mathématiques

```
create result = 10 + 5 * 2
say x - y
say a / b
```

Result / gestion d'erreurs

Les fonctions peuvent retourner une valeur ou une erreur :

```
function divide(a number, b number) returns number or error
    if b is equal to 0 then
        return error "Division par zero"
    end
    return a / b
end

create ok = divide(10, 2)
if ok is error then
    say ok.message
else
    say ok.value
end
```

- `returns T or error` — type de retour Result
- `return error "message"` — retourne une erreur
- `if x is error then` — teste si le Result est une erreur
- `.value` et `.message` — accès aux champs du Result

Interfaces (traits)

```
interface Speakable
    function speak()
end

interface Named
    function getName() returns text
end

class Dog extends Animal implements Speakable, Named
    function speak()
        say "Woof!"
    end

    function getName() returns text
        return "Rex"
    end
end
```

Les interfaces définissent des signatures de méthodes ; les classes qui les implémentent doivent fournir les méthodes correspondantes.

Tests et assertions

```
function add(a number, b number) returns number
    return a + b
end

test "addition works"
    create r = add(2, 3)
    assert r is equal to 5
end
```

Les blocs test sont compilés et exécutés automatiquement au lancement du programme. assert vérifie une condition et incrémente un compteur d'échecs en cas d'erreur.

Extension VS Code / Cursor

L'extension dans vscode-afrilang/ fournit :

- **LSP complet** — diagnostics en temps réel, complétion des mots-clés, formatage (`Shift+Alt+F`)
- **Coloration** — syntaxe anglaise + enums, match, null-safety
- **Commandes** — exécuter / vérifier un fichier `.afr` depuis l'éditeur
- **Icône** — carte d'Afrique dans Extensions et à côté des fichiers `.afr`

Installation :

```
cd build && cmake .. && make          # compiler afrilang
cd ../vscode-afrilang
npm install
code --install-extension .              # Cursor: cursor --install-extension .
```

Configuration (settings.json) :

```
{
  "afrilang.serverPath": "/chemin/vers/AFRILANG/build/afrilang",
  "editor.formatOnSave": true,
  "[afrilang]": { "editor.tabSize": 4 }
}
```

Voir vscode-afrilang/README.md pour le détail.

Formateur (`afrilang fmt`)

Normalise l'indentation (4 espaces) et la syntaxe naturelle :

```
afrilang fmt examples/hello.afr       # affiche sur stdout
afrilang fmt examples/hello.afr -w    # réécrit le fichier
```

REPL interactif

```
afrilang repl
>>> create x = 42
>>> say x + 8
50
>>> :help
>>> :quit
```

Commandes : :help, :reset, :show, :paste / :end, :quit.

Symboles de debug

Le code C++ généré inclut des directives #line pointant vers le fichier .afr source, et g++ est invoqué avec -g. Vous pouvez débugger avec GDB en suivant les numéros de ligne AFRILANG :

```
afrilang run examples/hello.afr --emit
gdb ./hello
```

FFI (Foreign Function Interface)

Appeler des fonctions C/C++ depuis AFRILANG :

```
extern from "m" function sin(x number) returns number
extern from "m" function sqrt(x number) returns number

say sin(3.14159 / 2)
say sqrt(16)
```

Bibliothèques supportées : m/libm → -lm, pthread → -lpthread, etc.

Types FFI : number, text (→ const char), pointer (→ void).

Gestionnaire de paquets (`afrilang pkg`)

```
afrilang pkg list              # paquets disponibles dans packages/
afrilang pkg search math       # recherche par nom ou description
afrilang pkg add math          # copie dans vendor/ + afrilang.toml
afrilang pkg install           # installe toutes les dépendances
afrilang pkg publish ./my_pkg  # publie dans packages/ (registre local)
```

Registre local : packages/index.json (généré à la publication).

Dans afrilang.toml :

```
[dependencies]
math = "0.1.0"
```

Import dans le code :

```
import "pkg/math/math.afr"
use Math
say square(5)
```

Cross-compilation

```
afrilang build --target linux-arm64
afrilang run examples/hello.afr --target wasm32
```

Cibles : native (défaut), linux-x64, linux-arm64, wasm32 (nécessite le toolchain correspondant).

Playground web

```
afrilang serve        # http://localhost:8080
afrilang serve 3000   # port personnalisé
```

Le site dans site/ propose un éditeur en ligne avec :

- exemples préchargés (Hello, OOP, enums/match, generics)
- exécution via `/api/run`
- formatage via `/api/fmt`
- raccourci **Ctrl+Enter** pour exécuter

Syntaxe anglaise

AFRILANG utilise une syntaxe en anglais à la manière du langage naturel :

```
if age is greater than 18 then
    say "Adult"
else
    say "Minor"
end

repeat 3 times
    say "Hello"
end
```

Mode éducatif

Dans le code — prefixe explain :

```
explain say "Hello"
explain create x = 10
```

Affiche une explication pédagogique puis exécute l'instruction.

En CLI :

```
afrilang explain examples/educational.afr
afrilang explain examples/hello.afr --line 1
```

Enums, pattern matching et null-safety

Enums — sommes algébriques avec champs optionnels par cas :

```
enum Status
    case Ok
    case Error with message text
end

create s = Status.Ok
create e = Status.Error with "failed"
```

Pattern matching — match sur une valeur enum :

```
match s
    case Ok then
        say "ok"
    end
    default
        say "other"
    end
end
```

Null-safety — types optionnels (text?) et nothing :

```
create nickname text? = nothing
if nickname is defined then
    say nickname
end
```

Generics

Type parameters on functions with inference at call sites:

```
function identity<T>(x T) returns T
    return x
end

function first<T>(items list of T) returns T
    return items at 0
end

say identity(42)
say identity("hello")

create nums = list of 10, 20, 30
say first(nums)

create words = list of "a", "b", "c"
say first(words)
```

Generics transpile to C++ template<typename T> functions. Types are inferred from arguments (42 → number, "hello" → text, list element types from list of ...).

Partie V — Entrée utilisateur (ask)

```
ask "Name:" into name
ask "Age:" into age as number
say "Hi " + name + " " + age
```

- Auto-déclaration de variable
- as number / as text
- say et + convertissent les nombres
- Mode console interactive (sans timeout sandbox)

Exemple PAHT

```
say "=== PAHT calculator ==="
say ""

ask "Last name: " into lastName
ask "First name: " into firstName
ask "Unit price (TTC): " into unitPrice as number
ask "VAT rate (%): " into vat as number

create paht = unitPrice / (1 + vat / 100)

say ""
say "Client: " + firstName + " " + lastName
say "Unit price TTC: " + unitPrice
say "VAT: " + vat + " %"
say "PAHT (purchase price excl. tax): " + paht
```

Partie VI — POO, modules, async, FFI, tests

```
import "std/json"
use json
```

```
extern from "m" function cos(x number) returns number
```

```
test "addition"
    assert 1 + 1 is equal to 2
end
```

GUI/jeux : std/ui, std/game2d, std/game3d, examples/snake.afr, gui_demo.afr.

Partie VII — Guides pédagogiques

Leçon 1 — Afficher

say écrit sur stdout.

```
say "Hello"
```

Leçon 2 — Variables

create et set.

```
create x = 1
set x = 2
```

Leçon 3 — Conditions

Comparaisons naturelles.

```
if x is greater than 0 then
    say "ok"
end
```

Leçon 4 — Boucles

repeat / while / for each.

```
repeat 3 times
    say "x"
end
```

Leçon 5 — Listes

list → vector.

```
create n = list of 1, 2
add 3 to n
```

Leçon 6 — Fonctions

Réutilisation.

```
function add(a number, b number) returns number
    return a + b
end
```

Leçon 7 — ask

Saisie typée.

```
ask "n:" into n as number
```

Leçon 8 — Classes

POO.

```
class A
    function hi()
        say "hi"
    end
end
```

Leçon 9 — std

Core stdlib.

```
import "std/math"
use math
say sqrt(9)
```

Leçon 10 — Projet

Cycle de vie.

```
afrilang init app
afrilang run src/main.afr
```

Intermédiaire / avancé

```
function id<T>(v T) returns T
    return v
end
```

```
afrilang run app.afr --target wasm32
```

Partie VIII — Bibliothèque standard

- Core stabilisé (runtime C++)
- Simple ~2150 modules (std/nom)
- Medium ~100 modules (std/m/nom)
- Complex ~5700 modules (std/c/nom)
- ~7971 modules documentés

Core

Core stdlib AFRILANG

La masse de fichiers sous stdlib/ (catalogues générés) sert surtout à la complétion / signatures IDE. Seuls les modules ci-dessous sont considérés « core » : backend C++ réel dans runtime/, compatibilité 1.x, documentation prioritaire.

Voir aussi STDLIB_API.md (API prioritaire Vague 3), STDLIB_GEN.md, et isLegacyStdlibModule dans src/utils/stdlib_registry.cpp. Suite de tests : afrilang test --specs.

Modules core (stabilisés)

```
| Module | Import typique | Domaine |
|--------|----------------|---------|
| io | `std/io` | Entrées / sorties |
| json | `std/json` | JSON |
| fs | `std/fs` | Fichiers |
| http | `std/http` | HTTP sync |
| str | `std/str` | Chaînes |
| log / logging | `std/log` | Journalisation |
| math | `std/math` | Math |
| time / chrono | `std/time` | Temps |
| re | `std/re` | Regex |
| collections | `std/collections` | Structures |
| args | `std/args` | CLI args |
| path | `std/path` | Chemins |
| async | `std/async` | Coroutines |
| ui | `std/ui` | Fenêtres (natif) |
| game2d / game3d | `std/game2d` | Jeux |
| env | `std/env` | Environnement |
| random | `std/random` | Aléatoire |
| crypto | `std/crypto` | Crypto de base |
| csv / yaml / html | `std/csv` … | Formats |
| uuid / base64 / hex / url | … | Utilitaires |
```

Hors core (expérimental / généré)

- `stdlib/*.afr`, `stdlib/m/`, `stdlib/c/` — stubs de signatures, **pas** une garantie de profondeur API
- Packs « ultra » (GIS, data, …) — utiles en démo ; stabilité au cas par cas

Politique

1. Bugs et docs prioritaires sur le core. 2. Un module généré n’entre dans le core qu’avec runtime réel + tests + entrée ici. 3. Les catalogues générés peuvent évoluer sans promesse semver stricte.

API core

Core stdlib API (Vague 3)

Documentation prioritaire des backends réels (runtime/*.hpp). Les catalogues générés (stdlib/m/, stdlib/c/, stubs massifs) restent expérimentaux.

Voir aussi CORE_STDLIB.md et afrilang test --specs.

`std/str`

```
| Fonction | Rôle |
|----------|------|
| `trim(text)` | Enlève espaces / tabs / newlines |
| `contains(text, needle)` | Sous-chaîne |
| `replace(text, from, to)` | Remplacement global |
| `split(text, sep)` | Découpe en liste |
| `join(parts, sep)` | Jointure |
| `toString(...)` | Conversion |
```

Tests : tests/stdlib/str.afr

`std/math`

```
| Fonction | Rôle |
|----------|------|
| `abs(n)` | Valeur absolue |
| `floor(n)` / `ceil(n)` | Entiers autour |
| `pow(base, exp)` | Puissance |
| `random()` | `[0, 1)` |
```

Tests : tests/stdlib/math.afr

`std/json`

```
| Fonction | Rôle |
|----------|------|
| `makeObject(key, value)` | Objet simple |
| `parse(text)` / `stringify(doc)` | Sérialisation |
| `getString` / `getNumber` | Lecture |
```

Tests : tests/stdlib/json.afr

WASM

Compatibles wasm32 (voir WASM_COMPAT.md) : str, math, json, collections, path, log, async (subset). Natif seulement : fs/io fichiers, http, ui, game*.

Démo : examples/tier8_stdlib.afr

SIMPLE — référence complète

Bibliothèques simples AFRILANG

2151 modules importables via import "std/nom" puis use nom.

std/absnum

import "std/absnum" · use absnum

- `absNumber(n number) → number`
- `absDiff(a number, b number) → number`

std/sqrtlib

import "std/sqrtlib" · use sqrtlib

- `sqrtNumber(n number) → number`
- `square(n number) → number`

std/cbrtlib

import "std/cbrtlib" · use cbrtlib

- `cbrtNumber(n number) → number`
- `cube(n number) → number`

std/min2

import "std/min2" · use min2

- `min2(a number, b number) → number`
- `min3(a number, b number, c number) → number`

std/max2

import "std/max2" · use max2

- `max2(a number, b number) → number`
- `max3(a number, b number, c number) → number`

std/clamp

import "std/clamp" · use clamp

- `clamp(n number, lo number, hi number) → number`
- `clamp01(n number) → number`

std/lerp

import "std/lerp" · use lerp

- `lerp(a number, b number, t number) → number`
- `inverseLerp(a number, b number, v number) → number`

std/sign

import "std/sign" · use sign

- `sign(n number) → number`
- `isPositive(n number) → bool`

std/parity

import "std/parity" · use parity

- `isEven(n number) → bool`
- `isOdd(n number) → bool`

std/trig

import "std/trig" · use trig

- `sinDeg(deg number) → number`
- `cosDeg(deg number) → number`

std/trig2

import "std/trig2" · use trig2

- `tanDeg(deg number) → number`
- `cotDeg(deg number) → number`

std/arc

import "std/arc" · use arc

- `asinSafe(n number) → number`
- `acosSafe(n number) → number`

std/hyper

import "std/hyper" · use hyper

- `sinhN(n number) → number`
- `coshN(n number) → number`

std/deg

import "std/deg" · use deg

- `degToRad(d number) → number`
- `radToDeg(r number) → number`

std/roundlib

import "std/roundlib" · use roundlib

- `roundN(n number) → number`
- `truncN(n number) → number`

std/modlib

import "std/modlib" · use modlib

- `mod(a number, b number) → number`
- `wrap(n number, lo number, hi number) → number`

std/gcdlib

import "std/gcdlib" · use gcdlib

- `gcd(a number, b number) → number`
- `lcm(a number, b number) → number`

std/fact

import "std/fact" · use fact

- `factorial(n number) → number`
- `doubleFact(n number) → number`

std/fib

import "std/fib" · use fib

- `fib(n number) → number`
- `fibB(n number) → number`

std/prime

import "std/prime" · use prime

- `isPrime(n number) → bool`
- `nextPrime(n number) → number`

std/mean

import "std/mean" · use mean

- `mean2(a number, b number) → number`
- `mean3(a number, b number, c number) → number`

std/percent

import "std/percent" · use percent

- `percent(part number, whole number) → number`
- `percentChange(oldVal number, newVal number) → number`

std/ratio

import "std/ratio" · use ratio

- `ratio(a number, b number) → number`
- `proportion(a number, b number, total number) → number`

std/dice

import "std/dice" · use dice

- `roll(sides number) → number`
- `roll2d6(ignored number) → number`

std/coin

import "std/coin" · use coin

- `flip(ignored number) → bool`
- `flipN(n number) → number`

std/upper

import "std/upper" · use upper

- `toUpper(s text) → text`
- `toLower(s text) → text`

std/capitalize

import "std/capitalize" · use capitalize

- `capitalize(s text) → text`
- `titleWord(s text) → text`

std/prefix

import "std/prefix" · use prefix

- `startsWith(s text, prefix text) → bool`
- `endsWith(s text, suffix text) → bool`

std/repeat

import "std/repeat" · use repeat

- `repeatText(s text, n number) → text`
- `repeatChar(c text, n number) → text`

std/pad

import "std/pad" · use pad

- `padLeft(s text, n number, ch text) → text`
- `padRight(s text, n number, ch text) → text`

std/reverse

import "std/reverse" · use reverse

- `reverseText(s text) → text`
- `reverseWords(s text) → text`

std/textlen

import "std/textlen" · use textlen

- `length(s text) → number`
- `isEmpty(s text) → bool`

std/concat

import "std/concat" · use concat

- `concat2(a text, b text) → text`
- `concat3(a text, b text, c text) → text`

std/compare

import "std/compare" · use compare

- `equals(a text, b text) → bool`
- `equalsIgnoreCase(a text, b text) → bool`

std/celsius

import "std/celsius" · use celsius

- `cToF(c number) → number`
- `fToC(f number) → number`

std/kelvin

import "std/kelvin" · use kelvin

- `cToK(c number) → number`
- `kToC(k number) → number`

std/distance

import "std/distance" · use distance

- `kmToMiles(km number) → number`
- `milesToKm(mi number) → number`

std/weight

import "std/weight" · use weight

- `kgToLb(kg number) → number`
- `lbToKg(lb number) → number`

std/bytes

import "std/bytes" · use bytes

- `bytesToKb(b number) → number`
- `kbToMb(kb number) → number`

std/timeconv

import "std/timeconv" · use timeconv

- `secToMin(s number) → number`
- `minToHour(m number) → number`

std/msconv

import "std/msconv" · use msconv

- `msToSec(ms number) → number`
- `secToMs(s number) → number`

std/angle

import "std/angle" · use angle

- `normalizeAngle(d number) → number`
- `angleDiff(a number, b number) → number`

std/hex

import "std/hex" · use hex

- `toHex(n number) → text`
- `fromHex(s text) → number`

std/binary

import "std/binary" · use binary

- `toBinary(n number) → text`
- `fromBinary(s text) → number`

std/octal

import "std/octal" · use octal

- `toOctal(n number) → text`
- `fromOctal(s text) → number`

std/rot13

import "std/rot13" · use rot13

- `rot13(s text) → text`
- `rot47(s text) → text`

std/hash

import "std/hash" · use hash

- `hashText(s text) → number`
- `hashNumber(n number) → number`

std/checksum

import "std/checksum" · use checksum

- `xorChecksum(s text) → number`
- `sumBytes(s text) → number`

std/slug

import "std/slug" · use slug

- `slugify(s text) → text`
- `deslugify(s text) → text`

std/email

import "std/email" · use email

- `isEmail(s text) → bool`
- `domainOf(s text) → text`

std/phone

import "std/phone" · use phone

- `isPhone(s text) → bool`
- `digitsOnly(s text) → text`

std/ipv4

import "std/ipv4" · use ipv4

- `isIpv4(s text) → bool`
- `countDots(s text) → number`

std/html

import "std/html" · use html

- `escapeHtml(s text) → text`
- `unescapeHtml(s text) → text`

std/point

import "std/point" · use point

- `distance2d(x1 number, y1 number, x2 number, y2 number) → number`
- `midpointX(x1 number, x2 number) → number`

std/rect

import "std/rect" · use rect

- `area(w number, h number) → number`
- `perimeter(w number, h number) → number`

std/circle

import "std/circle" · use circle

- `area(r number) → number`
- `circumference(r number) → number`

std/triangle

import "std/triangle" · use triangle

- `areaBaseHeight(b number, h number) → number`
- `heron(a number, b number, c number) → number`

std/vector2

import "std/vector2" · use vector2

- `dot2(x1 number, y1 number, x2 number, y2 number) → number`
- `length2(x number, y number) → number`

std/collision

import "std/collision" · use collision

- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`
- `rectsOverlap(x1 number, y1 number, w1 number, h1 number, x2 number, y2 number, w2 number, h2 number) → bool`

std/color

import "std/color" · use color

- `rgb(r number, g number, b number) → number`
- `redOf(packed number) → number`

std/easing

import "std/easing" · use easing

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/noise

import "std/noise" · use noise

- `noise1D(x number) → number`
- `noise2D(x number, y number) → number`

std/semver

import "std/semver" · use semver

- `parseMajor(s text) → number`
- `bumpMajor(s text) → text`

std/validate

import "std/validate" · use validate

- `isAlpha(s text) → bool`
- `isDigit(s text) → bool`

std/parse

import "std/parse" · use parse

- `parseInt(s text) → number`
- `parseFloat(s text) → number`

std/format

import "std/format" · use format

- `formatNumber(n number, decimals number) → text`
- `formatPercent(n number) → text`

std/currency

import "std/currency" · use currency

- `formatEuro(n number) → text`
- `formatDollar(n number) → text`

std/stopwatch

import "std/stopwatch" · use stopwatch

- `nowMs(ignored number) → number`
- `elapsedMs(start number) → number`

std/bits

import "std/bits" · use bits

- `bitAnd(a number, b number) → number`
- `bitOr(a number, b number) → number`

std/shift

import "std/shift" · use shift

- `shl(n number, b number) → number`
- `shr(n number, b number) → number`

std/flags

import "std/flags" · use flags

- `hasFlag(n number, f number) → bool`
- `setFlag(n number, f number) → number`

std/order

import "std/order" · use order

- `cmpNumber(a number, b number) → number`
- `cmpText(a text, b text) → number`

std/range

import "std/range" · use range

- `inRange(n number, lo number, hi number) → bool`
- `rangeSize(lo number, hi number) → number`

std/seq

import "std/seq" · use seq

- `arithmetic(a number, d number, n number) → number`
- `geometric(a number, r number, n number) → number`

std/loglib

import "std/loglib" · use loglib

- `log10(n number) → number`
- `log2(n number) → number`

std/powlib

import "std/powlib" · use powlib

- `pow2(n number) → number`
- `pow10(n number) → number`

std/hypot

import "std/hypot" · use hypot

- `hypot2(a number, b number) → number`
- `hypot3(a number, b number, c number) → number`

std/stats2

import "std/stats2" · use stats2

- `sum2(a number, b number) → number`
- `product2(a number, b number) → number`

std/stats3

import "std/stats3" · use stats3

- `sum3(a number, b number, c number) → number`
- `product3(a number, b number, c number) → number`

std/diff

import "std/diff" · use diff

- `delta(a number, b number) → number`
- `absDelta(a number, b number) → number`

std/avg

import "std/avg" · use avg

- `avg4(a number, b number, c number, d number) → number`
- `avg5(a number, b number, c number, d number, e number) → number`

std/margin

import "std/margin" · use margin

- `margin(cost number, price number) → number`
- `markup(cost number, price number) → number`

std/tax

import "std/tax" · use tax

- `addTax(amount number, rate number) → number`
- `taxAmount(amount number, rate number) → number`

std/discount

import "std/discount" · use discount

- `applyDiscount(price number, rate number) → number`
- `discountAmount(price number, rate number) → number`

std/interest

import "std/interest" · use interest

- `simpleInterest(principal number, rate number, years number) → number`
- `compoundOnce(principal number, rate number) → number`

std/loan

import "std/loan" · use loan

- `monthlyPayment(principal number, rate number, months number) → number`
- `totalPaid(payment number, months number) → number`

std/speed

import "std/speed" · use speed

- `kmhToMs(kmh number) → number`
- `msToKmh(ms number) → number`

std/volume

import "std/volume" · use volume

- `litersToMl(l number) → number`
- `mlToLiters(ml number) → number`

std/area_conv

import "std/area_conv" · use area_conv

- `m2ToCm2(m2 number) → number`
- `cm2ToM2(cm2 number) → number`

std/pressure

import "std/pressure" · use pressure

- `barToPsi(bar number) → number`
- `psiToBar(psi number) → number`

std/energy

import "std/energy" · use energy

- `joulesToCal(j number) → number`
- `calToJoules(cal number) → number`

std/bmi

import "std/bmi" · use bmi

- `bmi(kg number, m number) → number`
- `bmiCategory(bmi number) → text`

std/geo

import "std/geo" · use geo

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `midLat(a number, b number) → number`

std/compass

import "std/compass" · use compass

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `cardinal(deg number) → text`

std/uuid

import "std/uuid" · use uuid

- `randomHex(n number) → text`
- `simpleId(ignored number) → text`

std/counter

import "std/counter" · use counter

- `nextId(ignored number) → number`
- `resetId(ignored number) → number`

std/env

import "std/env" · use env

- `getEnv(key text) → text`
- `hasEnv(key text) → bool`

std/platform

import "std/platform" · use platform

- `osName(ignored number) → text`
- `archName(ignored number) → text`

std/pathutil

import "std/pathutil" · use pathutil

- `normalizeSlashes(s text) → text`
- `hasTrailingSlash(s text) → bool`

std/url

import "std/url" · use url

- `urlEncodeSpace(s text) → text`
- `urlDecodeSpace(s text) → text`

std/csv

import "std/csv" · use csv

- `splitCsv2(line text) → text`
- `splitCsvRest(line text) → text`

std/jsonlite

import "std/jsonlite" · use jsonlite

- `quote(s text) → text`
- `pair(key text, value text) → text`

std/xml

import "std/xml" · use xml

- `tag(name text, content text) → text`
- `escapeXml(s text) → text`

std/markdown

import "std/markdown" · use markdown

- `bold(s text) → text`
- `italic(s text) → text`

std/password

import "std/password" · use password

- `strengthScore(s text) → number`
- `isStrong(s text) → bool`

std/base64

import "std/base64" · use base64

- `encode64(s text) → text`
- `decode64(s text) → text`

std/mime

import "std/mime" · use mime

- `isTextPlain(mime text) → bool`
- `isJson(mime text) → bool`

std/httpcode

import "std/httpcode" · use httpcode

- `isSuccess(code number) → bool`
- `isRedirect(code number) → bool`

std/game

import "std/game" · use game

- `scoreBonus(score number, level number) → number`
- `levelFromXp(xp number) → number`
- `xpForScore(score number) → number`
- `speedMsForLevel(level number) → number`

std/music

import "std/music" · use music

- `midiNote(octave number, note number) → number`
- `freqA4(ignored number) → number`

std/color2

import "std/color2" · use color2

- `lerpColor(a number, b number, t number) → number`
- `invertRgb(packed number) → number`

std/matrix2

import "std/matrix2" · use matrix2

- `det2(a number, b number, c number, d number) → number`
- `trace2(a number, d number) → number`

std/physics

import "std/physics" · use physics

- `force(m number, a number) → number`
- `kineticEnergy(m number, v number) → number`

std/chemistry

import "std/chemistry" · use chemistry

- `moles(mass number, molarMass number) → number`
- `mass(moles number, molarMass number) → number`

std/units

import "std/units" · use units

- `inchToCm(inch number) → number`
- `cmToInch(cm number) → number`

std/paper

import "std/paper" · use paper

- `a4WidthCm(ignored number) → number`
- `a4HeightCm(ignored number) → number`

std/roman

import "std/roman" · use roman

- `toRoman(n number) → text`
- `fromRoman(s text) → number`

std/spell

import "std/spell" · use spell

- `spellDigit(n number) → text`
- `spellBool(b bool) → text`

std/listutil

import "std/listutil" · use listutil

- `firstIndex(ignored number) → number`
- `lastIndex(count number) → number`

std/queue

import "std/queue" · use queue

- `wrapIndex(i number, size number) → number`
- `nextIndex(i number, size number) → number`

std/stackutil

import "std/stackutil" · use stackutil

- `peekOffset(size number, offset number) → number`
- `isEmptyStack(size number) → bool`

std/graph

import "std/graph" · use graph

- `edgeKey(a number, b number) → number`
- `selfLoop(a number, b number) → bool`

std/search

import "std/search" · use search

- `binarySteps(n number) → number`
- `linearWorst(n number) → number`

std/sortkey

import "std/sortkey" · use sortkey

- `ascKey(n number) → number`
- `descKey(n number) → number`

std/textwrap

import "std/textwrap" · use textwrap

- `truncate(s text, n number) → text`
- `ellipsize(s text, n number) → text`

std/whitespace

import "std/whitespace" · use whitespace

- `collapseSpaces(s text) → text`
- `isWhitespace(s text) → bool`

std/charutil

import "std/charutil" · use charutil

- `charCode(s text) → number`
- `fromCharCode(n number) → text`

std/version

import "std/version" · use version

- `compareMajor(a text, b text) → number`
- `sameMajor(a text, b text) → bool`

std/config

import "std/config" · use config

- `parseBool(s text) → bool`
- `boolToText(b bool) → text`

std/retry

import "std/retry" · use retry

- `backoffMs(attempt number) → number`
- `shouldRetry(attempt number, max number) → bool`

std/ratelimit

import "std/ratelimit" · use ratelimit

- `allowBurst(count number, limit number) → bool`
- `tokensLeft(count number, limit number) → number`

std/cache

import "std/cache" · use cache

- `ttlExpired(ageMs number, ttlMs number) → bool`
- `freshness(ageMs number, ttlMs number) → number`

std/id

import "std/id" · use id

- `hashId(s text) → text`
- `shortHash(s text) → text`

std/locale

import "std/locale" · use locale

- `decimalComma(s text) → text`
- `decimalDot(s text) → text`

std/calendar

import "std/calendar" · use calendar

- `daysInYear(ignored number) → number`
- `isLeap(year number) → bool`

std/weekday

import "std/weekday" · use weekday

- `weekIndex(day number) → number`
- `isWeekendIndex(day number) → bool`

std/timezone

import "std/timezone" · use timezone

- `utcOffsetHours(ignored number) → number`
- `localHour(utcHour number) → number`

std/network

import "std/network" · use network

- `isPortValid(port number) → bool`
- `isLocalhost(host text) → bool`

std/dns

import "std/dns" · use dns

- `isValidHostname(s text) → bool`
- `hasSubdomain(s text) → bool`

std/socket

import "std/socket" · use socket

- `portPairKey(a number, b number) → number`
- `isEphemeralPort(port number) → bool`

std/process

import "std/process" · use process

- `exitOk(code number) → bool`
- `exitFailed(code number) → bool`

std/shell

import "std/shell" · use shell

- `quoteShell(s text) → text`
- `escapeShell(s text) → text`

std/memory

import "std/memory" · use memory

- `kbFromBytes(b number) → number`
- `mbFromBytes(b number) → number`

std/cpu

import "std/cpu" · use cpu

- `clampUsage(pct number) → number`
- `isHighLoad(pct number) → bool`

std/disk

import "std/disk" · use disk

- `usagePercent(used number, total number) → number`
- `freeSpace(used number, total number) → number`

std/security

import "std/security" · use security

- `maskSecret(s text) → text`
- `isMasked(s text) → bool`

std/audit

import "std/audit" · use audit

- `auditLine(action text, user text) → text`
- `timestampPrefix(msg text) → text`

std/tmpl

import "std/tmpl" · use tmpl

- `fill(pattern text, value text) → text`
- `fill2(pattern text, a text, b text) → text`

std/i18n

import "std/i18n" · use i18n

- `greetFr(name text) → text`
- `greetEn(name text) → text`

std/education

import "std/education" · use education

- `gradeLetter(score number) → text`
- `passed(score number, min number) → bool`

std/quiz

import "std/quiz" · use quiz

- `scorePercent(correct number, total number) → number`
- `isPerfect(correct number, total number) → bool`

std/gamekit001

import "std/gamekit001" · use gamekit001

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit002

import "std/gamekit002" · use gamekit002

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit003

import "std/gamekit003" · use gamekit003

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit004

import "std/gamekit004" · use gamekit004

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit005

import "std/gamekit005" · use gamekit005

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit006

import "std/gamekit006" · use gamekit006

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit007

import "std/gamekit007" · use gamekit007

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit008

import "std/gamekit008" · use gamekit008

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit009

import "std/gamekit009" · use gamekit009

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit010

import "std/gamekit010" · use gamekit010

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit011

import "std/gamekit011" · use gamekit011

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit012

import "std/gamekit012" · use gamekit012

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit013

import "std/gamekit013" · use gamekit013

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit014

import "std/gamekit014" · use gamekit014

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit015

import "std/gamekit015" · use gamekit015

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit016

import "std/gamekit016" · use gamekit016

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit017

import "std/gamekit017" · use gamekit017

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit018

import "std/gamekit018" · use gamekit018

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit019

import "std/gamekit019" · use gamekit019

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit020

import "std/gamekit020" · use gamekit020

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit021

import "std/gamekit021" · use gamekit021

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit022

import "std/gamekit022" · use gamekit022

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit023

import "std/gamekit023" · use gamekit023

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit024

import "std/gamekit024" · use gamekit024

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit025

import "std/gamekit025" · use gamekit025

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit026

import "std/gamekit026" · use gamekit026

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit027

import "std/gamekit027" · use gamekit027

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit028

import "std/gamekit028" · use gamekit028

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit029

import "std/gamekit029" · use gamekit029

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit030

import "std/gamekit030" · use gamekit030

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit031

import "std/gamekit031" · use gamekit031

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit032

import "std/gamekit032" · use gamekit032

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit033

import "std/gamekit033" · use gamekit033

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit034

import "std/gamekit034" · use gamekit034

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit035

import "std/gamekit035" · use gamekit035

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit036

import "std/gamekit036" · use gamekit036

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit037

import "std/gamekit037" · use gamekit037

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit038

import "std/gamekit038" · use gamekit038

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit039

import "std/gamekit039" · use gamekit039

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit040

import "std/gamekit040" · use gamekit040

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit041

import "std/gamekit041" · use gamekit041

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit042

import "std/gamekit042" · use gamekit042

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit043

import "std/gamekit043" · use gamekit043

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit044

import "std/gamekit044" · use gamekit044

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit045

import "std/gamekit045" · use gamekit045

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit046

import "std/gamekit046" · use gamekit046

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit047

import "std/gamekit047" · use gamekit047

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit048

import "std/gamekit048" · use gamekit048

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit049

import "std/gamekit049" · use gamekit049

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit050

import "std/gamekit050" · use gamekit050

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit051

import "std/gamekit051" · use gamekit051

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit052

import "std/gamekit052" · use gamekit052

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit053

import "std/gamekit053" · use gamekit053

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit054

import "std/gamekit054" · use gamekit054

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit055

import "std/gamekit055" · use gamekit055

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit056

import "std/gamekit056" · use gamekit056

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit057

import "std/gamekit057" · use gamekit057

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit058

import "std/gamekit058" · use gamekit058

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit059

import "std/gamekit059" · use gamekit059

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit060

import "std/gamekit060" · use gamekit060

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit061

import "std/gamekit061" · use gamekit061

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit062

import "std/gamekit062" · use gamekit062

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit063

import "std/gamekit063" · use gamekit063

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit064

import "std/gamekit064" · use gamekit064

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit065

import "std/gamekit065" · use gamekit065

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit066

import "std/gamekit066" · use gamekit066

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit067

import "std/gamekit067" · use gamekit067

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit068

import "std/gamekit068" · use gamekit068

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit069

import "std/gamekit069" · use gamekit069

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit070

import "std/gamekit070" · use gamekit070

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit071

import "std/gamekit071" · use gamekit071

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit072

import "std/gamekit072" · use gamekit072

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit073

import "std/gamekit073" · use gamekit073

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit074

import "std/gamekit074" · use gamekit074

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit075

import "std/gamekit075" · use gamekit075

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit076

import "std/gamekit076" · use gamekit076

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit077

import "std/gamekit077" · use gamekit077

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit078

import "std/gamekit078" · use gamekit078

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit079

import "std/gamekit079" · use gamekit079

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit080

import "std/gamekit080" · use gamekit080

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit081

import "std/gamekit081" · use gamekit081

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit082

import "std/gamekit082" · use gamekit082

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit083

import "std/gamekit083" · use gamekit083

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit084

import "std/gamekit084" · use gamekit084

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit085

import "std/gamekit085" · use gamekit085

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit086

import "std/gamekit086" · use gamekit086

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit087

import "std/gamekit087" · use gamekit087

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit088

import "std/gamekit088" · use gamekit088

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit089

import "std/gamekit089" · use gamekit089

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit090

import "std/gamekit090" · use gamekit090

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit091

import "std/gamekit091" · use gamekit091

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit092

import "std/gamekit092" · use gamekit092

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit093

import "std/gamekit093" · use gamekit093

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit094

import "std/gamekit094" · use gamekit094

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit095

import "std/gamekit095" · use gamekit095

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit096

import "std/gamekit096" · use gamekit096

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit097

import "std/gamekit097" · use gamekit097

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit098

import "std/gamekit098" · use gamekit098

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit099

import "std/gamekit099" · use gamekit099

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit100

import "std/gamekit100" · use gamekit100

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit101

import "std/gamekit101" · use gamekit101

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit102

import "std/gamekit102" · use gamekit102

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit103

import "std/gamekit103" · use gamekit103

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit104

import "std/gamekit104" · use gamekit104

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit105

import "std/gamekit105" · use gamekit105

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit106

import "std/gamekit106" · use gamekit106

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit107

import "std/gamekit107" · use gamekit107

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit108

import "std/gamekit108" · use gamekit108

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit109

import "std/gamekit109" · use gamekit109

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit110

import "std/gamekit110" · use gamekit110

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit111

import "std/gamekit111" · use gamekit111

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit112

import "std/gamekit112" · use gamekit112

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit113

import "std/gamekit113" · use gamekit113

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit114

import "std/gamekit114" · use gamekit114

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit115

import "std/gamekit115" · use gamekit115

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit116

import "std/gamekit116" · use gamekit116

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit117

import "std/gamekit117" · use gamekit117

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit118

import "std/gamekit118" · use gamekit118

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit119

import "std/gamekit119" · use gamekit119

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit120

import "std/gamekit120" · use gamekit120

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit121

import "std/gamekit121" · use gamekit121

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit122

import "std/gamekit122" · use gamekit122

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit123

import "std/gamekit123" · use gamekit123

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit124

import "std/gamekit124" · use gamekit124

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit125

import "std/gamekit125" · use gamekit125

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit126

import "std/gamekit126" · use gamekit126

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit127

import "std/gamekit127" · use gamekit127

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit128

import "std/gamekit128" · use gamekit128

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit129

import "std/gamekit129" · use gamekit129

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit130

import "std/gamekit130" · use gamekit130

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit131

import "std/gamekit131" · use gamekit131

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit132

import "std/gamekit132" · use gamekit132

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit133

import "std/gamekit133" · use gamekit133

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit134

import "std/gamekit134" · use gamekit134

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit135

import "std/gamekit135" · use gamekit135

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit136

import "std/gamekit136" · use gamekit136

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit137

import "std/gamekit137" · use gamekit137

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit138

import "std/gamekit138" · use gamekit138

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit139

import "std/gamekit139" · use gamekit139

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit140

import "std/gamekit140" · use gamekit140

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit141

import "std/gamekit141" · use gamekit141

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit142

import "std/gamekit142" · use gamekit142

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit143

import "std/gamekit143" · use gamekit143

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit144

import "std/gamekit144" · use gamekit144

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit145

import "std/gamekit145" · use gamekit145

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit146

import "std/gamekit146" · use gamekit146

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit147

import "std/gamekit147" · use gamekit147

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit148

import "std/gamekit148" · use gamekit148

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit149

import "std/gamekit149" · use gamekit149

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit150

import "std/gamekit150" · use gamekit150

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit151

import "std/gamekit151" · use gamekit151

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit152

import "std/gamekit152" · use gamekit152

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit153

import "std/gamekit153" · use gamekit153

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit154

import "std/gamekit154" · use gamekit154

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit155

import "std/gamekit155" · use gamekit155

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit156

import "std/gamekit156" · use gamekit156

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit157

import "std/gamekit157" · use gamekit157

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit158

import "std/gamekit158" · use gamekit158

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit159

import "std/gamekit159" · use gamekit159

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit160

import "std/gamekit160" · use gamekit160

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit161

import "std/gamekit161" · use gamekit161

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit162

import "std/gamekit162" · use gamekit162

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit163

import "std/gamekit163" · use gamekit163

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit164

import "std/gamekit164" · use gamekit164

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit165

import "std/gamekit165" · use gamekit165

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit166

import "std/gamekit166" · use gamekit166

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit167

import "std/gamekit167" · use gamekit167

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit168

import "std/gamekit168" · use gamekit168

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit169

import "std/gamekit169" · use gamekit169

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit170

import "std/gamekit170" · use gamekit170

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit171

import "std/gamekit171" · use gamekit171

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit172

import "std/gamekit172" · use gamekit172

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit173

import "std/gamekit173" · use gamekit173

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit174

import "std/gamekit174" · use gamekit174

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit175

import "std/gamekit175" · use gamekit175

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit176

import "std/gamekit176" · use gamekit176

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit177

import "std/gamekit177" · use gamekit177

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit178

import "std/gamekit178" · use gamekit178

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit179

import "std/gamekit179" · use gamekit179

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit180

import "std/gamekit180" · use gamekit180

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit181

import "std/gamekit181" · use gamekit181

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit182

import "std/gamekit182" · use gamekit182

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit183

import "std/gamekit183" · use gamekit183

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit184

import "std/gamekit184" · use gamekit184

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit185

import "std/gamekit185" · use gamekit185

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit186

import "std/gamekit186" · use gamekit186

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit187

import "std/gamekit187" · use gamekit187

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit188

import "std/gamekit188" · use gamekit188

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit189

import "std/gamekit189" · use gamekit189

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit190

import "std/gamekit190" · use gamekit190

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit191

import "std/gamekit191" · use gamekit191

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit192

import "std/gamekit192" · use gamekit192

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit193

import "std/gamekit193" · use gamekit193

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit194

import "std/gamekit194" · use gamekit194

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit195

import "std/gamekit195" · use gamekit195

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit196

import "std/gamekit196" · use gamekit196

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit197

import "std/gamekit197" · use gamekit197

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit198

import "std/gamekit198" · use gamekit198

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit199

import "std/gamekit199" · use gamekit199

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit200

import "std/gamekit200" · use gamekit200

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit201

import "std/gamekit201" · use gamekit201

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit202

import "std/gamekit202" · use gamekit202

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit203

import "std/gamekit203" · use gamekit203

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit204

import "std/gamekit204" · use gamekit204

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit205

import "std/gamekit205" · use gamekit205

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit206

import "std/gamekit206" · use gamekit206

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit207

import "std/gamekit207" · use gamekit207

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit208

import "std/gamekit208" · use gamekit208

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit209

import "std/gamekit209" · use gamekit209

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit210

import "std/gamekit210" · use gamekit210

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit211

import "std/gamekit211" · use gamekit211

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit212

import "std/gamekit212" · use gamekit212

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit213

import "std/gamekit213" · use gamekit213

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit214

import "std/gamekit214" · use gamekit214

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit215

import "std/gamekit215" · use gamekit215

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit216

import "std/gamekit216" · use gamekit216

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit217

import "std/gamekit217" · use gamekit217

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit218

import "std/gamekit218" · use gamekit218

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit219

import "std/gamekit219" · use gamekit219

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit220

import "std/gamekit220" · use gamekit220

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit221

import "std/gamekit221" · use gamekit221

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit222

import "std/gamekit222" · use gamekit222

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit223

import "std/gamekit223" · use gamekit223

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit224

import "std/gamekit224" · use gamekit224

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit225

import "std/gamekit225" · use gamekit225

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit226

import "std/gamekit226" · use gamekit226

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit227

import "std/gamekit227" · use gamekit227

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit228

import "std/gamekit228" · use gamekit228

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit229

import "std/gamekit229" · use gamekit229

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit230

import "std/gamekit230" · use gamekit230

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit231

import "std/gamekit231" · use gamekit231

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit232

import "std/gamekit232" · use gamekit232

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit233

import "std/gamekit233" · use gamekit233

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit234

import "std/gamekit234" · use gamekit234

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit235

import "std/gamekit235" · use gamekit235

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit236

import "std/gamekit236" · use gamekit236

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit237

import "std/gamekit237" · use gamekit237

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit238

import "std/gamekit238" · use gamekit238

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit239

import "std/gamekit239" · use gamekit239

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit240

import "std/gamekit240" · use gamekit240

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit241

import "std/gamekit241" · use gamekit241

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit242

import "std/gamekit242" · use gamekit242

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit243

import "std/gamekit243" · use gamekit243

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit244

import "std/gamekit244" · use gamekit244

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit245

import "std/gamekit245" · use gamekit245

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit246

import "std/gamekit246" · use gamekit246

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit247

import "std/gamekit247" · use gamekit247

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit248

import "std/gamekit248" · use gamekit248

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit249

import "std/gamekit249" · use gamekit249

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit250

import "std/gamekit250" · use gamekit250

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit251

import "std/gamekit251" · use gamekit251

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit252

import "std/gamekit252" · use gamekit252

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit253

import "std/gamekit253" · use gamekit253

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit254

import "std/gamekit254" · use gamekit254

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit255

import "std/gamekit255" · use gamekit255

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit256

import "std/gamekit256" · use gamekit256

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit257

import "std/gamekit257" · use gamekit257

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit258

import "std/gamekit258" · use gamekit258

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit259

import "std/gamekit259" · use gamekit259

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit260

import "std/gamekit260" · use gamekit260

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit261

import "std/gamekit261" · use gamekit261

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit262

import "std/gamekit262" · use gamekit262

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit263

import "std/gamekit263" · use gamekit263

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit264

import "std/gamekit264" · use gamekit264

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit265

import "std/gamekit265" · use gamekit265

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit266

import "std/gamekit266" · use gamekit266

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit267

import "std/gamekit267" · use gamekit267

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit268

import "std/gamekit268" · use gamekit268

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit269

import "std/gamekit269" · use gamekit269

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit270

import "std/gamekit270" · use gamekit270

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit271

import "std/gamekit271" · use gamekit271

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit272

import "std/gamekit272" · use gamekit272

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit273

import "std/gamekit273" · use gamekit273

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit274

import "std/gamekit274" · use gamekit274

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit275

import "std/gamekit275" · use gamekit275

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit276

import "std/gamekit276" · use gamekit276

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit277

import "std/gamekit277" · use gamekit277

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit278

import "std/gamekit278" · use gamekit278

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit279

import "std/gamekit279" · use gamekit279

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit280

import "std/gamekit280" · use gamekit280

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit281

import "std/gamekit281" · use gamekit281

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit282

import "std/gamekit282" · use gamekit282

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit283

import "std/gamekit283" · use gamekit283

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit284

import "std/gamekit284" · use gamekit284

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit285

import "std/gamekit285" · use gamekit285

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit286

import "std/gamekit286" · use gamekit286

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit287

import "std/gamekit287" · use gamekit287

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit288

import "std/gamekit288" · use gamekit288

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit289

import "std/gamekit289" · use gamekit289

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit290

import "std/gamekit290" · use gamekit290

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit291

import "std/gamekit291" · use gamekit291

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit292

import "std/gamekit292" · use gamekit292

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit293

import "std/gamekit293" · use gamekit293

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit294

import "std/gamekit294" · use gamekit294

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit295

import "std/gamekit295" · use gamekit295

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit296

import "std/gamekit296" · use gamekit296

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit297

import "std/gamekit297" · use gamekit297

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit298

import "std/gamekit298" · use gamekit298

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit299

import "std/gamekit299" · use gamekit299

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit300

import "std/gamekit300" · use gamekit300

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit301

import "std/gamekit301" · use gamekit301

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit302

import "std/gamekit302" · use gamekit302

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit303

import "std/gamekit303" · use gamekit303

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit304

import "std/gamekit304" · use gamekit304

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit305

import "std/gamekit305" · use gamekit305

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit306

import "std/gamekit306" · use gamekit306

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit307

import "std/gamekit307" · use gamekit307

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit308

import "std/gamekit308" · use gamekit308

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit309

import "std/gamekit309" · use gamekit309

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit310

import "std/gamekit310" · use gamekit310

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit311

import "std/gamekit311" · use gamekit311

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit312

import "std/gamekit312" · use gamekit312

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit313

import "std/gamekit313" · use gamekit313

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit314

import "std/gamekit314" · use gamekit314

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit315

import "std/gamekit315" · use gamekit315

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit316

import "std/gamekit316" · use gamekit316

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit317

import "std/gamekit317" · use gamekit317

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit318

import "std/gamekit318" · use gamekit318

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit319

import "std/gamekit319" · use gamekit319

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit320

import "std/gamekit320" · use gamekit320

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit321

import "std/gamekit321" · use gamekit321

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit322

import "std/gamekit322" · use gamekit322

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit323

import "std/gamekit323" · use gamekit323

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit324

import "std/gamekit324" · use gamekit324

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit325

import "std/gamekit325" · use gamekit325

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit326

import "std/gamekit326" · use gamekit326

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit327

import "std/gamekit327" · use gamekit327

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit328

import "std/gamekit328" · use gamekit328

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit329

import "std/gamekit329" · use gamekit329

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit330

import "std/gamekit330" · use gamekit330

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit331

import "std/gamekit331" · use gamekit331

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit332

import "std/gamekit332" · use gamekit332

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit333

import "std/gamekit333" · use gamekit333

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit334

import "std/gamekit334" · use gamekit334

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit335

import "std/gamekit335" · use gamekit335

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit336

import "std/gamekit336" · use gamekit336

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit337

import "std/gamekit337" · use gamekit337

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit338

import "std/gamekit338" · use gamekit338

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit339

import "std/gamekit339" · use gamekit339

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit340

import "std/gamekit340" · use gamekit340

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit341

import "std/gamekit341" · use gamekit341

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit342

import "std/gamekit342" · use gamekit342

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit343

import "std/gamekit343" · use gamekit343

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit344

import "std/gamekit344" · use gamekit344

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit345

import "std/gamekit345" · use gamekit345

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit346

import "std/gamekit346" · use gamekit346

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit347

import "std/gamekit347" · use gamekit347

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit348

import "std/gamekit348" · use gamekit348

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit349

import "std/gamekit349" · use gamekit349

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit350

import "std/gamekit350" · use gamekit350

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit351

import "std/gamekit351" · use gamekit351

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit352

import "std/gamekit352" · use gamekit352

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit353

import "std/gamekit353" · use gamekit353

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit354

import "std/gamekit354" · use gamekit354

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit355

import "std/gamekit355" · use gamekit355

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit356

import "std/gamekit356" · use gamekit356

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit357

import "std/gamekit357" · use gamekit357

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit358

import "std/gamekit358" · use gamekit358

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit359

import "std/gamekit359" · use gamekit359

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit360

import "std/gamekit360" · use gamekit360

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit361

import "std/gamekit361" · use gamekit361

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit362

import "std/gamekit362" · use gamekit362

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit363

import "std/gamekit363" · use gamekit363

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit364

import "std/gamekit364" · use gamekit364

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit365

import "std/gamekit365" · use gamekit365

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit366

import "std/gamekit366" · use gamekit366

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit367

import "std/gamekit367" · use gamekit367

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit368

import "std/gamekit368" · use gamekit368

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit369

import "std/gamekit369" · use gamekit369

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit370

import "std/gamekit370" · use gamekit370

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit371

import "std/gamekit371" · use gamekit371

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit372

import "std/gamekit372" · use gamekit372

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit373

import "std/gamekit373" · use gamekit373

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit374

import "std/gamekit374" · use gamekit374

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit375

import "std/gamekit375" · use gamekit375

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit376

import "std/gamekit376" · use gamekit376

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit377

import "std/gamekit377" · use gamekit377

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit378

import "std/gamekit378" · use gamekit378

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit379

import "std/gamekit379" · use gamekit379

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit380

import "std/gamekit380" · use gamekit380

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit381

import "std/gamekit381" · use gamekit381

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit382

import "std/gamekit382" · use gamekit382

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit383

import "std/gamekit383" · use gamekit383

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit384

import "std/gamekit384" · use gamekit384

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit385

import "std/gamekit385" · use gamekit385

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit386

import "std/gamekit386" · use gamekit386

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit387

import "std/gamekit387" · use gamekit387

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit388

import "std/gamekit388" · use gamekit388

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit389

import "std/gamekit389" · use gamekit389

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit390

import "std/gamekit390" · use gamekit390

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit391

import "std/gamekit391" · use gamekit391

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit392

import "std/gamekit392" · use gamekit392

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit393

import "std/gamekit393" · use gamekit393

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit394

import "std/gamekit394" · use gamekit394

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit395

import "std/gamekit395" · use gamekit395

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit396

import "std/gamekit396" · use gamekit396

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit397

import "std/gamekit397" · use gamekit397

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit398

import "std/gamekit398" · use gamekit398

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit399

import "std/gamekit399" · use gamekit399

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit400

import "std/gamekit400" · use gamekit400

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit401

import "std/gamekit401" · use gamekit401

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit402

import "std/gamekit402" · use gamekit402

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit403

import "std/gamekit403" · use gamekit403

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit404

import "std/gamekit404" · use gamekit404

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit405

import "std/gamekit405" · use gamekit405

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit406

import "std/gamekit406" · use gamekit406

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit407

import "std/gamekit407" · use gamekit407

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit408

import "std/gamekit408" · use gamekit408

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit409

import "std/gamekit409" · use gamekit409

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit410

import "std/gamekit410" · use gamekit410

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit411

import "std/gamekit411" · use gamekit411

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit412

import "std/gamekit412" · use gamekit412

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit413

import "std/gamekit413" · use gamekit413

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit414

import "std/gamekit414" · use gamekit414

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit415

import "std/gamekit415" · use gamekit415

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit416

import "std/gamekit416" · use gamekit416

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit417

import "std/gamekit417" · use gamekit417

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit418

import "std/gamekit418" · use gamekit418

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit419

import "std/gamekit419" · use gamekit419

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit420

import "std/gamekit420" · use gamekit420

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit421

import "std/gamekit421" · use gamekit421

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit422

import "std/gamekit422" · use gamekit422

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit423

import "std/gamekit423" · use gamekit423

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit424

import "std/gamekit424" · use gamekit424

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit425

import "std/gamekit425" · use gamekit425

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit426

import "std/gamekit426" · use gamekit426

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit427

import "std/gamekit427" · use gamekit427

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit428

import "std/gamekit428" · use gamekit428

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit429

import "std/gamekit429" · use gamekit429

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit430

import "std/gamekit430" · use gamekit430

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit431

import "std/gamekit431" · use gamekit431

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit432

import "std/gamekit432" · use gamekit432

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit433

import "std/gamekit433" · use gamekit433

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit434

import "std/gamekit434" · use gamekit434

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit435

import "std/gamekit435" · use gamekit435

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit436

import "std/gamekit436" · use gamekit436

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit437

import "std/gamekit437" · use gamekit437

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit438

import "std/gamekit438" · use gamekit438

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit439

import "std/gamekit439" · use gamekit439

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit440

import "std/gamekit440" · use gamekit440

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit441

import "std/gamekit441" · use gamekit441

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit442

import "std/gamekit442" · use gamekit442

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit443

import "std/gamekit443" · use gamekit443

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit444

import "std/gamekit444" · use gamekit444

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit445

import "std/gamekit445" · use gamekit445

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit446

import "std/gamekit446" · use gamekit446

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit447

import "std/gamekit447" · use gamekit447

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit448

import "std/gamekit448" · use gamekit448

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit449

import "std/gamekit449" · use gamekit449

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit450

import "std/gamekit450" · use gamekit450

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit451

import "std/gamekit451" · use gamekit451

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit452

import "std/gamekit452" · use gamekit452

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit453

import "std/gamekit453" · use gamekit453

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit454

import "std/gamekit454" · use gamekit454

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit455

import "std/gamekit455" · use gamekit455

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit456

import "std/gamekit456" · use gamekit456

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit457

import "std/gamekit457" · use gamekit457

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit458

import "std/gamekit458" · use gamekit458

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit459

import "std/gamekit459" · use gamekit459

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit460

import "std/gamekit460" · use gamekit460

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit461

import "std/gamekit461" · use gamekit461

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit462

import "std/gamekit462" · use gamekit462

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit463

import "std/gamekit463" · use gamekit463

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit464

import "std/gamekit464" · use gamekit464

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit465

import "std/gamekit465" · use gamekit465

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit466

import "std/gamekit466" · use gamekit466

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit467

import "std/gamekit467" · use gamekit467

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit468

import "std/gamekit468" · use gamekit468

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit469

import "std/gamekit469" · use gamekit469

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit470

import "std/gamekit470" · use gamekit470

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit471

import "std/gamekit471" · use gamekit471

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit472

import "std/gamekit472" · use gamekit472

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit473

import "std/gamekit473" · use gamekit473

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit474

import "std/gamekit474" · use gamekit474

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit475

import "std/gamekit475" · use gamekit475

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit476

import "std/gamekit476" · use gamekit476

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit477

import "std/gamekit477" · use gamekit477

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit478

import "std/gamekit478" · use gamekit478

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit479

import "std/gamekit479" · use gamekit479

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit480

import "std/gamekit480" · use gamekit480

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit481

import "std/gamekit481" · use gamekit481

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit482

import "std/gamekit482" · use gamekit482

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit483

import "std/gamekit483" · use gamekit483

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit484

import "std/gamekit484" · use gamekit484

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit485

import "std/gamekit485" · use gamekit485

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit486

import "std/gamekit486" · use gamekit486

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit487

import "std/gamekit487" · use gamekit487

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit488

import "std/gamekit488" · use gamekit488

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit489

import "std/gamekit489" · use gamekit489

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit490

import "std/gamekit490" · use gamekit490

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit491

import "std/gamekit491" · use gamekit491

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit492

import "std/gamekit492" · use gamekit492

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit493

import "std/gamekit493" · use gamekit493

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit494

import "std/gamekit494" · use gamekit494

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit495

import "std/gamekit495" · use gamekit495

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/gamekit496

import "std/gamekit496" · use gamekit496

- `falloff(dist number, radius number) → number`
- `shake(time number, amp number) → number`

std/gamekit497

import "std/gamekit497" · use gamekit497

- `easeIn(t number) → number`
- `falloff(dist number, radius number) → number`

std/gamekit498

import "std/gamekit498" · use gamekit498

- `easeOut(t number) → number`
- `spring(x number, v number, target number, dt number) → number`

std/gamekit499

import "std/gamekit499" · use gamekit499

- `spring(x number, v number, target number, dt number) → number`
- `shake(time number, amp number) → number`

std/gamekit500

import "std/gamekit500" · use gamekit500

- `easeIn(t number) → number`
- `easeOut(t number) → number`

std/game2dkit001

import "std/game2dkit001" · use game2dkit001

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit002

import "std/game2dkit002" · use game2dkit002

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit003

import "std/game2dkit003" · use game2dkit003

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit004

import "std/game2dkit004" · use game2dkit004

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit005

import "std/game2dkit005" · use game2dkit005

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit006

import "std/game2dkit006" · use game2dkit006

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit007

import "std/game2dkit007" · use game2dkit007

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit008

import "std/game2dkit008" · use game2dkit008

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit009

import "std/game2dkit009" · use game2dkit009

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit010

import "std/game2dkit010" · use game2dkit010

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit011

import "std/game2dkit011" · use game2dkit011

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit012

import "std/game2dkit012" · use game2dkit012

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit013

import "std/game2dkit013" · use game2dkit013

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit014

import "std/game2dkit014" · use game2dkit014

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit015

import "std/game2dkit015" · use game2dkit015

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit016

import "std/game2dkit016" · use game2dkit016

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit017

import "std/game2dkit017" · use game2dkit017

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit018

import "std/game2dkit018" · use game2dkit018

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit019

import "std/game2dkit019" · use game2dkit019

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit020

import "std/game2dkit020" · use game2dkit020

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit021

import "std/game2dkit021" · use game2dkit021

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit022

import "std/game2dkit022" · use game2dkit022

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit023

import "std/game2dkit023" · use game2dkit023

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit024

import "std/game2dkit024" · use game2dkit024

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit025

import "std/game2dkit025" · use game2dkit025

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit026

import "std/game2dkit026" · use game2dkit026

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit027

import "std/game2dkit027" · use game2dkit027

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit028

import "std/game2dkit028" · use game2dkit028

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit029

import "std/game2dkit029" · use game2dkit029

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit030

import "std/game2dkit030" · use game2dkit030

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit031

import "std/game2dkit031" · use game2dkit031

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit032

import "std/game2dkit032" · use game2dkit032

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit033

import "std/game2dkit033" · use game2dkit033

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit034

import "std/game2dkit034" · use game2dkit034

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit035

import "std/game2dkit035" · use game2dkit035

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit036

import "std/game2dkit036" · use game2dkit036

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit037

import "std/game2dkit037" · use game2dkit037

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit038

import "std/game2dkit038" · use game2dkit038

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit039

import "std/game2dkit039" · use game2dkit039

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit040

import "std/game2dkit040" · use game2dkit040

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit041

import "std/game2dkit041" · use game2dkit041

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit042

import "std/game2dkit042" · use game2dkit042

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit043

import "std/game2dkit043" · use game2dkit043

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit044

import "std/game2dkit044" · use game2dkit044

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit045

import "std/game2dkit045" · use game2dkit045

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit046

import "std/game2dkit046" · use game2dkit046

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit047

import "std/game2dkit047" · use game2dkit047

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit048

import "std/game2dkit048" · use game2dkit048

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit049

import "std/game2dkit049" · use game2dkit049

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit050

import "std/game2dkit050" · use game2dkit050

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit051

import "std/game2dkit051" · use game2dkit051

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit052

import "std/game2dkit052" · use game2dkit052

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit053

import "std/game2dkit053" · use game2dkit053

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit054

import "std/game2dkit054" · use game2dkit054

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit055

import "std/game2dkit055" · use game2dkit055

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit056

import "std/game2dkit056" · use game2dkit056

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit057

import "std/game2dkit057" · use game2dkit057

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit058

import "std/game2dkit058" · use game2dkit058

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit059

import "std/game2dkit059" · use game2dkit059

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit060

import "std/game2dkit060" · use game2dkit060

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit061

import "std/game2dkit061" · use game2dkit061

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit062

import "std/game2dkit062" · use game2dkit062

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit063

import "std/game2dkit063" · use game2dkit063

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit064

import "std/game2dkit064" · use game2dkit064

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit065

import "std/game2dkit065" · use game2dkit065

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit066

import "std/game2dkit066" · use game2dkit066

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit067

import "std/game2dkit067" · use game2dkit067

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit068

import "std/game2dkit068" · use game2dkit068

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit069

import "std/game2dkit069" · use game2dkit069

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit070

import "std/game2dkit070" · use game2dkit070

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit071

import "std/game2dkit071" · use game2dkit071

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit072

import "std/game2dkit072" · use game2dkit072

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit073

import "std/game2dkit073" · use game2dkit073

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit074

import "std/game2dkit074" · use game2dkit074

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit075

import "std/game2dkit075" · use game2dkit075

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit076

import "std/game2dkit076" · use game2dkit076

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit077

import "std/game2dkit077" · use game2dkit077

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit078

import "std/game2dkit078" · use game2dkit078

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit079

import "std/game2dkit079" · use game2dkit079

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit080

import "std/game2dkit080" · use game2dkit080

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit081

import "std/game2dkit081" · use game2dkit081

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit082

import "std/game2dkit082" · use game2dkit082

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit083

import "std/game2dkit083" · use game2dkit083

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit084

import "std/game2dkit084" · use game2dkit084

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit085

import "std/game2dkit085" · use game2dkit085

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit086

import "std/game2dkit086" · use game2dkit086

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit087

import "std/game2dkit087" · use game2dkit087

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit088

import "std/game2dkit088" · use game2dkit088

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit089

import "std/game2dkit089" · use game2dkit089

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit090

import "std/game2dkit090" · use game2dkit090

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit091

import "std/game2dkit091" · use game2dkit091

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit092

import "std/game2dkit092" · use game2dkit092

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit093

import "std/game2dkit093" · use game2dkit093

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit094

import "std/game2dkit094" · use game2dkit094

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit095

import "std/game2dkit095" · use game2dkit095

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit096

import "std/game2dkit096" · use game2dkit096

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit097

import "std/game2dkit097" · use game2dkit097

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit098

import "std/game2dkit098" · use game2dkit098

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit099

import "std/game2dkit099" · use game2dkit099

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit100

import "std/game2dkit100" · use game2dkit100

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit101

import "std/game2dkit101" · use game2dkit101

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit102

import "std/game2dkit102" · use game2dkit102

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit103

import "std/game2dkit103" · use game2dkit103

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit104

import "std/game2dkit104" · use game2dkit104

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit105

import "std/game2dkit105" · use game2dkit105

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit106

import "std/game2dkit106" · use game2dkit106

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit107

import "std/game2dkit107" · use game2dkit107

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit108

import "std/game2dkit108" · use game2dkit108

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit109

import "std/game2dkit109" · use game2dkit109

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit110

import "std/game2dkit110" · use game2dkit110

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit111

import "std/game2dkit111" · use game2dkit111

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit112

import "std/game2dkit112" · use game2dkit112

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit113

import "std/game2dkit113" · use game2dkit113

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit114

import "std/game2dkit114" · use game2dkit114

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit115

import "std/game2dkit115" · use game2dkit115

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit116

import "std/game2dkit116" · use game2dkit116

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit117

import "std/game2dkit117" · use game2dkit117

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit118

import "std/game2dkit118" · use game2dkit118

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit119

import "std/game2dkit119" · use game2dkit119

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit120

import "std/game2dkit120" · use game2dkit120

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit121

import "std/game2dkit121" · use game2dkit121

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit122

import "std/game2dkit122" · use game2dkit122

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit123

import "std/game2dkit123" · use game2dkit123

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit124

import "std/game2dkit124" · use game2dkit124

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit125

import "std/game2dkit125" · use game2dkit125

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit126

import "std/game2dkit126" · use game2dkit126

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit127

import "std/game2dkit127" · use game2dkit127

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit128

import "std/game2dkit128" · use game2dkit128

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit129

import "std/game2dkit129" · use game2dkit129

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit130

import "std/game2dkit130" · use game2dkit130

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit131

import "std/game2dkit131" · use game2dkit131

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit132

import "std/game2dkit132" · use game2dkit132

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit133

import "std/game2dkit133" · use game2dkit133

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit134

import "std/game2dkit134" · use game2dkit134

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit135

import "std/game2dkit135" · use game2dkit135

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit136

import "std/game2dkit136" · use game2dkit136

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit137

import "std/game2dkit137" · use game2dkit137

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit138

import "std/game2dkit138" · use game2dkit138

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit139

import "std/game2dkit139" · use game2dkit139

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit140

import "std/game2dkit140" · use game2dkit140

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit141

import "std/game2dkit141" · use game2dkit141

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit142

import "std/game2dkit142" · use game2dkit142

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit143

import "std/game2dkit143" · use game2dkit143

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit144

import "std/game2dkit144" · use game2dkit144

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit145

import "std/game2dkit145" · use game2dkit145

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit146

import "std/game2dkit146" · use game2dkit146

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit147

import "std/game2dkit147" · use game2dkit147

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit148

import "std/game2dkit148" · use game2dkit148

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit149

import "std/game2dkit149" · use game2dkit149

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit150

import "std/game2dkit150" · use game2dkit150

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit151

import "std/game2dkit151" · use game2dkit151

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit152

import "std/game2dkit152" · use game2dkit152

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit153

import "std/game2dkit153" · use game2dkit153

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit154

import "std/game2dkit154" · use game2dkit154

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit155

import "std/game2dkit155" · use game2dkit155

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit156

import "std/game2dkit156" · use game2dkit156

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit157

import "std/game2dkit157" · use game2dkit157

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit158

import "std/game2dkit158" · use game2dkit158

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit159

import "std/game2dkit159" · use game2dkit159

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit160

import "std/game2dkit160" · use game2dkit160

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit161

import "std/game2dkit161" · use game2dkit161

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit162

import "std/game2dkit162" · use game2dkit162

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit163

import "std/game2dkit163" · use game2dkit163

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit164

import "std/game2dkit164" · use game2dkit164

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit165

import "std/game2dkit165" · use game2dkit165

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit166

import "std/game2dkit166" · use game2dkit166

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit167

import "std/game2dkit167" · use game2dkit167

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit168

import "std/game2dkit168" · use game2dkit168

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit169

import "std/game2dkit169" · use game2dkit169

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit170

import "std/game2dkit170" · use game2dkit170

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit171

import "std/game2dkit171" · use game2dkit171

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit172

import "std/game2dkit172" · use game2dkit172

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit173

import "std/game2dkit173" · use game2dkit173

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit174

import "std/game2dkit174" · use game2dkit174

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit175

import "std/game2dkit175" · use game2dkit175

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit176

import "std/game2dkit176" · use game2dkit176

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit177

import "std/game2dkit177" · use game2dkit177

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit178

import "std/game2dkit178" · use game2dkit178

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit179

import "std/game2dkit179" · use game2dkit179

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit180

import "std/game2dkit180" · use game2dkit180

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit181

import "std/game2dkit181" · use game2dkit181

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit182

import "std/game2dkit182" · use game2dkit182

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit183

import "std/game2dkit183" · use game2dkit183

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit184

import "std/game2dkit184" · use game2dkit184

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit185

import "std/game2dkit185" · use game2dkit185

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit186

import "std/game2dkit186" · use game2dkit186

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit187

import "std/game2dkit187" · use game2dkit187

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit188

import "std/game2dkit188" · use game2dkit188

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit189

import "std/game2dkit189" · use game2dkit189

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit190

import "std/game2dkit190" · use game2dkit190

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit191

import "std/game2dkit191" · use game2dkit191

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit192

import "std/game2dkit192" · use game2dkit192

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit193

import "std/game2dkit193" · use game2dkit193

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit194

import "std/game2dkit194" · use game2dkit194

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit195

import "std/game2dkit195" · use game2dkit195

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit196

import "std/game2dkit196" · use game2dkit196

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit197

import "std/game2dkit197" · use game2dkit197

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit198

import "std/game2dkit198" · use game2dkit198

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit199

import "std/game2dkit199" · use game2dkit199

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit200

import "std/game2dkit200" · use game2dkit200

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit201

import "std/game2dkit201" · use game2dkit201

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit202

import "std/game2dkit202" · use game2dkit202

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit203

import "std/game2dkit203" · use game2dkit203

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit204

import "std/game2dkit204" · use game2dkit204

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit205

import "std/game2dkit205" · use game2dkit205

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit206

import "std/game2dkit206" · use game2dkit206

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit207

import "std/game2dkit207" · use game2dkit207

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit208

import "std/game2dkit208" · use game2dkit208

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit209

import "std/game2dkit209" · use game2dkit209

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit210

import "std/game2dkit210" · use game2dkit210

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit211

import "std/game2dkit211" · use game2dkit211

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit212

import "std/game2dkit212" · use game2dkit212

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit213

import "std/game2dkit213" · use game2dkit213

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit214

import "std/game2dkit214" · use game2dkit214

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit215

import "std/game2dkit215" · use game2dkit215

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit216

import "std/game2dkit216" · use game2dkit216

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit217

import "std/game2dkit217" · use game2dkit217

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit218

import "std/game2dkit218" · use game2dkit218

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit219

import "std/game2dkit219" · use game2dkit219

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit220

import "std/game2dkit220" · use game2dkit220

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit221

import "std/game2dkit221" · use game2dkit221

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit222

import "std/game2dkit222" · use game2dkit222

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit223

import "std/game2dkit223" · use game2dkit223

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit224

import "std/game2dkit224" · use game2dkit224

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit225

import "std/game2dkit225" · use game2dkit225

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit226

import "std/game2dkit226" · use game2dkit226

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit227

import "std/game2dkit227" · use game2dkit227

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit228

import "std/game2dkit228" · use game2dkit228

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit229

import "std/game2dkit229" · use game2dkit229

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit230

import "std/game2dkit230" · use game2dkit230

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit231

import "std/game2dkit231" · use game2dkit231

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit232

import "std/game2dkit232" · use game2dkit232

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit233

import "std/game2dkit233" · use game2dkit233

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit234

import "std/game2dkit234" · use game2dkit234

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit235

import "std/game2dkit235" · use game2dkit235

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit236

import "std/game2dkit236" · use game2dkit236

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit237

import "std/game2dkit237" · use game2dkit237

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit238

import "std/game2dkit238" · use game2dkit238

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit239

import "std/game2dkit239" · use game2dkit239

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit240

import "std/game2dkit240" · use game2dkit240

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit241

import "std/game2dkit241" · use game2dkit241

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit242

import "std/game2dkit242" · use game2dkit242

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit243

import "std/game2dkit243" · use game2dkit243

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit244

import "std/game2dkit244" · use game2dkit244

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit245

import "std/game2dkit245" · use game2dkit245

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit246

import "std/game2dkit246" · use game2dkit246

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit247

import "std/game2dkit247" · use game2dkit247

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit248

import "std/game2dkit248" · use game2dkit248

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit249

import "std/game2dkit249" · use game2dkit249

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit250

import "std/game2dkit250" · use game2dkit250

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit251

import "std/game2dkit251" · use game2dkit251

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit252

import "std/game2dkit252" · use game2dkit252

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit253

import "std/game2dkit253" · use game2dkit253

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit254

import "std/game2dkit254" · use game2dkit254

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit255

import "std/game2dkit255" · use game2dkit255

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit256

import "std/game2dkit256" · use game2dkit256

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit257

import "std/game2dkit257" · use game2dkit257

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit258

import "std/game2dkit258" · use game2dkit258

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit259

import "std/game2dkit259" · use game2dkit259

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit260

import "std/game2dkit260" · use game2dkit260

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit261

import "std/game2dkit261" · use game2dkit261

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit262

import "std/game2dkit262" · use game2dkit262

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit263

import "std/game2dkit263" · use game2dkit263

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit264

import "std/game2dkit264" · use game2dkit264

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit265

import "std/game2dkit265" · use game2dkit265

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit266

import "std/game2dkit266" · use game2dkit266

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit267

import "std/game2dkit267" · use game2dkit267

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit268

import "std/game2dkit268" · use game2dkit268

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit269

import "std/game2dkit269" · use game2dkit269

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit270

import "std/game2dkit270" · use game2dkit270

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit271

import "std/game2dkit271" · use game2dkit271

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit272

import "std/game2dkit272" · use game2dkit272

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit273

import "std/game2dkit273" · use game2dkit273

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit274

import "std/game2dkit274" · use game2dkit274

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit275

import "std/game2dkit275" · use game2dkit275

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit276

import "std/game2dkit276" · use game2dkit276

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit277

import "std/game2dkit277" · use game2dkit277

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit278

import "std/game2dkit278" · use game2dkit278

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit279

import "std/game2dkit279" · use game2dkit279

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit280

import "std/game2dkit280" · use game2dkit280

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit281

import "std/game2dkit281" · use game2dkit281

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit282

import "std/game2dkit282" · use game2dkit282

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit283

import "std/game2dkit283" · use game2dkit283

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit284

import "std/game2dkit284" · use game2dkit284

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit285

import "std/game2dkit285" · use game2dkit285

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit286

import "std/game2dkit286" · use game2dkit286

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit287

import "std/game2dkit287" · use game2dkit287

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit288

import "std/game2dkit288" · use game2dkit288

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit289

import "std/game2dkit289" · use game2dkit289

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit290

import "std/game2dkit290" · use game2dkit290

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit291

import "std/game2dkit291" · use game2dkit291

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit292

import "std/game2dkit292" · use game2dkit292

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit293

import "std/game2dkit293" · use game2dkit293

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit294

import "std/game2dkit294" · use game2dkit294

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit295

import "std/game2dkit295" · use game2dkit295

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit296

import "std/game2dkit296" · use game2dkit296

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit297

import "std/game2dkit297" · use game2dkit297

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit298

import "std/game2dkit298" · use game2dkit298

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit299

import "std/game2dkit299" · use game2dkit299

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit300

import "std/game2dkit300" · use game2dkit300

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit301

import "std/game2dkit301" · use game2dkit301

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit302

import "std/game2dkit302" · use game2dkit302

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit303

import "std/game2dkit303" · use game2dkit303

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit304

import "std/game2dkit304" · use game2dkit304

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit305

import "std/game2dkit305" · use game2dkit305

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit306

import "std/game2dkit306" · use game2dkit306

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit307

import "std/game2dkit307" · use game2dkit307

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit308

import "std/game2dkit308" · use game2dkit308

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit309

import "std/game2dkit309" · use game2dkit309

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit310

import "std/game2dkit310" · use game2dkit310

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit311

import "std/game2dkit311" · use game2dkit311

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit312

import "std/game2dkit312" · use game2dkit312

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit313

import "std/game2dkit313" · use game2dkit313

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit314

import "std/game2dkit314" · use game2dkit314

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit315

import "std/game2dkit315" · use game2dkit315

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit316

import "std/game2dkit316" · use game2dkit316

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit317

import "std/game2dkit317" · use game2dkit317

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit318

import "std/game2dkit318" · use game2dkit318

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit319

import "std/game2dkit319" · use game2dkit319

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit320

import "std/game2dkit320" · use game2dkit320

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit321

import "std/game2dkit321" · use game2dkit321

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit322

import "std/game2dkit322" · use game2dkit322

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit323

import "std/game2dkit323" · use game2dkit323

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit324

import "std/game2dkit324" · use game2dkit324

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit325

import "std/game2dkit325" · use game2dkit325

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit326

import "std/game2dkit326" · use game2dkit326

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit327

import "std/game2dkit327" · use game2dkit327

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit328

import "std/game2dkit328" · use game2dkit328

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit329

import "std/game2dkit329" · use game2dkit329

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit330

import "std/game2dkit330" · use game2dkit330

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit331

import "std/game2dkit331" · use game2dkit331

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit332

import "std/game2dkit332" · use game2dkit332

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit333

import "std/game2dkit333" · use game2dkit333

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit334

import "std/game2dkit334" · use game2dkit334

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit335

import "std/game2dkit335" · use game2dkit335

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit336

import "std/game2dkit336" · use game2dkit336

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit337

import "std/game2dkit337" · use game2dkit337

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit338

import "std/game2dkit338" · use game2dkit338

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit339

import "std/game2dkit339" · use game2dkit339

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit340

import "std/game2dkit340" · use game2dkit340

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit341

import "std/game2dkit341" · use game2dkit341

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit342

import "std/game2dkit342" · use game2dkit342

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit343

import "std/game2dkit343" · use game2dkit343

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit344

import "std/game2dkit344" · use game2dkit344

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit345

import "std/game2dkit345" · use game2dkit345

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit346

import "std/game2dkit346" · use game2dkit346

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit347

import "std/game2dkit347" · use game2dkit347

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit348

import "std/game2dkit348" · use game2dkit348

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit349

import "std/game2dkit349" · use game2dkit349

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit350

import "std/game2dkit350" · use game2dkit350

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit351

import "std/game2dkit351" · use game2dkit351

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit352

import "std/game2dkit352" · use game2dkit352

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit353

import "std/game2dkit353" · use game2dkit353

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit354

import "std/game2dkit354" · use game2dkit354

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit355

import "std/game2dkit355" · use game2dkit355

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit356

import "std/game2dkit356" · use game2dkit356

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit357

import "std/game2dkit357" · use game2dkit357

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit358

import "std/game2dkit358" · use game2dkit358

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit359

import "std/game2dkit359" · use game2dkit359

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit360

import "std/game2dkit360" · use game2dkit360

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit361

import "std/game2dkit361" · use game2dkit361

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit362

import "std/game2dkit362" · use game2dkit362

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit363

import "std/game2dkit363" · use game2dkit363

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit364

import "std/game2dkit364" · use game2dkit364

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit365

import "std/game2dkit365" · use game2dkit365

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit366

import "std/game2dkit366" · use game2dkit366

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit367

import "std/game2dkit367" · use game2dkit367

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit368

import "std/game2dkit368" · use game2dkit368

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit369

import "std/game2dkit369" · use game2dkit369

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit370

import "std/game2dkit370" · use game2dkit370

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit371

import "std/game2dkit371" · use game2dkit371

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit372

import "std/game2dkit372" · use game2dkit372

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit373

import "std/game2dkit373" · use game2dkit373

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit374

import "std/game2dkit374" · use game2dkit374

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit375

import "std/game2dkit375" · use game2dkit375

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit376

import "std/game2dkit376" · use game2dkit376

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit377

import "std/game2dkit377" · use game2dkit377

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit378

import "std/game2dkit378" · use game2dkit378

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit379

import "std/game2dkit379" · use game2dkit379

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit380

import "std/game2dkit380" · use game2dkit380

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit381

import "std/game2dkit381" · use game2dkit381

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit382

import "std/game2dkit382" · use game2dkit382

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit383

import "std/game2dkit383" · use game2dkit383

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit384

import "std/game2dkit384" · use game2dkit384

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit385

import "std/game2dkit385" · use game2dkit385

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit386

import "std/game2dkit386" · use game2dkit386

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit387

import "std/game2dkit387" · use game2dkit387

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit388

import "std/game2dkit388" · use game2dkit388

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit389

import "std/game2dkit389" · use game2dkit389

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit390

import "std/game2dkit390" · use game2dkit390

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit391

import "std/game2dkit391" · use game2dkit391

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit392

import "std/game2dkit392" · use game2dkit392

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit393

import "std/game2dkit393" · use game2dkit393

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit394

import "std/game2dkit394" · use game2dkit394

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit395

import "std/game2dkit395" · use game2dkit395

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit396

import "std/game2dkit396" · use game2dkit396

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit397

import "std/game2dkit397" · use game2dkit397

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit398

import "std/game2dkit398" · use game2dkit398

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit399

import "std/game2dkit399" · use game2dkit399

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit400

import "std/game2dkit400" · use game2dkit400

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit401

import "std/game2dkit401" · use game2dkit401

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit402

import "std/game2dkit402" · use game2dkit402

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit403

import "std/game2dkit403" · use game2dkit403

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit404

import "std/game2dkit404" · use game2dkit404

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit405

import "std/game2dkit405" · use game2dkit405

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit406

import "std/game2dkit406" · use game2dkit406

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit407

import "std/game2dkit407" · use game2dkit407

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit408

import "std/game2dkit408" · use game2dkit408

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit409

import "std/game2dkit409" · use game2dkit409

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit410

import "std/game2dkit410" · use game2dkit410

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit411

import "std/game2dkit411" · use game2dkit411

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit412

import "std/game2dkit412" · use game2dkit412

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit413

import "std/game2dkit413" · use game2dkit413

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit414

import "std/game2dkit414" · use game2dkit414

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit415

import "std/game2dkit415" · use game2dkit415

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit416

import "std/game2dkit416" · use game2dkit416

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit417

import "std/game2dkit417" · use game2dkit417

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit418

import "std/game2dkit418" · use game2dkit418

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit419

import "std/game2dkit419" · use game2dkit419

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit420

import "std/game2dkit420" · use game2dkit420

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit421

import "std/game2dkit421" · use game2dkit421

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit422

import "std/game2dkit422" · use game2dkit422

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit423

import "std/game2dkit423" · use game2dkit423

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit424

import "std/game2dkit424" · use game2dkit424

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit425

import "std/game2dkit425" · use game2dkit425

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit426

import "std/game2dkit426" · use game2dkit426

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit427

import "std/game2dkit427" · use game2dkit427

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit428

import "std/game2dkit428" · use game2dkit428

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit429

import "std/game2dkit429" · use game2dkit429

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit430

import "std/game2dkit430" · use game2dkit430

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit431

import "std/game2dkit431" · use game2dkit431

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit432

import "std/game2dkit432" · use game2dkit432

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit433

import "std/game2dkit433" · use game2dkit433

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit434

import "std/game2dkit434" · use game2dkit434

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit435

import "std/game2dkit435" · use game2dkit435

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit436

import "std/game2dkit436" · use game2dkit436

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit437

import "std/game2dkit437" · use game2dkit437

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit438

import "std/game2dkit438" · use game2dkit438

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit439

import "std/game2dkit439" · use game2dkit439

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit440

import "std/game2dkit440" · use game2dkit440

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit441

import "std/game2dkit441" · use game2dkit441

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit442

import "std/game2dkit442" · use game2dkit442

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit443

import "std/game2dkit443" · use game2dkit443

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit444

import "std/game2dkit444" · use game2dkit444

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit445

import "std/game2dkit445" · use game2dkit445

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit446

import "std/game2dkit446" · use game2dkit446

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit447

import "std/game2dkit447" · use game2dkit447

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit448

import "std/game2dkit448" · use game2dkit448

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit449

import "std/game2dkit449" · use game2dkit449

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit450

import "std/game2dkit450" · use game2dkit450

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit451

import "std/game2dkit451" · use game2dkit451

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit452

import "std/game2dkit452" · use game2dkit452

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit453

import "std/game2dkit453" · use game2dkit453

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit454

import "std/game2dkit454" · use game2dkit454

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit455

import "std/game2dkit455" · use game2dkit455

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit456

import "std/game2dkit456" · use game2dkit456

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit457

import "std/game2dkit457" · use game2dkit457

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit458

import "std/game2dkit458" · use game2dkit458

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit459

import "std/game2dkit459" · use game2dkit459

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit460

import "std/game2dkit460" · use game2dkit460

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit461

import "std/game2dkit461" · use game2dkit461

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit462

import "std/game2dkit462" · use game2dkit462

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit463

import "std/game2dkit463" · use game2dkit463

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit464

import "std/game2dkit464" · use game2dkit464

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit465

import "std/game2dkit465" · use game2dkit465

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit466

import "std/game2dkit466" · use game2dkit466

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit467

import "std/game2dkit467" · use game2dkit467

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit468

import "std/game2dkit468" · use game2dkit468

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit469

import "std/game2dkit469" · use game2dkit469

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit470

import "std/game2dkit470" · use game2dkit470

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit471

import "std/game2dkit471" · use game2dkit471

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit472

import "std/game2dkit472" · use game2dkit472

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit473

import "std/game2dkit473" · use game2dkit473

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit474

import "std/game2dkit474" · use game2dkit474

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit475

import "std/game2dkit475" · use game2dkit475

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit476

import "std/game2dkit476" · use game2dkit476

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit477

import "std/game2dkit477" · use game2dkit477

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit478

import "std/game2dkit478" · use game2dkit478

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit479

import "std/game2dkit479" · use game2dkit479

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit480

import "std/game2dkit480" · use game2dkit480

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit481

import "std/game2dkit481" · use game2dkit481

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit482

import "std/game2dkit482" · use game2dkit482

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit483

import "std/game2dkit483" · use game2dkit483

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit484

import "std/game2dkit484" · use game2dkit484

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit485

import "std/game2dkit485" · use game2dkit485

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit486

import "std/game2dkit486" · use game2dkit486

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit487

import "std/game2dkit487" · use game2dkit487

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit488

import "std/game2dkit488" · use game2dkit488

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit489

import "std/game2dkit489" · use game2dkit489

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit490

import "std/game2dkit490" · use game2dkit490

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit491

import "std/game2dkit491" · use game2dkit491

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit492

import "std/game2dkit492" · use game2dkit492

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit493

import "std/game2dkit493" · use game2dkit493

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit494

import "std/game2dkit494" · use game2dkit494

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit495

import "std/game2dkit495" · use game2dkit495

- `dist2(x0 number, y0 number, x1 number, y1 number) → number`
- `wrap(v number, size number) → number`

std/game2dkit496

import "std/game2dkit496" · use game2dkit496

- `shake(time number, amp number) → number`
- `follow(cam number, target number, dt number) → number`

std/game2dkit497

import "std/game2dkit497" · use game2dkit497

- `snapGrid(v number) → number`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit498

import "std/game2dkit498" · use game2dkit498

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `pointInRect(px number, py number, rx number, ry number, rw number, rh number) → bool`

std/game2dkit499

import "std/game2dkit499" · use game2dkit499

- `aabbHit(ax number, ay number, aw number, ah number, bx number, by number, bw number, bh number) → bool`
- `dist2(x0 number, y0 number, x1 number, y1 number) → number`

std/game2dkit500

import "std/game2dkit500" · use game2dkit500

- `approach(cur number, target number, maxDelta number) → number`
- `follow(cam number, target number, dt number) → number`

std/game3dkit001

import "std/game3dkit001" · use game3dkit001

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit002

import "std/game3dkit002" · use game3dkit002

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit003

import "std/game3dkit003" · use game3dkit003

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit004

import "std/game3dkit004" · use game3dkit004

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit005

import "std/game3dkit005" · use game3dkit005

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit006

import "std/game3dkit006" · use game3dkit006

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit007

import "std/game3dkit007" · use game3dkit007

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit008

import "std/game3dkit008" · use game3dkit008

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit009

import "std/game3dkit009" · use game3dkit009

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit010

import "std/game3dkit010" · use game3dkit010

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit011

import "std/game3dkit011" · use game3dkit011

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit012

import "std/game3dkit012" · use game3dkit012

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit013

import "std/game3dkit013" · use game3dkit013

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit014

import "std/game3dkit014" · use game3dkit014

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit015

import "std/game3dkit015" · use game3dkit015

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit016

import "std/game3dkit016" · use game3dkit016

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit017

import "std/game3dkit017" · use game3dkit017

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit018

import "std/game3dkit018" · use game3dkit018

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit019

import "std/game3dkit019" · use game3dkit019

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit020

import "std/game3dkit020" · use game3dkit020

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit021

import "std/game3dkit021" · use game3dkit021

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit022

import "std/game3dkit022" · use game3dkit022

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit023

import "std/game3dkit023" · use game3dkit023

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit024

import "std/game3dkit024" · use game3dkit024

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit025

import "std/game3dkit025" · use game3dkit025

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit026

import "std/game3dkit026" · use game3dkit026

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit027

import "std/game3dkit027" · use game3dkit027

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit028

import "std/game3dkit028" · use game3dkit028

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit029

import "std/game3dkit029" · use game3dkit029

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit030

import "std/game3dkit030" · use game3dkit030

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit031

import "std/game3dkit031" · use game3dkit031

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit032

import "std/game3dkit032" · use game3dkit032

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit033

import "std/game3dkit033" · use game3dkit033

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit034

import "std/game3dkit034" · use game3dkit034

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit035

import "std/game3dkit035" · use game3dkit035

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit036

import "std/game3dkit036" · use game3dkit036

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit037

import "std/game3dkit037" · use game3dkit037

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit038

import "std/game3dkit038" · use game3dkit038

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit039

import "std/game3dkit039" · use game3dkit039

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit040

import "std/game3dkit040" · use game3dkit040

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit041

import "std/game3dkit041" · use game3dkit041

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit042

import "std/game3dkit042" · use game3dkit042

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit043

import "std/game3dkit043" · use game3dkit043

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit044

import "std/game3dkit044" · use game3dkit044

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit045

import "std/game3dkit045" · use game3dkit045

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit046

import "std/game3dkit046" · use game3dkit046

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit047

import "std/game3dkit047" · use game3dkit047

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit048

import "std/game3dkit048" · use game3dkit048

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit049

import "std/game3dkit049" · use game3dkit049

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit050

import "std/game3dkit050" · use game3dkit050

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit051

import "std/game3dkit051" · use game3dkit051

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit052

import "std/game3dkit052" · use game3dkit052

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit053

import "std/game3dkit053" · use game3dkit053

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit054

import "std/game3dkit054" · use game3dkit054

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit055

import "std/game3dkit055" · use game3dkit055

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit056

import "std/game3dkit056" · use game3dkit056

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit057

import "std/game3dkit057" · use game3dkit057

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit058

import "std/game3dkit058" · use game3dkit058

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit059

import "std/game3dkit059" · use game3dkit059

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit060

import "std/game3dkit060" · use game3dkit060

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit061

import "std/game3dkit061" · use game3dkit061

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit062

import "std/game3dkit062" · use game3dkit062

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit063

import "std/game3dkit063" · use game3dkit063

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit064

import "std/game3dkit064" · use game3dkit064

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit065

import "std/game3dkit065" · use game3dkit065

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit066

import "std/game3dkit066" · use game3dkit066

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit067

import "std/game3dkit067" · use game3dkit067

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit068

import "std/game3dkit068" · use game3dkit068

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit069

import "std/game3dkit069" · use game3dkit069

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit070

import "std/game3dkit070" · use game3dkit070

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit071

import "std/game3dkit071" · use game3dkit071

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit072

import "std/game3dkit072" · use game3dkit072

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit073

import "std/game3dkit073" · use game3dkit073

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit074

import "std/game3dkit074" · use game3dkit074

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit075

import "std/game3dkit075" · use game3dkit075

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit076

import "std/game3dkit076" · use game3dkit076

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit077

import "std/game3dkit077" · use game3dkit077

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit078

import "std/game3dkit078" · use game3dkit078

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit079

import "std/game3dkit079" · use game3dkit079

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit080

import "std/game3dkit080" · use game3dkit080

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit081

import "std/game3dkit081" · use game3dkit081

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit082

import "std/game3dkit082" · use game3dkit082

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit083

import "std/game3dkit083" · use game3dkit083

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit084

import "std/game3dkit084" · use game3dkit084

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit085

import "std/game3dkit085" · use game3dkit085

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit086

import "std/game3dkit086" · use game3dkit086

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit087

import "std/game3dkit087" · use game3dkit087

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit088

import "std/game3dkit088" · use game3dkit088

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit089

import "std/game3dkit089" · use game3dkit089

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit090

import "std/game3dkit090" · use game3dkit090

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit091

import "std/game3dkit091" · use game3dkit091

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit092

import "std/game3dkit092" · use game3dkit092

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit093

import "std/game3dkit093" · use game3dkit093

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit094

import "std/game3dkit094" · use game3dkit094

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit095

import "std/game3dkit095" · use game3dkit095

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit096

import "std/game3dkit096" · use game3dkit096

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit097

import "std/game3dkit097" · use game3dkit097

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit098

import "std/game3dkit098" · use game3dkit098

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit099

import "std/game3dkit099" · use game3dkit099

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit100

import "std/game3dkit100" · use game3dkit100

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit101

import "std/game3dkit101" · use game3dkit101

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit102

import "std/game3dkit102" · use game3dkit102

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit103

import "std/game3dkit103" · use game3dkit103

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit104

import "std/game3dkit104" · use game3dkit104

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit105

import "std/game3dkit105" · use game3dkit105

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit106

import "std/game3dkit106" · use game3dkit106

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit107

import "std/game3dkit107" · use game3dkit107

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit108

import "std/game3dkit108" · use game3dkit108

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit109

import "std/game3dkit109" · use game3dkit109

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit110

import "std/game3dkit110" · use game3dkit110

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit111

import "std/game3dkit111" · use game3dkit111

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit112

import "std/game3dkit112" · use game3dkit112

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit113

import "std/game3dkit113" · use game3dkit113

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit114

import "std/game3dkit114" · use game3dkit114

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit115

import "std/game3dkit115" · use game3dkit115

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit116

import "std/game3dkit116" · use game3dkit116

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit117

import "std/game3dkit117" · use game3dkit117

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit118

import "std/game3dkit118" · use game3dkit118

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit119

import "std/game3dkit119" · use game3dkit119

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit120

import "std/game3dkit120" · use game3dkit120

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit121

import "std/game3dkit121" · use game3dkit121

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit122

import "std/game3dkit122" · use game3dkit122

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit123

import "std/game3dkit123" · use game3dkit123

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit124

import "std/game3dkit124" · use game3dkit124

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit125

import "std/game3dkit125" · use game3dkit125

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit126

import "std/game3dkit126" · use game3dkit126

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit127

import "std/game3dkit127" · use game3dkit127

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit128

import "std/game3dkit128" · use game3dkit128

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit129

import "std/game3dkit129" · use game3dkit129

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit130

import "std/game3dkit130" · use game3dkit130

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit131

import "std/game3dkit131" · use game3dkit131

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit132

import "std/game3dkit132" · use game3dkit132

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit133

import "std/game3dkit133" · use game3dkit133

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit134

import "std/game3dkit134" · use game3dkit134

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit135

import "std/game3dkit135" · use game3dkit135

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit136

import "std/game3dkit136" · use game3dkit136

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit137

import "std/game3dkit137" · use game3dkit137

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit138

import "std/game3dkit138" · use game3dkit138

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit139

import "std/game3dkit139" · use game3dkit139

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit140

import "std/game3dkit140" · use game3dkit140

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit141

import "std/game3dkit141" · use game3dkit141

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit142

import "std/game3dkit142" · use game3dkit142

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit143

import "std/game3dkit143" · use game3dkit143

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit144

import "std/game3dkit144" · use game3dkit144

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit145

import "std/game3dkit145" · use game3dkit145

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit146

import "std/game3dkit146" · use game3dkit146

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit147

import "std/game3dkit147" · use game3dkit147

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit148

import "std/game3dkit148" · use game3dkit148

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit149

import "std/game3dkit149" · use game3dkit149

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit150

import "std/game3dkit150" · use game3dkit150

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit151

import "std/game3dkit151" · use game3dkit151

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit152

import "std/game3dkit152" · use game3dkit152

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit153

import "std/game3dkit153" · use game3dkit153

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit154

import "std/game3dkit154" · use game3dkit154

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit155

import "std/game3dkit155" · use game3dkit155

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit156

import "std/game3dkit156" · use game3dkit156

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit157

import "std/game3dkit157" · use game3dkit157

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit158

import "std/game3dkit158" · use game3dkit158

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit159

import "std/game3dkit159" · use game3dkit159

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit160

import "std/game3dkit160" · use game3dkit160

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit161

import "std/game3dkit161" · use game3dkit161

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit162

import "std/game3dkit162" · use game3dkit162

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit163

import "std/game3dkit163" · use game3dkit163

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit164

import "std/game3dkit164" · use game3dkit164

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit165

import "std/game3dkit165" · use game3dkit165

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit166

import "std/game3dkit166" · use game3dkit166

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit167

import "std/game3dkit167" · use game3dkit167

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit168

import "std/game3dkit168" · use game3dkit168

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit169

import "std/game3dkit169" · use game3dkit169

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit170

import "std/game3dkit170" · use game3dkit170

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit171

import "std/game3dkit171" · use game3dkit171

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit172

import "std/game3dkit172" · use game3dkit172

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit173

import "std/game3dkit173" · use game3dkit173

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit174

import "std/game3dkit174" · use game3dkit174

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit175

import "std/game3dkit175" · use game3dkit175

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit176

import "std/game3dkit176" · use game3dkit176

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit177

import "std/game3dkit177" · use game3dkit177

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit178

import "std/game3dkit178" · use game3dkit178

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit179

import "std/game3dkit179" · use game3dkit179

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit180

import "std/game3dkit180" · use game3dkit180

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit181

import "std/game3dkit181" · use game3dkit181

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit182

import "std/game3dkit182" · use game3dkit182

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit183

import "std/game3dkit183" · use game3dkit183

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit184

import "std/game3dkit184" · use game3dkit184

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit185

import "std/game3dkit185" · use game3dkit185

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit186

import "std/game3dkit186" · use game3dkit186

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit187

import "std/game3dkit187" · use game3dkit187

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit188

import "std/game3dkit188" · use game3dkit188

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit189

import "std/game3dkit189" · use game3dkit189

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit190

import "std/game3dkit190" · use game3dkit190

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit191

import "std/game3dkit191" · use game3dkit191

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit192

import "std/game3dkit192" · use game3dkit192

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit193

import "std/game3dkit193" · use game3dkit193

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit194

import "std/game3dkit194" · use game3dkit194

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit195

import "std/game3dkit195" · use game3dkit195

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit196

import "std/game3dkit196" · use game3dkit196

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit197

import "std/game3dkit197" · use game3dkit197

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit198

import "std/game3dkit198" · use game3dkit198

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit199

import "std/game3dkit199" · use game3dkit199

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit200

import "std/game3dkit200" · use game3dkit200

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit201

import "std/game3dkit201" · use game3dkit201

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit202

import "std/game3dkit202" · use game3dkit202

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit203

import "std/game3dkit203" · use game3dkit203

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit204

import "std/game3dkit204" · use game3dkit204

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit205

import "std/game3dkit205" · use game3dkit205

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit206

import "std/game3dkit206" · use game3dkit206

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit207

import "std/game3dkit207" · use game3dkit207

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit208

import "std/game3dkit208" · use game3dkit208

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit209

import "std/game3dkit209" · use game3dkit209

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit210

import "std/game3dkit210" · use game3dkit210

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit211

import "std/game3dkit211" · use game3dkit211

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit212

import "std/game3dkit212" · use game3dkit212

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit213

import "std/game3dkit213" · use game3dkit213

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit214

import "std/game3dkit214" · use game3dkit214

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit215

import "std/game3dkit215" · use game3dkit215

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit216

import "std/game3dkit216" · use game3dkit216

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit217

import "std/game3dkit217" · use game3dkit217

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit218

import "std/game3dkit218" · use game3dkit218

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit219

import "std/game3dkit219" · use game3dkit219

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit220

import "std/game3dkit220" · use game3dkit220

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit221

import "std/game3dkit221" · use game3dkit221

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit222

import "std/game3dkit222" · use game3dkit222

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit223

import "std/game3dkit223" · use game3dkit223

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit224

import "std/game3dkit224" · use game3dkit224

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit225

import "std/game3dkit225" · use game3dkit225

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit226

import "std/game3dkit226" · use game3dkit226

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit227

import "std/game3dkit227" · use game3dkit227

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit228

import "std/game3dkit228" · use game3dkit228

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit229

import "std/game3dkit229" · use game3dkit229

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit230

import "std/game3dkit230" · use game3dkit230

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit231

import "std/game3dkit231" · use game3dkit231

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit232

import "std/game3dkit232" · use game3dkit232

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit233

import "std/game3dkit233" · use game3dkit233

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit234

import "std/game3dkit234" · use game3dkit234

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit235

import "std/game3dkit235" · use game3dkit235

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit236

import "std/game3dkit236" · use game3dkit236

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit237

import "std/game3dkit237" · use game3dkit237

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit238

import "std/game3dkit238" · use game3dkit238

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit239

import "std/game3dkit239" · use game3dkit239

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit240

import "std/game3dkit240" · use game3dkit240

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit241

import "std/game3dkit241" · use game3dkit241

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit242

import "std/game3dkit242" · use game3dkit242

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit243

import "std/game3dkit243" · use game3dkit243

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit244

import "std/game3dkit244" · use game3dkit244

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit245

import "std/game3dkit245" · use game3dkit245

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit246

import "std/game3dkit246" · use game3dkit246

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit247

import "std/game3dkit247" · use game3dkit247

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit248

import "std/game3dkit248" · use game3dkit248

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit249

import "std/game3dkit249" · use game3dkit249

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit250

import "std/game3dkit250" · use game3dkit250

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit251

import "std/game3dkit251" · use game3dkit251

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit252

import "std/game3dkit252" · use game3dkit252

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit253

import "std/game3dkit253" · use game3dkit253

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit254

import "std/game3dkit254" · use game3dkit254

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit255

import "std/game3dkit255" · use game3dkit255

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit256

import "std/game3dkit256" · use game3dkit256

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit257

import "std/game3dkit257" · use game3dkit257

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit258

import "std/game3dkit258" · use game3dkit258

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit259

import "std/game3dkit259" · use game3dkit259

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit260

import "std/game3dkit260" · use game3dkit260

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit261

import "std/game3dkit261" · use game3dkit261

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit262

import "std/game3dkit262" · use game3dkit262

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit263

import "std/game3dkit263" · use game3dkit263

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit264

import "std/game3dkit264" · use game3dkit264

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit265

import "std/game3dkit265" · use game3dkit265

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit266

import "std/game3dkit266" · use game3dkit266

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit267

import "std/game3dkit267" · use game3dkit267

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit268

import "std/game3dkit268" · use game3dkit268

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit269

import "std/game3dkit269" · use game3dkit269

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit270

import "std/game3dkit270" · use game3dkit270

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit271

import "std/game3dkit271" · use game3dkit271

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit272

import "std/game3dkit272" · use game3dkit272

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit273

import "std/game3dkit273" · use game3dkit273

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit274

import "std/game3dkit274" · use game3dkit274

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit275

import "std/game3dkit275" · use game3dkit275

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit276

import "std/game3dkit276" · use game3dkit276

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit277

import "std/game3dkit277" · use game3dkit277

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit278

import "std/game3dkit278" · use game3dkit278

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit279

import "std/game3dkit279" · use game3dkit279

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit280

import "std/game3dkit280" · use game3dkit280

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit281

import "std/game3dkit281" · use game3dkit281

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit282

import "std/game3dkit282" · use game3dkit282

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit283

import "std/game3dkit283" · use game3dkit283

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit284

import "std/game3dkit284" · use game3dkit284

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit285

import "std/game3dkit285" · use game3dkit285

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit286

import "std/game3dkit286" · use game3dkit286

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit287

import "std/game3dkit287" · use game3dkit287

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit288

import "std/game3dkit288" · use game3dkit288

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit289

import "std/game3dkit289" · use game3dkit289

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit290

import "std/game3dkit290" · use game3dkit290

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit291

import "std/game3dkit291" · use game3dkit291

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit292

import "std/game3dkit292" · use game3dkit292

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit293

import "std/game3dkit293" · use game3dkit293

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit294

import "std/game3dkit294" · use game3dkit294

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit295

import "std/game3dkit295" · use game3dkit295

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit296

import "std/game3dkit296" · use game3dkit296

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit297

import "std/game3dkit297" · use game3dkit297

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit298

import "std/game3dkit298" · use game3dkit298

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit299

import "std/game3dkit299" · use game3dkit299

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit300

import "std/game3dkit300" · use game3dkit300

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit301

import "std/game3dkit301" · use game3dkit301

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit302

import "std/game3dkit302" · use game3dkit302

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit303

import "std/game3dkit303" · use game3dkit303

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit304

import "std/game3dkit304" · use game3dkit304

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit305

import "std/game3dkit305" · use game3dkit305

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit306

import "std/game3dkit306" · use game3dkit306

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit307

import "std/game3dkit307" · use game3dkit307

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit308

import "std/game3dkit308" · use game3dkit308

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit309

import "std/game3dkit309" · use game3dkit309

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit310

import "std/game3dkit310" · use game3dkit310

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit311

import "std/game3dkit311" · use game3dkit311

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit312

import "std/game3dkit312" · use game3dkit312

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit313

import "std/game3dkit313" · use game3dkit313

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit314

import "std/game3dkit314" · use game3dkit314

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit315

import "std/game3dkit315" · use game3dkit315

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit316

import "std/game3dkit316" · use game3dkit316

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit317

import "std/game3dkit317" · use game3dkit317

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit318

import "std/game3dkit318" · use game3dkit318

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit319

import "std/game3dkit319" · use game3dkit319

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit320

import "std/game3dkit320" · use game3dkit320

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit321

import "std/game3dkit321" · use game3dkit321

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit322

import "std/game3dkit322" · use game3dkit322

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit323

import "std/game3dkit323" · use game3dkit323

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit324

import "std/game3dkit324" · use game3dkit324

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit325

import "std/game3dkit325" · use game3dkit325

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit326

import "std/game3dkit326" · use game3dkit326

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit327

import "std/game3dkit327" · use game3dkit327

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit328

import "std/game3dkit328" · use game3dkit328

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit329

import "std/game3dkit329" · use game3dkit329

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit330

import "std/game3dkit330" · use game3dkit330

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit331

import "std/game3dkit331" · use game3dkit331

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit332

import "std/game3dkit332" · use game3dkit332

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit333

import "std/game3dkit333" · use game3dkit333

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit334

import "std/game3dkit334" · use game3dkit334

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit335

import "std/game3dkit335" · use game3dkit335

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit336

import "std/game3dkit336" · use game3dkit336

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit337

import "std/game3dkit337" · use game3dkit337

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit338

import "std/game3dkit338" · use game3dkit338

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit339

import "std/game3dkit339" · use game3dkit339

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit340

import "std/game3dkit340" · use game3dkit340

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit341

import "std/game3dkit341" · use game3dkit341

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit342

import "std/game3dkit342" · use game3dkit342

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit343

import "std/game3dkit343" · use game3dkit343

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit344

import "std/game3dkit344" · use game3dkit344

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit345

import "std/game3dkit345" · use game3dkit345

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit346

import "std/game3dkit346" · use game3dkit346

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit347

import "std/game3dkit347" · use game3dkit347

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit348

import "std/game3dkit348" · use game3dkit348

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit349

import "std/game3dkit349" · use game3dkit349

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit350

import "std/game3dkit350" · use game3dkit350

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit351

import "std/game3dkit351" · use game3dkit351

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit352

import "std/game3dkit352" · use game3dkit352

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit353

import "std/game3dkit353" · use game3dkit353

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit354

import "std/game3dkit354" · use game3dkit354

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit355

import "std/game3dkit355" · use game3dkit355

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit356

import "std/game3dkit356" · use game3dkit356

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit357

import "std/game3dkit357" · use game3dkit357

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit358

import "std/game3dkit358" · use game3dkit358

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit359

import "std/game3dkit359" · use game3dkit359

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit360

import "std/game3dkit360" · use game3dkit360

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit361

import "std/game3dkit361" · use game3dkit361

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit362

import "std/game3dkit362" · use game3dkit362

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit363

import "std/game3dkit363" · use game3dkit363

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit364

import "std/game3dkit364" · use game3dkit364

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit365

import "std/game3dkit365" · use game3dkit365

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit366

import "std/game3dkit366" · use game3dkit366

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit367

import "std/game3dkit367" · use game3dkit367

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit368

import "std/game3dkit368" · use game3dkit368

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit369

import "std/game3dkit369" · use game3dkit369

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit370

import "std/game3dkit370" · use game3dkit370

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit371

import "std/game3dkit371" · use game3dkit371

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit372

import "std/game3dkit372" · use game3dkit372

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit373

import "std/game3dkit373" · use game3dkit373

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit374

import "std/game3dkit374" · use game3dkit374

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit375

import "std/game3dkit375" · use game3dkit375

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit376

import "std/game3dkit376" · use game3dkit376

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit377

import "std/game3dkit377" · use game3dkit377

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit378

import "std/game3dkit378" · use game3dkit378

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit379

import "std/game3dkit379" · use game3dkit379

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit380

import "std/game3dkit380" · use game3dkit380

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit381

import "std/game3dkit381" · use game3dkit381

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit382

import "std/game3dkit382" · use game3dkit382

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit383

import "std/game3dkit383" · use game3dkit383

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit384

import "std/game3dkit384" · use game3dkit384

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit385

import "std/game3dkit385" · use game3dkit385

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit386

import "std/game3dkit386" · use game3dkit386

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit387

import "std/game3dkit387" · use game3dkit387

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit388

import "std/game3dkit388" · use game3dkit388

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit389

import "std/game3dkit389" · use game3dkit389

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit390

import "std/game3dkit390" · use game3dkit390

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit391

import "std/game3dkit391" · use game3dkit391

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit392

import "std/game3dkit392" · use game3dkit392

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit393

import "std/game3dkit393" · use game3dkit393

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit394

import "std/game3dkit394" · use game3dkit394

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit395

import "std/game3dkit395" · use game3dkit395

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit396

import "std/game3dkit396" · use game3dkit396

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit397

import "std/game3dkit397" · use game3dkit397

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit398

import "std/game3dkit398" · use game3dkit398

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit399

import "std/game3dkit399" · use game3dkit399

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit400

import "std/game3dkit400" · use game3dkit400

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit401

import "std/game3dkit401" · use game3dkit401

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit402

import "std/game3dkit402" · use game3dkit402

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit403

import "std/game3dkit403" · use game3dkit403

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit404

import "std/game3dkit404" · use game3dkit404

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit405

import "std/game3dkit405" · use game3dkit405

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit406

import "std/game3dkit406" · use game3dkit406

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit407

import "std/game3dkit407" · use game3dkit407

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit408

import "std/game3dkit408" · use game3dkit408

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit409

import "std/game3dkit409" · use game3dkit409

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit410

import "std/game3dkit410" · use game3dkit410

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit411

import "std/game3dkit411" · use game3dkit411

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit412

import "std/game3dkit412" · use game3dkit412

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit413

import "std/game3dkit413" · use game3dkit413

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit414

import "std/game3dkit414" · use game3dkit414

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit415

import "std/game3dkit415" · use game3dkit415

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit416

import "std/game3dkit416" · use game3dkit416

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit417

import "std/game3dkit417" · use game3dkit417

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit418

import "std/game3dkit418" · use game3dkit418

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit419

import "std/game3dkit419" · use game3dkit419

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit420

import "std/game3dkit420" · use game3dkit420

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit421

import "std/game3dkit421" · use game3dkit421

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit422

import "std/game3dkit422" · use game3dkit422

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit423

import "std/game3dkit423" · use game3dkit423

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit424

import "std/game3dkit424" · use game3dkit424

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit425

import "std/game3dkit425" · use game3dkit425

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit426

import "std/game3dkit426" · use game3dkit426

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit427

import "std/game3dkit427" · use game3dkit427

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit428

import "std/game3dkit428" · use game3dkit428

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit429

import "std/game3dkit429" · use game3dkit429

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit430

import "std/game3dkit430" · use game3dkit430

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit431

import "std/game3dkit431" · use game3dkit431

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit432

import "std/game3dkit432" · use game3dkit432

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit433

import "std/game3dkit433" · use game3dkit433

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit434

import "std/game3dkit434" · use game3dkit434

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit435

import "std/game3dkit435" · use game3dkit435

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit436

import "std/game3dkit436" · use game3dkit436

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit437

import "std/game3dkit437" · use game3dkit437

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit438

import "std/game3dkit438" · use game3dkit438

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit439

import "std/game3dkit439" · use game3dkit439

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit440

import "std/game3dkit440" · use game3dkit440

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit441

import "std/game3dkit441" · use game3dkit441

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit442

import "std/game3dkit442" · use game3dkit442

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit443

import "std/game3dkit443" · use game3dkit443

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit444

import "std/game3dkit444" · use game3dkit444

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit445

import "std/game3dkit445" · use game3dkit445

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit446

import "std/game3dkit446" · use game3dkit446

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit447

import "std/game3dkit447" · use game3dkit447

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit448

import "std/game3dkit448" · use game3dkit448

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit449

import "std/game3dkit449" · use game3dkit449

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit450

import "std/game3dkit450" · use game3dkit450

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit451

import "std/game3dkit451" · use game3dkit451

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit452

import "std/game3dkit452" · use game3dkit452

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit453

import "std/game3dkit453" · use game3dkit453

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit454

import "std/game3dkit454" · use game3dkit454

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit455

import "std/game3dkit455" · use game3dkit455

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit456

import "std/game3dkit456" · use game3dkit456

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit457

import "std/game3dkit457" · use game3dkit457

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit458

import "std/game3dkit458" · use game3dkit458

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit459

import "std/game3dkit459" · use game3dkit459

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit460

import "std/game3dkit460" · use game3dkit460

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit461

import "std/game3dkit461" · use game3dkit461

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit462

import "std/game3dkit462" · use game3dkit462

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit463

import "std/game3dkit463" · use game3dkit463

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit464

import "std/game3dkit464" · use game3dkit464

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit465

import "std/game3dkit465" · use game3dkit465

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit466

import "std/game3dkit466" · use game3dkit466

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit467

import "std/game3dkit467" · use game3dkit467

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit468

import "std/game3dkit468" · use game3dkit468

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit469

import "std/game3dkit469" · use game3dkit469

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit470

import "std/game3dkit470" · use game3dkit470

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit471

import "std/game3dkit471" · use game3dkit471

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit472

import "std/game3dkit472" · use game3dkit472

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit473

import "std/game3dkit473" · use game3dkit473

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit474

import "std/game3dkit474" · use game3dkit474

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit475

import "std/game3dkit475" · use game3dkit475

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit476

import "std/game3dkit476" · use game3dkit476

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit477

import "std/game3dkit477" · use game3dkit477

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit478

import "std/game3dkit478" · use game3dkit478

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit479

import "std/game3dkit479" · use game3dkit479

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit480

import "std/game3dkit480" · use game3dkit480

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit481

import "std/game3dkit481" · use game3dkit481

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit482

import "std/game3dkit482" · use game3dkit482

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit483

import "std/game3dkit483" · use game3dkit483

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit484

import "std/game3dkit484" · use game3dkit484

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit485

import "std/game3dkit485" · use game3dkit485

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit486

import "std/game3dkit486" · use game3dkit486

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit487

import "std/game3dkit487" · use game3dkit487

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit488

import "std/game3dkit488" · use game3dkit488

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit489

import "std/game3dkit489" · use game3dkit489

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit490

import "std/game3dkit490" · use game3dkit490

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit491

import "std/game3dkit491" · use game3dkit491

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit492

import "std/game3dkit492" · use game3dkit492

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit493

import "std/game3dkit493" · use game3dkit493

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit494

import "std/game3dkit494" · use game3dkit494

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/game3dkit495

import "std/game3dkit495" · use game3dkit495

- `yawFromDir(dx number, dz number) → number`
- `pitchFromDir(dx number, dy number, dz number) → number`

std/game3dkit496

import "std/game3dkit496" · use game3dkit496

- `rayPlaneHitT(ox number, oy number, oz number, dx number, dy number, dz number, py number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit497

import "std/game3dkit497" · use game3dkit497

- `smooth(cur number, target number, dt number) → number`
- `lerp3x(ax number, bx number, t number) → number`

std/game3dkit498

import "std/game3dkit498" · use game3dkit498

- `dot3(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossX(ax number, ay number, az number, bx number, by number, bz number) → number`
- `crossY(ax number, ay number, az number, bx number, by number, bz number) → number`

std/game3dkit499

import "std/game3dkit499" · use game3dkit499

- `len3(x number, y number, z number) → number`
- `normalize3x(x number, y number, z number) → number`
- `normalize3y(x number, y number, z number) → number`

std/game3dkit500

import "std/game3dkit500" · use game3dkit500

- `normalize3z(x number, y number, z number) → number`
- `dist3(x0 number, y0 number, z0 number, x1 number, y1 number, z1 number) → number`

std/giskit001

import "std/giskit001" · use giskit001

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit002

import "std/giskit002" · use giskit002

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit003

import "std/giskit003" · use giskit003

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit004

import "std/giskit004" · use giskit004

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit005

import "std/giskit005" · use giskit005

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit006

import "std/giskit006" · use giskit006

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit007

import "std/giskit007" · use giskit007

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit008

import "std/giskit008" · use giskit008

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit009

import "std/giskit009" · use giskit009

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit010

import "std/giskit010" · use giskit010

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit011

import "std/giskit011" · use giskit011

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit012

import "std/giskit012" · use giskit012

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit013

import "std/giskit013" · use giskit013

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit014

import "std/giskit014" · use giskit014

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit015

import "std/giskit015" · use giskit015

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit016

import "std/giskit016" · use giskit016

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit017

import "std/giskit017" · use giskit017

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit018

import "std/giskit018" · use giskit018

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit019

import "std/giskit019" · use giskit019

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit020

import "std/giskit020" · use giskit020

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit021

import "std/giskit021" · use giskit021

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit022

import "std/giskit022" · use giskit022

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit023

import "std/giskit023" · use giskit023

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit024

import "std/giskit024" · use giskit024

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit025

import "std/giskit025" · use giskit025

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit026

import "std/giskit026" · use giskit026

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit027

import "std/giskit027" · use giskit027

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit028

import "std/giskit028" · use giskit028

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit029

import "std/giskit029" · use giskit029

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit030

import "std/giskit030" · use giskit030

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit031

import "std/giskit031" · use giskit031

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit032

import "std/giskit032" · use giskit032

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit033

import "std/giskit033" · use giskit033

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit034

import "std/giskit034" · use giskit034

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit035

import "std/giskit035" · use giskit035

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit036

import "std/giskit036" · use giskit036

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit037

import "std/giskit037" · use giskit037

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit038

import "std/giskit038" · use giskit038

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit039

import "std/giskit039" · use giskit039

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit040

import "std/giskit040" · use giskit040

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit041

import "std/giskit041" · use giskit041

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit042

import "std/giskit042" · use giskit042

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit043

import "std/giskit043" · use giskit043

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit044

import "std/giskit044" · use giskit044

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit045

import "std/giskit045" · use giskit045

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit046

import "std/giskit046" · use giskit046

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit047

import "std/giskit047" · use giskit047

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit048

import "std/giskit048" · use giskit048

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit049

import "std/giskit049" · use giskit049

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit050

import "std/giskit050" · use giskit050

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit051

import "std/giskit051" · use giskit051

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit052

import "std/giskit052" · use giskit052

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit053

import "std/giskit053" · use giskit053

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit054

import "std/giskit054" · use giskit054

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit055

import "std/giskit055" · use giskit055

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit056

import "std/giskit056" · use giskit056

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit057

import "std/giskit057" · use giskit057

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit058

import "std/giskit058" · use giskit058

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit059

import "std/giskit059" · use giskit059

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit060

import "std/giskit060" · use giskit060

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit061

import "std/giskit061" · use giskit061

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit062

import "std/giskit062" · use giskit062

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit063

import "std/giskit063" · use giskit063

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit064

import "std/giskit064" · use giskit064

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit065

import "std/giskit065" · use giskit065

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit066

import "std/giskit066" · use giskit066

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit067

import "std/giskit067" · use giskit067

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit068

import "std/giskit068" · use giskit068

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit069

import "std/giskit069" · use giskit069

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit070

import "std/giskit070" · use giskit070

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit071

import "std/giskit071" · use giskit071

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit072

import "std/giskit072" · use giskit072

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit073

import "std/giskit073" · use giskit073

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit074

import "std/giskit074" · use giskit074

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit075

import "std/giskit075" · use giskit075

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit076

import "std/giskit076" · use giskit076

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit077

import "std/giskit077" · use giskit077

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit078

import "std/giskit078" · use giskit078

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit079

import "std/giskit079" · use giskit079

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit080

import "std/giskit080" · use giskit080

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit081

import "std/giskit081" · use giskit081

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit082

import "std/giskit082" · use giskit082

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit083

import "std/giskit083" · use giskit083

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit084

import "std/giskit084" · use giskit084

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit085

import "std/giskit085" · use giskit085

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit086

import "std/giskit086" · use giskit086

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit087

import "std/giskit087" · use giskit087

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit088

import "std/giskit088" · use giskit088

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit089

import "std/giskit089" · use giskit089

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit090

import "std/giskit090" · use giskit090

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit091

import "std/giskit091" · use giskit091

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit092

import "std/giskit092" · use giskit092

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit093

import "std/giskit093" · use giskit093

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit094

import "std/giskit094" · use giskit094

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit095

import "std/giskit095" · use giskit095

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit096

import "std/giskit096" · use giskit096

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit097

import "std/giskit097" · use giskit097

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit098

import "std/giskit098" · use giskit098

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit099

import "std/giskit099" · use giskit099

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit100

import "std/giskit100" · use giskit100

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit101

import "std/giskit101" · use giskit101

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit102

import "std/giskit102" · use giskit102

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit103

import "std/giskit103" · use giskit103

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit104

import "std/giskit104" · use giskit104

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit105

import "std/giskit105" · use giskit105

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit106

import "std/giskit106" · use giskit106

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit107

import "std/giskit107" · use giskit107

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit108

import "std/giskit108" · use giskit108

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit109

import "std/giskit109" · use giskit109

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit110

import "std/giskit110" · use giskit110

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit111

import "std/giskit111" · use giskit111

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit112

import "std/giskit112" · use giskit112

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit113

import "std/giskit113" · use giskit113

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit114

import "std/giskit114" · use giskit114

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit115

import "std/giskit115" · use giskit115

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit116

import "std/giskit116" · use giskit116

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit117

import "std/giskit117" · use giskit117

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit118

import "std/giskit118" · use giskit118

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit119

import "std/giskit119" · use giskit119

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit120

import "std/giskit120" · use giskit120

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit121

import "std/giskit121" · use giskit121

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit122

import "std/giskit122" · use giskit122

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit123

import "std/giskit123" · use giskit123

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit124

import "std/giskit124" · use giskit124

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit125

import "std/giskit125" · use giskit125

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit126

import "std/giskit126" · use giskit126

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit127

import "std/giskit127" · use giskit127

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit128

import "std/giskit128" · use giskit128

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit129

import "std/giskit129" · use giskit129

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit130

import "std/giskit130" · use giskit130

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit131

import "std/giskit131" · use giskit131

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit132

import "std/giskit132" · use giskit132

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit133

import "std/giskit133" · use giskit133

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit134

import "std/giskit134" · use giskit134

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit135

import "std/giskit135" · use giskit135

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit136

import "std/giskit136" · use giskit136

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit137

import "std/giskit137" · use giskit137

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit138

import "std/giskit138" · use giskit138

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit139

import "std/giskit139" · use giskit139

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit140

import "std/giskit140" · use giskit140

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit141

import "std/giskit141" · use giskit141

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit142

import "std/giskit142" · use giskit142

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit143

import "std/giskit143" · use giskit143

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit144

import "std/giskit144" · use giskit144

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit145

import "std/giskit145" · use giskit145

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit146

import "std/giskit146" · use giskit146

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit147

import "std/giskit147" · use giskit147

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit148

import "std/giskit148" · use giskit148

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit149

import "std/giskit149" · use giskit149

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit150

import "std/giskit150" · use giskit150

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit151

import "std/giskit151" · use giskit151

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit152

import "std/giskit152" · use giskit152

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit153

import "std/giskit153" · use giskit153

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit154

import "std/giskit154" · use giskit154

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit155

import "std/giskit155" · use giskit155

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit156

import "std/giskit156" · use giskit156

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit157

import "std/giskit157" · use giskit157

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit158

import "std/giskit158" · use giskit158

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit159

import "std/giskit159" · use giskit159

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit160

import "std/giskit160" · use giskit160

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit161

import "std/giskit161" · use giskit161

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit162

import "std/giskit162" · use giskit162

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit163

import "std/giskit163" · use giskit163

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit164

import "std/giskit164" · use giskit164

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit165

import "std/giskit165" · use giskit165

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit166

import "std/giskit166" · use giskit166

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit167

import "std/giskit167" · use giskit167

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit168

import "std/giskit168" · use giskit168

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit169

import "std/giskit169" · use giskit169

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit170

import "std/giskit170" · use giskit170

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit171

import "std/giskit171" · use giskit171

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit172

import "std/giskit172" · use giskit172

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit173

import "std/giskit173" · use giskit173

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit174

import "std/giskit174" · use giskit174

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit175

import "std/giskit175" · use giskit175

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit176

import "std/giskit176" · use giskit176

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit177

import "std/giskit177" · use giskit177

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit178

import "std/giskit178" · use giskit178

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit179

import "std/giskit179" · use giskit179

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit180

import "std/giskit180" · use giskit180

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit181

import "std/giskit181" · use giskit181

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit182

import "std/giskit182" · use giskit182

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit183

import "std/giskit183" · use giskit183

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit184

import "std/giskit184" · use giskit184

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit185

import "std/giskit185" · use giskit185

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit186

import "std/giskit186" · use giskit186

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit187

import "std/giskit187" · use giskit187

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit188

import "std/giskit188" · use giskit188

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit189

import "std/giskit189" · use giskit189

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit190

import "std/giskit190" · use giskit190

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit191

import "std/giskit191" · use giskit191

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit192

import "std/giskit192" · use giskit192

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit193

import "std/giskit193" · use giskit193

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit194

import "std/giskit194" · use giskit194

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit195

import "std/giskit195" · use giskit195

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit196

import "std/giskit196" · use giskit196

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit197

import "std/giskit197" · use giskit197

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit198

import "std/giskit198" · use giskit198

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit199

import "std/giskit199" · use giskit199

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit200

import "std/giskit200" · use giskit200

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit201

import "std/giskit201" · use giskit201

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit202

import "std/giskit202" · use giskit202

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit203

import "std/giskit203" · use giskit203

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit204

import "std/giskit204" · use giskit204

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit205

import "std/giskit205" · use giskit205

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit206

import "std/giskit206" · use giskit206

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit207

import "std/giskit207" · use giskit207

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit208

import "std/giskit208" · use giskit208

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit209

import "std/giskit209" · use giskit209

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit210

import "std/giskit210" · use giskit210

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit211

import "std/giskit211" · use giskit211

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit212

import "std/giskit212" · use giskit212

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit213

import "std/giskit213" · use giskit213

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit214

import "std/giskit214" · use giskit214

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit215

import "std/giskit215" · use giskit215

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit216

import "std/giskit216" · use giskit216

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit217

import "std/giskit217" · use giskit217

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit218

import "std/giskit218" · use giskit218

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit219

import "std/giskit219" · use giskit219

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit220

import "std/giskit220" · use giskit220

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit221

import "std/giskit221" · use giskit221

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit222

import "std/giskit222" · use giskit222

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit223

import "std/giskit223" · use giskit223

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit224

import "std/giskit224" · use giskit224

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit225

import "std/giskit225" · use giskit225

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit226

import "std/giskit226" · use giskit226

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit227

import "std/giskit227" · use giskit227

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit228

import "std/giskit228" · use giskit228

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit229

import "std/giskit229" · use giskit229

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit230

import "std/giskit230" · use giskit230

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit231

import "std/giskit231" · use giskit231

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit232

import "std/giskit232" · use giskit232

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit233

import "std/giskit233" · use giskit233

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit234

import "std/giskit234" · use giskit234

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit235

import "std/giskit235" · use giskit235

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit236

import "std/giskit236" · use giskit236

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit237

import "std/giskit237" · use giskit237

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit238

import "std/giskit238" · use giskit238

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit239

import "std/giskit239" · use giskit239

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit240

import "std/giskit240" · use giskit240

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit241

import "std/giskit241" · use giskit241

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit242

import "std/giskit242" · use giskit242

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit243

import "std/giskit243" · use giskit243

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit244

import "std/giskit244" · use giskit244

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit245

import "std/giskit245" · use giskit245

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit246

import "std/giskit246" · use giskit246

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit247

import "std/giskit247" · use giskit247

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit248

import "std/giskit248" · use giskit248

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit249

import "std/giskit249" · use giskit249

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit250

import "std/giskit250" · use giskit250

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit251

import "std/giskit251" · use giskit251

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit252

import "std/giskit252" · use giskit252

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit253

import "std/giskit253" · use giskit253

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit254

import "std/giskit254" · use giskit254

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit255

import "std/giskit255" · use giskit255

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit256

import "std/giskit256" · use giskit256

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit257

import "std/giskit257" · use giskit257

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit258

import "std/giskit258" · use giskit258

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit259

import "std/giskit259" · use giskit259

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit260

import "std/giskit260" · use giskit260

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit261

import "std/giskit261" · use giskit261

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit262

import "std/giskit262" · use giskit262

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit263

import "std/giskit263" · use giskit263

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit264

import "std/giskit264" · use giskit264

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit265

import "std/giskit265" · use giskit265

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit266

import "std/giskit266" · use giskit266

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit267

import "std/giskit267" · use giskit267

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit268

import "std/giskit268" · use giskit268

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit269

import "std/giskit269" · use giskit269

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit270

import "std/giskit270" · use giskit270

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit271

import "std/giskit271" · use giskit271

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit272

import "std/giskit272" · use giskit272

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit273

import "std/giskit273" · use giskit273

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit274

import "std/giskit274" · use giskit274

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit275

import "std/giskit275" · use giskit275

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit276

import "std/giskit276" · use giskit276

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit277

import "std/giskit277" · use giskit277

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit278

import "std/giskit278" · use giskit278

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit279

import "std/giskit279" · use giskit279

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit280

import "std/giskit280" · use giskit280

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit281

import "std/giskit281" · use giskit281

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit282

import "std/giskit282" · use giskit282

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit283

import "std/giskit283" · use giskit283

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit284

import "std/giskit284" · use giskit284

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit285

import "std/giskit285" · use giskit285

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit286

import "std/giskit286" · use giskit286

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit287

import "std/giskit287" · use giskit287

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit288

import "std/giskit288" · use giskit288

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit289

import "std/giskit289" · use giskit289

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit290

import "std/giskit290" · use giskit290

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit291

import "std/giskit291" · use giskit291

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit292

import "std/giskit292" · use giskit292

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit293

import "std/giskit293" · use giskit293

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit294

import "std/giskit294" · use giskit294

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit295

import "std/giskit295" · use giskit295

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit296

import "std/giskit296" · use giskit296

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit297

import "std/giskit297" · use giskit297

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit298

import "std/giskit298" · use giskit298

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit299

import "std/giskit299" · use giskit299

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit300

import "std/giskit300" · use giskit300

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit301

import "std/giskit301" · use giskit301

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit302

import "std/giskit302" · use giskit302

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit303

import "std/giskit303" · use giskit303

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit304

import "std/giskit304" · use giskit304

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit305

import "std/giskit305" · use giskit305

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit306

import "std/giskit306" · use giskit306

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit307

import "std/giskit307" · use giskit307

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit308

import "std/giskit308" · use giskit308

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit309

import "std/giskit309" · use giskit309

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit310

import "std/giskit310" · use giskit310

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit311

import "std/giskit311" · use giskit311

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit312

import "std/giskit312" · use giskit312

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit313

import "std/giskit313" · use giskit313

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit314

import "std/giskit314" · use giskit314

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit315

import "std/giskit315" · use giskit315

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit316

import "std/giskit316" · use giskit316

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit317

import "std/giskit317" · use giskit317

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit318

import "std/giskit318" · use giskit318

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit319

import "std/giskit319" · use giskit319

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit320

import "std/giskit320" · use giskit320

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit321

import "std/giskit321" · use giskit321

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit322

import "std/giskit322" · use giskit322

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit323

import "std/giskit323" · use giskit323

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit324

import "std/giskit324" · use giskit324

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit325

import "std/giskit325" · use giskit325

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit326

import "std/giskit326" · use giskit326

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit327

import "std/giskit327" · use giskit327

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit328

import "std/giskit328" · use giskit328

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit329

import "std/giskit329" · use giskit329

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit330

import "std/giskit330" · use giskit330

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit331

import "std/giskit331" · use giskit331

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit332

import "std/giskit332" · use giskit332

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit333

import "std/giskit333" · use giskit333

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit334

import "std/giskit334" · use giskit334

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit335

import "std/giskit335" · use giskit335

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit336

import "std/giskit336" · use giskit336

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit337

import "std/giskit337" · use giskit337

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit338

import "std/giskit338" · use giskit338

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit339

import "std/giskit339" · use giskit339

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit340

import "std/giskit340" · use giskit340

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit341

import "std/giskit341" · use giskit341

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit342

import "std/giskit342" · use giskit342

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit343

import "std/giskit343" · use giskit343

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit344

import "std/giskit344" · use giskit344

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit345

import "std/giskit345" · use giskit345

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit346

import "std/giskit346" · use giskit346

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit347

import "std/giskit347" · use giskit347

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit348

import "std/giskit348" · use giskit348

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit349

import "std/giskit349" · use giskit349

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit350

import "std/giskit350" · use giskit350

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit351

import "std/giskit351" · use giskit351

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit352

import "std/giskit352" · use giskit352

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit353

import "std/giskit353" · use giskit353

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit354

import "std/giskit354" · use giskit354

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit355

import "std/giskit355" · use giskit355

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit356

import "std/giskit356" · use giskit356

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit357

import "std/giskit357" · use giskit357

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit358

import "std/giskit358" · use giskit358

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit359

import "std/giskit359" · use giskit359

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit360

import "std/giskit360" · use giskit360

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit361

import "std/giskit361" · use giskit361

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit362

import "std/giskit362" · use giskit362

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit363

import "std/giskit363" · use giskit363

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit364

import "std/giskit364" · use giskit364

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit365

import "std/giskit365" · use giskit365

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit366

import "std/giskit366" · use giskit366

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit367

import "std/giskit367" · use giskit367

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit368

import "std/giskit368" · use giskit368

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit369

import "std/giskit369" · use giskit369

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit370

import "std/giskit370" · use giskit370

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit371

import "std/giskit371" · use giskit371

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit372

import "std/giskit372" · use giskit372

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit373

import "std/giskit373" · use giskit373

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit374

import "std/giskit374" · use giskit374

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit375

import "std/giskit375" · use giskit375

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit376

import "std/giskit376" · use giskit376

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit377

import "std/giskit377" · use giskit377

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit378

import "std/giskit378" · use giskit378

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit379

import "std/giskit379" · use giskit379

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit380

import "std/giskit380" · use giskit380

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit381

import "std/giskit381" · use giskit381

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit382

import "std/giskit382" · use giskit382

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit383

import "std/giskit383" · use giskit383

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit384

import "std/giskit384" · use giskit384

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit385

import "std/giskit385" · use giskit385

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit386

import "std/giskit386" · use giskit386

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit387

import "std/giskit387" · use giskit387

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit388

import "std/giskit388" · use giskit388

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit389

import "std/giskit389" · use giskit389

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit390

import "std/giskit390" · use giskit390

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit391

import "std/giskit391" · use giskit391

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit392

import "std/giskit392" · use giskit392

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit393

import "std/giskit393" · use giskit393

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit394

import "std/giskit394" · use giskit394

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit395

import "std/giskit395" · use giskit395

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit396

import "std/giskit396" · use giskit396

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit397

import "std/giskit397" · use giskit397

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit398

import "std/giskit398" · use giskit398

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit399

import "std/giskit399" · use giskit399

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit400

import "std/giskit400" · use giskit400

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit401

import "std/giskit401" · use giskit401

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit402

import "std/giskit402" · use giskit402

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit403

import "std/giskit403" · use giskit403

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit404

import "std/giskit404" · use giskit404

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit405

import "std/giskit405" · use giskit405

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit406

import "std/giskit406" · use giskit406

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit407

import "std/giskit407" · use giskit407

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit408

import "std/giskit408" · use giskit408

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit409

import "std/giskit409" · use giskit409

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit410

import "std/giskit410" · use giskit410

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit411

import "std/giskit411" · use giskit411

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit412

import "std/giskit412" · use giskit412

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit413

import "std/giskit413" · use giskit413

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit414

import "std/giskit414" · use giskit414

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit415

import "std/giskit415" · use giskit415

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit416

import "std/giskit416" · use giskit416

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit417

import "std/giskit417" · use giskit417

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit418

import "std/giskit418" · use giskit418

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit419

import "std/giskit419" · use giskit419

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit420

import "std/giskit420" · use giskit420

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit421

import "std/giskit421" · use giskit421

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit422

import "std/giskit422" · use giskit422

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit423

import "std/giskit423" · use giskit423

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit424

import "std/giskit424" · use giskit424

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit425

import "std/giskit425" · use giskit425

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit426

import "std/giskit426" · use giskit426

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit427

import "std/giskit427" · use giskit427

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit428

import "std/giskit428" · use giskit428

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit429

import "std/giskit429" · use giskit429

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit430

import "std/giskit430" · use giskit430

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit431

import "std/giskit431" · use giskit431

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit432

import "std/giskit432" · use giskit432

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit433

import "std/giskit433" · use giskit433

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit434

import "std/giskit434" · use giskit434

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit435

import "std/giskit435" · use giskit435

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit436

import "std/giskit436" · use giskit436

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit437

import "std/giskit437" · use giskit437

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit438

import "std/giskit438" · use giskit438

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit439

import "std/giskit439" · use giskit439

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit440

import "std/giskit440" · use giskit440

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit441

import "std/giskit441" · use giskit441

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit442

import "std/giskit442" · use giskit442

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit443

import "std/giskit443" · use giskit443

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit444

import "std/giskit444" · use giskit444

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit445

import "std/giskit445" · use giskit445

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit446

import "std/giskit446" · use giskit446

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit447

import "std/giskit447" · use giskit447

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit448

import "std/giskit448" · use giskit448

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit449

import "std/giskit449" · use giskit449

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit450

import "std/giskit450" · use giskit450

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit451

import "std/giskit451" · use giskit451

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit452

import "std/giskit452" · use giskit452

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit453

import "std/giskit453" · use giskit453

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit454

import "std/giskit454" · use giskit454

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit455

import "std/giskit455" · use giskit455

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit456

import "std/giskit456" · use giskit456

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit457

import "std/giskit457" · use giskit457

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit458

import "std/giskit458" · use giskit458

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit459

import "std/giskit459" · use giskit459

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit460

import "std/giskit460" · use giskit460

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit461

import "std/giskit461" · use giskit461

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit462

import "std/giskit462" · use giskit462

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit463

import "std/giskit463" · use giskit463

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit464

import "std/giskit464" · use giskit464

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit465

import "std/giskit465" · use giskit465

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit466

import "std/giskit466" · use giskit466

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit467

import "std/giskit467" · use giskit467

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit468

import "std/giskit468" · use giskit468

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit469

import "std/giskit469" · use giskit469

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit470

import "std/giskit470" · use giskit470

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit471

import "std/giskit471" · use giskit471

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit472

import "std/giskit472" · use giskit472

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit473

import "std/giskit473" · use giskit473

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit474

import "std/giskit474" · use giskit474

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit475

import "std/giskit475" · use giskit475

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit476

import "std/giskit476" · use giskit476

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit477

import "std/giskit477" · use giskit477

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit478

import "std/giskit478" · use giskit478

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit479

import "std/giskit479" · use giskit479

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit480

import "std/giskit480" · use giskit480

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit481

import "std/giskit481" · use giskit481

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit482

import "std/giskit482" · use giskit482

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit483

import "std/giskit483" · use giskit483

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit484

import "std/giskit484" · use giskit484

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

std/giskit485

import "std/giskit485" · use giskit485

- `mToKm(m number) → number`
- `kmToM(km number) → number`

std/giskit486

import "std/giskit486" · use giskit486

- `normalizeLon(lon number) → number`
- `clampLat(lat number) → number`

std/giskit487

import "std/giskit487" · use giskit487

- `pixelIndex(row number, col number, width number) → number`
- `geoToPixelX(x number, originX number, resX number) → number`

std/giskit488

import "std/giskit488" · use giskit488

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit489

import "std/giskit489" · use giskit489

- `ndvi(nir number, red number) → number`
- `brightness(r number, g number, b number) → number`

std/giskit490

import "std/giskit490" · use giskit490

- `gsdFromAltitude(altM number, focalMm number, pixelUm number) → number`
- `groundWidth(pixels number, gsd number) → number`

std/giskit491

import "std/giskit491" · use giskit491

- `tileRow(lat number, z number) → number`
- `tileCol(lon number, z number) → number`

std/giskit492

import "std/giskit492" · use giskit492

- `utmZone(lon number) → number`
- `isNorthernHemisphere(lat number) → bool`

std/giskit493

import "std/giskit493" · use giskit493

- `planarDist(x1 number, y1 number, x2 number, y2 number) → number`
- `planarDist2(x1 number, y1 number, x2 number, y2 number) → number`

std/giskit494

import "std/giskit494" · use giskit494

- `percentSlope(rise number, run number) → number`
- `degreeSlope(rise number, run number) → number`

std/giskit495

import "std/giskit495" · use giskit495

- `tileOriginX(tileCol number, tileSize number) → number`
- `tileOriginY(tileRow number, tileSize number) → number`

std/giskit496

import "std/giskit496" · use giskit496

- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `haversineM(lat1 number, lon1 number, lat2 number, lon2 number) → number`

std/giskit497

import "std/giskit497" · use giskit497

- `bearingDeg(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `azimuthDiff(a number, b number) → number`

std/giskit498

import "std/giskit498" · use giskit498

- `bboxWidth(minX number, minY number, maxX number, maxY number) → number`
- `bboxArea(minX number, minY number, maxX number, maxY number) → number`

std/giskit499

import "std/giskit499" · use giskit499

- `bboxCenterX(minX number, minY number, maxX number, maxY number) → number`
- `bboxCenterY(minX number, minY number, maxX number, maxY number) → number`

std/giskit500

import "std/giskit500" · use giskit500

- `pointInBbox(x number, y number, minX number, minY number, maxX number, maxY number) → bool`
- `degToRad(deg number) → number`

MEDIUM

Bibliothèques moyennes AFRILANG

102 modules — std/m/{nom}

std/m/textstats

import "std/m/textstats" · use textstats

- `wordCount(s text) → number`
- `charCount(s text) → number`
- `lineCount(s text) → number`
- `avgWordLen(s text) → number`
- `digitCount(s text) → number`

std/m/charfreq

import "std/m/charfreq" · use charfreq

- `countChar(s text, ch text) → number`
- `mostCommonChar(s text) → text`
- `uniqueChars(s text) → number`
- `alphaRatio(s text) → number`
- `spaceRatio(s text) → number`

std/m/sentsplit

import "std/m/sentsplit" · use sentsplit

- `splitSentences(s text) → list text`
- `sentenceCount(s text) → number`
- `firstSentence(s text) → text`
- `lastSentence(s text) → text`

std/m/wordtok

import "std/m/wordtok" · use wordtok

- `tokenize(s text) → list text`
- `tokenCount(s text) → number`
- `longestToken(s text) → text`
- `shortestToken(s text) → text`
- `avgTokenLen(s text) → number`

std/m/ngrams

import "std/m/ngrams" · use ngrams

- `bigrams(s text) → list text`
- `trigrams(s text) → list text`
- `charBigrams(s text) → list text`
- `ngramCount(s text, n number) → number`

std/m/palindrom2

import "std/m/palindrom2" · use palindrom2

- `isPalindrome(s text) → bool`
- `longestPalindromeLen(s text) → number`
- `reverseAlphaNum(s text) → text`
- `isPalindromeWord(w text) → bool`

std/m/anagram2

import "std/m/anagram2" · use anagram2

- `normalizeWord(s text) → text`
- `areAnagrams(a text, b text) → bool`
- `anagramKey(s text) → text`
- `sortLetters(s text) → text`

std/m/textwrap2

import "std/m/textwrap2" · use textwrap2

- `wrapLines(s text, width number) → list text`
- `wrapCount(s text, width number) → number`
- `truncateWords(s text, maxWords number) → text`
- `padCenter(s text, width number) → text`

std/m/textsearch

import "std/m/textsearch" · use textsearch

- `findAll(s text, sub text) → list number`
- `countOccurrences(s text, sub text) → number`
- `containsIgnoreCase(s text, sub text) → bool`
- `replaceAll(s text, from text, to text) → text`
- `indexOf(s text, sub text) → number`

std/m/accentstrip

import "std/m/accentstrip" · use accentstrip

- `stripAccents(s text) → text`
- `toAsciiFold(s text) → text`
- `removeNonPrintable(s text) → text`
- `collapseSpaces(s text) → text`

std/m/casemap2

import "std/m/casemap2" · use casemap2

- `titleCase(s text) → text`
- `swapCase(s text) → text`
- `camelToSnake(s text) → text`
- `snakeToCamel(s text) → text`

std/m/textdiff2

import "std/m/textdiff2" · use textdiff2

- `commonPrefixLen(a text, b text) → number`
- `commonSuffixLen(a text, b text) → number`
- `levenshtein(a text, b text) → number`
- `similarityRatio(a text, b text) → number`

std/m/regexlite

import "std/m/regexlite" · use regexlite

- `matchDigits(s text) → bool`
- `matchAlpha(s text) → bool`
- `matchAlnum(s text) → bool`
- `extractDigits(s text) → text`
- `extractAlpha(s text) → text`

std/m/motfreq

import "std/m/motfreq" · use motfreq

- `wordFrequency(s text, word text) → number`
- `mostFrequentWord(s text) → text`
- `uniqueWords(s text) → number`
- `lexicalDensity(s text) → number`

std/m/numstats

import "std/m/numstats" · use numstats

- `sum(v list number) → number`
- `mean(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `range(v list number) → number`

std/m/listmedian

import "std/m/listmedian" · use listmedian

- `median(v list number) → number`
- `q1(v list number) → number`
- `q3(v list number) → number`

- `iqr(v list number) → number`

std/m/listvar

import "std/m/listvar" · use listvar

- `variance(v list number) → number`
- `stddev(v list number) → number`
- `popVariance(v list number) → number`
- `popStddev(v list number) → number`

std/m/listmode

import "std/m/listmode" · use listmode

- `mode(v list number) → number`
- `modeCount(v list number) → number`
- `uniqueCount(v list number) → number`
- `duplicateCount(v list number) → number`

std/m/listcum

import "std/m/listcum" · use listcum

- `cumSum(v list number) → list number`
- `cumProd(v list number) → list number`
- `diffs(v list number) → list number`
- `pctChange(v list number) → list number`

std/m/listnorm

import "std/m/listnorm" · use listnorm

- `normalize(v list number) → list number`
- `zScores(v list number) → list number`
- `scaleTo100(v list number) → list number`
- `demean(v list number) → list number`

std/m/listcorr

import "std/m/listcorr" · use listcorr

- `covariance(a list number, b list number) → number`
- `correlation(a list number, b list number) → number`
- `dotProduct(a list number, b list number) → number`
- `euclidean(a list number, b list number) → number`

std/m/listoutlier

import "std/m/listoutlier" · use listoutlier

- `outlierIndices(v list number) → list number`
- `outlierCount(v list number) → number`
- `withoutOutliers(v list number) → list number`

- `winsorize(v list number, pct number) → list number`

std/m/listquantile

import "std/m/listquantile" · use listquantile

- `quantile(v list number, q number) → number`
- `percentile(v list number, p number) → number`
- `decile(v list number, d number) → number`
- `rankOf(v list number) → list number`

std/m/listfreq

import "std/m/listfreq" · use listfreq

- `valueCounts(v list number) → list number`
- `histogram(v list number, bins number) → list number`
- `binEdges(v list number, bins number) → list number`
- `skewness(v list number) → number`

std/m/listmoving

import "std/m/listmoving" · use listmoving

- `movingAvg(v list number, window number) → list number`
- `movingMax(v list number, window number) → list number`
- `movingMin(v list number, window number) → list number`
- `movingSum(v list number, window number) → list number`

std/m/listscale

import "std/m/listscale" · use listscale

- `rescale(v list number, newMin number, newMax number) → list number`
- `clip(v list number, lo number, hi number) → list number`
- `absValues(v list number) → list number`
- `negateAll(v list number) → list number`

std/m/listsort

import "std/m/listsort" · use listsort

- `sortAsc(v list number) → list number`
- `sortDesc(v list number) → list number`
- `sortTextAsc(v list text) → list text`
- `sortTextDesc(v list text) → list text`
- `isSorted(v list number) → bool`

std/m/listsearch

import "std/m/listsearch" · use listsearch

- `linearSearch(v list number, target number) → number`
- `contains(v list number, target number) → bool`

- `countValue(v list number, target number) → number`
- `lastIndexOf(v list number, target number) → number`

std/m/listbinary

import "std/m/listbinary" · use listbinary

- `binarySearch(v list number, target number) → number`
- `lowerBound(v list number, target number) → number`
- `upperBound(v list number, target number) → number`
- `insertSorted(v list number, val number) → list number`

std/m/listunique2

import "std/m/listunique2" · use listunique2

- `unique(v list number) → list number`
- `uniqueText(v list text) → list text`
- `duplicates(v list number) → list number`
- `frequency(v list number, val number) → number`

std/m/listpartition

import "std/m/listpartition" · use listpartition

- `partitionAt(v list number, pivot number) → list number`
- `splitHalf(v list number) → list number`
- `secondHalf(v list number) → list number`
- `filterGt(v list number, threshold number) → list number`
- `filterLt(v list number, threshold number) → list number`

std/m/listtopk

import "std/m/listtopk" · use listtopk

- `topK(v list number, k number) → list number`
- `bottomK(v list number, k number) → list number`
- `kthLargest(v list number, k number) → number`
- `kthSmallest(v list number, k number) → number`

std/m/listmerge2

import "std/m/listmerge2" · use listmerge2

- `mergeSorted(a list number, b list number) → list number`
- `mergeAll(a list number, b list number) → list number`
- `intersect(a list number, b list number) → list number`
- `unionAll(a list number, b list number) → list number`

std/m/listrotate2

import "std/m/listrotate2" · use listrotate2

- `rotateLeft(v list number, n number) → list number`
- `rotateRight(v list number, n number) → list number`
- `reverseList(v list number) → list number`
- `shuffleSeed(v list number, seed number) → list number`

std/m/listchunk

import "std/m/listchunk" · use listchunk

- `chunkCount(v list number, size number) → number`
- `take(v list number, n number) → list number`
- `drop(v list number, n number) → list number`
- `slice(v list number, start number, end number) → list number`
- `chunkAt(v list number, index number, size number) → list number`

std/m/listzip2

import "std/m/listzip2" · use listzip2

- `zipAdd(a list number, b list number) → list number`
- `zipMul(a list number, b list number) → list number`
- `zipSub(a list number, b list number) → list number`
- `zipMax(a list number, b list number) → list number`

std/m/textsplit2

import "std/m/textsplit2" · use textsplit2

- `splitLines(s text) → list text`
- `splitBy(s text, delim text) → list text`
- `splitWords(s text) → list text`
- `splitChars(s text) → list text`

std/m/textjoin2

import "std/m/textjoin2" · use textjoin2

- `joinLines(parts list text) → text`
- `joinWith(parts list text, delim text) → text`
- `joinComma(parts list text) → text`
- `joinSpace(parts list text) → text`

std/m/listfilter2

import "std/m/listfilter2" · use listfilter2

- `filterEmpty(v list text) → list text`
- `filterLonger(v list text, minLen number) → list text`
- `filterContains(v list text, sub text) → list text`
- `filterStarts(v list text, prefix text) → list text`

std/m/listmap2

import "std/m/listmap2" · use listmap2

- `mapUpper(v list text) → list text`
- `mapLower(v list text) → list text`
- `mapTrim(v list text) → list text`
- `mapLength(v list text) → list number`

std/m/listgroup2

import "std/m/listgroup2" · use listgroup2

- `groupByLen(v list text, len number) → list text`
- `groupByPrefix(v list text, prefix text) → list text`
- `longestString(v list text) → text`
- `shortestString(v list text) → text`

std/m/listpick2

import "std/m/listpick2" · use listpick2

- `pickFirst(v list text) → text`
- `pickLast(v list text) → text`
- `pickAt(v list text, index number) → text`
- `pickRandom(v list text, seed number) → text`

std/m/datecalc

import "std/m/datecalc" · use datecalc

- `daysBetween(d1 text, d2 text) → number`
- `addDays(date text, days number) → text`
- `isLeapYear(year number) → bool`
- `daysInMonth(year number, month number) → number`
- `weekday(date text) → number`

std/m/datefmt2

import "std/m/datefmt2" · use datefmt2

- `formatIso(year number, month number, day number) → text`
- `parseYear(date text) → number`
- `parseMonth(date text) → number`
- `parseDay(date text) → number`

std/m/duration2

import "std/m/duration2" · use duration2

- `hoursToMinutes(h number) → number`
- `minutesToSeconds(m number) → number`
- `daysToHours(d number) → number`
- `secondsToHours(s number) → number`

std/m/timestamp2

import "std/m/timestamp2" · use timestamp2

- `epochDays(date text) → number`
- `fromEpochDays(days number) → text`
- `isValidDate(date text) → bool`
- `todayOffset(date text, offset number) → number`

std/m/calendar2

import "std/m/calendar2" · use calendar2

- `quarterOf(month number) → number`
- `weekOfYear(date text) → number`
- `monthName(month number) → text`
- `dayName(date text) → text`

std/m/agecalc

import "std/m/agecalc" · use agecalc

- `ageYears(birth text, today text) → number`
- `ageMonths(birth text, today text) → number`
- `ageDays(birth text, today text) → number`
- `isBirthday(birth text, today text) → bool`

std/m/timezone2

import "std/m/timezone2" · use timezone2

- `utcOffsetHours(offset number) → number`
- `toUtc(localTime text, offsetHours number) → text`
- `fromUtc(utcTime text, offsetHours number) → text`
- `formatTime(hours number, minutes number) → text`

std/m/period2

import "std/m/period2" · use period2

- `monthsBetween(d1 text, d2 text) → number`
- `yearsBetween(d1 text, d2 text) → number`
- `addMonths(date text, months number) → text`
- `endOfMonth(date text) → text`

std/m/chrono2

import "std/m/chrono2" · use chrono2

- `elapsedDays(start text, end text) → number`
- `isWeekend(date text) → bool`
- `nextWeekday(date text) → text`
- `businessDays(start text, end text) → number`

std/m/season2

import "std/m/season2" · use season2

- `seasonOf(month number) → text`
- `isSummer(month number) → bool`
- `isWinter(month number) → bool`
- `daylightHours(month number, lat number) → number`

std/m/uriparse

import "std/m/uriparse" · use uriparse

- `getScheme(url text) → text`
- `getHost(url text) → text`
- `getPath(url text) → text`
- `getQuery(url text) → text`
- `hasQuery(url text) → bool`

std/m/urlbuild

import "std/m/urlbuild" · use urlbuild

- `buildUrl(scheme text, host text, path text) → text`
- `buildWithQuery(base text, query text) → text`
- `encodeParam(key text, value text) → text`
- `joinParams(params list text) → text`

std/m/queryparse

import "std/m/queryparse" · use queryparse

- `parseQuery(query text) → list text`
- `getParam(query text, key text) → text`
- `paramCount(query text) → number`
- `hasParam(query text, key text) → bool`

std/m/jsonparse2

import "std/m/jsonparse2" · use jsonparse2

- `parseString(json text, key text) → text`
- `parseNumber(json text, key text) → number`
- `isValidJson(json text) → bool`
- `keyCount(json text) → number`

std/m/jsonbuild2

import "std/m/jsonbuild2" · use jsonbuild2

- `buildObject(pairs list text) → text`
- `buildArray(items list text) → text`
- `quoteKey(key text, value text) → text`
- `quoteNum(key text, value number) → text`

std/m/csvparse2

import "std/m/csvparse2" · use csvparse2

- `parseRow(line text) → list text`
- `getField(line text, index number) → text`
- `fieldCount(line text) → number`
- `parseAll(csv text) → list text`

std/m/csvbuild2

import "std/m/csvbuild2" · use csvbuild2

- `buildRow(fields list text) → text`
- `escapeField(field text) → text`
- `buildCsv(rows list text) → text`
- `quoteField(field text) → text`

std/m/pathparse2

import "std/m/pathparse2" · use pathparse2

- `basename(path text) → text`
- `dirname(path text) → text`
- `extension(path text) → text`
- `stem(path text) → text`
- `joinPath(a text, b text) → text`

std/m/mimeparse2

import "std/m/mimeparse2" · use mimeparse2

- `mainType(mime text) → text`
- `subType(mime text) → text`
- `isText(mime text) → bool`
- `isImage(mime text) → bool`

std/m/httpparse2

import "std/m/httpparse2" · use httpparse2

- `statusClass(code number) → number`
- `isClientError(code number) → bool`
- `isServerError(code number) → bool`
- `statusText(code number) → text`

std/m/emailparse2

import "std/m/emailparse2" · use emailparse2

- `localPart(email text) → text`
- `domainPart(email text) → text`
- `isValidEmail(email text) → bool`
- `normalizeEmail(email text) → text`

std/m/base64med

import "std/m/base64med" · use base64med

- `encodeBasic(s text) → text`
- `decodeBasic(s text) → text`
- `isBase64(s text) → bool`
- `paddedLength(s text) → number`

std/m/matrix2

import "std/m/matrix2" · use matrix2

- `create2x2(a number, b number, c number, d number) → list number`
- `det2x2(m list number) → number`
- `trace2x2(m list number) → number`
- `scale2x2(m list number, s number) → list number`

std/m/matadd

import "std/m/matadd" · use matadd

- `add(a list number, b list number) → list number`
- `sub(a list number, b list number) → list number`
- `neg(m list number) → list number`
- `addScalar(m list number, s number) → list number`

std/m/matmul2

import "std/m/matmul2" · use matmul2

- `mul2x2(a list number, b list number) → list number`
- `mulScalar(m list number, s number) → list number`
- `dot2(a list number, b list number) → number`
- `norm2(v list number) → number`

std/m/mattranspose

import "std/m/mattranspose" · use mattranspose

- `transpose2x2(m list number) → list number`
- `transpose3x3(m list number) → list number`
- `isSquare(m list number, n number) → bool`
- `identity2(ignored number) → list number`

std/m/matdet2

import "std/m/matdet2" · use matdet2

- `det3x3(m list number) → number`
- `det2x2(m list number) → number`
- `isSingular(m list number) → bool`
- `cofactor2(m list number) → list number`

std/m/matinv2

import "std/m/matinv2" · use matinv2

- `inv2x2(m list number) → list number`
- `adj2x2(m list number) → list number`
- `solve2x2(a list number, b list number) → list number`
- `condition2x2(m list number) → number`

std/m/vector3

import "std/m/vector3" · use vector3

- `cross3(a list number, b list number) → list number`
- `dot3(a list number, b list number) → number`
- `length3(v list number) → number`
- `normalize3(v list number) → list number`

std/m/matrotate

import "std/m/matrotate" · use matrotate

- `rotate2d(angle number) → list number`
- `apply2d(m list number, v list number) → list number`
- `rotatePoint(x number, y number, angle number) → list number`
- `degreesToRad(deg number) → number`

std/m/matstats

import "std/m/matstats" · use matstats

- `rowSum(m list number, cols number) → list number`
- `colSum(m list number, rows number, cols number) → list number`
- `frobeniusNorm(m list number) → number`
- `maxAbs(m list number) → number`

std/m/linsolve2

import "std/m/linsolve2" · use linsolve2

- `solveLinear2(a1 number, b1 number, c1 number, a2 number, b2 number, c2 number) → list number`
- `solve3eq(m list number, b list number) → list number`
- `residual(a list number, x list number, b list number) → number`
- `isConsistent(m list number) → bool`

std/m/finance2

import "std/m/finance2" · use finance2

- `futureValue(pv number, rate number, periods number) → number`
- `presentValue(fv number, rate number, periods number) → number`
- `annuity(payment number, rate number, periods number) → number`
- `payment(pv number, rate number, periods number) → number`

std/m/amortize2

import "std/m/amortize2" · use amortize2

- `monthlyInterest(balance number, rate number) → number`
- `principalPart(payment number, interest number) → number`
- `remainingBalance(balance number, payment number, interest number) → number`
- `amortSchedule(balance number, rate number, payment number, months number) → list number`

std/m/npv2

import "std/m/npv2" · use npv2

- `npv(rate number, cashflows list number) → number`
- `npvSimple(rate number, initial number, returns list number) → number`
- `profitability(npvVal number, investment number) → number`
- `discountedCashflow(cf number, rate number, period number) → number`

std/m/irr2

import "std/m/irr2" · use irr2

- `irrApprox(cashflows list number) → number`
- `mirr(cashflows list number, financeRate number, reinvestRate number) → number`
- `paybackPeriod(cashflows list number) → number`
- `roiPercent(gain number, cost number) → number`

std/m/breakEven2

import "std/m/breakEven2" · use breakEven2

- `breakEvenUnits(fixedCost number, price number, varCost number) → number`
- `breakEvenRevenue(fixedCost number, margin number) → number`
- `contributionMargin(price number, varCost number) → number`
- `marginOfSafety(currentSales number, breakEven number) → number`

std/m/compound2

import "std/m/compound2" · use compound2

- `compoundAnnual(principal number, rate number, years number) → number`
- `compoundMonthly(principal number, rate number, months number) → number`
- `effectiveRate(nominal number, periods number) → number`
- `doublingTime(rate number) → number`

std/m/inflation2

import "std/m/inflation2" · use inflation2

- `inflate(amount number, rate number, years number) → number`
- `deflate(amount number, rate number, years number) → number`
- `realReturn(nominal number, inflation number) → number`
- `purchasingPower(amount number, inflation number, years number) → number`

std/m/currency2

import "std/m/currency2" · use currency2

- `convert(amount number, rate number) → number`
- `crossRate(fromUsd number, toUsd number) → number`
- `spread(bid number, ask number) → number`
- `midRate(bid number, ask number) → number`

std/m/portfolio2

import "std/m/portfolio2" · use portfolio2

- `weightedReturn(returns list number, weights list number) → number`
- `portfolioValue(shares list number, prices list number) → number`
- `rebalance(values list number, target list number) → list number`
- `diversification(weights list number) → number`

std/m/taxcalc2

import "std/m/taxcalc2" · use taxcalc2

- `incomeTax(income number, rate number) → number`
- `netIncome(gross number, rate number) → number`
- `vatAmount(price number, rate number) → number`
- `priceWithVat(price number, rate number) → number`

std/m/depreciation2

import "std/m/depreciation2" · use depreciation2

- `straightLine(cost number, salvage number, years number) → number`
- `decliningBalance(bookValue number, rate number) → number`
- `sumOfYears(cost number, salvage number, years number, year number) → number`
- `accumDepreciation(annual number, yearsElapsed number) → number`

std/m/bondcalc2

import "std/m/bondcalc2" · use bondcalc2

- `bondPrice(face number, coupon number, rate number, years number) → number`
- `yieldToMaturity(price number, face number, coupon number, years number) → number`
- `currentYield(coupon number, price number) → number`
- `duration(years number, coupon number) → number`

std/m/polygon2

import "std/m/polygon2" · use polygon2

- `areaTriangle(base number, height number) → number`
- `perimeterRect(w number, h number) → number`
- `areaRect(w number, h number) → number`
- `diagonal(w number, h number) → number`

std/m/statsreg2

import "std/m/statsreg2" · use statsreg2

- `linearSlope(x list number, y list number) → number`
- `linearIntercept(x list number, y list number) → number`
- `predict(slope number, intercept number, x number) → number`
- `rSquared(x list number, y list number) → number`

std/m/interpolate2

import "std/m/interpolate2" · use interpolate2

- `lerp(a number, b number, t number) → number`
- `lerpList(a list number, b list number, t number) → list number`
- `inverseLerp(a number, b number, v number) → number`
- `remap(v number, inLo number, inHi number, outLo number, outHi number) → number`

std/m/bezier2

import "std/m/bezier2" · use bezier2

- `quadBezier(p0 number, p1 number, p2 number, t number) → number`
- `cubicBezier(p0 number, p1 number, p2 number, p3 number, t number) → number`
- `bezierTangent(p0 number, p1 number, p2 number, t number) → number`
- `bezierPoint2d(x0 number, y0 number, x1 number, y1 number, x2 number, y2 number, t number) → list number`

std/m/distance3

import "std/m/distance3" · use distance3

- `manhattan2d(x1 number, y1 number, x2 number, y2 number) → number`
- `chebyshev2d(x1 number, y1 number, x2 number, y2 number) → number`
- `cosineSimilarity(a list number, b list number) → number`
- `hammingDist(a text, b text) → number`

std/m/histogram2

import "std/m/histogram2" · use histogram2

- `binIndex(value number, minVal number, maxVal number, bins number) → number`
- `normalizeHist(counts list number) → list number`
- `cumulativeHist(counts list number) → list number`
- `peakBin(counts list number) → number`

std/m/percentile2

import "std/m/percentile2" · use percentile2

- `percentileRank(v list number, value number) → number`
- `percentileValue(v list number, p number) → number`
- `trimMean(v list number, pct number) → number`
- `geometricMean(v list number) → number`

std/m/checksum2

import "std/m/checksum2" · use checksum2

- `crcSimple(s text) → number`
- `adler32lite(s text) → number`
- `fnvHash(s text) → number`
- `sumChecksum(s text) → number`

std/m/hash2

import "std/m/hash2" · use hash2

- `djb2(s text) → number`
- `sdbm(s text) → number`
- `javaHash(s text) → number`
- `hashCombine(a number, b number) → number`

std/m/rotN

import "std/m/rotN" · use rotN

- `rotEncode(s text, n number) → text`
- `rotDecode(s text, n number) → text`
- `caesarShift(s text, n number) → text`
- `xorCipher(s text, key text) → text`

std/m/uuid2

import "std/m/uuid2" · use uuid2

- `randomHex(n number) → text`
- `simpleUuid(seed number) → text`
- `formatUuid(hex32 text) → text`
- `isUuidFormat(s text) → bool`

std/m/random2

import "std/m/random2" · use random2

- `seedRandom(seed number) → number`
- `randomRange(lo number, hi number) → number`
- `randomList(count number, lo number, hi number) → list number`
- `shuffleList(v list number, seed number) → list number`

std/m/entropy2

import "std/m/entropy2" · use entropy2

- `shannonEntropy(s text) → number`
- `uniqueRatio(s text) → number`
- `compressionRatio(original number, compressed number) → number`
- `bitEntropy(s text) → number`

std/m/units2

import "std/m/units2" · use units2

- `celsiusToFahrenheit(c number) → number`
- `fahrenheitToCelsius(f number) → number`
- `metersToFeet(m number) → number`
- `feetToMeters(ft number) → number`

std/m/physics2

import "std/m/physics2" · use physics2

- `kineticEnergy(mass number, velocity number) → number`
- `potentialEnergy(mass number, height number) → number`
- `momentum(mass number, velocity number) → number`
- `force(mass number, accel number) → number`

std/m/gamestats2

import "std/m/gamestats2" · use gamestats2

- `scoreMultiplier(score number, multiplier number) → number`
- `xpForLevel(level number) → number`
- `levelFromXp(xp number) → number`
- `damageCalc(attack number, defense number) → number`

COMPLEX — catalogue

Bibliothèques complexes AFRILANG

5710 modules — std/c/{nom}

std/c/graphbfs

import "std/c/graphbfs" · use graphbfs

- `bfsDistances(adj list number, n number, start number) → list number`
- `bfsOrder(adj list number, n number, start number) → list number`
- `bfsReachable(adj list number, n number, start number) → number`
- `bfsLayers(adj list number, n number, start number) → list number`
- `isConnected(adj list number, n number) → bool`
- `shortestPathLen(adj list number, n number, start number, goal number) → number`
- `multiSourceBfs(adj list number, n number, sources list number) → list number`

std/c/graphdfs

import "std/c/graphdfs" · use graphdfs

- `dfsOrder(adj list number, n number, start number) → list number`
- `dfsRecursiveMark(adj list number, n number) → list number`
- `hasCycleUndirected(adj list number, n number) → bool`
- `countComponents(adj list number, n number) → number`

- `topoSortDfs(adj list number, n number) → list number`
- `isTree(adj list number, n number) → bool`
- `dfsPathExists(adj list number, n number, start number, goal number) → bool`

std/c/graphdijk

import "std/c/graphdijk" · use graphdijk

- `dijkstra(adj list number, n number, start number) → list number`
- `shortestWeighted(adj list number, n number, start number, goal number) → number`
- `dijkstraPath(adj list number, n number, start number, goal number) → list number`
- `minEdgeWeight(adj list number, n number) → number`
- `maxEdgeWeight(adj list number, n number) → number`
- `avgEdgeWeight(adj list number, n number) → number`
- `edgeCount(adj list number, n number) → number`

std/c/graphtopo

import "std/c/graphtopo" · use graphtopo

- `topoSort(adj list number, n number) → list number`
- `hasCycleDirected(adj list number, n number) → bool`
- `inDegrees(adj list number, n number) → list number`
- `outDegrees(adj list number, n number) → list number`
- `sources(adj list number, n number) → list number`
- `sinks(adj list number, n number) → list number`
- `isDag(adj list number, n number) → bool`

std/c/graphcc

import "std/c/graphcc" · use graphcc

- `connectedComponents(adj list number, n number) → list number`
- `componentCount(adj list number, n number) → number`
- `largestComponentSize(adj list number, n number) → number`
- `sameComponent(adj list number, n number, a number, b number) → bool`
- `componentSizes(adj list number, n number) → list number`
- `outDegreeSum(adj list number, n number) → list number`
- `isolateCount(adj list number, n number) → number`

std/c/graphcycle

import "std/c/graphcycle" · use graphcycle

- `detectCycleDirected(adj list number, n number) → bool`
- `detectCycleUndirected(adj list number, n number) → bool`
- `cycleEdgeCount(adj list number, n number) → number`
- `isAcyclic(adj list number, n number) → bool`
- `selfLoopCount(adj list number, n number) → number`
- `removeSelfLoops(adj list number, n number) → list number`
- `hasParallelEdges(adj list number, n number) → bool`

std/c/graphmst

import "std/c/graphmst" · use graphmst

- `mstWeightPrim(adj list number, n number) → number`
- `mstWeightKruskal(adj list number, n number) → number`
- `mstEdges(adj list number, n number) → number`
- `compareMstAlgos(adj list number, n number) → number`
- `forestMstWeight(adj list number, n number) → number`
- `isConnectedForMst(adj list number, n number) → bool`

std/c/graphshort

import "std/c/graphshort" · use graphshort

- `allPairsBfs(adj list number, n number) → list number`
- `eccentricity(adj list number, n number) → list number`
- `graphDiameter(adj list number, n number) → number`
- `graphRadius(adj list number, n number) → number`
- `centerNodes(adj list number, n number) → list number`
- `peripheryNodes(adj list number, n number) → list number`

std/c/graphbellman

import "std/c/graphbellman" · use graphbellman

- `bellmanFord(adj list number, n number, start number) → list number`
- `hasNegCycle(adj list number, n number) → bool`
- `negEdgeCount(adj list number, n number) → number`
- `relaxOnce(adj list number, n number, dist list number) → list number`
- `pathCost(adj list number, n number, path list number) → number`
- `reachableWeighted(adj list number, n number, start number) → number`

std/c/graphfloyd

import "std/c/graphfloyd" · use graphfloyd

- `floydWarshall(adj list number, n number) → list number`
- `pairDistance(adj list number, n number, a number, b number) → number`
- `closureReachable(adj list number, n number) → list number`
- `graphDensity(adj list number, n number) → number`
- `avgShortestPath(adj list number, n number) → number`
- `transitiveClosure(adj list number, n number) → list number`

std/c/graphprim

import "std/c/graphprim" · use graphprim

- `primMst(adj list number, n number) → number`
- `primParents(adj list number, n number) → list number`
- `primOrder(adj list number, n number) → list number`
- `mstEdgeCount(adj list number, n number) → number`
- `lightestEdge(adj list number, n number) → number`

- `heaviestMstEdge(adj list number, n number) → number`
- `isPrimConnected(adj list number, n number) → bool`

std/c/graphkruskal

import "std/c/graphkruskal" · use graphkruskal

- `kruskalMst(adj list number, n number) → number`
- `kruskalEdges(adj list number, n number) → list number`
- `unionFindComponents(adj list number, n number) → number`
- `edgeListSize(adj list number, n number) → number`
- `kruskalForestWeight(adj list number, n number) → number`
- `maxSpanningForest(adj list number, n number) → number`
- `compareKruskalPrim(adj list number, n number) → number`

std/c/graphbipart

import "std/c/graphbipart" · use graphbipart

- `isBipartite(adj list number, n number) → bool`
- `bipartiteColors(adj list number, n number) → list number`
- `partitionSizes(adj list number, n number) → list number`
- `crossEdgeCount(adj list number, n number) → number`
- `sameSideEdges(adj list number, n number) → number`
- `bipartiteDensity(adj list number, n number) → number`
- `hasOddCycle(adj list number, n number) → bool`

std/c/grapheuler

import "std/c/grapheuler" · use grapheuler

- `vertexDegrees(adj list number, n number) → list number`
- `hasEulerCircuit(adj list number, n number) → bool`
- `hasEulerPath(adj list number, n number) → bool`
- `oddDegreeCount(adj list number, n number) → number`
- `totalDegree(adj list number, n number) → number`
- `eulerTrailLen(adj list number, n number) → number`
- `isEulerian(adj list number, n number) → bool`

std/c/graphham

import "std/c/graphham" · use graphham

- `hamiltonianExists(adj list number, n number) → bool`
- `permuteCheck(adj list number, n number) → number`
- `longestPath(adj list number, n number) → number`
- `hamiltonianCycle(adj list number, n number) → bool`
- `pathCoverSize(adj list number, n number) → number`
- `tspLowerBound(adj list number, n number) → number`
- `nearestNeighbor(adj list number, n number, start number) → list number`

std/c/graphclique

import "std/c/graphclique" · use graphclique

- `maxCliqueSize(adj list number, n number) → number`
- `cliqueNumber(adj list number, n number) → number`
- `isClique(adj list number, n number, nodes list number) → bool`
- `cliqueCover(adj list number, n number) → number`
- `triangleCount(adj list number, n number) → number`
- `clusteringCoeff(adj list number, n number) → list number`
- `density(adj list number, n number) → number`

std/c/graphcolor

import "std/c/graphcolor" · use graphcolor

- `greedyColor(adj list number, n number) → list number`
- `chromaticNumber(adj list number, n number) → number`
- `isValidColoring(adj list number, n number, colors list number) → bool`
- `colorClasses(adj list number, n number) → list number`
- `conflictCount(adj list number, n number, colors list number) → number`
- `welshPowell(adj list number, n number) → list number`
- `sameColorNeighbors(adj list number, n number, node number) → number`

std/c/graphflow

import "std/c/graphflow" · use graphflow

- `maxFlow(adj list number, n number, source number, sink number) → number`
- `minCutValue(adj list number, n number, source number, sink number) → number`
- `flowMatrix(adj list number, n number, source number, sink number) → list number`
- `bottleneckCapacity(adj list number, n number) → number`
- `outFlowSum(adj list number, n number, node number) → number`
- `inFlowSum(adj list number, n number, node number) → number`
- `isFlowConserved(adj list number, n number, source number, sink number) → bool`

std/c/graphmatch

import "std/c/graphmatch" · use graphmatch

- `maxMatching(adj list number, n number) → number`
- `matchingPairs(adj list number, n number) → list number`
- `isPerfectMatching(adj list number, n number) → bool`
- `unmatchedCount(adj list number, n number) → number`
- `matchingWeight(adj list number, n number) → number`
- `vertexCoverSize(adj list number, n number) → number`
- `isMatched(adj list number, n number, node number) → bool`

std/c/graphcent

import "std/c/graphcent" · use graphcent

- `degreeCentrality(adj list number, n number) → list number`
- `closenessCentrality(adj list number, n number) → list number`
- `betweennessApprox(adj list number, n number) → list number`
- `eigenvectorCentrality(adj list number, n number) → list number`
- `pageRank(adj list number, n number, damping number) → list number`
- `hubScore(adj list number, n number) → list number`
- `mostCentral(adj list number, n number) → number`

std/c/statregress

import "std/c/statregress" · use statregress

- `regSum(v list number) → number`
- `regMean(v list number) → number`
- `regMin(v list number) → number`
- `regMax(v list number) → number`
- `regVar(v list number) → number`
- `regStd(v list number) → number`
- `regNorm(v list number) → list number`
- `linearRegress(x list number, y list number) → list number`

std/c/statcorrel

import "std/c/statcorrel" · use statcorrel

- `pearson(a list number, b list number) → number`
- `covariance(a list number, b list number) → number`
- `crossCorr(a list number, b list number, lag number) → number`
- `autocorr1(v list number) → number`
- `partialCorr(a list number, b list number, c list number) → number`
- `corrMatrix2(a list number, b list number) → list number`
- `rankCorr(a list number, b list number) → number`

std/c/statquartile

import "std/c/statquartile" · use statquartile

- `q1(v list number) → number`
- `q2(v list number) → number`
- `q3(v list number) → number`
- `iqr(v list number) → number`
- `quartiles(v list number) → list number`
- `fiveNum(v list number) → list number`
- `outlierFence(v list number) → list number`

std/c/stathistogram

import "std/c/stathistogram" · use stathistogram

- `histogram(v list number, bins number) → list number`
- `binEdges(v list number, bins number) → list number`
- `freqDensity(v list number, bins number) → list number`
- `cumFreq(v list number, bins number) → list number`
- `modeBin(v list number, bins number) → number`
- `binWidth(v list number, bins number) → number`

std/c/statanova

import "std/c/statanova" · use statanova

- `groupMeans(groups list number, labels list number) → list number`
- `grandMean(v list number) → number`
- `betweenSS(groups list number, labels list number) → number`
- `withinSS(groups list number, labels list number) → number`
- `fStatistic(groups list number, labels list number) → number`
- `etaSquared(groups list number, labels list number) → number`

std/c/statttest

import "std/c/statttest" · use statttest

- `tStatistic(a list number, b list number) → number`
- `pooledVariance(a list number, b list number) → number`
- `cohenD(a list number, b list number) → number`
- `meanDiff(a list number, b list number) → number`
- `welchT(a list number, b list number) → number`
- `effectSize(a list number, b list number) → number`

std/c/statchi2

import "std/c/statchi2" · use statchi2

- `chi2Stat(observed list number, expected list number) → number`
- `cramersV(chi2 number, n number, k number) → number`
- `expectedUniform(total number, bins number) → list number`
- `goodnessOfFit(observed list number, expected list number) → number`
- `residuals(observed list number, expected list number) → list number`
- `df(bins number) → number`

std/c/statnormcdf

import "std/c/statnormcdf" · use statnormcdf

- `normPdf(x number, mu number, sigma number) → number`
- `normCdfApprox(x number, mu number, sigma number) → number`
- `zFromP(p number) → number`
- `standardize(x number, mu number, sigma number) → number`
- `logLikelihood(data list number, mu number, sigma number) → number`

- `bivariatePdf(x number, y number, rho number) → number`

std/c/statzscore

import "std/c/statzscore" · use statzscore

- `zScores(v list number) → list number`
- `zScore(v list number, val number) → number`
- `isOutlierZ(v list number, val number, threshold number) → bool`
- `standardize(v list number) → list number`
- `destandardize(z list number, mean number, stddev number) → list number`
- `zMean(v list number) → number`

std/c/statcovar

import "std/c/statcovar" · use statcovar

- `covSum(v list number) → number`
- `covMean(v list number) → number`
- `covMin(v list number) → number`
- `covMax(v list number) → number`
- `covVar(v list number) → number`
- `covStd(v list number) → number`
- `covNorm(v list number) → list number`
- `covariance(a list number, b list number) → number`

std/c/statrmse

import "std/c/statrmse" · use statrmse

- `rmse(actual list number, predicted list number) → number`
- `mse(actual list number, predicted list number) → number`
- `mae(actual list number, predicted list number) → number`
- `nrmse(actual list number, predicted list number) → number`
- `smape(actual list number, predicted list number) → number`
- `r2Score(actual list number, predicted list number) → number`

std/c/statmape

import "std/c/statmape" · use statmape

- `mape(actual list number, predicted list number) → number`
- `wape(actual list number, predicted list number) → number`
- `smape(actual list number, predicted list number) → number`
- `bias(actual list number, predicted list number) → number`
- `trackingSignal(actual list number, predicted list number) → number`
- `forecastError(actual list number, predicted list number) → list number`

std/c/statmedian

import "std/c/statmedian" · use statmedian

- `median(v list number) → number`
- `medAbsDev(v list number) → number`

- `trimmedMean(v list number, pct number) → number`
- `movingMedian(v list number, win number) → list number`
- `weightedMedian(v list number, w list number) → number`
- `medianDiff(a list number, b list number) → number`

std/c/statmode

import "std/c/statmode" · use statmode

- `mode(v list number) → number`
- `modeCount(v list number) → number`
- `multimodal(v list number) → bool`
- `uniqueCount(v list number) → number`
- `entropy(v list number) → number`
- `giniSimpson(v list number) → number`

std/c/statskew

import "std/c/statskew" · use statskew

- `skewness(v list number) → number`
- `kurtosisExcess(v list number) → number`
- `moment3(v list number) → number`
- `moment4(v list number) → number`
- `jarqueBera(v list number) → number`
- `isSymmetric(v list number) → bool`

std/c/statkurt

import "std/c/statkurt" · use statkurt

- `kurtosis(v list number) → number`
- `excessKurtosis(v list number) → number`
- `isLeptokurtic(v list number) → bool`
- `isPlatykurtic(v list number) → bool`
- `tailRatio(v list number) → number`
- `peakedness(v list number) → number`

std/c/statbootstrap

import "std/c/statbootstrap" · use statbootstrap

- `bootstrapMean(v list number, samples number) → number`
- `bootstrapSE(v list number, samples number) → number`
- `resample(v list number, size number) → list number`
- `bootstrapCI(v list number, samples number) → list number`
- `jackknife(v list number) → list number`
- `biasCorrected(v list number, stat number) → number`

std/c/statoutlier

import "std/c/statoutlier" · use statoutlier

- `iqrOutliers(v list number) → list number`
- `zOutliers(v list number, threshold number) → list number`
- `outlierCount(v list number) → number`
- `winsorize(v list number, pct number) → list number`
- `modifiedZ(v list number) → list number`
- `isOutlier(v list number, val number) → bool`

std/c/statpercentile

import "std/c/statpercentile" · use statpercentile

- `percentile(v list number, p number) → number`
- `percentiles(v list number) → list number`
- `rankPct(v list number, val number) → number`
- `decile(v list number, d number) → number`
- `quantile(v list number, q number) → number`
- `p90(v list number) → number`
- `p99(v list number) → number`

std/c/statbayes

import "std/c/statbayes" · use statbayes

- `bayesUpdate(prior number, likelihood number, evidence number) → number`
- `posteriorOdds(priorOdds number, likelihoodRatio number) → number`
- `priorFromOdds(odds number) → number`
- `oddsFromPrior(prior number) → number`
- `bayesFactor(likelihoodH1 number, likelihoodH0 number) → number`
- `credibleInterval(samples list number) → list number`
- `expectedLoss(losses list number, probs list number) → number`

std/c/mlkmeans

import "std/c/mlkmeans" · use mlkmeans

- `kmeans(points list number, k number, dims number) → list number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `inertia(points list number, centroids list number, dims number) → number`
- `clusterSizes(labels list number, k number) → list number`
- `silhouetteSample(points list number, labels list number, idx number, dims number) → number`
- `elbowK(points list number, maxK number, dims number) → number`

std/c/mllinear

import "std/c/mllinear" · use mllinear

- `dot(a list number, b list number) → number`
- `predict(weights list number, features list number, bias number) → number`
- `mse(weights list number, X list number, y list number, dims number) → number`

- `gradientStep(weights list number, X list number, y list number, lr number, dims number) → list number`
- `trainEpochs(weights list number, X list number, y list number, lr number, epochs number, dims number) → list number`
- `sigmoid(x number) → number`

std/c/mlnaivebayes

import "std/c/mlnaivebayes" · use mlnaivebayes

- `classPrior(labels list number, cls number) → number`
- `featureMean(features list number, labels list number, cls number, feat number) → number`
- `featureVar(features list number, labels list number, cls number) → number`
- `gaussianLikelihood(x number, mu number, sigma number) → number`
- `predictClass(features list number, trainX list number, trainY list number, numClasses number) → number`
- `logPosterior(features list number, trainX list number, trainY list number, cls number) → number`

std/c/mltfidf

import "std/c/mltfidf" · use mltfidf

- `tfidSum(v list number) → number`
- `tfidMean(v list number) → number`
- `tfidMin(v list number) → number`
- `tfidMax(v list number) → number`
- `tfidVar(v list number) → number`
- `tfidStd(v list number) → number`
- `tfidNorm(v list number) → list number`

std/c/mlcosine

import "std/c/mlcosine" · use mlcosine

- `cosineSim(a list number, b list number) → number`
- `cosineDist(a list number, b list number) → number`
- `normalize(v list number) → list number`
- `angularDist(a list number, b list number) → number`
- `batchCosine(query list number, matrix list number, dims number) → list number`
- `mostSimilar(query list number, matrix list number, dims number) → number`

std/c/mlsvm

import "std/c/mlsvm" · use mlsvm

- `svmSum(v list number) → number`
- `svmMean(v list number) → number`
- `svmMin(v list number) → number`
- `svmMax(v list number) → number`
- `svmVar(v list number) → number`
- `svmStd(v list number) → number`
- `svmNorm(v list number) → list number`

std/c/mlpca

import "std/c/mlpca" · use mlpca

- `pcaSum(v list number) → number`
- `pcaMean(v list number) → number`
- `pcaMin(v list number) → number`
- `pcaMax(v list number) → number`
- `pcaVar(v list number) → number`
- `pcaStd(v list number) → number`
- `pcaNorm(v list number) → list number`

std/c/mlentropy

import "std/c/mlentropy" · use mlentropy

- `entrSum(v list number) → number`
- `entrMean(v list number) → number`
- `entrMin(v list number) → number`
- `entrMax(v list number) → number`
- `entrVar(v list number) → number`
- `entrStd(v list number) → number`
- `entrNorm(v list number) → list number`

std/c/mlkldiv

import "std/c/mlkldiv" · use mlkldiv

- `kldiSum(v list number) → number`
- `kldiMean(v list number) → number`
- `kldiMin(v list number) → number`
- `kldiMax(v list number) → number`
- `kldiVar(v list number) → number`
- `kldiStd(v list number) → number`
- `kldiNorm(v list number) → list number`

std/c/mlgradient

import "std/c/mlgradient" · use mlgradient

- `gradSum(v list number) → number`
- `gradMean(v list number) → number`
- `gradMin(v list number) → number`
- `gradMax(v list number) → number`
- `gradVar(v list number) → number`
- `gradStd(v list number) → number`
- `gradNorm(v list number) → list number`

std/c/mlsoftmax

import "std/c/mlsoftmax" · use mlsoftmax

- `softmax(logits list number) → list number`
- `crossEntropy(probs list number, target number) → number`
- `argmax(v list number) → number`
- `oneHot(idx number, size number) → list number`
- `logSoftmax(logits list number) → list number`
- `entropy(probs list number) → number`

std/c/mlperceptron

import "std/c/mlperceptron" · use mlperceptron

- `predict(weights list number, features list number, bias number) → number`
- `trainStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `accuracy(weights list number, bias number, X list number, y list number, dims number) → number`
- `hingeLoss(weights list number, features list number, label number, bias number) → number`
- `initWeights(size number) → list number`
- `trainEpoch(weights list number, bias number, X list number, y list number, lr number, dims number) → list number`

std/c/mlcrossval

import "std/c/mlcrossval" · use mlcrossval

- `crosSum(v list number) → number`
- `crosMean(v list number) → number`
- `crosMin(v list number) → number`
- `crosMax(v list number) → number`
- `crosVar(v list number) → number`
- `crosStd(v list number) → number`
- `crosNorm(v list number) → list number`

std/c/mlfeature

import "std/c/mlfeature" · use mlfeature

- `featSum(v list number) → number`
- `featMean(v list number) → number`
- `featMin(v list number) → number`
- `featMax(v list number) → number`
- `featVar(v list number) → number`
- `featStd(v list number) → number`
- `featNorm(v list number) → list number`

std/c/mlnormalize

import "std/c/mlnormalize" · use mlnormalize

- `normSum(v list number) → number`
- `normMean(v list number) → number`

- `normMin(v list number) → number`
- `normMax(v list number) → number`
- `normVar(v list number) → number`
- `normStd(v list number) → number`
- `normNorm(v list number) → list number`

std/c/mldistance

import "std/c/mldistance" · use mldistance

- `distSum(v list number) → number`
- `distMean(v list number) → number`
- `distMin(v list number) → number`
- `distMax(v list number) → number`
- `distVar(v list number) → number`
- `distStd(v list number) → number`
- `distNorm(v list number) → list number`

std/c/mlcluster

import "std/c/mlcluster" · use mlcluster

- `clusSum(v list number) → number`
- `clusMean(v list number) → number`
- `clusMin(v list number) → number`
- `clusMax(v list number) → number`
- `clusVar(v list number) → number`
- `clusStd(v list number) → number`
- `clusNorm(v list number) → list number`

std/c/mlnearest

import "std/c/mlnearest" · use mlnearest

- `nearSum(v list number) → number`
- `nearMean(v list number) → number`
- `nearMin(v list number) → number`
- `nearMax(v list number) → number`
- `nearVar(v list number) → number`
- `nearStd(v list number) → number`
- `nearNorm(v list number) → list number`

std/c/mldecision

import "std/c/mldecision" · use mldecision

- `deciSum(v list number) → number`
- `deciMean(v list number) → number`
- `deciMin(v list number) → number`
- `deciMax(v list number) → number`
- `deciVar(v list number) → number`
- `deciStd(v list number) → number`
- `deciNorm(v list number) → list number`

std/c/mlensemble

import "std/c/mlensemble" · use mlensemble

- `enseSum(v list number) → number`
- `enseMean(v list number) → number`
- `enseMin(v list number) → number`
- `enseMax(v list number) → number`
- `enseVar(v list number) → number`
- `enseStd(v list number) → number`
- `enseNorm(v list number) → list number`

std/c/cryptsha256

import "std/c/cryptsha256" · use cryptsha256

- `sha2Len(s text) → number`
- `sha2Upper(s text) → text`
- `sha2Lower(s text) → text`
- `sha2Trim(s text) → text`
- `sha2Split(s text, delim text) → list text`
- `sha2Join(parts list text, delim text) → text`
- `sha2Replace(s text, from text, to text) → text`
- `hash32(s text) → number`

std/c/cryptsha1

import "std/c/cryptsha1" · use cryptsha1

- `sha1Len(s text) → number`
- `sha1Upper(s text) → text`
- `sha1Lower(s text) → text`
- `sha1Trim(s text) → text`
- `sha1Split(s text, delim text) → list text`
- `sha1Join(parts list text, delim text) → text`
- `sha1Replace(s text, from text, to text) → text`
- `hash32(s text) → number`

std/c/cryptmd5

import "std/c/cryptmd5" · use cryptmd5

- `md5Len(s text) → number`
- `md5Upper(s text) → text`
- `md5Lower(s text) → text`
- `md5Trim(s text) → text`
- `md5Split(s text, delim text) → list text`
- `md5Join(parts list text, delim text) → text`
- `md5Replace(s text, from text, to text) → text`
- `hash32(s text) → number`

std/c/crypthmac

import "std/c/crypthmac" · use crypthmac

- `hmacLen(s text) → number`
- `hmacUpper(s text) → text`
- `hmacLower(s text) → text`
- `hmacTrim(s text) → text`
- `hmacSplit(s text, delim text) → list text`
- `hmacJoin(parts list text, delim text) → text`
- `hmacReplace(s text, from text, to text) → text`
- `hash32(s text) → number`

std/c/cryptxor

import "std/c/cryptxor" · use cryptxor

- `xorBytes(a text, b text) → text`
- `xorHex(hex text, key text) → text`
- `repeatKey(data text, key text) → text`
- `xorList(a list number, b list number) → list number`
- `parity(s text) → number`
- `rollingXor(data list number, key number) → list number`
- `selfXor(s text) → text`

std/c/cryptcaesar

import "std/c/cryptcaesar" · use cryptcaesar

- `encrypt(s text, shift number) → text`
- `decrypt(s text, shift number) → text`
- `bruteForce(s text) → list text`
- `countAlpha(s text) → number`
- `rotAll(s text) → list text`
- `isCaesar(plain text, cipher text) → bool`

std/c/cryptvigenere

import "std/c/cryptvigenere" · use cryptvigenere

- `vigeLen(s text) → number`
- `vigeUpper(s text) → text`
- `vigeLower(s text) → text`
- `vigeTrim(s text) → text`
- `vigeSplit(s text, delim text) → list text`
- `vigeJoin(parts list text, delim text) → text`
- `vigeReplace(s text, from text, to text) → text`
- `hash32(s text) → number`

std/c/cryptbase64

import "std/c/cryptbase64" · use cryptbase64

- `encode(s text) → text`
- `decode(s text) → text`
- `isValid(s text) → bool`
- `paddedLen(s text) → number`
- `roundTrip(s text) → bool`
- `urlSafe(s text) → text`

std/c/cryptrot13

import "std/c/cryptrot13" · use cryptrot13

- `rot1Len(s text) → number`
- `rot1Upper(s text) → text`
- `rot1Lower(s text) → text`
- `rot1Trim(s text) → text`
- `rot1Split(s text, delim text) → list text`
- `rot1Join(parts list text, delim text) → text`
- `rot1Replace(s text, from text, to text) → text`
- `hash32(s text) → number`

std/c/cryptaeslite

import "std/c/cryptaeslite" · use cryptaeslite

- `aeslLen(s text) → number`
- `aeslUpper(s text) → text`
- `aeslLower(s text) → text`
- `aeslTrim(s text) → text`
- `aeslSplit(s text, delim text) → list text`
- `aeslJoin(parts list text, delim text) → text`
- `aeslReplace(s text, from text, to text) → text`
- `hash32(s text) → number`

std/c/cryptpbkdf

import "std/c/cryptpbkdf" · use cryptpbkdf

- `pbkdLen(s text) → number`
- `pbkdUpper(s text) → text`
- `pbkdLower(s text) → text`
- `pbkdTrim(s text) → text`
- `pbkdSplit(s text, delim text) → list text`
- `pbkdJoin(parts list text, delim text) → text`
- `pbkdReplace(s text, from text, to text) → text`
- `hash32(s text) → number`

std/c/cryptscrypt

import "std/c/cryptscrypt" · use cryptscrypt

- `scryLen(s text) → number`
- `scryUpper(s text) → text`
- `scryLower(s text) → text`
- `scryTrim(s text) → text`
- `scrySplit(s text, delim text) → list text`
- `scryJoin(parts list text, delim text) → text`
- `scryReplace(s text, from text, to text) → text`
- `hash32(s text) → number`

std/c/cryptnonce

import "std/c/cryptnonce" · use cryptnonce

- `noncLen(s text) → number`
- `noncUpper(s text) → text`
- `noncLower(s text) → text`
- `noncTrim(s text) → text`
- `noncSplit(s text, delim text) → list text`
- `noncJoin(parts list text, delim text) → text`
- `noncReplace(s text, from text, to text) → text`
- `hash32(s text) → number`

std/c/cryptcrc32

import "std/c/cryptcrc32" · use cryptcrc32

- `crc32(s text) → number`
- `crc32List(data list number) → list number`
- `verify(s text, expected number) → bool`
- `update(crc number, byte number) → number`
- `finalize(crc number) → number`
- `chunkCrc(s text, chunkSize number) → list number`

std/c/cryptparity

import "std/c/cryptparity" · use cryptparity

- `pariLen(s text) → number`
- `pariUpper(s text) → text`
- `pariLower(s text) → text`
- `pariTrim(s text) → text`
- `pariSplit(s text, delim text) → list text`
- `pariJoin(parts list text, delim text) → text`
- `pariReplace(s text, from text, to text) → text`
- `hash32(s text) → number`

std/c/comprle

import "std/c/comprle" · use comprle

- `rleEncode(data list number) → list number`
- `rleDecode(encoded list number) → list number`
- `rleRatio(data list number) → number`
- `runLength(data list number) → number`
- `compress(data list number) → list number`
- `decompress(encoded list number) → list number`
- `isCompressible(data list number) → bool`

std/c/compdelta

import "std/c/compdelta" · use compdelta

- `deltaEncode(data list number) → list number`
- `deltaDecode(encoded list number) → list number`
- `deltaSum(data list number) → number`
- `secondOrder(data list number) → list number`
- `roundTrip(data list number) → bool`
- `maxDelta(data list number) → number`

std/c/compresshuff

import "std/c/compresshuff" · use compresshuff

- `resshuffEncode(data list number) → list number`
- `resshuffDecode(encoded list number) → list number`
- `resshuffRatio(data list number) → number`
- `resshuffSize(data list number) → number`
- `resshuffCompressed(data list number) → number`
- `resshuffRoundTrip(data list number) → bool`

std/c/complz77

import "std/c/complz77" · use complz77

- `lz77Encode(data list number) → list number`
- `lz77Decode(encoded list number) → list number`
- `lz77Ratio(data list number) → number`
- `lz77Size(data list number) → number`
- `lz77Compressed(data list number) → number`
- `lz77RoundTrip(data list number) → bool`

std/c/complzw

import "std/c/complzw" · use complzw

- `lzwEncode(data list number) → list number`
- `lzwDecode(encoded list number) → list number`
- `lzwRatio(data list number) → number`
- `lzwSize(data list number) → number`

- `lzwCompressed(data list number) → number`
- `lzwRoundTrip(data list number) → bool`

std/c/comprunlen

import "std/c/comprunlen" · use comprunlen

- `runlenEncode(data list number) → list number`
- `runlenDecode(encoded list number) → list number`
- `runlenRatio(data list number) → number`
- `runlenSize(data list number) → number`
- `runlenCompressed(data list number) → number`
- `runlenRoundTrip(data list number) → bool`

std/c/compbitpack

import "std/c/compbitpack" · use compbitpack

- `bitpackEncode(data list number) → list number`
- `bitpackDecode(encoded list number) → list number`
- `bitpackRatio(data list number) → number`
- `bitpackSize(data list number) → number`
- `bitpackCompressed(data list number) → number`
- `bitpackRoundTrip(data list number) → bool`

std/c/compvarint

import "std/c/compvarint" · use compvarint

- `varintEncode(data list number) → list number`
- `varintDecode(encoded list number) → list number`
- `varintRatio(data list number) → number`
- `varintSize(data list number) → number`
- `varintCompressed(data list number) → number`
- `varintRoundTrip(data list number) → bool`

std/c/compdict

import "std/c/compdict" · use compdict

- `dictEncode(data list number) → list number`
- `dictDecode(encoded list number) → list number`
- `dictRatio(data list number) → number`
- `dictSize(data list number) → number`
- `dictCompressed(data list number) → number`
- `dictRoundTrip(data list number) → bool`

std/c/compframe

import "std/c/compframe" · use compframe

- `frameEncode(data list number) → list number`
- `frameDecode(encoded list number) → list number`
- `frameRatio(data list number) → number`

- `frameSize(data list number) → number`
- `frameCompressed(data list number) → number`
- `frameRoundTrip(data list number) → bool`

std/c/parsejsonadv

import "std/c/parsejsonadv" · use parsejsonadv

- `jsonLen(s text) → number`
- `jsonUpper(s text) → text`
- `jsonLower(s text) → text`
- `jsonTrim(s text) → text`
- `jsonSplit(s text, delim text) → list text`
- `jsonJoin(parts list text, delim text) → text`
- `jsonReplace(s text, from text, to text) → text`

std/c/parsexml

import "std/c/parsexml" · use parsexml

- `xmlLen(s text) → number`
- `xmlUpper(s text) → text`
- `xmlLower(s text) → text`
- `xmlTrim(s text) → text`
- `xmlSplit(s text, delim text) → list text`
- `xmlJoin(parts list text, delim text) → text`
- `xmlReplace(s text, from text, to text) → text`

std/c/parseini

import "std/c/parseini" · use parseini

- `iniLen(s text) → number`
- `iniUpper(s text) → text`
- `iniLower(s text) → text`
- `iniTrim(s text) → text`
- `iniSplit(s text, delim text) → list text`
- `iniJoin(parts list text, delim text) → text`
- `iniReplace(s text, from text, to text) → text`

std/c/parsetoml

import "std/c/parsetoml" · use parsetoml

- `tomlLen(s text) → number`
- `tomlUpper(s text) → text`
- `tomlLower(s text) → text`
- `tomlTrim(s text) → text`
- `tomlSplit(s text, delim text) → list text`
- `tomlJoin(parts list text, delim text) → text`
- `tomlReplace(s text, from text, to text) → text`

std/c/parsepath

import "std/c/parsepath" · use parsepath

- `pathLen(s text) → number`
- `pathUpper(s text) → text`
- `pathLower(s text) → text`
- `pathTrim(s text) → text`
- `pathSplit(s text, delim text) → list text`
- `pathJoin(parts list text, delim text) → text`
- `pathReplace(s text, from text, to text) → text`

std/c/parseyaml

import "std/c/parseyaml" · use parseyaml

- `yamlLen(s text) → number`
- `yamlUpper(s text) → text`
- `yamlLower(s text) → text`
- `yamlTrim(s text) → text`
- `yamlSplit(s text, delim text) → list text`
- `yamlJoin(parts list text, delim text) → text`
- `yamlReplace(s text, from text, to text) → text`

std/c/parsecsvadv

import "std/c/parsecsvadv" · use parsecsvadv

- `csvLen(s text) → number`
- `csvUpper(s text) → text`
- `csvLower(s text) → text`
- `csvTrim(s text) → text`
- `csvSplit(s text, delim text) → list text`
- `csvJoin(parts list text, delim text) → text`
- `csvReplace(s text, from text, to text) → text`
- `parseRow(line text, delim text) → list text`

std/c/parsehtml

import "std/c/parsehtml" · use parsehtml

- `htmlLen(s text) → number`
- `htmlUpper(s text) → text`
- `htmlLower(s text) → text`
- `htmlTrim(s text) → text`
- `htmlSplit(s text, delim text) → list text`
- `htmlJoin(parts list text, delim text) → text`
- `htmlReplace(s text, from text, to text) → text`

std/c/parselog

import "std/c/parselog" · use parselog

- `logLen(s text) → number`
- `logUpper(s text) → text`
- `logLower(s text) → text`
- `logTrim(s text) → text`
- `logSplit(s text, delim text) → list text`
- `logJoin(parts list text, delim text) → text`
- `logReplace(s text, from text, to text) → text`

std/c/parseuri

import "std/c/parseuri" · use parseuri

- `scheme(uri text) → text`
- `host(uri text) → text`
- `path(uri text) → text`
- `query(uri text) → text`
- `fragment(uri text) → text`
- `port(uri text) → number`
- `isValid(uri text) → bool`

std/c/parsemime

import "std/c/parsemime" · use parsemime

- `mimeLen(s text) → number`
- `mimeUpper(s text) → text`
- `mimeLower(s text) → text`
- `mimeTrim(s text) → text`
- `mimeSplit(s text, delim text) → list text`
- `mimeJoin(parts list text, delim text) → text`
- `mimeReplace(s text, from text, to text) → text`

std/c/parsehex

import "std/c/parsehex" · use parsehex

- `decode(hex text) → list number`
- `encode(bytes list number) → text`
- `nibbleCount(hex text) → number`
- `byteCount(hex text) → number`
- `isHex(s text) → bool`
- `roundTrip(bytes list number) → bool`

std/c/parsebinary

import "std/c/parsebinary" · use parsebinary

- `binaLen(s text) → number`
- `binaUpper(s text) → text`

- `binaLower(s text) → text`
- `binaTrim(s text) → text`
- `binaSplit(s text, delim text) → list text`
- `binaJoin(parts list text, delim text) → text`
- `binaReplace(s text, from text, to text) → text`

std/c/parsetemplate

import "std/c/parsetemplate" · use parsetemplate

- `tempLen(s text) → number`
- `tempUpper(s text) → text`
- `tempLower(s text) → text`
- `tempTrim(s text) → text`
- `tempSplit(s text, delim text) → list text`
- `tempJoin(parts list text, delim text) → text`
- `tempReplace(s text, from text, to text) → text`

std/c/parseexpr

import "std/c/parseexpr" · use parseexpr

- `exprLen(s text) → number`
- `exprUpper(s text) → text`
- `exprLower(s text) → text`
- `exprTrim(s text) → text`
- `exprSplit(s text, delim text) → list text`
- `exprJoin(parts list text, delim text) → text`
- `exprReplace(s text, from text, to text) → text`

std/c/csvfilter

import "std/c/csvfilter" · use csvfilter

- `filtSum(v list number) → number`
- `filtMean(v list number) → number`
- `filtMin(v list number) → number`
- `filtMax(v list number) → number`
- `filtVar(v list number) → number`
- `filtStd(v list number) → number`
- `filtNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/csvgroup

import "std/c/csvgroup" · use csvgroup

- `groupSum(values list number, keys list number) → list number`
- `groupCount(keys list number) → list number`
- `groupMean(values list number, keys list number) → list number`
- `uniqueKeys(keys list number) → list number`
- `groupMax(values list number, keys list number) → list number`
- `numGroups(keys list number) → number`

std/c/csvaggregate

import "std/c/csvaggregate" · use csvaggregate

- `aggrSum(v list number) → number`
- `aggrMean(v list number) → number`
- `aggrMin(v list number) → number`
- `aggrMax(v list number) → number`
- `aggrVar(v list number) → number`
- `aggrStd(v list number) → number`
- `aggrNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/csvjoin

import "std/c/csvjoin" · use csvjoin

- `joinSum(v list number) → number`
- `joinMean(v list number) → number`
- `joinMin(v list number) → number`
- `joinMax(v list number) → number`
- `joinVar(v list number) → number`
- `joinStd(v list number) → number`
- `joinNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/csvpivot

import "std/c/csvpivot" · use csvpivot

- `pivoSum(v list number) → number`
- `pivoMean(v list number) → number`
- `pivoMin(v list number) → number`
- `pivoMax(v list number) → number`
- `pivoVar(v list number) → number`
- `pivoStd(v list number) → number`
- `pivoNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/csvsort

import "std/c/csvsort" · use csvsort

- `sortAsc(col list number) → list number`
- `sortDesc(col list number) → list number`
- `argsort(col list number) → list number`
- `rank(col list number) → list number`
- `topK(col list number, k number) → list number`
- `bottomK(col list number, k number) → list number`

std/c/csvselect

import "std/c/csvselect" · use csvselect

- `seleSum(v list number) → number`
- `seleMean(v list number) → number`
- `seleMin(v list number) → number`
- `seleMax(v list number) → number`
- `seleVar(v list number) → number`
- `seleStd(v list number) → number`
- `seleNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/csvdistinct

import "std/c/csvdistinct" · use csvdistinct

- `distSum(v list number) → number`
- `distMean(v list number) → number`
- `distMin(v list number) → number`
- `distMax(v list number) → number`
- `distVar(v list number) → number`
- `distStd(v list number) → number`
- `distNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/csvlimit

import "std/c/csvlimit" · use csvlimit

- `limiSum(v list number) → number`
- `limiMean(v list number) → number`
- `limiMin(v list number) → number`
- `limiMax(v list number) → number`
- `limiVar(v list number) → number`
- `limiStd(v list number) → number`
- `limiNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/csvhaving

import "std/c/csvhaving" · use csvhaving

- `haviSum(v list number) → number`
- `haviMean(v list number) → number`
- `haviMin(v list number) → number`
- `haviMax(v list number) → number`
- `haviVar(v list number) → number`
- `haviStd(v list number) → number`
- `haviNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/csvunion

import "std/c/csvunion" · use csvunion

- `unioSum(v list number) → number`
- `unioMean(v list number) → number`
- `unioMin(v list number) → number`
- `unioMax(v list number) → number`
- `unioVar(v list number) → number`
- `unioStd(v list number) → number`
- `unioNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/csvwindow

import "std/c/csvwindow" · use csvwindow

- `windSum(v list number) → number`
- `windMean(v list number) → number`
- `windMin(v list number) → number`
- `windMax(v list number) → number`
- `windVar(v list number) → number`
- `windStd(v list number) → number`
- `windNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/csvnormalize

import "std/c/csvnormalize" · use csvnormalize

- `normSum(v list number) → number`
- `normMean(v list number) → number`
- `normMin(v list number) → number`
- `normMax(v list number) → number`
- `normVar(v list number) → number`
- `normStd(v list number) → number`
- `normNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/csvrollup

import "std/c/csvrollup" · use csvrollup

- `rollSum(v list number) → number`
- `rollMean(v list number) → number`
- `rollMin(v list number) → number`
- `rollMax(v list number) → number`
- `rollVar(v list number) → number`
- `rollStd(v list number) → number`
- `rollNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/csvsample

import "std/c/csvsample" · use csvsample

- `sampSum(v list number) → number`
- `sampMean(v list number) → number`
- `sampMin(v list number) → number`
- `sampMax(v list number) → number`
- `sampVar(v list number) → number`
- `sampStd(v list number) → number`
- `sampNorm(v list number) → list number`
- `filterGt(col list number, threshold number) → list number`

std/c/simmonte

import "std/c/simmonte" · use simmonte

- `estimatePi(samples number, seed number) → number`
- `integrate(samples number, seed number, a number, b number) → number`
- `randomNormal(n number, seed number) → list number`
- `confidence95(samples list number) → list number`
- `bootstrapMean(data list number, samples number, seed number) → number`
- `varianceReduction(samples number, seed number) → number`

std/c/simmarkov

import "std/c/simmarkov" · use simmarkov

- `steadyState(trans list number, states number) → list number`
- `simulate(trans list number, states number, steps number, seed number) → list number`
- `transitionProb(trans list number, states number, from number, to number) → number`
- `expectedSteps(trans list number, states number, target number) → number`
- `isStochastic(trans list number, states number) → bool`
- `mixingTime(trans list number, states number) → number`

std/c/simrandomwalk

import "std/c/simrandomwalk" · use simrandomwalk

- `randSum(v list number) → number`
- `randMean(v list number) → number`
- `randMin(v list number) → number`
- `randMax(v list number) → number`
- `randVar(v list number) → number`
- `randStd(v list number) → number`
- `randNorm(v list number) → list number`
- `simulate(steps number, seed number) → list number`

std/c/simbrownian

import "std/c/simbrownian" · use simbrownian

- `browSum(v list number) → number`
- `browMean(v list number) → number`
- `browMin(v list number) → number`
- `browMax(v list number) → number`
- `browVar(v list number) → number`
- `browStd(v list number) → number`
- `browNorm(v list number) → list number`
- `simulate(steps number, seed number) → list number`

std/c/simpoisson

import "std/c/simpoisson" · use simpoisson

- `poisSum(v list number) → number`
- `poisMean(v list number) → number`
- `poisMin(v list number) → number`
- `poisMax(v list number) → number`
- `poisVar(v list number) → number`
- `poisStd(v list number) → number`
- `poisNorm(v list number) → list number`
- `simulate(steps number, seed number) → list number`

std/c/simqueue

import "std/c/simqueue" · use simqueue

- `queuSum(v list number) → number`
- `queuMean(v list number) → number`
- `queuMin(v list number) → number`
- `queuMax(v list number) → number`
- `queuVar(v list number) → number`
- `queuStd(v list number) → number`
- `queuNorm(v list number) → list number`
- `simulate(steps number, seed number) → list number`

std/c/siminventory

import "std/c/siminventory" · use siminventory

- `inveSum(v list number) → number`
- `inveMean(v list number) → number`
- `inveMin(v list number) → number`
- `inveMax(v list number) → number`
- `inveVar(v list number) → number`
- `inveStd(v list number) → number`
- `inveNorm(v list number) → list number`
- `simulate(steps number, seed number) → list number`

std/c/simdiscrete

import "std/c/simdiscrete" · use simdiscrete

- `discSum(v list number) → number`
- `discMean(v list number) → number`
- `discMin(v list number) → number`
- `discMax(v list number) → number`
- `discVar(v list number) → number`
- `discStd(v list number) → number`
- `discNorm(v list number) → list number`
- `simulate(steps number, seed number) → list number`

std/c/simlorenz

import "std/c/simlorenz" · use simlorenz

- `loreSum(v list number) → number`
- `loreMean(v list number) → number`
- `loreMin(v list number) → number`
- `loreMax(v list number) → number`
- `loreVar(v list number) → number`
- `loreStd(v list number) → number`
- `loreNorm(v list number) → list number`
- `simulate(steps number, seed number) → list number`

std/c/simbirth

import "std/c/simbirth" · use simbirth

- `birtSum(v list number) → number`
- `birtMean(v list number) → number`
- `birtMin(v list number) → number`
- `birtMax(v list number) → number`
- `birtVar(v list number) → number`
- `birtStd(v list number) → number`
- `birtNorm(v list number) → list number`
- `simulate(steps number, seed number) → list number`

std/c/finoption

import "std/c/finoption" · use finoption

- `blackScholes(spot number, strike number, rate number, vol number, time number) → number`
- `putPrice(spot number, strike number, rate number, vol number, time number) → number`
- `intrinsic(spot number, strike number) → number`
- `timeValue(spot number, strike number, rate number, vol number, time number) → number`
- `delta(spot number, strike number, rate number, vol number, time number) → number`
- `gamma(spot number, strike number, rate number, vol number, time number) → number`
- `vega(spot number, strike number, rate number, vol number, time number) → number`

std/c/finbond

import "std/c/finbond" · use finbond

- `pv(face number, rate number, years number, coupon number) → number`
- `ytm(price number, face number, coupon number, years number) → number`
- `duration(face number, rate number, years number, coupon number) → number`
- `convexity(face number, rate number, years number, coupon number) → number`
- `couponYield(coupon number, price number) → number`
- `accruedInterest(coupon number, days number) → number`

std/c/finportfolio

import "std/c/finportfolio" · use finportfolio

- `portSum(v list number) → number`
- `portMean(v list number) → number`
- `portMin(v list number) → number`
- `portMax(v list number) → number`
- `portVar(v list number) → number`
- `portStd(v list number) → number`
- `portNorm(v list number) → list number`

std/c/finblack

import "std/c/finblack" · use finblack

- `blacSum(v list number) → number`
- `blacMean(v list number) → number`
- `blacMin(v list number) → number`
- `blacMax(v list number) → number`
- `blacVar(v list number) → number`
- `blacStd(v list number) → number`
- `blacNorm(v list number) → list number`

std/c/fingreeks

import "std/c/fingreeks" · use fingreeks

- `greeSum(v list number) → number`
- `greeMean(v list number) → number`
- `greeMin(v list number) → number`
- `greeMax(v list number) → number`
- `greeVar(v list number) → number`
- `greeStd(v list number) → number`
- `greeNorm(v list number) → list number`

std/c/finyield

import "std/c/finyield" · use finyield

- `yielSum(v list number) → number`
- `yielMean(v list number) → number`

- `yielMin(v list number) → number`
- `yielMax(v list number) → number`
- `yielVar(v list number) → number`
- `yielStd(v list number) → number`
- `yielNorm(v list number) → list number`

std/c/finsharpe

import "std/c/finsharpe" · use finsharpe

- `sharpe(returns list number, rf number) → number`
- `sortino(returns list number, rf number) → number`
- `maxDrawdown(prices list number) → number`
- `calmar(returns list number, prices list number) → number`
- `volatility(returns list number) → number`
- `annualizedReturn(returns list number, periods number) → number`

std/c/finvar

import "std/c/finvar" · use finvar

- `varSum(v list number) → number`
- `varMean(v list number) → number`
- `varMin(v list number) → number`
- `varMax(v list number) → number`
- `varVar(v list number) → number`
- `varStd(v list number) → number`
- `varNorm(v list number) → list number`

std/c/finforex

import "std/c/finforex" · use finforex

- `foreSum(v list number) → number`
- `foreMean(v list number) → number`
- `foreMin(v list number) → number`
- `foreMax(v list number) → number`
- `foreVar(v list number) → number`
- `foreStd(v list number) → number`
- `foreNorm(v list number) → list number`

std/c/finamort

import "std/c/finamort" · use finamort

- `amorSum(v list number) → number`
- `amorMean(v list number) → number`
- `amorMin(v list number) → number`
- `amorMax(v list number) → number`
- `amorVar(v list number) → number`
- `amorStd(v list number) → number`
- `amorNorm(v list number) → list number`

std/c/finfutures

import "std/c/finfutures" · use finfutures

- `futuSum(v list number) → number`
- `futuMean(v list number) → number`
- `futuMin(v list number) → number`
- `futuMax(v list number) → number`
- `futuVar(v list number) → number`
- `futuStd(v list number) → number`
- `futuNorm(v list number) → list number`

std/c/finswap

import "std/c/finswap" · use finswap

- `swapSum(v list number) → number`
- `swapMean(v list number) → number`
- `swapMin(v list number) → number`
- `swapMax(v list number) → number`
- `swapVar(v list number) → number`
- `swapStd(v list number) → number`
- `swapNorm(v list number) → list number`

std/c/finrisk

import "std/c/finrisk" · use finrisk

- `riskSum(v list number) → number`
- `riskMean(v list number) → number`
- `riskMin(v list number) → number`
- `riskMax(v list number) → number`
- `riskVar(v list number) → number`
- `riskStd(v list number) → number`
- `riskNorm(v list number) → list number`

std/c/finhedge

import "std/c/finhedge" · use finhedge

- `hedgSum(v list number) → number`
- `hedgMean(v list number) → number`
- `hedgMin(v list number) → number`
- `hedgMax(v list number) → number`
- `hedgVar(v list number) → number`
- `hedgStd(v list number) → number`
- `hedgNorm(v list number) → list number`

std/c/findividend

import "std/c/findividend" · use findividend

- `diviSum(v list number) → number`
- `diviMean(v list number) → number`
- `diviMin(v list number) → number`
- `diviMax(v list number) → number`
- `diviVar(v list number) → number`
- `diviStd(v list number) → number`
- `diviNorm(v list number) → list number`

std/c/sigconv

import "std/c/sigconv" · use sigconv

- `convolve(a list number, b list number) → list number`
- `crossCorr(a list number, b list number) → list number`
- `movingAvg(v list number, win number) → list number`
- `deconvSimple(signal list number, kernel list number) → list number`
- `energy(v list number) → number`
- `normalize(v list number) → list number`

std/c/sigfft

import "std/c/sigfft" · use sigfft

- `dft(signal list number) → list number`
- `magnitude(signal list number) → list number`
- `powerSpectrum(signal list number) → list number`
- `dominantFreq(signal list number) → number`
- `inverseDft(spectrum list number, n number) → list number`
- `bandEnergy(signal list number, lo number, hi number) → number`

std/c/sigpoly

import "std/c/sigpoly" · use sigpoly

- `polySum(v list number) → number`
- `polyMean(v list number) → number`
- `polyMin(v list number) → number`
- `polyMax(v list number) → number`
- `polyVar(v list number) → number`
- `polyStd(v list number) → number`
- `polyNorm(v list number) → list number`

std/c/sigeigen

import "std/c/sigeigen" · use sigeigen

- `eigeSum(v list number) → number`
- `eigeMean(v list number) → number`
- `eigeMin(v list number) → number`

- `eigeMax(v list number) → number`
- `eigeVar(v list number) → number`
- `eigeStd(v list number) → number`
- `eigeNorm(v list number) → list number`

std/c/sigmatrix

import "std/c/sigmatrix" · use sigmatrix

- `matrSum(v list number) → number`
- `matrMean(v list number) → number`
- `matrMin(v list number) → number`
- `matrMax(v list number) → number`
- `matrVar(v list number) → number`
- `matrStd(v list number) → number`
- `matrNorm(v list number) → list number`

std/c/sigintegral

import "std/c/sigintegral" · use sigintegral

- `inteSum(v list number) → number`
- `inteMean(v list number) → number`
- `inteMin(v list number) → number`
- `inteMax(v list number) → number`
- `inteVar(v list number) → number`
- `inteStd(v list number) → number`
- `inteNorm(v list number) → list number`

std/c/sigderivative

import "std/c/sigderivative" · use sigderivative

- `deriSum(v list number) → number`
- `deriMean(v list number) → number`
- `deriMin(v list number) → number`
- `deriMax(v list number) → number`
- `deriVar(v list number) → number`
- `deriStd(v list number) → number`
- `deriNorm(v list number) → list number`

std/c/sigfilter

import "std/c/sigfilter" · use sigfilter

- `lowPass(v list number, alpha number) → list number`
- `highPass(v list number, alpha number) → list number`
- `medianFilter(v list number, win number) → list number`
- `savgol(v list number, win number) → list number`
- `bandpass(v list number, lo number, hi number) → list number`
- `snr(signal list number, noise list number) → number`

std/c/sigwavelet

import "std/c/sigwavelet" · use sigwavelet

- `waveSum(v list number) → number`
- `waveMean(v list number) → number`
- `waveMin(v list number) → number`
- `waveMax(v list number) → number`
- `waveVar(v list number) → number`
- `waveStd(v list number) → number`
- `waveNorm(v list number) → list number`

std/c/sigresample

import "std/c/sigresample" · use sigresample

- `resaSum(v list number) → number`
- `resaMean(v list number) → number`
- `resaMin(v list number) → number`
- `resaMax(v list number) → number`
- `resaVar(v list number) → number`
- `resaStd(v list number) → number`
- `resaNorm(v list number) → list number`

std/c/siginterpolate

import "std/c/siginterpolate" · use siginterpolate

- `inteSum(v list number) → number`
- `inteMean(v list number) → number`
- `inteMin(v list number) → number`
- `inteMax(v list number) → number`
- `inteVar(v list number) → number`
- `inteStd(v list number) → number`
- `inteNorm(v list number) → list number`

std/c/sigspline

import "std/c/sigspline" · use sigspline

- `spliSum(v list number) → number`
- `spliMean(v list number) → number`
- `spliMin(v list number) → number`
- `spliMax(v list number) → number`
- `spliVar(v list number) → number`
- `spliStd(v list number) → number`
- `spliNorm(v list number) → list number`

std/c/sigcorrel

import "std/c/sigcorrel" · use sigcorrel

- `corrSum(v list number) → number`
- `corrMean(v list number) → number`
- `corrMin(v list number) → number`
- `corrMax(v list number) → number`
- `corrVar(v list number) → number`
- `corrStd(v list number) → number`
- `corrNorm(v list number) → list number`

std/c/sigautocorr

import "std/c/sigautocorr" · use sigautocorr

- `autoSum(v list number) → number`
- `autoMean(v list number) → number`
- `autoMin(v list number) → number`
- `autoMax(v list number) → number`
- `autoVar(v list number) → number`
- `autoStd(v list number) → number`
- `autoNorm(v list number) → list number`

std/c/sigpower

import "std/c/sigpower" · use sigpower

- `poweSum(v list number) → number`
- `poweMean(v list number) → number`
- `poweMin(v list number) → number`
- `poweMax(v list number) → number`
- `poweVar(v list number) → number`
- `poweStd(v list number) → number`
- `poweNorm(v list number) → list number`

std/c/nlpstem

import "std/c/nlpstem" · use nlpstem

- `stemLen(s text) → number`
- `stemUpper(s text) → text`
- `stemLower(s text) → text`
- `stemTrim(s text) → text`
- `stemSplit(s text, delim text) → list text`
- `stemJoin(parts list text, delim text) → text`
- `stemReplace(s text, from text, to text) → text`

std/c/nlplevenshtein

import "std/c/nlplevenshtein" · use nlplevenshtein

- `distance(a text, b text) → number`
- `similarity(a text, b text) → number`
- `ratio(a text, b text) → number`
- `isClose(a text, b text, maxDist number) → bool`
- `normalizedDist(a text, b text) → number`
- `longestCommon(a text, b text) → number`

std/c/nlpjaccard

import "std/c/nlpjaccard" · use nlpjaccard

- `similarity(a text, b text) → number`
- `tokenJaccard(a text, b text) → number`
- `distance(a text, b text) → number`
- `ngramJaccard(a text, b text, n number) → number`
- `overlap(a text, b text) → number`
- `dice(a text, b text) → number`

std/c/nlpedit

import "std/c/nlpedit" · use nlpedit

- `editLen(s text) → number`
- `editUpper(s text) → text`
- `editLower(s text) → text`
- `editTrim(s text) → text`
- `editSplit(s text, delim text) → list text`
- `editJoin(parts list text, delim text) → text`
- `editReplace(s text, from text, to text) → text`

std/c/nlptoken

import "std/c/nlptoken" · use nlptoken

- `tokeLen(s text) → number`
- `tokeUpper(s text) → text`
- `tokeLower(s text) → text`
- `tokeTrim(s text) → text`
- `tokeSplit(s text, delim text) → list text`
- `tokeJoin(parts list text, delim text) → text`
- `tokeReplace(s text, from text, to text) → text`

std/c/nlpngram

import "std/c/nlpngram" · use nlpngram

- `ngraLen(s text) → number`
- `ngraUpper(s text) → text`
- `ngraLower(s text) → text`

- `ngraTrim(s text) → text`
- `ngraSplit(s text, delim text) → list text`
- `ngraJoin(parts list text, delim text) → text`
- `ngraReplace(s text, from text, to text) → text`

std/c/nlpbow

import "std/c/nlpbow" · use nlpbow

- `bowLen(s text) → number`
- `bowUpper(s text) → text`
- `bowLower(s text) → text`
- `bowTrim(s text) → text`
- `bowSplit(s text, delim text) → list text`
- `bowJoin(parts list text, delim text) → text`
- `bowReplace(s text, from text, to text) → text`

std/c/nlpstop

import "std/c/nlpstop" · use nlpstop

- `stopLen(s text) → number`
- `stopUpper(s text) → text`
- `stopLower(s text) → text`
- `stopTrim(s text) → text`
- `stopSplit(s text, delim text) → list text`
- `stopJoin(parts list text, delim text) → text`
- `stopReplace(s text, from text, to text) → text`

std/c/nlpphrase

import "std/c/nlpphrase" · use nlpphrase

- `phraLen(s text) → number`
- `phraUpper(s text) → text`
- `phraLower(s text) → text`
- `phraTrim(s text) → text`
- `phraSplit(s text, delim text) → list text`
- `phraJoin(parts list text, delim text) → text`
- `phraReplace(s text, from text, to text) → text`

std/c/nlpsentiment

import "std/c/nlpsentiment" · use nlpsentiment

- `sentLen(s text) → number`
- `sentUpper(s text) → text`
- `sentLower(s text) → text`
- `sentTrim(s text) → text`
- `sentSplit(s text, delim text) → list text`
- `sentJoin(parts list text, delim text) → text`
- `sentReplace(s text, from text, to text) → text`

std/c/nlpkeyword

import "std/c/nlpkeyword" · use nlpkeyword

- `keywLen(s text) → number`
- `keywUpper(s text) → text`
- `keywLower(s text) → text`
- `keywTrim(s text) → text`
- `keywSplit(s text, delim text) → list text`
- `keywJoin(parts list text, delim text) → text`
- `keywReplace(s text, from text, to text) → text`

std/c/nlptfidf

import "std/c/nlptfidf" · use nlptfidf

- `tf(doc text, term text) → number`
- `idf(term text, corpus list text) → number`
- `tfidf(doc text, term text, corpus list text) → number`
- `docVector(doc text, terms list text, corpus list text) → list number`
- `cosineSim(a list number, b list number) → number`
- `rankTerms(doc text, corpus list text) → list text`

std/c/nlpoverlap

import "std/c/nlpoverlap" · use nlpoverlap

- `overLen(s text) → number`
- `overUpper(s text) → text`
- `overLower(s text) → text`
- `overTrim(s text) → text`
- `overSplit(s text, delim text) → list text`
- `overJoin(parts list text, delim text) → text`
- `overReplace(s text, from text, to text) → text`

std/c/nlpsimhash

import "std/c/nlpsimhash" · use nlpsimhash

- `simhLen(s text) → number`
- `simhUpper(s text) → text`
- `simhLower(s text) → text`
- `simhTrim(s text) → text`
- `simhSplit(s text, delim text) → list text`
- `simhJoin(parts list text, delim text) → text`
- `simhReplace(s text, from text, to text) → text`

std/c/nlpcompress

import "std/c/nlpcompress" · use nlpcompress

- `compLen(s text) → number`
- `compUpper(s text) → text`

- `compLower(s text) → text`
- `compTrim(s text) → text`
- `compSplit(s text, delim text) → list text`
- `compJoin(parts list text, delim text) → text`
- `compReplace(s text, from text, to text) → text`

std/c/tsma

import "std/c/tsma" · use tsma

- `sma(v list number, win number) → list number`
- `wma(v list number, win number) → list number`
- `dema(v list number, win number) → list number`
- `crossover(v list number, fast number, slow number) → list number`
- `signal(v list number, fast number, slow number) → number`
- `trend(v list number, win number) → number`

std/c/tsema

import "std/c/tsema" · use tsema

- `ema(v list number, alpha number) → list number`
- `emaSpan(v list number, span number) → list number`
- `macd(v list number) → list number`
- `macdSignal(v list number) → list number`
- `histogram(v list number) → list number`
- `lastEma(v list number, alpha number) → number`

std/c/tsautocorr

import "std/c/tsautocorr" · use tsautocorr

- `autoSum(v list number) → number`
- `autoMean(v list number) → number`
- `autoMin(v list number) → number`
- `autoMax(v list number) → number`
- `autoVar(v list number) → number`
- `autoStd(v list number) → number`
- `autoNorm(v list number) → list number`

std/c/tsseason

import "std/c/tsseason" · use tsseason

- `seasSum(v list number) → number`
- `seasMean(v list number) → number`
- `seasMin(v list number) → number`
- `seasMax(v list number) → number`
- `seasVar(v list number) → number`
- `seasStd(v list number) → number`
- `seasNorm(v list number) → list number`

std/c/tsdecompose

import "std/c/tsdecompose" · use tsdecompose

- `decoSum(v list number) → number`
- `decoMean(v list number) → number`
- `decoMin(v list number) → number`
- `decoMax(v list number) → number`
- `decoVar(v list number) → number`
- `decoStd(v list number) → number`
- `decoNorm(v list number) → list number`

std/c/tsforecast

import "std/c/tsforecast" · use tsforecast

- `foreSum(v list number) → number`
- `foreMean(v list number) → number`
- `foreMin(v list number) → number`
- `foreMax(v list number) → number`
- `foreVar(v list number) → number`
- `foreStd(v list number) → number`
- `foreNorm(v list number) → list number`

std/c/tsanomaly

import "std/c/tsanomaly" · use tsanomaly

- `anomSum(v list number) → number`
- `anomMean(v list number) → number`
- `anomMin(v list number) → number`
- `anomMax(v list number) → number`
- `anomVar(v list number) → number`
- `anomStd(v list number) → number`
- `anomNorm(v list number) → list number`

std/c/tsrolling

import "std/c/tsrolling" · use tsrolling

- `rollSum(v list number) → number`
- `rollMean(v list number) → number`
- `rollMin(v list number) → number`
- `rollMax(v list number) → number`
- `rollVar(v list number) → number`
- `rollStd(v list number) → number`
- `rollNorm(v list number) → list number`

std/c/tscumulative

import "std/c/tscumulative" · use tscumulative

- `cumuSum(v list number) → number`
- `cumuMean(v list number) → number`
- `cumuMin(v list number) → number`
- `cumuMax(v list number) → number`
- `cumuVar(v list number) → number`
- `cumuStd(v list number) → number`
- `cumuNorm(v list number) → list number`

std/c/tsdiff

import "std/c/tsdiff" · use tsdiff

- `diff(v list number, order number) → list number`
- `pctChange(v list number) → list number`
- `logReturn(v list number) → list number`
- `cumReturn(v list number) → list number`
- `seasonalDiff(v list number, period number) → list number`
- `integrate(v list number, start number) → list number`

std/c/geomhull

import "std/c/geomhull" · use geomhull

- `hullSum(v list number) → number`
- `hullMean(v list number) → number`
- `hullMin(v list number) → number`
- `hullMax(v list number) → number`
- `hullVar(v list number) → number`
- `hullStd(v list number) → number`
- `hullNorm(v list number) → list number`

std/c/geompolygon

import "std/c/geompolygon" · use geompolygon

- `polySum(v list number) → number`
- `polyMean(v list number) → number`
- `polyMin(v list number) → number`
- `polyMax(v list number) → number`
- `polyVar(v list number) → number`
- `polyStd(v list number) → number`
- `polyNorm(v list number) → list number`

std/c/geomline

import "std/c/geomline" · use geomline

- `lineSum(v list number) → number`
- `lineMean(v list number) → number`

- `lineMin(v list number) → number`
- `lineMax(v list number) → number`
- `lineVar(v list number) → number`
- `lineStd(v list number) → number`
- `lineNorm(v list number) → list number`

std/c/geomcircle

import "std/c/geomcircle" · use geomcircle

- `euclidean2d(x1 number, y1 number, x2 number, y2 number) → number`
- `area(r number) → number`
- `circumference(r number) → number`
- `contains(cx number, cy number, r number, px number, py number) → bool`
- `pointAt(cx number, cy number, r number, angle number) → list number`
- `intersectArea(r1 number, r2 number, d number) → number`

std/c/geombezier

import "std/c/geombezier" · use geombezier

- `beziSum(v list number) → number`
- `beziMean(v list number) → number`
- `beziMin(v list number) → number`
- `beziMax(v list number) → number`
- `beziVar(v list number) → number`
- `beziStd(v list number) → number`
- `beziNorm(v list number) → list number`

std/c/geomclip

import "std/c/geomclip" · use geomclip

- `clipSum(v list number) → number`
- `clipMean(v list number) → number`
- `clipMin(v list number) → number`
- `clipMax(v list number) → number`
- `clipVar(v list number) → number`
- `clipStd(v list number) → number`
- `clipNorm(v list number) → list number`

std/c/geomrotate

import "std/c/geomrotate" · use geomrotate

- `rotaSum(v list number) → number`
- `rotaMean(v list number) → number`
- `rotaMin(v list number) → number`
- `rotaMax(v list number) → number`
- `rotaVar(v list number) → number`
- `rotaStd(v list number) → number`
- `rotaNorm(v list number) → list number`

std/c/geomscale

import "std/c/geomscale" · use geomscale

- `scalSum(v list number) → number`
- `scalMean(v list number) → number`
- `scalMin(v list number) → number`
- `scalMax(v list number) → number`
- `scalVar(v list number) → number`
- `scalStd(v list number) → number`
- `scalNorm(v list number) → list number`

std/c/geomintersect

import "std/c/geomintersect" · use geomintersect

- `inteSum(v list number) → number`
- `inteMean(v list number) → number`
- `inteMin(v list number) → number`
- `inteMax(v list number) → number`
- `inteVar(v list number) → number`
- `inteStd(v list number) → number`
- `inteNorm(v list number) → list number`

std/c/geomdistance

import "std/c/geomdistance" · use geomdistance

- `euclidean2d(x1 number, y1 number, x2 number, y2 number) → number`
- `manhattan2d(x1 number, y1 number, x2 number, y2 number) → number`
- `chebyshev2d(x1 number, y1 number, x2 number, y2 number) → number`
- `pointToLine(px number, py number, x1 number, y1 number, x2 number, y2 number) → number`
- `pairwiseDist(points list number) → list number`
- `centroid2d(points list number) → list number`
- `bbox2d(points list number) → list number`

std/c/netip

import "std/c/netip" · use netip

- `parseOctets(ip text) → list number`
- `toInt(ip text) → number`
- `fromInt(n number) → text`
- `isPrivate(ip text) → bool`
- `isValid(ip text) → bool`
- `sameSubnet(a text, b text, mask number) → bool`
- `broadcast(ip text, mask number) → text`

std/c/netsubnet

import "std/c/netsubnet" · use netsubnet

- `subnLen(s text) → number`
- `subnUpper(s text) → text`
- `subnLower(s text) → text`
- `subnTrim(s text) → text`
- `subnSplit(s text, delim text) → list text`
- `subnJoin(parts list text, delim text) → text`
- `subnReplace(s text, from text, to text) → text`

std/c/netcidr

import "std/c/netcidr" · use netcidr

- `prefixLen(cidr text) → number`
- `hostCount(cidr text) → number`
- `maskFromPrefix(prefix number) → number`
- `ipToInt(ip text) → number`
- `networkInt(cidr text) → number`
- `containsIp(cidr text, ip text) → bool`
- `broadcastInt(cidr text) → number`

std/c/netpacket

import "std/c/netpacket" · use netpacket

- `packLen(s text) → number`
- `packUpper(s text) → text`
- `packLower(s text) → text`
- `packTrim(s text) → text`
- `packSplit(s text, delim text) → list text`
- `packJoin(parts list text, delim text) → text`
- `packReplace(s text, from text, to text) → text`

std/c/netmac

import "std/c/netmac" · use netmac

- `macLen(s text) → number`
- `macUpper(s text) → text`
- `macLower(s text) → text`
- `macTrim(s text) → text`
- `macSplit(s text, delim text) → list text`
- `macJoin(parts list text, delim text) → text`
- `macReplace(s text, from text, to text) → text`

std/c/netdns

import "std/c/netdns" · use netdns

- `dnsLen(s text) → number`
- `dnsUpper(s text) → text`
- `dnsLower(s text) → text`
- `dnsTrim(s text) → text`
- `dnsSplit(s text, delim text) → list text`
- `dnsJoin(parts list text, delim text) → text`
- `dnsReplace(s text, from text, to text) → text`

std/c/netport

import "std/c/netport" · use netport

- `portLen(s text) → number`
- `portUpper(s text) → text`
- `portLower(s text) → text`
- `portTrim(s text) → text`
- `portSplit(s text, delim text) → list text`
- `portJoin(parts list text, delim text) → text`
- `portReplace(s text, from text, to text) → text`

std/c/netroute

import "std/c/netroute" · use netroute

- `routLen(s text) → number`
- `routUpper(s text) → text`
- `routLower(s text) → text`
- `routTrim(s text) → text`
- `routSplit(s text, delim text) → list text`
- `routJoin(parts list text, delim text) → text`
- `routReplace(s text, from text, to text) → text`

std/c/netping

import "std/c/netping" · use netping

- `pingLen(s text) → number`
- `pingUpper(s text) → text`
- `pingLower(s text) → text`
- `pingTrim(s text) → text`
- `pingSplit(s text, delim text) → list text`
- `pingJoin(parts list text, delim text) → text`
- `pingReplace(s text, from text, to text) → text`

std/c/netchecksum

import "std/c/netchecksum" · use netchecksum

- `ipChecksum(data list number) → number`
- `onesComplement(n number) → number`
- `fold32(sum number) → number`
- `verify(data list number, expected number) → bool`
- `pseudoHeader(src list number, dst list number, proto number, len number) → list number`
- `tcpChecksum(header list number, payload list number) → number`

std/c/gameminimax

import "std/c/gameminimax" · use gameminimax

- `minimax(board list number, depth number, maximizing bool) → number`
- `bestMove(board list number, depth number) → number`
- `evaluate(board list number) → number`
- `isTerminal(board list number) → bool`
- `negamax(board list number, depth number, color number) → number`
- `depthReached(depth number) → number`
- `scoreDiff(board list number) → number`

std/c/gamealphabeta

import "std/c/gamealphabeta" · use gamealphabeta

- `alphabeta(board list number, depth number, alpha number, beta number, maximizing bool) → number`
- `bestMove(board list number, depth number) → number`
- `pruneCount(board list number, depth number) → number`
- `evaluate(board list number) → number`
- `search(board list number, depth number) → number`
- `isWinning(board list number) → bool`
- `windowSearch(board list number, depth number, window number) → number`

std/c/gameastar

import "std/c/gameastar" · use gameastar

- `heuristic(ax number, ay number, bx number, by number) → number`
- `astar(grid list number, width number, start number, goal number) → list number`
- `pathCost(grid list number, width number, start number, goal number) → number`
- `reconstruct(cameFrom list number, current number) → list number`
- `manhattan(ax number, ay number, bx number, by number) → number`
- `octile(ax number, ay number, bx number, by number) → number`
- `openSetSize(grid list number) → number`

std/c/gamedijkstra

import "std/c/gamedijkstra" · use gamedijkstra

- `gridDijkstra(grid list number, width number, start number) → list number`
- `shortestCost(grid list number, width number, start number, goal number) → number`

- `reachableCells(grid list number, width number, start number) → number`
- `multiGoal(grid list number, width number, start number, goals list number) → list number`
- `relaxStep(dist list number, grid list number, width number, u number) → list number`
- `maxDist(grid list number, width number, start number) → number`

std/c/gamemcts

import "std/c/gamemcts" · use gamemcts

- `uctSelect(wins list number, visits list number, totalVisits number, explore number) → number`
- `rollout(state list number, depth number) → number`
- `simulate(state list number, sims number) → number`
- `bestAction(wins list number, visits list number) → number`
- `updateStats(wins list number, visits list number, action number, reward number) → list number`
- `treePolicy(state list number, actions list number) → number`
- `mctsValue(wins list number, visits list number) → number`

std/c/gamepathfind

import "std/c/gamepathfind" · use gamepathfind

- `bfsPath(grid list number, width number, start number, goal number) → list number`
- `pathLen(grid list number, width number, start number, goal number) → number`
- `lineOfSight(grid list number, width number, x0 number, y0 number, x1 number, y1 number) → bool`
- `smoothPath(path list number, grid list number, width number) → list number`
- `waypoints(path list number, step number) → list number`
- `navCost(grid list number, width number, start number, goal number) → number`

std/c/gamenavmesh

import "std/c/gamenavmesh" · use gamenavmesh

- `pointInTri(px number, py number, ax number, ay number, bx number, by number, cx number, cy number) → bool`
- `triArea(ax number, ay number, bx number, by number, cx number, cy number) → number`
- `portalMid(a list number, b list number) → list number`
- `navDistance(a list number, b list number) → number`
- `funnelCost(path list number) → number`
- `centroid(ax number, ay number, bx number, by number, cx number, cy number) → list number`
- `meshQuality(ax number, ay number, bx number, by number, cx number, cy number) → number`

std/c/gamebehavior

import "std/c/gamebehavior" · use gamebehavior

- `evalSelector(scores list number) → number`
- `evalSequence(scores list number) → number`
- `evalParallel(scores list number) → number`
- `blackboardGet(keys list text, values list number, key text) → number`
- `blackboardSet(keys list text, values list number, key text, val number) → list number`
- `cooldownReady(last number, now number, cd number) → bool`
- `weightedPick(weights list number) → number`

std/c/gameflood

import "std/c/gameflood" · use gameflood

- `floodFill(grid list number, width number, start number, newVal number) → list number`
- `connectedRegion(grid list number, width number, start number) → number`
- `countRegions(grid list number, width number) → number`
- `fillDist(grid list number, width number, start number) → list number`
- `reachable(grid list number, width number, start number) → number`
- `boundary(grid list number, width number, start number) → list number`

std/c/gametree

import "std/c/gametree" · use gametree

- `branchingFactor(children list number) → number`
- `treeDepth(depths list number) → number`
- `minimaxLeaf(values list number, maxNode bool) → number`
- `expectimax(values list number, probs list number) → number`
- `alphaBetaWindow(alpha number, beta number) → list number`
- `gameTreeSize(branching number, depth number) → number`
- `plyCount(moves list number) → number`
- `bestChild(scores list number) → number`

std/c/gameultra001

import "std/c/gameultra001" · use gameultra001

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra002

import "std/c/gameultra002" · use gameultra002

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra003

import "std/c/gameultra003" · use gameultra003

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra004

import "std/c/gameultra004" · use gameultra004

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra005

import "std/c/gameultra005" · use gameultra005

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra006

import "std/c/gameultra006" · use gameultra006

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra007

import "std/c/gameultra007" · use gameultra007

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra008

import "std/c/gameultra008" · use gameultra008

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra009

import "std/c/gameultra009" · use gameultra009

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra010

import "std/c/gameultra010" · use gameultra010

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra011

import "std/c/gameultra011" · use gameultra011

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra012

import "std/c/gameultra012" · use gameultra012

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra013

import "std/c/gameultra013" · use gameultra013

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra014

import "std/c/gameultra014" · use gameultra014

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra015

import "std/c/gameultra015" · use gameultra015

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra016

import "std/c/gameultra016" · use gameultra016

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra017

import "std/c/gameultra017" · use gameultra017

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra018

import "std/c/gameultra018" · use gameultra018

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra019

import "std/c/gameultra019" · use gameultra019

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra020

import "std/c/gameultra020" · use gameultra020

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra021

import "std/c/gameultra021" · use gameultra021

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra022

import "std/c/gameultra022" · use gameultra022

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra023

import "std/c/gameultra023" · use gameultra023

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra024

import "std/c/gameultra024" · use gameultra024

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra025

import "std/c/gameultra025" · use gameultra025

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra026

import "std/c/gameultra026" · use gameultra026

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra027

import "std/c/gameultra027" · use gameultra027

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra028

import "std/c/gameultra028" · use gameultra028

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra029

import "std/c/gameultra029" · use gameultra029

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra030

import "std/c/gameultra030" · use gameultra030

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra031

import "std/c/gameultra031" · use gameultra031

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra032

import "std/c/gameultra032" · use gameultra032

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra033

import "std/c/gameultra033" · use gameultra033

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra034

import "std/c/gameultra034" · use gameultra034

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra035

import "std/c/gameultra035" · use gameultra035

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra036

import "std/c/gameultra036" · use gameultra036

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra037

import "std/c/gameultra037" · use gameultra037

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra038

import "std/c/gameultra038" · use gameultra038

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra039

import "std/c/gameultra039" · use gameultra039

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra040

import "std/c/gameultra040" · use gameultra040

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra041

import "std/c/gameultra041" · use gameultra041

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra042

import "std/c/gameultra042" · use gameultra042

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra043

import "std/c/gameultra043" · use gameultra043

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra044

import "std/c/gameultra044" · use gameultra044

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra045

import "std/c/gameultra045" · use gameultra045

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra046

import "std/c/gameultra046" · use gameultra046

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra047

import "std/c/gameultra047" · use gameultra047

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra048

import "std/c/gameultra048" · use gameultra048

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra049

import "std/c/gameultra049" · use gameultra049

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra050

import "std/c/gameultra050" · use gameultra050

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra051

import "std/c/gameultra051" · use gameultra051

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra052

import "std/c/gameultra052" · use gameultra052

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra053

import "std/c/gameultra053" · use gameultra053

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra054

import "std/c/gameultra054" · use gameultra054

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra055

import "std/c/gameultra055" · use gameultra055

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra056

import "std/c/gameultra056" · use gameultra056

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra057

import "std/c/gameultra057" · use gameultra057

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra058

import "std/c/gameultra058" · use gameultra058

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra059

import "std/c/gameultra059" · use gameultra059

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra060

import "std/c/gameultra060" · use gameultra060

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra061

import "std/c/gameultra061" · use gameultra061

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra062

import "std/c/gameultra062" · use gameultra062

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra063

import "std/c/gameultra063" · use gameultra063

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra064

import "std/c/gameultra064" · use gameultra064

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra065

import "std/c/gameultra065" · use gameultra065

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra066

import "std/c/gameultra066" · use gameultra066

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra067

import "std/c/gameultra067" · use gameultra067

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra068

import "std/c/gameultra068" · use gameultra068

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra069

import "std/c/gameultra069" · use gameultra069

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra070

import "std/c/gameultra070" · use gameultra070

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra071

import "std/c/gameultra071" · use gameultra071

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra072

import "std/c/gameultra072" · use gameultra072

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra073

import "std/c/gameultra073" · use gameultra073

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra074

import "std/c/gameultra074" · use gameultra074

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra075

import "std/c/gameultra075" · use gameultra075

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra076

import "std/c/gameultra076" · use gameultra076

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra077

import "std/c/gameultra077" · use gameultra077

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra078

import "std/c/gameultra078" · use gameultra078

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra079

import "std/c/gameultra079" · use gameultra079

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra080

import "std/c/gameultra080" · use gameultra080

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra081

import "std/c/gameultra081" · use gameultra081

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra082

import "std/c/gameultra082" · use gameultra082

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra083

import "std/c/gameultra083" · use gameultra083

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra084

import "std/c/gameultra084" · use gameultra084

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra085

import "std/c/gameultra085" · use gameultra085

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra086

import "std/c/gameultra086" · use gameultra086

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra087

import "std/c/gameultra087" · use gameultra087

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra088

import "std/c/gameultra088" · use gameultra088

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra089

import "std/c/gameultra089" · use gameultra089

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra090

import "std/c/gameultra090" · use gameultra090

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra091

import "std/c/gameultra091" · use gameultra091

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra092

import "std/c/gameultra092" · use gameultra092

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra093

import "std/c/gameultra093" · use gameultra093

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra094

import "std/c/gameultra094" · use gameultra094

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra095

import "std/c/gameultra095" · use gameultra095

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra096

import "std/c/gameultra096" · use gameultra096

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra097

import "std/c/gameultra097" · use gameultra097

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra098

import "std/c/gameultra098" · use gameultra098

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra099

import "std/c/gameultra099" · use gameultra099

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra100

import "std/c/gameultra100" · use gameultra100

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra101

import "std/c/gameultra101" · use gameultra101

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra102

import "std/c/gameultra102" · use gameultra102

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra103

import "std/c/gameultra103" · use gameultra103

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra104

import "std/c/gameultra104" · use gameultra104

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra105

import "std/c/gameultra105" · use gameultra105

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra106

import "std/c/gameultra106" · use gameultra106

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra107

import "std/c/gameultra107" · use gameultra107

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra108

import "std/c/gameultra108" · use gameultra108

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra109

import "std/c/gameultra109" · use gameultra109

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra110

import "std/c/gameultra110" · use gameultra110

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra111

import "std/c/gameultra111" · use gameultra111

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra112

import "std/c/gameultra112" · use gameultra112

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra113

import "std/c/gameultra113" · use gameultra113

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra114

import "std/c/gameultra114" · use gameultra114

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra115

import "std/c/gameultra115" · use gameultra115

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra116

import "std/c/gameultra116" · use gameultra116

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra117

import "std/c/gameultra117" · use gameultra117

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra118

import "std/c/gameultra118" · use gameultra118

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra119

import "std/c/gameultra119" · use gameultra119

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra120

import "std/c/gameultra120" · use gameultra120

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra121

import "std/c/gameultra121" · use gameultra121

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra122

import "std/c/gameultra122" · use gameultra122

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra123

import "std/c/gameultra123" · use gameultra123

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra124

import "std/c/gameultra124" · use gameultra124

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra125

import "std/c/gameultra125" · use gameultra125

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra126

import "std/c/gameultra126" · use gameultra126

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra127

import "std/c/gameultra127" · use gameultra127

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra128

import "std/c/gameultra128" · use gameultra128

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra129

import "std/c/gameultra129" · use gameultra129

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra130

import "std/c/gameultra130" · use gameultra130

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra131

import "std/c/gameultra131" · use gameultra131

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra132

import "std/c/gameultra132" · use gameultra132

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra133

import "std/c/gameultra133" · use gameultra133

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra134

import "std/c/gameultra134" · use gameultra134

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra135

import "std/c/gameultra135" · use gameultra135

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra136

import "std/c/gameultra136" · use gameultra136

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra137

import "std/c/gameultra137" · use gameultra137

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra138

import "std/c/gameultra138" · use gameultra138

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra139

import "std/c/gameultra139" · use gameultra139

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra140

import "std/c/gameultra140" · use gameultra140

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra141

import "std/c/gameultra141" · use gameultra141

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra142

import "std/c/gameultra142" · use gameultra142

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra143

import "std/c/gameultra143" · use gameultra143

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra144

import "std/c/gameultra144" · use gameultra144

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra145

import "std/c/gameultra145" · use gameultra145

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra146

import "std/c/gameultra146" · use gameultra146

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra147

import "std/c/gameultra147" · use gameultra147

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra148

import "std/c/gameultra148" · use gameultra148

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra149

import "std/c/gameultra149" · use gameultra149

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra150

import "std/c/gameultra150" · use gameultra150

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra151

import "std/c/gameultra151" · use gameultra151

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra152

import "std/c/gameultra152" · use gameultra152

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra153

import "std/c/gameultra153" · use gameultra153

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra154

import "std/c/gameultra154" · use gameultra154

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra155

import "std/c/gameultra155" · use gameultra155

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra156

import "std/c/gameultra156" · use gameultra156

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra157

import "std/c/gameultra157" · use gameultra157

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra158

import "std/c/gameultra158" · use gameultra158

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra159

import "std/c/gameultra159" · use gameultra159

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra160

import "std/c/gameultra160" · use gameultra160

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra161

import "std/c/gameultra161" · use gameultra161

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra162

import "std/c/gameultra162" · use gameultra162

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra163

import "std/c/gameultra163" · use gameultra163

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra164

import "std/c/gameultra164" · use gameultra164

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra165

import "std/c/gameultra165" · use gameultra165

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra166

import "std/c/gameultra166" · use gameultra166

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra167

import "std/c/gameultra167" · use gameultra167

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra168

import "std/c/gameultra168" · use gameultra168

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra169

import "std/c/gameultra169" · use gameultra169

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra170

import "std/c/gameultra170" · use gameultra170

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra171

import "std/c/gameultra171" · use gameultra171

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra172

import "std/c/gameultra172" · use gameultra172

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra173

import "std/c/gameultra173" · use gameultra173

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra174

import "std/c/gameultra174" · use gameultra174

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra175

import "std/c/gameultra175" · use gameultra175

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra176

import "std/c/gameultra176" · use gameultra176

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra177

import "std/c/gameultra177" · use gameultra177

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra178

import "std/c/gameultra178" · use gameultra178

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra179

import "std/c/gameultra179" · use gameultra179

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra180

import "std/c/gameultra180" · use gameultra180

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra181

import "std/c/gameultra181" · use gameultra181

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra182

import "std/c/gameultra182" · use gameultra182

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra183

import "std/c/gameultra183" · use gameultra183

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra184

import "std/c/gameultra184" · use gameultra184

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra185

import "std/c/gameultra185" · use gameultra185

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra186

import "std/c/gameultra186" · use gameultra186

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra187

import "std/c/gameultra187" · use gameultra187

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra188

import "std/c/gameultra188" · use gameultra188

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra189

import "std/c/gameultra189" · use gameultra189

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra190

import "std/c/gameultra190" · use gameultra190

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra191

import "std/c/gameultra191" · use gameultra191

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra192

import "std/c/gameultra192" · use gameultra192

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra193

import "std/c/gameultra193" · use gameultra193

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra194

import "std/c/gameultra194" · use gameultra194

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra195

import "std/c/gameultra195" · use gameultra195

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra196

import "std/c/gameultra196" · use gameultra196

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra197

import "std/c/gameultra197" · use gameultra197

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra198

import "std/c/gameultra198" · use gameultra198

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra199

import "std/c/gameultra199" · use gameultra199

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra200

import "std/c/gameultra200" · use gameultra200

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra201

import "std/c/gameultra201" · use gameultra201

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra202

import "std/c/gameultra202" · use gameultra202

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra203

import "std/c/gameultra203" · use gameultra203

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra204

import "std/c/gameultra204" · use gameultra204

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra205

import "std/c/gameultra205" · use gameultra205

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra206

import "std/c/gameultra206" · use gameultra206

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra207

import "std/c/gameultra207" · use gameultra207

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra208

import "std/c/gameultra208" · use gameultra208

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra209

import "std/c/gameultra209" · use gameultra209

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra210

import "std/c/gameultra210" · use gameultra210

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra211

import "std/c/gameultra211" · use gameultra211

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra212

import "std/c/gameultra212" · use gameultra212

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra213

import "std/c/gameultra213" · use gameultra213

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra214

import "std/c/gameultra214" · use gameultra214

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra215

import "std/c/gameultra215" · use gameultra215

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra216

import "std/c/gameultra216" · use gameultra216

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra217

import "std/c/gameultra217" · use gameultra217

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra218

import "std/c/gameultra218" · use gameultra218

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra219

import "std/c/gameultra219" · use gameultra219

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra220

import "std/c/gameultra220" · use gameultra220

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra221

import "std/c/gameultra221" · use gameultra221

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra222

import "std/c/gameultra222" · use gameultra222

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra223

import "std/c/gameultra223" · use gameultra223

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra224

import "std/c/gameultra224" · use gameultra224

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra225

import "std/c/gameultra225" · use gameultra225

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra226

import "std/c/gameultra226" · use gameultra226

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra227

import "std/c/gameultra227" · use gameultra227

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra228

import "std/c/gameultra228" · use gameultra228

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra229

import "std/c/gameultra229" · use gameultra229

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra230

import "std/c/gameultra230" · use gameultra230

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra231

import "std/c/gameultra231" · use gameultra231

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra232

import "std/c/gameultra232" · use gameultra232

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra233

import "std/c/gameultra233" · use gameultra233

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra234

import "std/c/gameultra234" · use gameultra234

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra235

import "std/c/gameultra235" · use gameultra235

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra236

import "std/c/gameultra236" · use gameultra236

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra237

import "std/c/gameultra237" · use gameultra237

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra238

import "std/c/gameultra238" · use gameultra238

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra239

import "std/c/gameultra239" · use gameultra239

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra240

import "std/c/gameultra240" · use gameultra240

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra241

import "std/c/gameultra241" · use gameultra241

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra242

import "std/c/gameultra242" · use gameultra242

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra243

import "std/c/gameultra243" · use gameultra243

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra244

import "std/c/gameultra244" · use gameultra244

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra245

import "std/c/gameultra245" · use gameultra245

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra246

import "std/c/gameultra246" · use gameultra246

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra247

import "std/c/gameultra247" · use gameultra247

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra248

import "std/c/gameultra248" · use gameultra248

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra249

import "std/c/gameultra249" · use gameultra249

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra250

import "std/c/gameultra250" · use gameultra250

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra251

import "std/c/gameultra251" · use gameultra251

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra252

import "std/c/gameultra252" · use gameultra252

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra253

import "std/c/gameultra253" · use gameultra253

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra254

import "std/c/gameultra254" · use gameultra254

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra255

import "std/c/gameultra255" · use gameultra255

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra256

import "std/c/gameultra256" · use gameultra256

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra257

import "std/c/gameultra257" · use gameultra257

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra258

import "std/c/gameultra258" · use gameultra258

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra259

import "std/c/gameultra259" · use gameultra259

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra260

import "std/c/gameultra260" · use gameultra260

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra261

import "std/c/gameultra261" · use gameultra261

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra262

import "std/c/gameultra262" · use gameultra262

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra263

import "std/c/gameultra263" · use gameultra263

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra264

import "std/c/gameultra264" · use gameultra264

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra265

import "std/c/gameultra265" · use gameultra265

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra266

import "std/c/gameultra266" · use gameultra266

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra267

import "std/c/gameultra267" · use gameultra267

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra268

import "std/c/gameultra268" · use gameultra268

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra269

import "std/c/gameultra269" · use gameultra269

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra270

import "std/c/gameultra270" · use gameultra270

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra271

import "std/c/gameultra271" · use gameultra271

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra272

import "std/c/gameultra272" · use gameultra272

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra273

import "std/c/gameultra273" · use gameultra273

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra274

import "std/c/gameultra274" · use gameultra274

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra275

import "std/c/gameultra275" · use gameultra275

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra276

import "std/c/gameultra276" · use gameultra276

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra277

import "std/c/gameultra277" · use gameultra277

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra278

import "std/c/gameultra278" · use gameultra278

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra279

import "std/c/gameultra279" · use gameultra279

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra280

import "std/c/gameultra280" · use gameultra280

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra281

import "std/c/gameultra281" · use gameultra281

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra282

import "std/c/gameultra282" · use gameultra282

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra283

import "std/c/gameultra283" · use gameultra283

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra284

import "std/c/gameultra284" · use gameultra284

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra285

import "std/c/gameultra285" · use gameultra285

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra286

import "std/c/gameultra286" · use gameultra286

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra287

import "std/c/gameultra287" · use gameultra287

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra288

import "std/c/gameultra288" · use gameultra288

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra289

import "std/c/gameultra289" · use gameultra289

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra290

import "std/c/gameultra290" · use gameultra290

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra291

import "std/c/gameultra291" · use gameultra291

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra292

import "std/c/gameultra292" · use gameultra292

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra293

import "std/c/gameultra293" · use gameultra293

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra294

import "std/c/gameultra294" · use gameultra294

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra295

import "std/c/gameultra295" · use gameultra295

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra296

import "std/c/gameultra296" · use gameultra296

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra297

import "std/c/gameultra297" · use gameultra297

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra298

import "std/c/gameultra298" · use gameultra298

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra299

import "std/c/gameultra299" · use gameultra299

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra300

import "std/c/gameultra300" · use gameultra300

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra301

import "std/c/gameultra301" · use gameultra301

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra302

import "std/c/gameultra302" · use gameultra302

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra303

import "std/c/gameultra303" · use gameultra303

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra304

import "std/c/gameultra304" · use gameultra304

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra305

import "std/c/gameultra305" · use gameultra305

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra306

import "std/c/gameultra306" · use gameultra306

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra307

import "std/c/gameultra307" · use gameultra307

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra308

import "std/c/gameultra308" · use gameultra308

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra309

import "std/c/gameultra309" · use gameultra309

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra310

import "std/c/gameultra310" · use gameultra310

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra311

import "std/c/gameultra311" · use gameultra311

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra312

import "std/c/gameultra312" · use gameultra312

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra313

import "std/c/gameultra313" · use gameultra313

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra314

import "std/c/gameultra314" · use gameultra314

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra315

import "std/c/gameultra315" · use gameultra315

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra316

import "std/c/gameultra316" · use gameultra316

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra317

import "std/c/gameultra317" · use gameultra317

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra318

import "std/c/gameultra318" · use gameultra318

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra319

import "std/c/gameultra319" · use gameultra319

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra320

import "std/c/gameultra320" · use gameultra320

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra321

import "std/c/gameultra321" · use gameultra321

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra322

import "std/c/gameultra322" · use gameultra322

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra323

import "std/c/gameultra323" · use gameultra323

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra324

import "std/c/gameultra324" · use gameultra324

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra325

import "std/c/gameultra325" · use gameultra325

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra326

import "std/c/gameultra326" · use gameultra326

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra327

import "std/c/gameultra327" · use gameultra327

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra328

import "std/c/gameultra328" · use gameultra328

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra329

import "std/c/gameultra329" · use gameultra329

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra330

import "std/c/gameultra330" · use gameultra330

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra331

import "std/c/gameultra331" · use gameultra331

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra332

import "std/c/gameultra332" · use gameultra332

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra333

import "std/c/gameultra333" · use gameultra333

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra334

import "std/c/gameultra334" · use gameultra334

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra335

import "std/c/gameultra335" · use gameultra335

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra336

import "std/c/gameultra336" · use gameultra336

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra337

import "std/c/gameultra337" · use gameultra337

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra338

import "std/c/gameultra338" · use gameultra338

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra339

import "std/c/gameultra339" · use gameultra339

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra340

import "std/c/gameultra340" · use gameultra340

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra341

import "std/c/gameultra341" · use gameultra341

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra342

import "std/c/gameultra342" · use gameultra342

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra343

import "std/c/gameultra343" · use gameultra343

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra344

import "std/c/gameultra344" · use gameultra344

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra345

import "std/c/gameultra345" · use gameultra345

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra346

import "std/c/gameultra346" · use gameultra346

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra347

import "std/c/gameultra347" · use gameultra347

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra348

import "std/c/gameultra348" · use gameultra348

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra349

import "std/c/gameultra349" · use gameultra349

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra350

import "std/c/gameultra350" · use gameultra350

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra351

import "std/c/gameultra351" · use gameultra351

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra352

import "std/c/gameultra352" · use gameultra352

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra353

import "std/c/gameultra353" · use gameultra353

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra354

import "std/c/gameultra354" · use gameultra354

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra355

import "std/c/gameultra355" · use gameultra355

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra356

import "std/c/gameultra356" · use gameultra356

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra357

import "std/c/gameultra357" · use gameultra357

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra358

import "std/c/gameultra358" · use gameultra358

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra359

import "std/c/gameultra359" · use gameultra359

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra360

import "std/c/gameultra360" · use gameultra360

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra361

import "std/c/gameultra361" · use gameultra361

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra362

import "std/c/gameultra362" · use gameultra362

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra363

import "std/c/gameultra363" · use gameultra363

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra364

import "std/c/gameultra364" · use gameultra364

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra365

import "std/c/gameultra365" · use gameultra365

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra366

import "std/c/gameultra366" · use gameultra366

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra367

import "std/c/gameultra367" · use gameultra367

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra368

import "std/c/gameultra368" · use gameultra368

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra369

import "std/c/gameultra369" · use gameultra369

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra370

import "std/c/gameultra370" · use gameultra370

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra371

import "std/c/gameultra371" · use gameultra371

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra372

import "std/c/gameultra372" · use gameultra372

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra373

import "std/c/gameultra373" · use gameultra373

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra374

import "std/c/gameultra374" · use gameultra374

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra375

import "std/c/gameultra375" · use gameultra375

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra376

import "std/c/gameultra376" · use gameultra376

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra377

import "std/c/gameultra377" · use gameultra377

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra378

import "std/c/gameultra378" · use gameultra378

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra379

import "std/c/gameultra379" · use gameultra379

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra380

import "std/c/gameultra380" · use gameultra380

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra381

import "std/c/gameultra381" · use gameultra381

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra382

import "std/c/gameultra382" · use gameultra382

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra383

import "std/c/gameultra383" · use gameultra383

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra384

import "std/c/gameultra384" · use gameultra384

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra385

import "std/c/gameultra385" · use gameultra385

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra386

import "std/c/gameultra386" · use gameultra386

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra387

import "std/c/gameultra387" · use gameultra387

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra388

import "std/c/gameultra388" · use gameultra388

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra389

import "std/c/gameultra389" · use gameultra389

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra390

import "std/c/gameultra390" · use gameultra390

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra391

import "std/c/gameultra391" · use gameultra391

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra392

import "std/c/gameultra392" · use gameultra392

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra393

import "std/c/gameultra393" · use gameultra393

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra394

import "std/c/gameultra394" · use gameultra394

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra395

import "std/c/gameultra395" · use gameultra395

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra396

import "std/c/gameultra396" · use gameultra396

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra397

import "std/c/gameultra397" · use gameultra397

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra398

import "std/c/gameultra398" · use gameultra398

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra399

import "std/c/gameultra399" · use gameultra399

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra400

import "std/c/gameultra400" · use gameultra400

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra401

import "std/c/gameultra401" · use gameultra401

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra402

import "std/c/gameultra402" · use gameultra402

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra403

import "std/c/gameultra403" · use gameultra403

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra404

import "std/c/gameultra404" · use gameultra404

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra405

import "std/c/gameultra405" · use gameultra405

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra406

import "std/c/gameultra406" · use gameultra406

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra407

import "std/c/gameultra407" · use gameultra407

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra408

import "std/c/gameultra408" · use gameultra408

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra409

import "std/c/gameultra409" · use gameultra409

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra410

import "std/c/gameultra410" · use gameultra410

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra411

import "std/c/gameultra411" · use gameultra411

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra412

import "std/c/gameultra412" · use gameultra412

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra413

import "std/c/gameultra413" · use gameultra413

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra414

import "std/c/gameultra414" · use gameultra414

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra415

import "std/c/gameultra415" · use gameultra415

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra416

import "std/c/gameultra416" · use gameultra416

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra417

import "std/c/gameultra417" · use gameultra417

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra418

import "std/c/gameultra418" · use gameultra418

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra419

import "std/c/gameultra419" · use gameultra419

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra420

import "std/c/gameultra420" · use gameultra420

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra421

import "std/c/gameultra421" · use gameultra421

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra422

import "std/c/gameultra422" · use gameultra422

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra423

import "std/c/gameultra423" · use gameultra423

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra424

import "std/c/gameultra424" · use gameultra424

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra425

import "std/c/gameultra425" · use gameultra425

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra426

import "std/c/gameultra426" · use gameultra426

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra427

import "std/c/gameultra427" · use gameultra427

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra428

import "std/c/gameultra428" · use gameultra428

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra429

import "std/c/gameultra429" · use gameultra429

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra430

import "std/c/gameultra430" · use gameultra430

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra431

import "std/c/gameultra431" · use gameultra431

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra432

import "std/c/gameultra432" · use gameultra432

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra433

import "std/c/gameultra433" · use gameultra433

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra434

import "std/c/gameultra434" · use gameultra434

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra435

import "std/c/gameultra435" · use gameultra435

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra436

import "std/c/gameultra436" · use gameultra436

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra437

import "std/c/gameultra437" · use gameultra437

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra438

import "std/c/gameultra438" · use gameultra438

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra439

import "std/c/gameultra439" · use gameultra439

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra440

import "std/c/gameultra440" · use gameultra440

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra441

import "std/c/gameultra441" · use gameultra441

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra442

import "std/c/gameultra442" · use gameultra442

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra443

import "std/c/gameultra443" · use gameultra443

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra444

import "std/c/gameultra444" · use gameultra444

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra445

import "std/c/gameultra445" · use gameultra445

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra446

import "std/c/gameultra446" · use gameultra446

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra447

import "std/c/gameultra447" · use gameultra447

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra448

import "std/c/gameultra448" · use gameultra448

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra449

import "std/c/gameultra449" · use gameultra449

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra450

import "std/c/gameultra450" · use gameultra450

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra451

import "std/c/gameultra451" · use gameultra451

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra452

import "std/c/gameultra452" · use gameultra452

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra453

import "std/c/gameultra453" · use gameultra453

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra454

import "std/c/gameultra454" · use gameultra454

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra455

import "std/c/gameultra455" · use gameultra455

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra456

import "std/c/gameultra456" · use gameultra456

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra457

import "std/c/gameultra457" · use gameultra457

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra458

import "std/c/gameultra458" · use gameultra458

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra459

import "std/c/gameultra459" · use gameultra459

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra460

import "std/c/gameultra460" · use gameultra460

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra461

import "std/c/gameultra461" · use gameultra461

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra462

import "std/c/gameultra462" · use gameultra462

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra463

import "std/c/gameultra463" · use gameultra463

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra464

import "std/c/gameultra464" · use gameultra464

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra465

import "std/c/gameultra465" · use gameultra465

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra466

import "std/c/gameultra466" · use gameultra466

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra467

import "std/c/gameultra467" · use gameultra467

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra468

import "std/c/gameultra468" · use gameultra468

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra469

import "std/c/gameultra469" · use gameultra469

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra470

import "std/c/gameultra470" · use gameultra470

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra471

import "std/c/gameultra471" · use gameultra471

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra472

import "std/c/gameultra472" · use gameultra472

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra473

import "std/c/gameultra473" · use gameultra473

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra474

import "std/c/gameultra474" · use gameultra474

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra475

import "std/c/gameultra475" · use gameultra475

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra476

import "std/c/gameultra476" · use gameultra476

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra477

import "std/c/gameultra477" · use gameultra477

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra478

import "std/c/gameultra478" · use gameultra478

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra479

import "std/c/gameultra479" · use gameultra479

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra480

import "std/c/gameultra480" · use gameultra480

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra481

import "std/c/gameultra481" · use gameultra481

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra482

import "std/c/gameultra482" · use gameultra482

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra483

import "std/c/gameultra483" · use gameultra483

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra484

import "std/c/gameultra484" · use gameultra484

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra485

import "std/c/gameultra485" · use gameultra485

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra486

import "std/c/gameultra486" · use gameultra486

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra487

import "std/c/gameultra487" · use gameultra487

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra488

import "std/c/gameultra488" · use gameultra488

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra489

import "std/c/gameultra489" · use gameultra489

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra490

import "std/c/gameultra490" · use gameultra490

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra491

import "std/c/gameultra491" · use gameultra491

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra492

import "std/c/gameultra492" · use gameultra492

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra493

import "std/c/gameultra493" · use gameultra493

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra494

import "std/c/gameultra494" · use gameultra494

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra495

import "std/c/gameultra495" · use gameultra495

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra496

import "std/c/gameultra496" · use gameultra496

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra497

import "std/c/gameultra497" · use gameultra497

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra498

import "std/c/gameultra498" · use gameultra498

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra499

import "std/c/gameultra499" · use gameultra499

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra500

import "std/c/gameultra500" · use gameultra500

- `astarGridPath(grid list number, w number, h number, sx number, sy number, gx number, gy number, blockedAbove number) → list number`
- `floodCount(grid list number, w number, h number, sx number, sy number, blockedAbove number) → number`
- `simplifyGridPath(path list number, w number) → list number`

std/c/gameultra3d001

import "std/c/gameultra3d001" · use gameultra3d001

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d002

import "std/c/gameultra3d002" · use gameultra3d002

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d003

import "std/c/gameultra3d003" · use gameultra3d003

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`

• `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d004

import "std/c/gameultra3d004" · use gameultra3d004

• `integratePos(pos list number, vel list number, n number, dt number) → list number`
• `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
• `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d005

import "std/c/gameultra3d005" · use gameultra3d005

• `integratePos(pos list number, vel list number, n number, dt number) → list number`
• `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
• `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d006

import "std/c/gameultra3d006" · use gameultra3d006

• `integratePos(pos list number, vel list number, n number, dt number) → list number`
• `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
• `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d007

import "std/c/gameultra3d007" · use gameultra3d007

• `integratePos(pos list number, vel list number, n number, dt number) → list number`
• `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
• `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d008

import "std/c/gameultra3d008" · use gameultra3d008

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d009

import "std/c/gameultra3d009" · use gameultra3d009

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d010

import "std/c/gameultra3d010" · use gameultra3d010

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d011

import "std/c/gameultra3d011" · use gameultra3d011

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d012

import "std/c/gameultra3d012" · use gameultra3d012

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d013

import "std/c/gameultra3d013" · use gameultra3d013

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d014

import "std/c/gameultra3d014" · use gameultra3d014

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d015

import "std/c/gameultra3d015" · use gameultra3d015

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d016

import "std/c/gameultra3d016" · use gameultra3d016

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d017

import "std/c/gameultra3d017" · use gameultra3d017

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d018

import "std/c/gameultra3d018" · use gameultra3d018

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d019

import "std/c/gameultra3d019" · use gameultra3d019

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d020

import "std/c/gameultra3d020" · use gameultra3d020

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d021

import "std/c/gameultra3d021" · use gameultra3d021

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d022

import "std/c/gameultra3d022" · use gameultra3d022

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d023

import "std/c/gameultra3d023" · use gameultra3d023

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d024

import "std/c/gameultra3d024" · use gameultra3d024

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d025

import "std/c/gameultra3d025" · use gameultra3d025

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d026

import "std/c/gameultra3d026" · use gameultra3d026

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d027

import "std/c/gameultra3d027" · use gameultra3d027

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d028

import "std/c/gameultra3d028" · use gameultra3d028

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d029

import "std/c/gameultra3d029" · use gameultra3d029

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d030

import "std/c/gameultra3d030" · use gameultra3d030

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d031

import "std/c/gameultra3d031" · use gameultra3d031

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d032

import "std/c/gameultra3d032" · use gameultra3d032

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d033

import "std/c/gameultra3d033" · use gameultra3d033

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d034

import "std/c/gameultra3d034" · use gameultra3d034

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d035

import "std/c/gameultra3d035" · use gameultra3d035

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d036

import "std/c/gameultra3d036" · use gameultra3d036

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d037

import "std/c/gameultra3d037" · use gameultra3d037

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d038

import "std/c/gameultra3d038" · use gameultra3d038

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d039

import "std/c/gameultra3d039" · use gameultra3d039

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d040

import "std/c/gameultra3d040" · use gameultra3d040

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d041

import "std/c/gameultra3d041" · use gameultra3d041

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d042

import "std/c/gameultra3d042" · use gameultra3d042

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d043

import "std/c/gameultra3d043" · use gameultra3d043

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d044

import "std/c/gameultra3d044" · use gameultra3d044

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d045

import "std/c/gameultra3d045" · use gameultra3d045

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d046

import "std/c/gameultra3d046" · use gameultra3d046

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d047

import "std/c/gameultra3d047" · use gameultra3d047

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d048

import "std/c/gameultra3d048" · use gameultra3d048

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d049

import "std/c/gameultra3d049" · use gameultra3d049

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d050

import "std/c/gameultra3d050" · use gameultra3d050

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d051

import "std/c/gameultra3d051" · use gameultra3d051

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d052

import "std/c/gameultra3d052" · use gameultra3d052

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d053

import "std/c/gameultra3d053" · use gameultra3d053

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d054

import "std/c/gameultra3d054" · use gameultra3d054

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d055

import "std/c/gameultra3d055" · use gameultra3d055

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d056

import "std/c/gameultra3d056" · use gameultra3d056

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d057

import "std/c/gameultra3d057" · use gameultra3d057

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d058

import "std/c/gameultra3d058" · use gameultra3d058

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d059

import "std/c/gameultra3d059" · use gameultra3d059

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d060

import "std/c/gameultra3d060" · use gameultra3d060

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d061

import "std/c/gameultra3d061" · use gameultra3d061

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d062

import "std/c/gameultra3d062" · use gameultra3d062

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d063

import "std/c/gameultra3d063" · use gameultra3d063

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d064

import "std/c/gameultra3d064" · use gameultra3d064

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d065

import "std/c/gameultra3d065" · use gameultra3d065

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d066

import "std/c/gameultra3d066" · use gameultra3d066

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d067

import "std/c/gameultra3d067" · use gameultra3d067

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d068

import "std/c/gameultra3d068" · use gameultra3d068

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d069

import "std/c/gameultra3d069" · use gameultra3d069

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d070

import "std/c/gameultra3d070" · use gameultra3d070

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d071

import "std/c/gameultra3d071" · use gameultra3d071

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d072

import "std/c/gameultra3d072" · use gameultra3d072

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d073

import "std/c/gameultra3d073" · use gameultra3d073

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d074

import "std/c/gameultra3d074" · use gameultra3d074

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d075

import "std/c/gameultra3d075" · use gameultra3d075

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d076

import "std/c/gameultra3d076" · use gameultra3d076

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d077

import "std/c/gameultra3d077" · use gameultra3d077

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d078

import "std/c/gameultra3d078" · use gameultra3d078

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d079

import "std/c/gameultra3d079" · use gameultra3d079

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d080

import "std/c/gameultra3d080" · use gameultra3d080

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d081

import "std/c/gameultra3d081" · use gameultra3d081

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d082

import "std/c/gameultra3d082" · use gameultra3d082

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d083

import "std/c/gameultra3d083" · use gameultra3d083

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d084

import "std/c/gameultra3d084" · use gameultra3d084

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d085

import "std/c/gameultra3d085" · use gameultra3d085

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d086

import "std/c/gameultra3d086" · use gameultra3d086

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d087

import "std/c/gameultra3d087" · use gameultra3d087

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d088

import "std/c/gameultra3d088" · use gameultra3d088

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d089

import "std/c/gameultra3d089" · use gameultra3d089

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d090

import "std/c/gameultra3d090" · use gameultra3d090

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d091

import "std/c/gameultra3d091" · use gameultra3d091

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d092

import "std/c/gameultra3d092" · use gameultra3d092

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d093

import "std/c/gameultra3d093" · use gameultra3d093

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d094

import "std/c/gameultra3d094" · use gameultra3d094

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d095

import "std/c/gameultra3d095" · use gameultra3d095

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d096

import "std/c/gameultra3d096" · use gameultra3d096

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d097

import "std/c/gameultra3d097" · use gameultra3d097

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d098

import "std/c/gameultra3d098" · use gameultra3d098

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d099

import "std/c/gameultra3d099" · use gameultra3d099

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d100

import "std/c/gameultra3d100" · use gameultra3d100

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d101

import "std/c/gameultra3d101" · use gameultra3d101

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d102

import "std/c/gameultra3d102" · use gameultra3d102

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d103

import "std/c/gameultra3d103" · use gameultra3d103

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d104

import "std/c/gameultra3d104" · use gameultra3d104

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d105

import "std/c/gameultra3d105" · use gameultra3d105

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d106

import "std/c/gameultra3d106" · use gameultra3d106

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d107

import "std/c/gameultra3d107" · use gameultra3d107

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d108

import "std/c/gameultra3d108" · use gameultra3d108

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d109

import "std/c/gameultra3d109" · use gameultra3d109

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d110

import "std/c/gameultra3d110" · use gameultra3d110

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d111

import "std/c/gameultra3d111" · use gameultra3d111

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d112

import "std/c/gameultra3d112" · use gameultra3d112

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d113

import "std/c/gameultra3d113" · use gameultra3d113

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d114

import "std/c/gameultra3d114" · use gameultra3d114

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d115

import "std/c/gameultra3d115" · use gameultra3d115

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d116

import "std/c/gameultra3d116" · use gameultra3d116

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d117

import "std/c/gameultra3d117" · use gameultra3d117

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d118

import "std/c/gameultra3d118" · use gameultra3d118

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d119

import "std/c/gameultra3d119" · use gameultra3d119

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d120

import "std/c/gameultra3d120" · use gameultra3d120

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d121

import "std/c/gameultra3d121" · use gameultra3d121

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d122

import "std/c/gameultra3d122" · use gameultra3d122

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d123

import "std/c/gameultra3d123" · use gameultra3d123

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d124

import "std/c/gameultra3d124" · use gameultra3d124

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d125

import "std/c/gameultra3d125" · use gameultra3d125

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d126

import "std/c/gameultra3d126" · use gameultra3d126

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d127

import "std/c/gameultra3d127" · use gameultra3d127

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d128

import "std/c/gameultra3d128" · use gameultra3d128

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d129

import "std/c/gameultra3d129" · use gameultra3d129

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d130

import "std/c/gameultra3d130" · use gameultra3d130

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d131

import "std/c/gameultra3d131" · use gameultra3d131

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d132

import "std/c/gameultra3d132" · use gameultra3d132

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d133

import "std/c/gameultra3d133" · use gameultra3d133

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d134

import "std/c/gameultra3d134" · use gameultra3d134

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d135

import "std/c/gameultra3d135" · use gameultra3d135

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d136

import "std/c/gameultra3d136" · use gameultra3d136

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d137

import "std/c/gameultra3d137" · use gameultra3d137

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d138

import "std/c/gameultra3d138" · use gameultra3d138

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d139

import "std/c/gameultra3d139" · use gameultra3d139

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d140

import "std/c/gameultra3d140" · use gameultra3d140

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d141

import "std/c/gameultra3d141" · use gameultra3d141

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d142

import "std/c/gameultra3d142" · use gameultra3d142

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d143

import "std/c/gameultra3d143" · use gameultra3d143

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d144

import "std/c/gameultra3d144" · use gameultra3d144

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d145

import "std/c/gameultra3d145" · use gameultra3d145

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d146

import "std/c/gameultra3d146" · use gameultra3d146

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d147

import "std/c/gameultra3d147" · use gameultra3d147

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d148

import "std/c/gameultra3d148" · use gameultra3d148

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d149

import "std/c/gameultra3d149" · use gameultra3d149

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d150

import "std/c/gameultra3d150" · use gameultra3d150

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d151

import "std/c/gameultra3d151" · use gameultra3d151

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d152

import "std/c/gameultra3d152" · use gameultra3d152

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d153

import "std/c/gameultra3d153" · use gameultra3d153

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d154

import "std/c/gameultra3d154" · use gameultra3d154

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d155

import "std/c/gameultra3d155" · use gameultra3d155

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d156

import "std/c/gameultra3d156" · use gameultra3d156

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d157

import "std/c/gameultra3d157" · use gameultra3d157

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d158

import "std/c/gameultra3d158" · use gameultra3d158

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d159

import "std/c/gameultra3d159" · use gameultra3d159

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d160

import "std/c/gameultra3d160" · use gameultra3d160

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d161

import "std/c/gameultra3d161" · use gameultra3d161

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d162

import "std/c/gameultra3d162" · use gameultra3d162

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d163

import "std/c/gameultra3d163" · use gameultra3d163

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d164

import "std/c/gameultra3d164" · use gameultra3d164

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d165

import "std/c/gameultra3d165" · use gameultra3d165

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d166

import "std/c/gameultra3d166" · use gameultra3d166

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d167

import "std/c/gameultra3d167" · use gameultra3d167

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d168

import "std/c/gameultra3d168" · use gameultra3d168

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d169

import "std/c/gameultra3d169" · use gameultra3d169

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d170

import "std/c/gameultra3d170" · use gameultra3d170

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d171

import "std/c/gameultra3d171" · use gameultra3d171

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d172

import "std/c/gameultra3d172" · use gameultra3d172

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d173

import "std/c/gameultra3d173" · use gameultra3d173

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d174

import "std/c/gameultra3d174" · use gameultra3d174

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d175

import "std/c/gameultra3d175" · use gameultra3d175

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d176

import "std/c/gameultra3d176" · use gameultra3d176

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d177

import "std/c/gameultra3d177" · use gameultra3d177

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d178

import "std/c/gameultra3d178" · use gameultra3d178

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d179

import "std/c/gameultra3d179" · use gameultra3d179

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d180

import "std/c/gameultra3d180" · use gameultra3d180

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d181

import "std/c/gameultra3d181" · use gameultra3d181

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d182

import "std/c/gameultra3d182" · use gameultra3d182

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d183

import "std/c/gameultra3d183" · use gameultra3d183

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d184

import "std/c/gameultra3d184" · use gameultra3d184

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d185

import "std/c/gameultra3d185" · use gameultra3d185

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d186

import "std/c/gameultra3d186" · use gameultra3d186

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d187

import "std/c/gameultra3d187" · use gameultra3d187

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d188

import "std/c/gameultra3d188" · use gameultra3d188

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d189

import "std/c/gameultra3d189" · use gameultra3d189

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d190

import "std/c/gameultra3d190" · use gameultra3d190

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d191

import "std/c/gameultra3d191" · use gameultra3d191

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d192

import "std/c/gameultra3d192" · use gameultra3d192

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d193

import "std/c/gameultra3d193" · use gameultra3d193

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d194

import "std/c/gameultra3d194" · use gameultra3d194

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d195

import "std/c/gameultra3d195" · use gameultra3d195

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d196

import "std/c/gameultra3d196" · use gameultra3d196

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d197

import "std/c/gameultra3d197" · use gameultra3d197

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d198

import "std/c/gameultra3d198" · use gameultra3d198

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d199

import "std/c/gameultra3d199" · use gameultra3d199

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d200

import "std/c/gameultra3d200" · use gameultra3d200

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d201

import "std/c/gameultra3d201" · use gameultra3d201

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d202

import "std/c/gameultra3d202" · use gameultra3d202

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d203

import "std/c/gameultra3d203" · use gameultra3d203

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d204

import "std/c/gameultra3d204" · use gameultra3d204

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d205

import "std/c/gameultra3d205" · use gameultra3d205

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d206

import "std/c/gameultra3d206" · use gameultra3d206

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d207

import "std/c/gameultra3d207" · use gameultra3d207

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d208

import "std/c/gameultra3d208" · use gameultra3d208

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d209

import "std/c/gameultra3d209" · use gameultra3d209

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d210

import "std/c/gameultra3d210" · use gameultra3d210

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d211

import "std/c/gameultra3d211" · use gameultra3d211

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d212

import "std/c/gameultra3d212" · use gameultra3d212

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d213

import "std/c/gameultra3d213" · use gameultra3d213

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d214

import "std/c/gameultra3d214" · use gameultra3d214

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d215

import "std/c/gameultra3d215" · use gameultra3d215

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d216

import "std/c/gameultra3d216" · use gameultra3d216

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d217

import "std/c/gameultra3d217" · use gameultra3d217

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d218

import "std/c/gameultra3d218" · use gameultra3d218

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d219

import "std/c/gameultra3d219" · use gameultra3d219

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d220

import "std/c/gameultra3d220" · use gameultra3d220

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d221

import "std/c/gameultra3d221" · use gameultra3d221

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d222

import "std/c/gameultra3d222" · use gameultra3d222

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d223

import "std/c/gameultra3d223" · use gameultra3d223

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d224

import "std/c/gameultra3d224" · use gameultra3d224

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d225

import "std/c/gameultra3d225" · use gameultra3d225

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d226

import "std/c/gameultra3d226" · use gameultra3d226

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d227

import "std/c/gameultra3d227" · use gameultra3d227

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d228

import "std/c/gameultra3d228" · use gameultra3d228

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d229

import "std/c/gameultra3d229" · use gameultra3d229

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d230

import "std/c/gameultra3d230" · use gameultra3d230

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d231

import "std/c/gameultra3d231" · use gameultra3d231

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d232

import "std/c/gameultra3d232" · use gameultra3d232

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d233

import "std/c/gameultra3d233" · use gameultra3d233

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d234

import "std/c/gameultra3d234" · use gameultra3d234

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d235

import "std/c/gameultra3d235" · use gameultra3d235

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d236

import "std/c/gameultra3d236" · use gameultra3d236

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d237

import "std/c/gameultra3d237" · use gameultra3d237

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d238

import "std/c/gameultra3d238" · use gameultra3d238

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d239

import "std/c/gameultra3d239" · use gameultra3d239

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d240

import "std/c/gameultra3d240" · use gameultra3d240

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d241

import "std/c/gameultra3d241" · use gameultra3d241

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d242

import "std/c/gameultra3d242" · use gameultra3d242

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d243

import "std/c/gameultra3d243" · use gameultra3d243

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d244

import "std/c/gameultra3d244" · use gameultra3d244

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d245

import "std/c/gameultra3d245" · use gameultra3d245

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d246

import "std/c/gameultra3d246" · use gameultra3d246

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d247

import "std/c/gameultra3d247" · use gameultra3d247

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d248

import "std/c/gameultra3d248" · use gameultra3d248

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d249

import "std/c/gameultra3d249" · use gameultra3d249

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d250

import "std/c/gameultra3d250" · use gameultra3d250

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d251

import "std/c/gameultra3d251" · use gameultra3d251

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d252

import "std/c/gameultra3d252" · use gameultra3d252

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d253

import "std/c/gameultra3d253" · use gameultra3d253

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d254

import "std/c/gameultra3d254" · use gameultra3d254

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d255

import "std/c/gameultra3d255" · use gameultra3d255

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d256

import "std/c/gameultra3d256" · use gameultra3d256

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d257

import "std/c/gameultra3d257" · use gameultra3d257

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d258

import "std/c/gameultra3d258" · use gameultra3d258

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d259

import "std/c/gameultra3d259" · use gameultra3d259

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d260

import "std/c/gameultra3d260" · use gameultra3d260

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d261

import "std/c/gameultra3d261" · use gameultra3d261

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d262

import "std/c/gameultra3d262" · use gameultra3d262

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d263

import "std/c/gameultra3d263" · use gameultra3d263

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d264

import "std/c/gameultra3d264" · use gameultra3d264

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d265

import "std/c/gameultra3d265" · use gameultra3d265

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d266

import "std/c/gameultra3d266" · use gameultra3d266

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d267

import "std/c/gameultra3d267" · use gameultra3d267

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d268

import "std/c/gameultra3d268" · use gameultra3d268

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d269

import "std/c/gameultra3d269" · use gameultra3d269

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d270

import "std/c/gameultra3d270" · use gameultra3d270

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d271

import "std/c/gameultra3d271" · use gameultra3d271

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d272

import "std/c/gameultra3d272" · use gameultra3d272

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d273

import "std/c/gameultra3d273" · use gameultra3d273

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d274

import "std/c/gameultra3d274" · use gameultra3d274

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d275

import "std/c/gameultra3d275" · use gameultra3d275

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d276

import "std/c/gameultra3d276" · use gameultra3d276

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d277

import "std/c/gameultra3d277" · use gameultra3d277

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d278

import "std/c/gameultra3d278" · use gameultra3d278

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d279

import "std/c/gameultra3d279" · use gameultra3d279

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d280

import "std/c/gameultra3d280" · use gameultra3d280

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d281

import "std/c/gameultra3d281" · use gameultra3d281

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d282

import "std/c/gameultra3d282" · use gameultra3d282

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d283

import "std/c/gameultra3d283" · use gameultra3d283

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d284

import "std/c/gameultra3d284" · use gameultra3d284

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d285

import "std/c/gameultra3d285" · use gameultra3d285

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d286

import "std/c/gameultra3d286" · use gameultra3d286

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d287

import "std/c/gameultra3d287" · use gameultra3d287

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d288

import "std/c/gameultra3d288" · use gameultra3d288

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d289

import "std/c/gameultra3d289" · use gameultra3d289

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d290

import "std/c/gameultra3d290" · use gameultra3d290

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d291

import "std/c/gameultra3d291" · use gameultra3d291

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d292

import "std/c/gameultra3d292" · use gameultra3d292

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d293

import "std/c/gameultra3d293" · use gameultra3d293

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d294

import "std/c/gameultra3d294" · use gameultra3d294

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d295

import "std/c/gameultra3d295" · use gameultra3d295

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d296

import "std/c/gameultra3d296" · use gameultra3d296

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d297

import "std/c/gameultra3d297" · use gameultra3d297

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d298

import "std/c/gameultra3d298" · use gameultra3d298

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d299

import "std/c/gameultra3d299" · use gameultra3d299

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d300

import "std/c/gameultra3d300" · use gameultra3d300

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d301

import "std/c/gameultra3d301" · use gameultra3d301

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d302

import "std/c/gameultra3d302" · use gameultra3d302

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d303

import "std/c/gameultra3d303" · use gameultra3d303

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d304

import "std/c/gameultra3d304" · use gameultra3d304

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d305

import "std/c/gameultra3d305" · use gameultra3d305

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d306

import "std/c/gameultra3d306" · use gameultra3d306

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d307

import "std/c/gameultra3d307" · use gameultra3d307

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d308

import "std/c/gameultra3d308" · use gameultra3d308

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d309

import "std/c/gameultra3d309" · use gameultra3d309

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d310

import "std/c/gameultra3d310" · use gameultra3d310

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d311

import "std/c/gameultra3d311" · use gameultra3d311

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d312

import "std/c/gameultra3d312" · use gameultra3d312

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d313

import "std/c/gameultra3d313" · use gameultra3d313

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d314

import "std/c/gameultra3d314" · use gameultra3d314

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d315

import "std/c/gameultra3d315" · use gameultra3d315

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d316

import "std/c/gameultra3d316" · use gameultra3d316

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d317

import "std/c/gameultra3d317" · use gameultra3d317

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d318

import "std/c/gameultra3d318" · use gameultra3d318

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d319

import "std/c/gameultra3d319" · use gameultra3d319

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d320

import "std/c/gameultra3d320" · use gameultra3d320

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d321

import "std/c/gameultra3d321" · use gameultra3d321

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d322

import "std/c/gameultra3d322" · use gameultra3d322

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d323

import "std/c/gameultra3d323" · use gameultra3d323

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d324

import "std/c/gameultra3d324" · use gameultra3d324

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d325

import "std/c/gameultra3d325" · use gameultra3d325

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d326

import "std/c/gameultra3d326" · use gameultra3d326

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d327

import "std/c/gameultra3d327" · use gameultra3d327

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d328

import "std/c/gameultra3d328" · use gameultra3d328

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d329

import "std/c/gameultra3d329" · use gameultra3d329

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d330

import "std/c/gameultra3d330" · use gameultra3d330

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d331

import "std/c/gameultra3d331" · use gameultra3d331

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d332

import "std/c/gameultra3d332" · use gameultra3d332

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d333

import "std/c/gameultra3d333" · use gameultra3d333

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d334

import "std/c/gameultra3d334" · use gameultra3d334

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d335

import "std/c/gameultra3d335" · use gameultra3d335

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d336

import "std/c/gameultra3d336" · use gameultra3d336

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d337

import "std/c/gameultra3d337" · use gameultra3d337

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d338

import "std/c/gameultra3d338" · use gameultra3d338

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d339

import "std/c/gameultra3d339" · use gameultra3d339

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d340

import "std/c/gameultra3d340" · use gameultra3d340

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d341

import "std/c/gameultra3d341" · use gameultra3d341

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d342

import "std/c/gameultra3d342" · use gameultra3d342

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d343

import "std/c/gameultra3d343" · use gameultra3d343

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d344

import "std/c/gameultra3d344" · use gameultra3d344

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d345

import "std/c/gameultra3d345" · use gameultra3d345

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d346

import "std/c/gameultra3d346" · use gameultra3d346

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d347

import "std/c/gameultra3d347" · use gameultra3d347

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d348

import "std/c/gameultra3d348" · use gameultra3d348

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d349

import "std/c/gameultra3d349" · use gameultra3d349

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d350

import "std/c/gameultra3d350" · use gameultra3d350

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d351

import "std/c/gameultra3d351" · use gameultra3d351

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d352

import "std/c/gameultra3d352" · use gameultra3d352

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d353

import "std/c/gameultra3d353" · use gameultra3d353

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d354

import "std/c/gameultra3d354" · use gameultra3d354

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d355

import "std/c/gameultra3d355" · use gameultra3d355

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d356

import "std/c/gameultra3d356" · use gameultra3d356

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d357

import "std/c/gameultra3d357" · use gameultra3d357

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d358

import "std/c/gameultra3d358" · use gameultra3d358

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d359

import "std/c/gameultra3d359" · use gameultra3d359

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d360

import "std/c/gameultra3d360" · use gameultra3d360

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d361

import "std/c/gameultra3d361" · use gameultra3d361

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d362

import "std/c/gameultra3d362" · use gameultra3d362

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d363

import "std/c/gameultra3d363" · use gameultra3d363

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d364

import "std/c/gameultra3d364" · use gameultra3d364

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d365

import "std/c/gameultra3d365" · use gameultra3d365

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d366

import "std/c/gameultra3d366" · use gameultra3d366

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d367

import "std/c/gameultra3d367" · use gameultra3d367

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d368

import "std/c/gameultra3d368" · use gameultra3d368

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d369

import "std/c/gameultra3d369" · use gameultra3d369

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d370

import "std/c/gameultra3d370" · use gameultra3d370

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d371

import "std/c/gameultra3d371" · use gameultra3d371

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d372

import "std/c/gameultra3d372" · use gameultra3d372

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d373

import "std/c/gameultra3d373" · use gameultra3d373

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d374

import "std/c/gameultra3d374" · use gameultra3d374

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d375

import "std/c/gameultra3d375" · use gameultra3d375

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d376

import "std/c/gameultra3d376" · use gameultra3d376

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d377

import "std/c/gameultra3d377" · use gameultra3d377

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d378

import "std/c/gameultra3d378" · use gameultra3d378

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d379

import "std/c/gameultra3d379" · use gameultra3d379

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d380

import "std/c/gameultra3d380" · use gameultra3d380

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d381

import "std/c/gameultra3d381" · use gameultra3d381

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d382

import "std/c/gameultra3d382" · use gameultra3d382

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d383

import "std/c/gameultra3d383" · use gameultra3d383

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d384

import "std/c/gameultra3d384" · use gameultra3d384

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d385

import "std/c/gameultra3d385" · use gameultra3d385

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d386

import "std/c/gameultra3d386" · use gameultra3d386

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d387

import "std/c/gameultra3d387" · use gameultra3d387

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d388

import "std/c/gameultra3d388" · use gameultra3d388

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d389

import "std/c/gameultra3d389" · use gameultra3d389

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d390

import "std/c/gameultra3d390" · use gameultra3d390

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d391

import "std/c/gameultra3d391" · use gameultra3d391

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d392

import "std/c/gameultra3d392" · use gameultra3d392

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d393

import "std/c/gameultra3d393" · use gameultra3d393

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d394

import "std/c/gameultra3d394" · use gameultra3d394

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d395

import "std/c/gameultra3d395" · use gameultra3d395

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d396

import "std/c/gameultra3d396" · use gameultra3d396

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d397

import "std/c/gameultra3d397" · use gameultra3d397

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d398

import "std/c/gameultra3d398" · use gameultra3d398

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d399

import "std/c/gameultra3d399" · use gameultra3d399

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d400

import "std/c/gameultra3d400" · use gameultra3d400

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d401

import "std/c/gameultra3d401" · use gameultra3d401

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d402

import "std/c/gameultra3d402" · use gameultra3d402

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d403

import "std/c/gameultra3d403" · use gameultra3d403

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d404

import "std/c/gameultra3d404" · use gameultra3d404

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d405

import "std/c/gameultra3d405" · use gameultra3d405

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d406

import "std/c/gameultra3d406" · use gameultra3d406

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d407

import "std/c/gameultra3d407" · use gameultra3d407

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d408

import "std/c/gameultra3d408" · use gameultra3d408

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d409

import "std/c/gameultra3d409" · use gameultra3d409

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d410

import "std/c/gameultra3d410" · use gameultra3d410

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d411

import "std/c/gameultra3d411" · use gameultra3d411

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d412

import "std/c/gameultra3d412" · use gameultra3d412

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d413

import "std/c/gameultra3d413" · use gameultra3d413

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d414

import "std/c/gameultra3d414" · use gameultra3d414

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d415

import "std/c/gameultra3d415" · use gameultra3d415

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d416

import "std/c/gameultra3d416" · use gameultra3d416

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d417

import "std/c/gameultra3d417" · use gameultra3d417

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d418

import "std/c/gameultra3d418" · use gameultra3d418

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d419

import "std/c/gameultra3d419" · use gameultra3d419

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d420

import "std/c/gameultra3d420" · use gameultra3d420

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d421

import "std/c/gameultra3d421" · use gameultra3d421

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d422

import "std/c/gameultra3d422" · use gameultra3d422

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d423

import "std/c/gameultra3d423" · use gameultra3d423

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d424

import "std/c/gameultra3d424" · use gameultra3d424

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d425

import "std/c/gameultra3d425" · use gameultra3d425

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d426

import "std/c/gameultra3d426" · use gameultra3d426

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d427

import "std/c/gameultra3d427" · use gameultra3d427

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d428

import "std/c/gameultra3d428" · use gameultra3d428

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d429

import "std/c/gameultra3d429" · use gameultra3d429

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d430

import "std/c/gameultra3d430" · use gameultra3d430

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d431

import "std/c/gameultra3d431" · use gameultra3d431

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d432

import "std/c/gameultra3d432" · use gameultra3d432

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d433

import "std/c/gameultra3d433" · use gameultra3d433

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d434

import "std/c/gameultra3d434" · use gameultra3d434

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d435

import "std/c/gameultra3d435" · use gameultra3d435

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d436

import "std/c/gameultra3d436" · use gameultra3d436

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d437

import "std/c/gameultra3d437" · use gameultra3d437

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d438

import "std/c/gameultra3d438" · use gameultra3d438

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d439

import "std/c/gameultra3d439" · use gameultra3d439

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d440

import "std/c/gameultra3d440" · use gameultra3d440

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d441

import "std/c/gameultra3d441" · use gameultra3d441

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d442

import "std/c/gameultra3d442" · use gameultra3d442

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d443

import "std/c/gameultra3d443" · use gameultra3d443

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d444

import "std/c/gameultra3d444" · use gameultra3d444

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d445

import "std/c/gameultra3d445" · use gameultra3d445

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d446

import "std/c/gameultra3d446" · use gameultra3d446

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d447

import "std/c/gameultra3d447" · use gameultra3d447

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d448

import "std/c/gameultra3d448" · use gameultra3d448

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d449

import "std/c/gameultra3d449" · use gameultra3d449

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d450

import "std/c/gameultra3d450" · use gameultra3d450

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d451

import "std/c/gameultra3d451" · use gameultra3d451

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d452

import "std/c/gameultra3d452" · use gameultra3d452

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d453

import "std/c/gameultra3d453" · use gameultra3d453

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d454

import "std/c/gameultra3d454" · use gameultra3d454

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d455

import "std/c/gameultra3d455" · use gameultra3d455

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d456

import "std/c/gameultra3d456" · use gameultra3d456

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d457

import "std/c/gameultra3d457" · use gameultra3d457

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d458

import "std/c/gameultra3d458" · use gameultra3d458

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d459

import "std/c/gameultra3d459" · use gameultra3d459

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d460

import "std/c/gameultra3d460" · use gameultra3d460

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d461

import "std/c/gameultra3d461" · use gameultra3d461

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d462

import "std/c/gameultra3d462" · use gameultra3d462

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d463

import "std/c/gameultra3d463" · use gameultra3d463

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d464

import "std/c/gameultra3d464" · use gameultra3d464

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d465

import "std/c/gameultra3d465" · use gameultra3d465

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d466

import "std/c/gameultra3d466" · use gameultra3d466

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d467

import "std/c/gameultra3d467" · use gameultra3d467

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d468

import "std/c/gameultra3d468" · use gameultra3d468

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d469

import "std/c/gameultra3d469" · use gameultra3d469

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d470

import "std/c/gameultra3d470" · use gameultra3d470

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d471

import "std/c/gameultra3d471" · use gameultra3d471

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d472

import "std/c/gameultra3d472" · use gameultra3d472

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d473

import "std/c/gameultra3d473" · use gameultra3d473

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d474

import "std/c/gameultra3d474" · use gameultra3d474

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d475

import "std/c/gameultra3d475" · use gameultra3d475

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d476

import "std/c/gameultra3d476" · use gameultra3d476

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d477

import "std/c/gameultra3d477" · use gameultra3d477

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d478

import "std/c/gameultra3d478" · use gameultra3d478

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d479

import "std/c/gameultra3d479" · use gameultra3d479

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d480

import "std/c/gameultra3d480" · use gameultra3d480

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d481

import "std/c/gameultra3d481" · use gameultra3d481

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d482

import "std/c/gameultra3d482" · use gameultra3d482

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d483

import "std/c/gameultra3d483" · use gameultra3d483

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d484

import "std/c/gameultra3d484" · use gameultra3d484

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d485

import "std/c/gameultra3d485" · use gameultra3d485

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d486

import "std/c/gameultra3d486" · use gameultra3d486

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d487

import "std/c/gameultra3d487" · use gameultra3d487

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d488

import "std/c/gameultra3d488" · use gameultra3d488

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d489

import "std/c/gameultra3d489" · use gameultra3d489

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d490

import "std/c/gameultra3d490" · use gameultra3d490

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d491

import "std/c/gameultra3d491" · use gameultra3d491

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d492

import "std/c/gameultra3d492" · use gameultra3d492

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d493

import "std/c/gameultra3d493" · use gameultra3d493

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d494

import "std/c/gameultra3d494" · use gameultra3d494

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d495

import "std/c/gameultra3d495" · use gameultra3d495

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d496

import "std/c/gameultra3d496" · use gameultra3d496

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d497

import "std/c/gameultra3d497" · use gameultra3d497

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d498

import "std/c/gameultra3d498" · use gameultra3d498

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d499

import "std/c/gameultra3d499" · use gameultra3d499

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3d500

import "std/c/gameultra3d500" · use gameultra3d500

- `integratePos(pos list number, vel list number, n number, dt number) → list number`
- `steerSeek(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `boidsVelStep(pos list number, vel list number, n number, dt number, neighborDist number, sepDist number, maxSpeed number) → list number`
- `raySphereHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, r number) → list number`

std/c/gameultra3dpro001

import "std/c/gameultra3dpro001" · use gameultra3dpro001

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro002

import "std/c/gameultra3dpro002" · use gameultra3dpro002

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro003

import "std/c/gameultra3dpro003" · use gameultra3dpro003

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro004

import "std/c/gameultra3dpro004" · use gameultra3dpro004

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro005

import "std/c/gameultra3dpro005" · use gameultra3dpro005

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro006

import "std/c/gameultra3dpro006" · use gameultra3dpro006

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro007

import "std/c/gameultra3dpro007" · use gameultra3dpro007

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro008

import "std/c/gameultra3dpro008" · use gameultra3dpro008

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro009

import "std/c/gameultra3dpro009" · use gameultra3dpro009

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro010

import "std/c/gameultra3dpro010" · use gameultra3dpro010

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro011

import "std/c/gameultra3dpro011" · use gameultra3dpro011

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro012

import "std/c/gameultra3dpro012" · use gameultra3dpro012

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro013

import "std/c/gameultra3dpro013" · use gameultra3dpro013

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro014

import "std/c/gameultra3dpro014" · use gameultra3dpro014

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro015

import "std/c/gameultra3dpro015" · use gameultra3dpro015

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro016

import "std/c/gameultra3dpro016" · use gameultra3dpro016

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro017

import "std/c/gameultra3dpro017" · use gameultra3dpro017

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro018

import "std/c/gameultra3dpro018" · use gameultra3dpro018

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro019

import "std/c/gameultra3dpro019" · use gameultra3dpro019

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro020

import "std/c/gameultra3dpro020" · use gameultra3dpro020

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro021

import "std/c/gameultra3dpro021" · use gameultra3dpro021

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro022

import "std/c/gameultra3dpro022" · use gameultra3dpro022

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro023

import "std/c/gameultra3dpro023" · use gameultra3dpro023

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro024

import "std/c/gameultra3dpro024" · use gameultra3dpro024

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro025

import "std/c/gameultra3dpro025" · use gameultra3dpro025

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro026

import "std/c/gameultra3dpro026" · use gameultra3dpro026

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro027

import "std/c/gameultra3dpro027" · use gameultra3dpro027

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro028

import "std/c/gameultra3dpro028" · use gameultra3dpro028

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro029

import "std/c/gameultra3dpro029" · use gameultra3dpro029

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro030

import "std/c/gameultra3dpro030" · use gameultra3dpro030

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro031

import "std/c/gameultra3dpro031" · use gameultra3dpro031

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro032

import "std/c/gameultra3dpro032" · use gameultra3dpro032

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro033

import "std/c/gameultra3dpro033" · use gameultra3dpro033

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro034

import "std/c/gameultra3dpro034" · use gameultra3dpro034

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro035

import "std/c/gameultra3dpro035" · use gameultra3dpro035

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro036

import "std/c/gameultra3dpro036" · use gameultra3dpro036

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro037

import "std/c/gameultra3dpro037" · use gameultra3dpro037

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro038

import "std/c/gameultra3dpro038" · use gameultra3dpro038

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro039

import "std/c/gameultra3dpro039" · use gameultra3dpro039

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro040

import "std/c/gameultra3dpro040" · use gameultra3dpro040

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro041

import "std/c/gameultra3dpro041" · use gameultra3dpro041

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro042

import "std/c/gameultra3dpro042" · use gameultra3dpro042

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro043

import "std/c/gameultra3dpro043" · use gameultra3dpro043

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro044

import "std/c/gameultra3dpro044" · use gameultra3dpro044

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro045

import "std/c/gameultra3dpro045" · use gameultra3dpro045

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro046

import "std/c/gameultra3dpro046" · use gameultra3dpro046

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro047

import "std/c/gameultra3dpro047" · use gameultra3dpro047

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro048

import "std/c/gameultra3dpro048" · use gameultra3dpro048

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro049

import "std/c/gameultra3dpro049" · use gameultra3dpro049

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro050

import "std/c/gameultra3dpro050" · use gameultra3dpro050

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro051

import "std/c/gameultra3dpro051" · use gameultra3dpro051

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro052

import "std/c/gameultra3dpro052" · use gameultra3dpro052

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro053

import "std/c/gameultra3dpro053" · use gameultra3dpro053

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro054

import "std/c/gameultra3dpro054" · use gameultra3dpro054

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro055

import "std/c/gameultra3dpro055" · use gameultra3dpro055

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro056

import "std/c/gameultra3dpro056" · use gameultra3dpro056

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro057

import "std/c/gameultra3dpro057" · use gameultra3dpro057

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro058

import "std/c/gameultra3dpro058" · use gameultra3dpro058

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro059

import "std/c/gameultra3dpro059" · use gameultra3dpro059

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro060

import "std/c/gameultra3dpro060" · use gameultra3dpro060

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro061

import "std/c/gameultra3dpro061" · use gameultra3dpro061

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro062

import "std/c/gameultra3dpro062" · use gameultra3dpro062

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro063

import "std/c/gameultra3dpro063" · use gameultra3dpro063

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro064

import "std/c/gameultra3dpro064" · use gameultra3dpro064

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro065

import "std/c/gameultra3dpro065" · use gameultra3dpro065

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro066

import "std/c/gameultra3dpro066" · use gameultra3dpro066

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro067

import "std/c/gameultra3dpro067" · use gameultra3dpro067

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro068

import "std/c/gameultra3dpro068" · use gameultra3dpro068

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro069

import "std/c/gameultra3dpro069" · use gameultra3dpro069

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro070

import "std/c/gameultra3dpro070" · use gameultra3dpro070

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro071

import "std/c/gameultra3dpro071" · use gameultra3dpro071

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro072

import "std/c/gameultra3dpro072" · use gameultra3dpro072

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro073

import "std/c/gameultra3dpro073" · use gameultra3dpro073

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro074

import "std/c/gameultra3dpro074" · use gameultra3dpro074

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro075

import "std/c/gameultra3dpro075" · use gameultra3dpro075

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro076

import "std/c/gameultra3dpro076" · use gameultra3dpro076

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro077

import "std/c/gameultra3dpro077" · use gameultra3dpro077

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro078

import "std/c/gameultra3dpro078" · use gameultra3dpro078

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro079

import "std/c/gameultra3dpro079" · use gameultra3dpro079

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro080

import "std/c/gameultra3dpro080" · use gameultra3dpro080

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro081

import "std/c/gameultra3dpro081" · use gameultra3dpro081

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro082

import "std/c/gameultra3dpro082" · use gameultra3dpro082

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro083

import "std/c/gameultra3dpro083" · use gameultra3dpro083

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro084

import "std/c/gameultra3dpro084" · use gameultra3dpro084

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro085

import "std/c/gameultra3dpro085" · use gameultra3dpro085

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro086

import "std/c/gameultra3dpro086" · use gameultra3dpro086

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro087

import "std/c/gameultra3dpro087" · use gameultra3dpro087

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro088

import "std/c/gameultra3dpro088" · use gameultra3dpro088

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro089

import "std/c/gameultra3dpro089" · use gameultra3dpro089

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro090

import "std/c/gameultra3dpro090" · use gameultra3dpro090

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro091

import "std/c/gameultra3dpro091" · use gameultra3dpro091

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro092

import "std/c/gameultra3dpro092" · use gameultra3dpro092

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro093

import "std/c/gameultra3dpro093" · use gameultra3dpro093

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro094

import "std/c/gameultra3dpro094" · use gameultra3dpro094

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro095

import "std/c/gameultra3dpro095" · use gameultra3dpro095

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro096

import "std/c/gameultra3dpro096" · use gameultra3dpro096

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro097

import "std/c/gameultra3dpro097" · use gameultra3dpro097

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro098

import "std/c/gameultra3dpro098" · use gameultra3dpro098

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro099

import "std/c/gameultra3dpro099" · use gameultra3dpro099

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro100

import "std/c/gameultra3dpro100" · use gameultra3dpro100

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro101

import "std/c/gameultra3dpro101" · use gameultra3dpro101

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro102

import "std/c/gameultra3dpro102" · use gameultra3dpro102

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro103

import "std/c/gameultra3dpro103" · use gameultra3dpro103

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro104

import "std/c/gameultra3dpro104" · use gameultra3dpro104

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro105

import "std/c/gameultra3dpro105" · use gameultra3dpro105

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro106

import "std/c/gameultra3dpro106" · use gameultra3dpro106

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro107

import "std/c/gameultra3dpro107" · use gameultra3dpro107

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro108

import "std/c/gameultra3dpro108" · use gameultra3dpro108

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro109

import "std/c/gameultra3dpro109" · use gameultra3dpro109

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro110

import "std/c/gameultra3dpro110" · use gameultra3dpro110

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro111

import "std/c/gameultra3dpro111" · use gameultra3dpro111

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro112

import "std/c/gameultra3dpro112" · use gameultra3dpro112

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro113

import "std/c/gameultra3dpro113" · use gameultra3dpro113

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro114

import "std/c/gameultra3dpro114" · use gameultra3dpro114

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro115

import "std/c/gameultra3dpro115" · use gameultra3dpro115

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro116

import "std/c/gameultra3dpro116" · use gameultra3dpro116

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro117

import "std/c/gameultra3dpro117" · use gameultra3dpro117

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro118

import "std/c/gameultra3dpro118" · use gameultra3dpro118

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro119

import "std/c/gameultra3dpro119" · use gameultra3dpro119

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro120

import "std/c/gameultra3dpro120" · use gameultra3dpro120

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro121

import "std/c/gameultra3dpro121" · use gameultra3dpro121

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro122

import "std/c/gameultra3dpro122" · use gameultra3dpro122

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro123

import "std/c/gameultra3dpro123" · use gameultra3dpro123

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro124

import "std/c/gameultra3dpro124" · use gameultra3dpro124

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro125

import "std/c/gameultra3dpro125" · use gameultra3dpro125

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro126

import "std/c/gameultra3dpro126" · use gameultra3dpro126

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro127

import "std/c/gameultra3dpro127" · use gameultra3dpro127

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro128

import "std/c/gameultra3dpro128" · use gameultra3dpro128

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro129

import "std/c/gameultra3dpro129" · use gameultra3dpro129

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro130

import "std/c/gameultra3dpro130" · use gameultra3dpro130

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro131

import "std/c/gameultra3dpro131" · use gameultra3dpro131

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro132

import "std/c/gameultra3dpro132" · use gameultra3dpro132

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro133

import "std/c/gameultra3dpro133" · use gameultra3dpro133

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro134

import "std/c/gameultra3dpro134" · use gameultra3dpro134

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro135

import "std/c/gameultra3dpro135" · use gameultra3dpro135

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro136

import "std/c/gameultra3dpro136" · use gameultra3dpro136

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro137

import "std/c/gameultra3dpro137" · use gameultra3dpro137

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro138

import "std/c/gameultra3dpro138" · use gameultra3dpro138

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro139

import "std/c/gameultra3dpro139" · use gameultra3dpro139

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro140

import "std/c/gameultra3dpro140" · use gameultra3dpro140

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro141

import "std/c/gameultra3dpro141" · use gameultra3dpro141

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro142

import "std/c/gameultra3dpro142" · use gameultra3dpro142

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro143

import "std/c/gameultra3dpro143" · use gameultra3dpro143

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro144

import "std/c/gameultra3dpro144" · use gameultra3dpro144

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro145

import "std/c/gameultra3dpro145" · use gameultra3dpro145

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro146

import "std/c/gameultra3dpro146" · use gameultra3dpro146

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro147

import "std/c/gameultra3dpro147" · use gameultra3dpro147

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro148

import "std/c/gameultra3dpro148" · use gameultra3dpro148

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro149

import "std/c/gameultra3dpro149" · use gameultra3dpro149

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro150

import "std/c/gameultra3dpro150" · use gameultra3dpro150

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro151

import "std/c/gameultra3dpro151" · use gameultra3dpro151

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro152

import "std/c/gameultra3dpro152" · use gameultra3dpro152

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro153

import "std/c/gameultra3dpro153" · use gameultra3dpro153

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro154

import "std/c/gameultra3dpro154" · use gameultra3dpro154

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro155

import "std/c/gameultra3dpro155" · use gameultra3dpro155

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro156

import "std/c/gameultra3dpro156" · use gameultra3dpro156

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro157

import "std/c/gameultra3dpro157" · use gameultra3dpro157

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro158

import "std/c/gameultra3dpro158" · use gameultra3dpro158

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro159

import "std/c/gameultra3dpro159" · use gameultra3dpro159

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro160

import "std/c/gameultra3dpro160" · use gameultra3dpro160

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro161

import "std/c/gameultra3dpro161" · use gameultra3dpro161

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro162

import "std/c/gameultra3dpro162" · use gameultra3dpro162

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro163

import "std/c/gameultra3dpro163" · use gameultra3dpro163

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro164

import "std/c/gameultra3dpro164" · use gameultra3dpro164

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro165

import "std/c/gameultra3dpro165" · use gameultra3dpro165

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro166

import "std/c/gameultra3dpro166" · use gameultra3dpro166

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro167

import "std/c/gameultra3dpro167" · use gameultra3dpro167

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro168

import "std/c/gameultra3dpro168" · use gameultra3dpro168

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro169

import "std/c/gameultra3dpro169" · use gameultra3dpro169

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro170

import "std/c/gameultra3dpro170" · use gameultra3dpro170

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro171

import "std/c/gameultra3dpro171" · use gameultra3dpro171

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro172

import "std/c/gameultra3dpro172" · use gameultra3dpro172

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro173

import "std/c/gameultra3dpro173" · use gameultra3dpro173

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro174

import "std/c/gameultra3dpro174" · use gameultra3dpro174

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro175

import "std/c/gameultra3dpro175" · use gameultra3dpro175

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro176

import "std/c/gameultra3dpro176" · use gameultra3dpro176

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro177

import "std/c/gameultra3dpro177" · use gameultra3dpro177

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro178

import "std/c/gameultra3dpro178" · use gameultra3dpro178

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro179

import "std/c/gameultra3dpro179" · use gameultra3dpro179

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro180

import "std/c/gameultra3dpro180" · use gameultra3dpro180

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro181

import "std/c/gameultra3dpro181" · use gameultra3dpro181

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro182

import "std/c/gameultra3dpro182" · use gameultra3dpro182

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro183

import "std/c/gameultra3dpro183" · use gameultra3dpro183

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro184

import "std/c/gameultra3dpro184" · use gameultra3dpro184

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro185

import "std/c/gameultra3dpro185" · use gameultra3dpro185

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro186

import "std/c/gameultra3dpro186" · use gameultra3dpro186

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro187

import "std/c/gameultra3dpro187" · use gameultra3dpro187

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro188

import "std/c/gameultra3dpro188" · use gameultra3dpro188

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro189

import "std/c/gameultra3dpro189" · use gameultra3dpro189

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro190

import "std/c/gameultra3dpro190" · use gameultra3dpro190

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro191

import "std/c/gameultra3dpro191" · use gameultra3dpro191

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro192

import "std/c/gameultra3dpro192" · use gameultra3dpro192

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro193

import "std/c/gameultra3dpro193" · use gameultra3dpro193

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro194

import "std/c/gameultra3dpro194" · use gameultra3dpro194

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro195

import "std/c/gameultra3dpro195" · use gameultra3dpro195

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro196

import "std/c/gameultra3dpro196" · use gameultra3dpro196

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro197

import "std/c/gameultra3dpro197" · use gameultra3dpro197

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro198

import "std/c/gameultra3dpro198" · use gameultra3dpro198

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro199

import "std/c/gameultra3dpro199" · use gameultra3dpro199

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro200

import "std/c/gameultra3dpro200" · use gameultra3dpro200

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro201

import "std/c/gameultra3dpro201" · use gameultra3dpro201

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro202

import "std/c/gameultra3dpro202" · use gameultra3dpro202

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro203

import "std/c/gameultra3dpro203" · use gameultra3dpro203

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro204

import "std/c/gameultra3dpro204" · use gameultra3dpro204

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro205

import "std/c/gameultra3dpro205" · use gameultra3dpro205

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro206

import "std/c/gameultra3dpro206" · use gameultra3dpro206

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro207

import "std/c/gameultra3dpro207" · use gameultra3dpro207

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro208

import "std/c/gameultra3dpro208" · use gameultra3dpro208

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro209

import "std/c/gameultra3dpro209" · use gameultra3dpro209

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro210

import "std/c/gameultra3dpro210" · use gameultra3dpro210

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro211

import "std/c/gameultra3dpro211" · use gameultra3dpro211

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro212

import "std/c/gameultra3dpro212" · use gameultra3dpro212

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro213

import "std/c/gameultra3dpro213" · use gameultra3dpro213

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro214

import "std/c/gameultra3dpro214" · use gameultra3dpro214

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro215

import "std/c/gameultra3dpro215" · use gameultra3dpro215

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro216

import "std/c/gameultra3dpro216" · use gameultra3dpro216

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro217

import "std/c/gameultra3dpro217" · use gameultra3dpro217

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro218

import "std/c/gameultra3dpro218" · use gameultra3dpro218

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro219

import "std/c/gameultra3dpro219" · use gameultra3dpro219

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro220

import "std/c/gameultra3dpro220" · use gameultra3dpro220

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro221

import "std/c/gameultra3dpro221" · use gameultra3dpro221

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro222

import "std/c/gameultra3dpro222" · use gameultra3dpro222

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro223

import "std/c/gameultra3dpro223" · use gameultra3dpro223

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro224

import "std/c/gameultra3dpro224" · use gameultra3dpro224

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro225

import "std/c/gameultra3dpro225" · use gameultra3dpro225

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro226

import "std/c/gameultra3dpro226" · use gameultra3dpro226

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro227

import "std/c/gameultra3dpro227" · use gameultra3dpro227

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro228

import "std/c/gameultra3dpro228" · use gameultra3dpro228

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro229

import "std/c/gameultra3dpro229" · use gameultra3dpro229

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro230

import "std/c/gameultra3dpro230" · use gameultra3dpro230

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro231

import "std/c/gameultra3dpro231" · use gameultra3dpro231

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro232

import "std/c/gameultra3dpro232" · use gameultra3dpro232

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro233

import "std/c/gameultra3dpro233" · use gameultra3dpro233

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro234

import "std/c/gameultra3dpro234" · use gameultra3dpro234

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro235

import "std/c/gameultra3dpro235" · use gameultra3dpro235

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro236

import "std/c/gameultra3dpro236" · use gameultra3dpro236

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro237

import "std/c/gameultra3dpro237" · use gameultra3dpro237

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro238

import "std/c/gameultra3dpro238" · use gameultra3dpro238

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro239

import "std/c/gameultra3dpro239" · use gameultra3dpro239

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro240

import "std/c/gameultra3dpro240" · use gameultra3dpro240

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro241

import "std/c/gameultra3dpro241" · use gameultra3dpro241

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro242

import "std/c/gameultra3dpro242" · use gameultra3dpro242

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro243

import "std/c/gameultra3dpro243" · use gameultra3dpro243

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro244

import "std/c/gameultra3dpro244" · use gameultra3dpro244

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro245

import "std/c/gameultra3dpro245" · use gameultra3dpro245

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro246

import "std/c/gameultra3dpro246" · use gameultra3dpro246

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro247

import "std/c/gameultra3dpro247" · use gameultra3dpro247

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro248

import "std/c/gameultra3dpro248" · use gameultra3dpro248

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro249

import "std/c/gameultra3dpro249" · use gameultra3dpro249

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro250

import "std/c/gameultra3dpro250" · use gameultra3dpro250

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro251

import "std/c/gameultra3dpro251" · use gameultra3dpro251

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro252

import "std/c/gameultra3dpro252" · use gameultra3dpro252

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro253

import "std/c/gameultra3dpro253" · use gameultra3dpro253

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro254

import "std/c/gameultra3dpro254" · use gameultra3dpro254

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro255

import "std/c/gameultra3dpro255" · use gameultra3dpro255

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro256

import "std/c/gameultra3dpro256" · use gameultra3dpro256

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro257

import "std/c/gameultra3dpro257" · use gameultra3dpro257

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro258

import "std/c/gameultra3dpro258" · use gameultra3dpro258

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro259

import "std/c/gameultra3dpro259" · use gameultra3dpro259

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro260

import "std/c/gameultra3dpro260" · use gameultra3dpro260

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro261

import "std/c/gameultra3dpro261" · use gameultra3dpro261

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro262

import "std/c/gameultra3dpro262" · use gameultra3dpro262

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro263

import "std/c/gameultra3dpro263" · use gameultra3dpro263

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro264

import "std/c/gameultra3dpro264" · use gameultra3dpro264

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro265

import "std/c/gameultra3dpro265" · use gameultra3dpro265

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro266

import "std/c/gameultra3dpro266" · use gameultra3dpro266

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro267

import "std/c/gameultra3dpro267" · use gameultra3dpro267

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro268

import "std/c/gameultra3dpro268" · use gameultra3dpro268

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro269

import "std/c/gameultra3dpro269" · use gameultra3dpro269

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro270

import "std/c/gameultra3dpro270" · use gameultra3dpro270

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro271

import "std/c/gameultra3dpro271" · use gameultra3dpro271

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro272

import "std/c/gameultra3dpro272" · use gameultra3dpro272

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro273

import "std/c/gameultra3dpro273" · use gameultra3dpro273

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro274

import "std/c/gameultra3dpro274" · use gameultra3dpro274

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro275

import "std/c/gameultra3dpro275" · use gameultra3dpro275

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro276

import "std/c/gameultra3dpro276" · use gameultra3dpro276

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro277

import "std/c/gameultra3dpro277" · use gameultra3dpro277

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro278

import "std/c/gameultra3dpro278" · use gameultra3dpro278

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro279

import "std/c/gameultra3dpro279" · use gameultra3dpro279

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro280

import "std/c/gameultra3dpro280" · use gameultra3dpro280

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro281

import "std/c/gameultra3dpro281" · use gameultra3dpro281

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro282

import "std/c/gameultra3dpro282" · use gameultra3dpro282

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro283

import "std/c/gameultra3dpro283" · use gameultra3dpro283

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro284

import "std/c/gameultra3dpro284" · use gameultra3dpro284

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro285

import "std/c/gameultra3dpro285" · use gameultra3dpro285

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro286

import "std/c/gameultra3dpro286" · use gameultra3dpro286

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro287

import "std/c/gameultra3dpro287" · use gameultra3dpro287

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro288

import "std/c/gameultra3dpro288" · use gameultra3dpro288

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro289

import "std/c/gameultra3dpro289" · use gameultra3dpro289

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro290

import "std/c/gameultra3dpro290" · use gameultra3dpro290

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro291

import "std/c/gameultra3dpro291" · use gameultra3dpro291

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro292

import "std/c/gameultra3dpro292" · use gameultra3dpro292

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro293

import "std/c/gameultra3dpro293" · use gameultra3dpro293

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro294

import "std/c/gameultra3dpro294" · use gameultra3dpro294

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro295

import "std/c/gameultra3dpro295" · use gameultra3dpro295

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro296

import "std/c/gameultra3dpro296" · use gameultra3dpro296

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro297

import "std/c/gameultra3dpro297" · use gameultra3dpro297

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro298

import "std/c/gameultra3dpro298" · use gameultra3dpro298

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro299

import "std/c/gameultra3dpro299" · use gameultra3dpro299

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro300

import "std/c/gameultra3dpro300" · use gameultra3dpro300

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro301

import "std/c/gameultra3dpro301" · use gameultra3dpro301

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro302

import "std/c/gameultra3dpro302" · use gameultra3dpro302

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro303

import "std/c/gameultra3dpro303" · use gameultra3dpro303

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro304

import "std/c/gameultra3dpro304" · use gameultra3dpro304

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro305

import "std/c/gameultra3dpro305" · use gameultra3dpro305

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro306

import "std/c/gameultra3dpro306" · use gameultra3dpro306

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro307

import "std/c/gameultra3dpro307" · use gameultra3dpro307

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro308

import "std/c/gameultra3dpro308" · use gameultra3dpro308

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro309

import "std/c/gameultra3dpro309" · use gameultra3dpro309

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro310

import "std/c/gameultra3dpro310" · use gameultra3dpro310

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro311

import "std/c/gameultra3dpro311" · use gameultra3dpro311

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro312

import "std/c/gameultra3dpro312" · use gameultra3dpro312

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro313

import "std/c/gameultra3dpro313" · use gameultra3dpro313

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro314

import "std/c/gameultra3dpro314" · use gameultra3dpro314

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro315

import "std/c/gameultra3dpro315" · use gameultra3dpro315

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro316

import "std/c/gameultra3dpro316" · use gameultra3dpro316

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro317

import "std/c/gameultra3dpro317" · use gameultra3dpro317

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro318

import "std/c/gameultra3dpro318" · use gameultra3dpro318

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro319

import "std/c/gameultra3dpro319" · use gameultra3dpro319

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro320

import "std/c/gameultra3dpro320" · use gameultra3dpro320

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro321

import "std/c/gameultra3dpro321" · use gameultra3dpro321

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro322

import "std/c/gameultra3dpro322" · use gameultra3dpro322

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro323

import "std/c/gameultra3dpro323" · use gameultra3dpro323

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro324

import "std/c/gameultra3dpro324" · use gameultra3dpro324

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro325

import "std/c/gameultra3dpro325" · use gameultra3dpro325

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro326

import "std/c/gameultra3dpro326" · use gameultra3dpro326

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro327

import "std/c/gameultra3dpro327" · use gameultra3dpro327

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro328

import "std/c/gameultra3dpro328" · use gameultra3dpro328

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro329

import "std/c/gameultra3dpro329" · use gameultra3dpro329

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro330

import "std/c/gameultra3dpro330" · use gameultra3dpro330

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro331

import "std/c/gameultra3dpro331" · use gameultra3dpro331

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro332

import "std/c/gameultra3dpro332" · use gameultra3dpro332

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro333

import "std/c/gameultra3dpro333" · use gameultra3dpro333

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro334

import "std/c/gameultra3dpro334" · use gameultra3dpro334

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro335

import "std/c/gameultra3dpro335" · use gameultra3dpro335

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro336

import "std/c/gameultra3dpro336" · use gameultra3dpro336

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro337

import "std/c/gameultra3dpro337" · use gameultra3dpro337

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro338

import "std/c/gameultra3dpro338" · use gameultra3dpro338

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro339

import "std/c/gameultra3dpro339" · use gameultra3dpro339

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro340

import "std/c/gameultra3dpro340" · use gameultra3dpro340

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro341

import "std/c/gameultra3dpro341" · use gameultra3dpro341

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro342

import "std/c/gameultra3dpro342" · use gameultra3dpro342

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro343

import "std/c/gameultra3dpro343" · use gameultra3dpro343

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro344

import "std/c/gameultra3dpro344" · use gameultra3dpro344

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro345

import "std/c/gameultra3dpro345" · use gameultra3dpro345

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro346

import "std/c/gameultra3dpro346" · use gameultra3dpro346

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro347

import "std/c/gameultra3dpro347" · use gameultra3dpro347

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro348

import "std/c/gameultra3dpro348" · use gameultra3dpro348

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro349

import "std/c/gameultra3dpro349" · use gameultra3dpro349

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro350

import "std/c/gameultra3dpro350" · use gameultra3dpro350

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro351

import "std/c/gameultra3dpro351" · use gameultra3dpro351

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro352

import "std/c/gameultra3dpro352" · use gameultra3dpro352

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro353

import "std/c/gameultra3dpro353" · use gameultra3dpro353

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro354

import "std/c/gameultra3dpro354" · use gameultra3dpro354

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro355

import "std/c/gameultra3dpro355" · use gameultra3dpro355

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro356

import "std/c/gameultra3dpro356" · use gameultra3dpro356

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro357

import "std/c/gameultra3dpro357" · use gameultra3dpro357

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro358

import "std/c/gameultra3dpro358" · use gameultra3dpro358

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro359

import "std/c/gameultra3dpro359" · use gameultra3dpro359

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro360

import "std/c/gameultra3dpro360" · use gameultra3dpro360

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro361

import "std/c/gameultra3dpro361" · use gameultra3dpro361

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro362

import "std/c/gameultra3dpro362" · use gameultra3dpro362

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro363

import "std/c/gameultra3dpro363" · use gameultra3dpro363

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro364

import "std/c/gameultra3dpro364" · use gameultra3dpro364

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro365

import "std/c/gameultra3dpro365" · use gameultra3dpro365

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro366

import "std/c/gameultra3dpro366" · use gameultra3dpro366

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro367

import "std/c/gameultra3dpro367" · use gameultra3dpro367

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro368

import "std/c/gameultra3dpro368" · use gameultra3dpro368

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro369

import "std/c/gameultra3dpro369" · use gameultra3dpro369

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro370

import "std/c/gameultra3dpro370" · use gameultra3dpro370

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro371

import "std/c/gameultra3dpro371" · use gameultra3dpro371

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro372

import "std/c/gameultra3dpro372" · use gameultra3dpro372

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro373

import "std/c/gameultra3dpro373" · use gameultra3dpro373

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro374

import "std/c/gameultra3dpro374" · use gameultra3dpro374

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro375

import "std/c/gameultra3dpro375" · use gameultra3dpro375

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro376

import "std/c/gameultra3dpro376" · use gameultra3dpro376

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro377

import "std/c/gameultra3dpro377" · use gameultra3dpro377

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro378

import "std/c/gameultra3dpro378" · use gameultra3dpro378

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro379

import "std/c/gameultra3dpro379" · use gameultra3dpro379

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro380

import "std/c/gameultra3dpro380" · use gameultra3dpro380

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro381

import "std/c/gameultra3dpro381" · use gameultra3dpro381

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro382

import "std/c/gameultra3dpro382" · use gameultra3dpro382

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro383

import "std/c/gameultra3dpro383" · use gameultra3dpro383

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro384

import "std/c/gameultra3dpro384" · use gameultra3dpro384

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro385

import "std/c/gameultra3dpro385" · use gameultra3dpro385

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro386

import "std/c/gameultra3dpro386" · use gameultra3dpro386

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro387

import "std/c/gameultra3dpro387" · use gameultra3dpro387

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro388

import "std/c/gameultra3dpro388" · use gameultra3dpro388

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro389

import "std/c/gameultra3dpro389" · use gameultra3dpro389

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro390

import "std/c/gameultra3dpro390" · use gameultra3dpro390

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro391

import "std/c/gameultra3dpro391" · use gameultra3dpro391

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro392

import "std/c/gameultra3dpro392" · use gameultra3dpro392

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro393

import "std/c/gameultra3dpro393" · use gameultra3dpro393

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro394

import "std/c/gameultra3dpro394" · use gameultra3dpro394

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro395

import "std/c/gameultra3dpro395" · use gameultra3dpro395

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro396

import "std/c/gameultra3dpro396" · use gameultra3dpro396

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro397

import "std/c/gameultra3dpro397" · use gameultra3dpro397

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro398

import "std/c/gameultra3dpro398" · use gameultra3dpro398

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro399

import "std/c/gameultra3dpro399" · use gameultra3dpro399

• `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
• `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
• `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
• `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro400

import "std/c/gameultra3dpro400" · use gameultra3dpro400

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro401

import "std/c/gameultra3dpro401" · use gameultra3dpro401

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro402

import "std/c/gameultra3dpro402" · use gameultra3dpro402

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro403

import "std/c/gameultra3dpro403" · use gameultra3dpro403

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro404

import "std/c/gameultra3dpro404" · use gameultra3dpro404

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro405

import "std/c/gameultra3dpro405" · use gameultra3dpro405

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro406

import "std/c/gameultra3dpro406" · use gameultra3dpro406

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro407

import "std/c/gameultra3dpro407" · use gameultra3dpro407

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro408

import "std/c/gameultra3dpro408" · use gameultra3dpro408

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro409

import "std/c/gameultra3dpro409" · use gameultra3dpro409

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro410

import "std/c/gameultra3dpro410" · use gameultra3dpro410

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro411

import "std/c/gameultra3dpro411" · use gameultra3dpro411

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro412

import "std/c/gameultra3dpro412" · use gameultra3dpro412

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro413

import "std/c/gameultra3dpro413" · use gameultra3dpro413

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro414

import "std/c/gameultra3dpro414" · use gameultra3dpro414

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro415

import "std/c/gameultra3dpro415" · use gameultra3dpro415

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro416

import "std/c/gameultra3dpro416" · use gameultra3dpro416

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro417

import "std/c/gameultra3dpro417" · use gameultra3dpro417

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro418

import "std/c/gameultra3dpro418" · use gameultra3dpro418

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro419

import "std/c/gameultra3dpro419" · use gameultra3dpro419

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro420

import "std/c/gameultra3dpro420" · use gameultra3dpro420

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro421

import "std/c/gameultra3dpro421" · use gameultra3dpro421

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro422

import "std/c/gameultra3dpro422" · use gameultra3dpro422

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro423

import "std/c/gameultra3dpro423" · use gameultra3dpro423

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro424

import "std/c/gameultra3dpro424" · use gameultra3dpro424

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro425

import "std/c/gameultra3dpro425" · use gameultra3dpro425

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro426

import "std/c/gameultra3dpro426" · use gameultra3dpro426

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro427

import "std/c/gameultra3dpro427" · use gameultra3dpro427

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro428

import "std/c/gameultra3dpro428" · use gameultra3dpro428

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro429

import "std/c/gameultra3dpro429" · use gameultra3dpro429

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro430

import "std/c/gameultra3dpro430" · use gameultra3dpro430

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro431

import "std/c/gameultra3dpro431" · use gameultra3dpro431

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro432

import "std/c/gameultra3dpro432" · use gameultra3dpro432

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro433

import "std/c/gameultra3dpro433" · use gameultra3dpro433

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro434

import "std/c/gameultra3dpro434" · use gameultra3dpro434

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro435

import "std/c/gameultra3dpro435" · use gameultra3dpro435

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro436

import "std/c/gameultra3dpro436" · use gameultra3dpro436

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro437

import "std/c/gameultra3dpro437" · use gameultra3dpro437

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro438

import "std/c/gameultra3dpro438" · use gameultra3dpro438

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro439

import "std/c/gameultra3dpro439" · use gameultra3dpro439

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro440

import "std/c/gameultra3dpro440" · use gameultra3dpro440

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro441

import "std/c/gameultra3dpro441" · use gameultra3dpro441

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro442

import "std/c/gameultra3dpro442" · use gameultra3dpro442

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro443

import "std/c/gameultra3dpro443" · use gameultra3dpro443

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro444

import "std/c/gameultra3dpro444" · use gameultra3dpro444

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro445

import "std/c/gameultra3dpro445" · use gameultra3dpro445

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro446

import "std/c/gameultra3dpro446" · use gameultra3dpro446

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro447

import "std/c/gameultra3dpro447" · use gameultra3dpro447

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro448

import "std/c/gameultra3dpro448" · use gameultra3dpro448

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro449

import "std/c/gameultra3dpro449" · use gameultra3dpro449

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro450

import "std/c/gameultra3dpro450" · use gameultra3dpro450

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro451

import "std/c/gameultra3dpro451" · use gameultra3dpro451

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro452

import "std/c/gameultra3dpro452" · use gameultra3dpro452

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro453

import "std/c/gameultra3dpro453" · use gameultra3dpro453

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro454

import "std/c/gameultra3dpro454" · use gameultra3dpro454

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro455

import "std/c/gameultra3dpro455" · use gameultra3dpro455

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro456

import "std/c/gameultra3dpro456" · use gameultra3dpro456

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro457

import "std/c/gameultra3dpro457" · use gameultra3dpro457

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro458

import "std/c/gameultra3dpro458" · use gameultra3dpro458

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro459

import "std/c/gameultra3dpro459" · use gameultra3dpro459

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro460

import "std/c/gameultra3dpro460" · use gameultra3dpro460

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro461

import "std/c/gameultra3dpro461" · use gameultra3dpro461

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro462

import "std/c/gameultra3dpro462" · use gameultra3dpro462

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro463

import "std/c/gameultra3dpro463" · use gameultra3dpro463

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro464

import "std/c/gameultra3dpro464" · use gameultra3dpro464

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro465

import "std/c/gameultra3dpro465" · use gameultra3dpro465

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro466

import "std/c/gameultra3dpro466" · use gameultra3dpro466

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro467

import "std/c/gameultra3dpro467" · use gameultra3dpro467

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro468

import "std/c/gameultra3dpro468" · use gameultra3dpro468

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro469

import "std/c/gameultra3dpro469" · use gameultra3dpro469

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro470

import "std/c/gameultra3dpro470" · use gameultra3dpro470

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro471

import "std/c/gameultra3dpro471" · use gameultra3dpro471

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro472

import "std/c/gameultra3dpro472" · use gameultra3dpro472

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro473

import "std/c/gameultra3dpro473" · use gameultra3dpro473

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro474

import "std/c/gameultra3dpro474" · use gameultra3dpro474

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro475

import "std/c/gameultra3dpro475" · use gameultra3dpro475

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro476

import "std/c/gameultra3dpro476" · use gameultra3dpro476

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro477

import "std/c/gameultra3dpro477" · use gameultra3dpro477

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro478

import "std/c/gameultra3dpro478" · use gameultra3dpro478

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro479

import "std/c/gameultra3dpro479" · use gameultra3dpro479

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro480

import "std/c/gameultra3dpro480" · use gameultra3dpro480

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro481

import "std/c/gameultra3dpro481" · use gameultra3dpro481

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro482

import "std/c/gameultra3dpro482" · use gameultra3dpro482

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro483

import "std/c/gameultra3dpro483" · use gameultra3dpro483

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro484

import "std/c/gameultra3dpro484" · use gameultra3dpro484

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro485

import "std/c/gameultra3dpro485" · use gameultra3dpro485

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro486

import "std/c/gameultra3dpro486" · use gameultra3dpro486

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro487

import "std/c/gameultra3dpro487" · use gameultra3dpro487

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro488

import "std/c/gameultra3dpro488" · use gameultra3dpro488

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro489

import "std/c/gameultra3dpro489" · use gameultra3dpro489

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro490

import "std/c/gameultra3dpro490" · use gameultra3dpro490

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro491

import "std/c/gameultra3dpro491" · use gameultra3dpro491

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro492

import "std/c/gameultra3dpro492" · use gameultra3dpro492

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro493

import "std/c/gameultra3dpro493" · use gameultra3dpro493

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro494

import "std/c/gameultra3dpro494" · use gameultra3dpro494

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro495

import "std/c/gameultra3dpro495" · use gameultra3dpro495

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro496

import "std/c/gameultra3dpro496" · use gameultra3dpro496

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro497

import "std/c/gameultra3dpro497" · use gameultra3dpro497

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro498

import "std/c/gameultra3dpro498" · use gameultra3dpro498

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro499

import "std/c/gameultra3dpro499" · use gameultra3dpro499

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gameultra3dpro500

import "std/c/gameultra3dpro500" · use gameultra3dpro500

- `astarVoxelPath(grid list number, w number, h number, d number, sx number, sy number, sz number, gx number, gy number, gz number, blockedAbove number) → list number`
- `rayAabbHit(ox number, oy number, oz number, dx number, dy number, dz number, cx number, cy number, cz number, hx number, hy number, hz number) → list number`
- `steerArrive(px number, py number, pz number, tx number, ty number, tz number, maxSpeed number) → list number`
- `avoidSphereForce(px number, py number, pz number, cx number, cy number, cz number, radius number) → list number`

std/c/gisultra001

import "std/c/gisultra001" · use gisultra001

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra002

import "std/c/gisultra002" · use gisultra002

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra003

import "std/c/gisultra003" · use gisultra003

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra004

import "std/c/gisultra004" · use gisultra004

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra005

import "std/c/gisultra005" · use gisultra005

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra006

import "std/c/gisultra006" · use gisultra006

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra007

import "std/c/gisultra007" · use gisultra007

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra008

import "std/c/gisultra008" · use gisultra008

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra009

import "std/c/gisultra009" · use gisultra009

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra010

import "std/c/gisultra010" · use gisultra010

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra011

import "std/c/gisultra011" · use gisultra011

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra012

import "std/c/gisultra012" · use gisultra012

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra013

import "std/c/gisultra013" · use gisultra013

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra014

import "std/c/gisultra014" · use gisultra014

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra015

import "std/c/gisultra015" · use gisultra015

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra016

import "std/c/gisultra016" · use gisultra016

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra017

import "std/c/gisultra017" · use gisultra017

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra018

import "std/c/gisultra018" · use gisultra018

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra019

import "std/c/gisultra019" · use gisultra019

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra020

import "std/c/gisultra020" · use gisultra020

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra021

import "std/c/gisultra021" · use gisultra021

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra022

import "std/c/gisultra022" · use gisultra022

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra023

import "std/c/gisultra023" · use gisultra023

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra024

import "std/c/gisultra024" · use gisultra024

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra025

import "std/c/gisultra025" · use gisultra025

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra026

import "std/c/gisultra026" · use gisultra026

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra027

import "std/c/gisultra027" · use gisultra027

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra028

import "std/c/gisultra028" · use gisultra028

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra029

import "std/c/gisultra029" · use gisultra029

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra030

import "std/c/gisultra030" · use gisultra030

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra031

import "std/c/gisultra031" · use gisultra031

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra032

import "std/c/gisultra032" · use gisultra032

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra033

import "std/c/gisultra033" · use gisultra033

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra034

import "std/c/gisultra034" · use gisultra034

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra035

import "std/c/gisultra035" · use gisultra035

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra036

import "std/c/gisultra036" · use gisultra036

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra037

import "std/c/gisultra037" · use gisultra037

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra038

import "std/c/gisultra038" · use gisultra038

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra039

import "std/c/gisultra039" · use gisultra039

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra040

import "std/c/gisultra040" · use gisultra040

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra041

import "std/c/gisultra041" · use gisultra041

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra042

import "std/c/gisultra042" · use gisultra042

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra043

import "std/c/gisultra043" · use gisultra043

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra044

import "std/c/gisultra044" · use gisultra044

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra045

import "std/c/gisultra045" · use gisultra045

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra046

import "std/c/gisultra046" · use gisultra046

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra047

import "std/c/gisultra047" · use gisultra047

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra048

import "std/c/gisultra048" · use gisultra048

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra049

import "std/c/gisultra049" · use gisultra049

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra050

import "std/c/gisultra050" · use gisultra050

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra051

import "std/c/gisultra051" · use gisultra051

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra052

import "std/c/gisultra052" · use gisultra052

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra053

import "std/c/gisultra053" · use gisultra053

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra054

import "std/c/gisultra054" · use gisultra054

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra055

import "std/c/gisultra055" · use gisultra055

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra056

import "std/c/gisultra056" · use gisultra056

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra057

import "std/c/gisultra057" · use gisultra057

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra058

import "std/c/gisultra058" · use gisultra058

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra059

import "std/c/gisultra059" · use gisultra059

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra060

import "std/c/gisultra060" · use gisultra060

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra061

import "std/c/gisultra061" · use gisultra061

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra062

import "std/c/gisultra062" · use gisultra062

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra063

import "std/c/gisultra063" · use gisultra063

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra064

import "std/c/gisultra064" · use gisultra064

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra065

import "std/c/gisultra065" · use gisultra065

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra066

import "std/c/gisultra066" · use gisultra066

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra067

import "std/c/gisultra067" · use gisultra067

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra068

import "std/c/gisultra068" · use gisultra068

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra069

import "std/c/gisultra069" · use gisultra069

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra070

import "std/c/gisultra070" · use gisultra070

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra071

import "std/c/gisultra071" · use gisultra071

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra072

import "std/c/gisultra072" · use gisultra072

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra073

import "std/c/gisultra073" · use gisultra073

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra074

import "std/c/gisultra074" · use gisultra074

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra075

import "std/c/gisultra075" · use gisultra075

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra076

import "std/c/gisultra076" · use gisultra076

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra077

import "std/c/gisultra077" · use gisultra077

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra078

import "std/c/gisultra078" · use gisultra078

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra079

import "std/c/gisultra079" · use gisultra079

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra080

import "std/c/gisultra080" · use gisultra080

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra081

import "std/c/gisultra081" · use gisultra081

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra082

import "std/c/gisultra082" · use gisultra082

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra083

import "std/c/gisultra083" · use gisultra083

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra084

import "std/c/gisultra084" · use gisultra084

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra085

import "std/c/gisultra085" · use gisultra085

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra086

import "std/c/gisultra086" · use gisultra086

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra087

import "std/c/gisultra087" · use gisultra087

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra088

import "std/c/gisultra088" · use gisultra088

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra089

import "std/c/gisultra089" · use gisultra089

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra090

import "std/c/gisultra090" · use gisultra090

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra091

import "std/c/gisultra091" · use gisultra091

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra092

import "std/c/gisultra092" · use gisultra092

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra093

import "std/c/gisultra093" · use gisultra093

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra094

import "std/c/gisultra094" · use gisultra094

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra095

import "std/c/gisultra095" · use gisultra095

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra096

import "std/c/gisultra096" · use gisultra096

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra097

import "std/c/gisultra097" · use gisultra097

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra098

import "std/c/gisultra098" · use gisultra098

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra099

import "std/c/gisultra099" · use gisultra099

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra100

import "std/c/gisultra100" · use gisultra100

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra101

import "std/c/gisultra101" · use gisultra101

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra102

import "std/c/gisultra102" · use gisultra102

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra103

import "std/c/gisultra103" · use gisultra103

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra104

import "std/c/gisultra104" · use gisultra104

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra105

import "std/c/gisultra105" · use gisultra105

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra106

import "std/c/gisultra106" · use gisultra106

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra107

import "std/c/gisultra107" · use gisultra107

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra108

import "std/c/gisultra108" · use gisultra108

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra109

import "std/c/gisultra109" · use gisultra109

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra110

import "std/c/gisultra110" · use gisultra110

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra111

import "std/c/gisultra111" · use gisultra111

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra112

import "std/c/gisultra112" · use gisultra112

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra113

import "std/c/gisultra113" · use gisultra113

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra114

import "std/c/gisultra114" · use gisultra114

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra115

import "std/c/gisultra115" · use gisultra115

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra116

import "std/c/gisultra116" · use gisultra116

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra117

import "std/c/gisultra117" · use gisultra117

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra118

import "std/c/gisultra118" · use gisultra118

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra119

import "std/c/gisultra119" · use gisultra119

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra120

import "std/c/gisultra120" · use gisultra120

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra121

import "std/c/gisultra121" · use gisultra121

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra122

import "std/c/gisultra122" · use gisultra122

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra123

import "std/c/gisultra123" · use gisultra123

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra124

import "std/c/gisultra124" · use gisultra124

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra125

import "std/c/gisultra125" · use gisultra125

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra126

import "std/c/gisultra126" · use gisultra126

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra127

import "std/c/gisultra127" · use gisultra127

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra128

import "std/c/gisultra128" · use gisultra128

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra129

import "std/c/gisultra129" · use gisultra129

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra130

import "std/c/gisultra130" · use gisultra130

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra131

import "std/c/gisultra131" · use gisultra131

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra132

import "std/c/gisultra132" · use gisultra132

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra133

import "std/c/gisultra133" · use gisultra133

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra134

import "std/c/gisultra134" · use gisultra134

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra135

import "std/c/gisultra135" · use gisultra135

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra136

import "std/c/gisultra136" · use gisultra136

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra137

import "std/c/gisultra137" · use gisultra137

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra138

import "std/c/gisultra138" · use gisultra138

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra139

import "std/c/gisultra139" · use gisultra139

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra140

import "std/c/gisultra140" · use gisultra140

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra141

import "std/c/gisultra141" · use gisultra141

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra142

import "std/c/gisultra142" · use gisultra142

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra143

import "std/c/gisultra143" · use gisultra143

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra144

import "std/c/gisultra144" · use gisultra144

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra145

import "std/c/gisultra145" · use gisultra145

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra146

import "std/c/gisultra146" · use gisultra146

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra147

import "std/c/gisultra147" · use gisultra147

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra148

import "std/c/gisultra148" · use gisultra148

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra149

import "std/c/gisultra149" · use gisultra149

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra150

import "std/c/gisultra150" · use gisultra150

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra151

import "std/c/gisultra151" · use gisultra151

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra152

import "std/c/gisultra152" · use gisultra152

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra153

import "std/c/gisultra153" · use gisultra153

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra154

import "std/c/gisultra154" · use gisultra154

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra155

import "std/c/gisultra155" · use gisultra155

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra156

import "std/c/gisultra156" · use gisultra156

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra157

import "std/c/gisultra157" · use gisultra157

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra158

import "std/c/gisultra158" · use gisultra158

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra159

import "std/c/gisultra159" · use gisultra159

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra160

import "std/c/gisultra160" · use gisultra160

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra161

import "std/c/gisultra161" · use gisultra161

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra162

import "std/c/gisultra162" · use gisultra162

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra163

import "std/c/gisultra163" · use gisultra163

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra164

import "std/c/gisultra164" · use gisultra164

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra165

import "std/c/gisultra165" · use gisultra165

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra166

import "std/c/gisultra166" · use gisultra166

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra167

import "std/c/gisultra167" · use gisultra167

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra168

import "std/c/gisultra168" · use gisultra168

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra169

import "std/c/gisultra169" · use gisultra169

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra170

import "std/c/gisultra170" · use gisultra170

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra171

import "std/c/gisultra171" · use gisultra171

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra172

import "std/c/gisultra172" · use gisultra172

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra173

import "std/c/gisultra173" · use gisultra173

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra174

import "std/c/gisultra174" · use gisultra174

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra175

import "std/c/gisultra175" · use gisultra175

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra176

import "std/c/gisultra176" · use gisultra176

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra177

import "std/c/gisultra177" · use gisultra177

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra178

import "std/c/gisultra178" · use gisultra178

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra179

import "std/c/gisultra179" · use gisultra179

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra180

import "std/c/gisultra180" · use gisultra180

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra181

import "std/c/gisultra181" · use gisultra181

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra182

import "std/c/gisultra182" · use gisultra182

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra183

import "std/c/gisultra183" · use gisultra183

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra184

import "std/c/gisultra184" · use gisultra184

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra185

import "std/c/gisultra185" · use gisultra185

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra186

import "std/c/gisultra186" · use gisultra186

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra187

import "std/c/gisultra187" · use gisultra187

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra188

import "std/c/gisultra188" · use gisultra188

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra189

import "std/c/gisultra189" · use gisultra189

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra190

import "std/c/gisultra190" · use gisultra190

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra191

import "std/c/gisultra191" · use gisultra191

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra192

import "std/c/gisultra192" · use gisultra192

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra193

import "std/c/gisultra193" · use gisultra193

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra194

import "std/c/gisultra194" · use gisultra194

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra195

import "std/c/gisultra195" · use gisultra195

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra196

import "std/c/gisultra196" · use gisultra196

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra197

import "std/c/gisultra197" · use gisultra197

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra198

import "std/c/gisultra198" · use gisultra198

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra199

import "std/c/gisultra199" · use gisultra199

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra200

import "std/c/gisultra200" · use gisultra200

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra201

import "std/c/gisultra201" · use gisultra201

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra202

import "std/c/gisultra202" · use gisultra202

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra203

import "std/c/gisultra203" · use gisultra203

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra204

import "std/c/gisultra204" · use gisultra204

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra205

import "std/c/gisultra205" · use gisultra205

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra206

import "std/c/gisultra206" · use gisultra206

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra207

import "std/c/gisultra207" · use gisultra207

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra208

import "std/c/gisultra208" · use gisultra208

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra209

import "std/c/gisultra209" · use gisultra209

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra210

import "std/c/gisultra210" · use gisultra210

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra211

import "std/c/gisultra211" · use gisultra211

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra212

import "std/c/gisultra212" · use gisultra212

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra213

import "std/c/gisultra213" · use gisultra213

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra214

import "std/c/gisultra214" · use gisultra214

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra215

import "std/c/gisultra215" · use gisultra215

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra216

import "std/c/gisultra216" · use gisultra216

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra217

import "std/c/gisultra217" · use gisultra217

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra218

import "std/c/gisultra218" · use gisultra218

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra219

import "std/c/gisultra219" · use gisultra219

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra220

import "std/c/gisultra220" · use gisultra220

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra221

import "std/c/gisultra221" · use gisultra221

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra222

import "std/c/gisultra222" · use gisultra222

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra223

import "std/c/gisultra223" · use gisultra223

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra224

import "std/c/gisultra224" · use gisultra224

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra225

import "std/c/gisultra225" · use gisultra225

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra226

import "std/c/gisultra226" · use gisultra226

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra227

import "std/c/gisultra227" · use gisultra227

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra228

import "std/c/gisultra228" · use gisultra228

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra229

import "std/c/gisultra229" · use gisultra229

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra230

import "std/c/gisultra230" · use gisultra230

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra231

import "std/c/gisultra231" · use gisultra231

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra232

import "std/c/gisultra232" · use gisultra232

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra233

import "std/c/gisultra233" · use gisultra233

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra234

import "std/c/gisultra234" · use gisultra234

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra235

import "std/c/gisultra235" · use gisultra235

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra236

import "std/c/gisultra236" · use gisultra236

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra237

import "std/c/gisultra237" · use gisultra237

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra238

import "std/c/gisultra238" · use gisultra238

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra239

import "std/c/gisultra239" · use gisultra239

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra240

import "std/c/gisultra240" · use gisultra240

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra241

import "std/c/gisultra241" · use gisultra241

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra242

import "std/c/gisultra242" · use gisultra242

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra243

import "std/c/gisultra243" · use gisultra243

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra244

import "std/c/gisultra244" · use gisultra244

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra245

import "std/c/gisultra245" · use gisultra245

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra246

import "std/c/gisultra246" · use gisultra246

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra247

import "std/c/gisultra247" · use gisultra247

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra248

import "std/c/gisultra248" · use gisultra248

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra249

import "std/c/gisultra249" · use gisultra249

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra250

import "std/c/gisultra250" · use gisultra250

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra251

import "std/c/gisultra251" · use gisultra251

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra252

import "std/c/gisultra252" · use gisultra252

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra253

import "std/c/gisultra253" · use gisultra253

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra254

import "std/c/gisultra254" · use gisultra254

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra255

import "std/c/gisultra255" · use gisultra255

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra256

import "std/c/gisultra256" · use gisultra256

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra257

import "std/c/gisultra257" · use gisultra257

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra258

import "std/c/gisultra258" · use gisultra258

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra259

import "std/c/gisultra259" · use gisultra259

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra260

import "std/c/gisultra260" · use gisultra260

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra261

import "std/c/gisultra261" · use gisultra261

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra262

import "std/c/gisultra262" · use gisultra262

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra263

import "std/c/gisultra263" · use gisultra263

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra264

import "std/c/gisultra264" · use gisultra264

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra265

import "std/c/gisultra265" · use gisultra265

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra266

import "std/c/gisultra266" · use gisultra266

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra267

import "std/c/gisultra267" · use gisultra267

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra268

import "std/c/gisultra268" · use gisultra268

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra269

import "std/c/gisultra269" · use gisultra269

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra270

import "std/c/gisultra270" · use gisultra270

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra271

import "std/c/gisultra271" · use gisultra271

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra272

import "std/c/gisultra272" · use gisultra272

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra273

import "std/c/gisultra273" · use gisultra273

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra274

import "std/c/gisultra274" · use gisultra274

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra275

import "std/c/gisultra275" · use gisultra275

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra276

import "std/c/gisultra276" · use gisultra276

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra277

import "std/c/gisultra277" · use gisultra277

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra278

import "std/c/gisultra278" · use gisultra278

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra279

import "std/c/gisultra279" · use gisultra279

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra280

import "std/c/gisultra280" · use gisultra280

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra281

import "std/c/gisultra281" · use gisultra281

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra282

import "std/c/gisultra282" · use gisultra282

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra283

import "std/c/gisultra283" · use gisultra283

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra284

import "std/c/gisultra284" · use gisultra284

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra285

import "std/c/gisultra285" · use gisultra285

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra286

import "std/c/gisultra286" · use gisultra286

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra287

import "std/c/gisultra287" · use gisultra287

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra288

import "std/c/gisultra288" · use gisultra288

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra289

import "std/c/gisultra289" · use gisultra289

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra290

import "std/c/gisultra290" · use gisultra290

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra291

import "std/c/gisultra291" · use gisultra291

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra292

import "std/c/gisultra292" · use gisultra292

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra293

import "std/c/gisultra293" · use gisultra293

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra294

import "std/c/gisultra294" · use gisultra294

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra295

import "std/c/gisultra295" · use gisultra295

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra296

import "std/c/gisultra296" · use gisultra296

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra297

import "std/c/gisultra297" · use gisultra297

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra298

import "std/c/gisultra298" · use gisultra298

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra299

import "std/c/gisultra299" · use gisultra299

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra300

import "std/c/gisultra300" · use gisultra300

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra301

import "std/c/gisultra301" · use gisultra301

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra302

import "std/c/gisultra302" · use gisultra302

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra303

import "std/c/gisultra303" · use gisultra303

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra304

import "std/c/gisultra304" · use gisultra304

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra305

import "std/c/gisultra305" · use gisultra305

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra306

import "std/c/gisultra306" · use gisultra306

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra307

import "std/c/gisultra307" · use gisultra307

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra308

import "std/c/gisultra308" · use gisultra308

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra309

import "std/c/gisultra309" · use gisultra309

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra310

import "std/c/gisultra310" · use gisultra310

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra311

import "std/c/gisultra311" · use gisultra311

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra312

import "std/c/gisultra312" · use gisultra312

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra313

import "std/c/gisultra313" · use gisultra313

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra314

import "std/c/gisultra314" · use gisultra314

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra315

import "std/c/gisultra315" · use gisultra315

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra316

import "std/c/gisultra316" · use gisultra316

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra317

import "std/c/gisultra317" · use gisultra317

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra318

import "std/c/gisultra318" · use gisultra318

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra319

import "std/c/gisultra319" · use gisultra319

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra320

import "std/c/gisultra320" · use gisultra320

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra321

import "std/c/gisultra321" · use gisultra321

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra322

import "std/c/gisultra322" · use gisultra322

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra323

import "std/c/gisultra323" · use gisultra323

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra324

import "std/c/gisultra324" · use gisultra324

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra325

import "std/c/gisultra325" · use gisultra325

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra326

import "std/c/gisultra326" · use gisultra326

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra327

import "std/c/gisultra327" · use gisultra327

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra328

import "std/c/gisultra328" · use gisultra328

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra329

import "std/c/gisultra329" · use gisultra329

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra330

import "std/c/gisultra330" · use gisultra330

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra331

import "std/c/gisultra331" · use gisultra331

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra332

import "std/c/gisultra332" · use gisultra332

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra333

import "std/c/gisultra333" · use gisultra333

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra334

import "std/c/gisultra334" · use gisultra334

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra335

import "std/c/gisultra335" · use gisultra335

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra336

import "std/c/gisultra336" · use gisultra336

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra337

import "std/c/gisultra337" · use gisultra337

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra338

import "std/c/gisultra338" · use gisultra338

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra339

import "std/c/gisultra339" · use gisultra339

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra340

import "std/c/gisultra340" · use gisultra340

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra341

import "std/c/gisultra341" · use gisultra341

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra342

import "std/c/gisultra342" · use gisultra342

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra343

import "std/c/gisultra343" · use gisultra343

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra344

import "std/c/gisultra344" · use gisultra344

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra345

import "std/c/gisultra345" · use gisultra345

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra346

import "std/c/gisultra346" · use gisultra346

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra347

import "std/c/gisultra347" · use gisultra347

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra348

import "std/c/gisultra348" · use gisultra348

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra349

import "std/c/gisultra349" · use gisultra349

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra350

import "std/c/gisultra350" · use gisultra350

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra351

import "std/c/gisultra351" · use gisultra351

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra352

import "std/c/gisultra352" · use gisultra352

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra353

import "std/c/gisultra353" · use gisultra353

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra354

import "std/c/gisultra354" · use gisultra354

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra355

import "std/c/gisultra355" · use gisultra355

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra356

import "std/c/gisultra356" · use gisultra356

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra357

import "std/c/gisultra357" · use gisultra357

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra358

import "std/c/gisultra358" · use gisultra358

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra359

import "std/c/gisultra359" · use gisultra359

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra360

import "std/c/gisultra360" · use gisultra360

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra361

import "std/c/gisultra361" · use gisultra361

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra362

import "std/c/gisultra362" · use gisultra362

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra363

import "std/c/gisultra363" · use gisultra363

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra364

import "std/c/gisultra364" · use gisultra364

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra365

import "std/c/gisultra365" · use gisultra365

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra366

import "std/c/gisultra366" · use gisultra366

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra367

import "std/c/gisultra367" · use gisultra367

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra368

import "std/c/gisultra368" · use gisultra368

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra369

import "std/c/gisultra369" · use gisultra369

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra370

import "std/c/gisultra370" · use gisultra370

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra371

import "std/c/gisultra371" · use gisultra371

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra372

import "std/c/gisultra372" · use gisultra372

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra373

import "std/c/gisultra373" · use gisultra373

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra374

import "std/c/gisultra374" · use gisultra374

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra375

import "std/c/gisultra375" · use gisultra375

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra376

import "std/c/gisultra376" · use gisultra376

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra377

import "std/c/gisultra377" · use gisultra377

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra378

import "std/c/gisultra378" · use gisultra378

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra379

import "std/c/gisultra379" · use gisultra379

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra380

import "std/c/gisultra380" · use gisultra380

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra381

import "std/c/gisultra381" · use gisultra381

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra382

import "std/c/gisultra382" · use gisultra382

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra383

import "std/c/gisultra383" · use gisultra383

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra384

import "std/c/gisultra384" · use gisultra384

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra385

import "std/c/gisultra385" · use gisultra385

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra386

import "std/c/gisultra386" · use gisultra386

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra387

import "std/c/gisultra387" · use gisultra387

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra388

import "std/c/gisultra388" · use gisultra388

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra389

import "std/c/gisultra389" · use gisultra389

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra390

import "std/c/gisultra390" · use gisultra390

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra391

import "std/c/gisultra391" · use gisultra391

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra392

import "std/c/gisultra392" · use gisultra392

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra393

import "std/c/gisultra393" · use gisultra393

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra394

import "std/c/gisultra394" · use gisultra394

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra395

import "std/c/gisultra395" · use gisultra395

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra396

import "std/c/gisultra396" · use gisultra396

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra397

import "std/c/gisultra397" · use gisultra397

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra398

import "std/c/gisultra398" · use gisultra398

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra399

import "std/c/gisultra399" · use gisultra399

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra400

import "std/c/gisultra400" · use gisultra400

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra401

import "std/c/gisultra401" · use gisultra401

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra402

import "std/c/gisultra402" · use gisultra402

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra403

import "std/c/gisultra403" · use gisultra403

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra404

import "std/c/gisultra404" · use gisultra404

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra405

import "std/c/gisultra405" · use gisultra405

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra406

import "std/c/gisultra406" · use gisultra406

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra407

import "std/c/gisultra407" · use gisultra407

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra408

import "std/c/gisultra408" · use gisultra408

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra409

import "std/c/gisultra409" · use gisultra409

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra410

import "std/c/gisultra410" · use gisultra410

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra411

import "std/c/gisultra411" · use gisultra411

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra412

import "std/c/gisultra412" · use gisultra412

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra413

import "std/c/gisultra413" · use gisultra413

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra414

import "std/c/gisultra414" · use gisultra414

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra415

import "std/c/gisultra415" · use gisultra415

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra416

import "std/c/gisultra416" · use gisultra416

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra417

import "std/c/gisultra417" · use gisultra417

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra418

import "std/c/gisultra418" · use gisultra418

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra419

import "std/c/gisultra419" · use gisultra419

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra420

import "std/c/gisultra420" · use gisultra420

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra421

import "std/c/gisultra421" · use gisultra421

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra422

import "std/c/gisultra422" · use gisultra422

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra423

import "std/c/gisultra423" · use gisultra423

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra424

import "std/c/gisultra424" · use gisultra424

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra425

import "std/c/gisultra425" · use gisultra425

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra426

import "std/c/gisultra426" · use gisultra426

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra427

import "std/c/gisultra427" · use gisultra427

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra428

import "std/c/gisultra428" · use gisultra428

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra429

import "std/c/gisultra429" · use gisultra429

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra430

import "std/c/gisultra430" · use gisultra430

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra431

import "std/c/gisultra431" · use gisultra431

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra432

import "std/c/gisultra432" · use gisultra432

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra433

import "std/c/gisultra433" · use gisultra433

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra434

import "std/c/gisultra434" · use gisultra434

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra435

import "std/c/gisultra435" · use gisultra435

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra436

import "std/c/gisultra436" · use gisultra436

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra437

import "std/c/gisultra437" · use gisultra437

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra438

import "std/c/gisultra438" · use gisultra438

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra439

import "std/c/gisultra439" · use gisultra439

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra440

import "std/c/gisultra440" · use gisultra440

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra441

import "std/c/gisultra441" · use gisultra441

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra442

import "std/c/gisultra442" · use gisultra442

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra443

import "std/c/gisultra443" · use gisultra443

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra444

import "std/c/gisultra444" · use gisultra444

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra445

import "std/c/gisultra445" · use gisultra445

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra446

import "std/c/gisultra446" · use gisultra446

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra447

import "std/c/gisultra447" · use gisultra447

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra448

import "std/c/gisultra448" · use gisultra448

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra449

import "std/c/gisultra449" · use gisultra449

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra450

import "std/c/gisultra450" · use gisultra450

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra451

import "std/c/gisultra451" · use gisultra451

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra452

import "std/c/gisultra452" · use gisultra452

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra453

import "std/c/gisultra453" · use gisultra453

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra454

import "std/c/gisultra454" · use gisultra454

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra455

import "std/c/gisultra455" · use gisultra455

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra456

import "std/c/gisultra456" · use gisultra456

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra457

import "std/c/gisultra457" · use gisultra457

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra458

import "std/c/gisultra458" · use gisultra458

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra459

import "std/c/gisultra459" · use gisultra459

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra460

import "std/c/gisultra460" · use gisultra460

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra461

import "std/c/gisultra461" · use gisultra461

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra462

import "std/c/gisultra462" · use gisultra462

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra463

import "std/c/gisultra463" · use gisultra463

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra464

import "std/c/gisultra464" · use gisultra464

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra465

import "std/c/gisultra465" · use gisultra465

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra466

import "std/c/gisultra466" · use gisultra466

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra467

import "std/c/gisultra467" · use gisultra467

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra468

import "std/c/gisultra468" · use gisultra468

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra469

import "std/c/gisultra469" · use gisultra469

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra470

import "std/c/gisultra470" · use gisultra470

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra471

import "std/c/gisultra471" · use gisultra471

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra472

import "std/c/gisultra472" · use gisultra472

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra473

import "std/c/gisultra473" · use gisultra473

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra474

import "std/c/gisultra474" · use gisultra474

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra475

import "std/c/gisultra475" · use gisultra475

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra476

import "std/c/gisultra476" · use gisultra476

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra477

import "std/c/gisultra477" · use gisultra477

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra478

import "std/c/gisultra478" · use gisultra478

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra479

import "std/c/gisultra479" · use gisultra479

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra480

import "std/c/gisultra480" · use gisultra480

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra481

import "std/c/gisultra481" · use gisultra481

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra482

import "std/c/gisultra482" · use gisultra482

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra483

import "std/c/gisultra483" · use gisultra483

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra484

import "std/c/gisultra484" · use gisultra484

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra485

import "std/c/gisultra485" · use gisultra485

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra486

import "std/c/gisultra486" · use gisultra486

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra487

import "std/c/gisultra487" · use gisultra487

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra488

import "std/c/gisultra488" · use gisultra488

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra489

import "std/c/gisultra489" · use gisultra489

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra490

import "std/c/gisultra490" · use gisultra490

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra491

import "std/c/gisultra491" · use gisultra491

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra492

import "std/c/gisultra492" · use gisultra492

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra493

import "std/c/gisultra493" · use gisultra493

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`

- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra494

import "std/c/gisultra494" · use gisultra494

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra495

import "std/c/gisultra495" · use gisultra495

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra496

import "std/c/gisultra496" · use gisultra496

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra497

import "std/c/gisultra497" · use gisultra497

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra498

import "std/c/gisultra498" · use gisultra498

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra499

import "std/c/gisultra499" · use gisultra499

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/gisultra500

import "std/c/gisultra500" · use gisultra500

- `ndviRaster(nir list number, red list number) → list number`
- `rasterWindowMean(grid list number, w number, h number, x number, y number, r number) → number`
- `hillshadeRaster(slopeDeg list number, aspectDeg list number) → list number`

std/c/rasterultra001

import "std/c/rasterultra001" · use rasterultra001

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra002

import "std/c/rasterultra002" · use rasterultra002

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra003

import "std/c/rasterultra003" · use rasterultra003

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra004

import "std/c/rasterultra004" · use rasterultra004

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra005

import "std/c/rasterultra005" · use rasterultra005

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`

- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra006

import "std/c/rasterultra006" · use rasterultra006

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra007

import "std/c/rasterultra007" · use rasterultra007

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra008

import "std/c/rasterultra008" · use rasterultra008

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra009

import "std/c/rasterultra009" · use rasterultra009

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra010

import "std/c/rasterultra010" · use rasterultra010

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra011

import "std/c/rasterultra011" · use rasterultra011

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra012

import "std/c/rasterultra012" · use rasterultra012

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra013

import "std/c/rasterultra013" · use rasterultra013

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra014

import "std/c/rasterultra014" · use rasterultra014

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra015

import "std/c/rasterultra015" · use rasterultra015

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra016

import "std/c/rasterultra016" · use rasterultra016

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra017

import "std/c/rasterultra017" · use rasterultra017

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra018

import "std/c/rasterultra018" · use rasterultra018

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra019

import "std/c/rasterultra019" · use rasterultra019

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra020

import "std/c/rasterultra020" · use rasterultra020

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra021

import "std/c/rasterultra021" · use rasterultra021

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra022

import "std/c/rasterultra022" · use rasterultra022

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra023

import "std/c/rasterultra023" · use rasterultra023

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra024

import "std/c/rasterultra024" · use rasterultra024

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra025

import "std/c/rasterultra025" · use rasterultra025

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra026

import "std/c/rasterultra026" · use rasterultra026

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra027

import "std/c/rasterultra027" · use rasterultra027

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra028

import "std/c/rasterultra028" · use rasterultra028

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra029

import "std/c/rasterultra029" · use rasterultra029

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra030

import "std/c/rasterultra030" · use rasterultra030

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra031

import "std/c/rasterultra031" · use rasterultra031

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra032

import "std/c/rasterultra032" · use rasterultra032

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra033

import "std/c/rasterultra033" · use rasterultra033

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra034

import "std/c/rasterultra034" · use rasterultra034

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra035

import "std/c/rasterultra035" · use rasterultra035

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra036

import "std/c/rasterultra036" · use rasterultra036

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra037

import "std/c/rasterultra037" · use rasterultra037

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra038

import "std/c/rasterultra038" · use rasterultra038

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra039

import "std/c/rasterultra039" · use rasterultra039

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra040

import "std/c/rasterultra040" · use rasterultra040

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra041

import "std/c/rasterultra041" · use rasterultra041

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra042

import "std/c/rasterultra042" · use rasterultra042

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra043

import "std/c/rasterultra043" · use rasterultra043

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra044

import "std/c/rasterultra044" · use rasterultra044

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra045

import "std/c/rasterultra045" · use rasterultra045

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra046

import "std/c/rasterultra046" · use rasterultra046

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra047

import "std/c/rasterultra047" · use rasterultra047

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra048

import "std/c/rasterultra048" · use rasterultra048

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra049

import "std/c/rasterultra049" · use rasterultra049

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra050

import "std/c/rasterultra050" · use rasterultra050

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra051

import "std/c/rasterultra051" · use rasterultra051

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra052

import "std/c/rasterultra052" · use rasterultra052

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra053

import "std/c/rasterultra053" · use rasterultra053

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra054

import "std/c/rasterultra054" · use rasterultra054

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra055

import "std/c/rasterultra055" · use rasterultra055

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra056

import "std/c/rasterultra056" · use rasterultra056

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra057

import "std/c/rasterultra057" · use rasterultra057

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra058

import "std/c/rasterultra058" · use rasterultra058

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra059

import "std/c/rasterultra059" · use rasterultra059

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra060

import "std/c/rasterultra060" · use rasterultra060

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra061

import "std/c/rasterultra061" · use rasterultra061

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra062

import "std/c/rasterultra062" · use rasterultra062

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra063

import "std/c/rasterultra063" · use rasterultra063

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra064

import "std/c/rasterultra064" · use rasterultra064

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra065

import "std/c/rasterultra065" · use rasterultra065

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra066

import "std/c/rasterultra066" · use rasterultra066

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra067

import "std/c/rasterultra067" · use rasterultra067

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra068

import "std/c/rasterultra068" · use rasterultra068

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra069

import "std/c/rasterultra069" · use rasterultra069

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra070

import "std/c/rasterultra070" · use rasterultra070

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra071

import "std/c/rasterultra071" · use rasterultra071

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra072

import "std/c/rasterultra072" · use rasterultra072

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra073

import "std/c/rasterultra073" · use rasterultra073

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra074

import "std/c/rasterultra074" · use rasterultra074

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra075

import "std/c/rasterultra075" · use rasterultra075

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra076

import "std/c/rasterultra076" · use rasterultra076

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra077

import "std/c/rasterultra077" · use rasterultra077

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra078

import "std/c/rasterultra078" · use rasterultra078

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra079

import "std/c/rasterultra079" · use rasterultra079

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra080

import "std/c/rasterultra080" · use rasterultra080

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra081

import "std/c/rasterultra081" · use rasterultra081

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra082

import "std/c/rasterultra082" · use rasterultra082

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra083

import "std/c/rasterultra083" · use rasterultra083

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra084

import "std/c/rasterultra084" · use rasterultra084

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra085

import "std/c/rasterultra085" · use rasterultra085

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra086

import "std/c/rasterultra086" · use rasterultra086

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra087

import "std/c/rasterultra087" · use rasterultra087

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra088

import "std/c/rasterultra088" · use rasterultra088

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra089

import "std/c/rasterultra089" · use rasterultra089

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra090

import "std/c/rasterultra090" · use rasterultra090

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra091

import "std/c/rasterultra091" · use rasterultra091

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra092

import "std/c/rasterultra092" · use rasterultra092

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra093

import "std/c/rasterultra093" · use rasterultra093

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra094

import "std/c/rasterultra094" · use rasterultra094

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra095

import "std/c/rasterultra095" · use rasterultra095

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra096

import "std/c/rasterultra096" · use rasterultra096

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra097

import "std/c/rasterultra097" · use rasterultra097

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra098

import "std/c/rasterultra098" · use rasterultra098

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra099

import "std/c/rasterultra099" · use rasterultra099

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra100

import "std/c/rasterultra100" · use rasterultra100

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra101

import "std/c/rasterultra101" · use rasterultra101

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra102

import "std/c/rasterultra102" · use rasterultra102

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra103

import "std/c/rasterultra103" · use rasterultra103

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra104

import "std/c/rasterultra104" · use rasterultra104

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra105

import "std/c/rasterultra105" · use rasterultra105

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra106

import "std/c/rasterultra106" · use rasterultra106

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra107

import "std/c/rasterultra107" · use rasterultra107

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra108

import "std/c/rasterultra108" · use rasterultra108

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra109

import "std/c/rasterultra109" · use rasterultra109

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra110

import "std/c/rasterultra110" · use rasterultra110

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra111

import "std/c/rasterultra111" · use rasterultra111

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra112

import "std/c/rasterultra112" · use rasterultra112

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra113

import "std/c/rasterultra113" · use rasterultra113

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra114

import "std/c/rasterultra114" · use rasterultra114

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra115

import "std/c/rasterultra115" · use rasterultra115

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra116

import "std/c/rasterultra116" · use rasterultra116

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra117

import "std/c/rasterultra117" · use rasterultra117

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra118

import "std/c/rasterultra118" · use rasterultra118

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra119

import "std/c/rasterultra119" · use rasterultra119

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra120

import "std/c/rasterultra120" · use rasterultra120

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra121

import "std/c/rasterultra121" · use rasterultra121

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra122

import "std/c/rasterultra122" · use rasterultra122

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra123

import "std/c/rasterultra123" · use rasterultra123

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra124

import "std/c/rasterultra124" · use rasterultra124

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra125

import "std/c/rasterultra125" · use rasterultra125

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra126

import "std/c/rasterultra126" · use rasterultra126

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra127

import "std/c/rasterultra127" · use rasterultra127

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra128

import "std/c/rasterultra128" · use rasterultra128

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra129

import "std/c/rasterultra129" · use rasterultra129

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra130

import "std/c/rasterultra130" · use rasterultra130

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra131

import "std/c/rasterultra131" · use rasterultra131

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra132

import "std/c/rasterultra132" · use rasterultra132

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra133

import "std/c/rasterultra133" · use rasterultra133

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra134

import "std/c/rasterultra134" · use rasterultra134

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra135

import "std/c/rasterultra135" · use rasterultra135

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra136

import "std/c/rasterultra136" · use rasterultra136

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra137

import "std/c/rasterultra137" · use rasterultra137

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra138

import "std/c/rasterultra138" · use rasterultra138

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra139

import "std/c/rasterultra139" · use rasterultra139

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra140

import "std/c/rasterultra140" · use rasterultra140

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra141

import "std/c/rasterultra141" · use rasterultra141

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra142

import "std/c/rasterultra142" · use rasterultra142

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra143

import "std/c/rasterultra143" · use rasterultra143

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra144

import "std/c/rasterultra144" · use rasterultra144

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra145

import "std/c/rasterultra145" · use rasterultra145

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra146

import "std/c/rasterultra146" · use rasterultra146

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra147

import "std/c/rasterultra147" · use rasterultra147

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra148

import "std/c/rasterultra148" · use rasterultra148

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra149

import "std/c/rasterultra149" · use rasterultra149

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra150

import "std/c/rasterultra150" · use rasterultra150

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra151

import "std/c/rasterultra151" · use rasterultra151

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra152

import "std/c/rasterultra152" · use rasterultra152

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra153

import "std/c/rasterultra153" · use rasterultra153

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra154

import "std/c/rasterultra154" · use rasterultra154

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra155

import "std/c/rasterultra155" · use rasterultra155

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra156

import "std/c/rasterultra156" · use rasterultra156

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra157

import "std/c/rasterultra157" · use rasterultra157

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra158

import "std/c/rasterultra158" · use rasterultra158

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra159

import "std/c/rasterultra159" · use rasterultra159

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra160

import "std/c/rasterultra160" · use rasterultra160

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra161

import "std/c/rasterultra161" · use rasterultra161

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra162

import "std/c/rasterultra162" · use rasterultra162

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra163

import "std/c/rasterultra163" · use rasterultra163

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra164

import "std/c/rasterultra164" · use rasterultra164

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra165

import "std/c/rasterultra165" · use rasterultra165

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra166

import "std/c/rasterultra166" · use rasterultra166

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra167

import "std/c/rasterultra167" · use rasterultra167

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra168

import "std/c/rasterultra168" · use rasterultra168

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra169

import "std/c/rasterultra169" · use rasterultra169

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra170

import "std/c/rasterultra170" · use rasterultra170

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra171

import "std/c/rasterultra171" · use rasterultra171

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra172

import "std/c/rasterultra172" · use rasterultra172

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra173

import "std/c/rasterultra173" · use rasterultra173

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra174

import "std/c/rasterultra174" · use rasterultra174

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra175

import "std/c/rasterultra175" · use rasterultra175

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra176

import "std/c/rasterultra176" · use rasterultra176

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra177

import "std/c/rasterultra177" · use rasterultra177

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra178

import "std/c/rasterultra178" · use rasterultra178

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra179

import "std/c/rasterultra179" · use rasterultra179

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra180

import "std/c/rasterultra180" · use rasterultra180

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra181

import "std/c/rasterultra181" · use rasterultra181

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra182

import "std/c/rasterultra182" · use rasterultra182

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra183

import "std/c/rasterultra183" · use rasterultra183

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra184

import "std/c/rasterultra184" · use rasterultra184

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra185

import "std/c/rasterultra185" · use rasterultra185

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra186

import "std/c/rasterultra186" · use rasterultra186

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra187

import "std/c/rasterultra187" · use rasterultra187

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra188

import "std/c/rasterultra188" · use rasterultra188

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra189

import "std/c/rasterultra189" · use rasterultra189

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra190

import "std/c/rasterultra190" · use rasterultra190

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra191

import "std/c/rasterultra191" · use rasterultra191

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra192

import "std/c/rasterultra192" · use rasterultra192

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra193

import "std/c/rasterultra193" · use rasterultra193

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra194

import "std/c/rasterultra194" · use rasterultra194

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra195

import "std/c/rasterultra195" · use rasterultra195

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra196

import "std/c/rasterultra196" · use rasterultra196

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra197

import "std/c/rasterultra197" · use rasterultra197

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra198

import "std/c/rasterultra198" · use rasterultra198

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra199

import "std/c/rasterultra199" · use rasterultra199

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra200

import "std/c/rasterultra200" · use rasterultra200

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra201

import "std/c/rasterultra201" · use rasterultra201

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra202

import "std/c/rasterultra202" · use rasterultra202

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra203

import "std/c/rasterultra203" · use rasterultra203

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra204

import "std/c/rasterultra204" · use rasterultra204

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra205

import "std/c/rasterultra205" · use rasterultra205

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra206

import "std/c/rasterultra206" · use rasterultra206

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra207

import "std/c/rasterultra207" · use rasterultra207

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra208

import "std/c/rasterultra208" · use rasterultra208

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra209

import "std/c/rasterultra209" · use rasterultra209

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra210

import "std/c/rasterultra210" · use rasterultra210

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra211

import "std/c/rasterultra211" · use rasterultra211

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra212

import "std/c/rasterultra212" · use rasterultra212

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra213

import "std/c/rasterultra213" · use rasterultra213

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra214

import "std/c/rasterultra214" · use rasterultra214

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra215

import "std/c/rasterultra215" · use rasterultra215

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra216

import "std/c/rasterultra216" · use rasterultra216

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra217

import "std/c/rasterultra217" · use rasterultra217

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra218

import "std/c/rasterultra218" · use rasterultra218

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra219

import "std/c/rasterultra219" · use rasterultra219

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra220

import "std/c/rasterultra220" · use rasterultra220

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra221

import "std/c/rasterultra221" · use rasterultra221

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra222

import "std/c/rasterultra222" · use rasterultra222

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra223

import "std/c/rasterultra223" · use rasterultra223

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra224

import "std/c/rasterultra224" · use rasterultra224

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra225

import "std/c/rasterultra225" · use rasterultra225

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra226

import "std/c/rasterultra226" · use rasterultra226

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra227

import "std/c/rasterultra227" · use rasterultra227

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra228

import "std/c/rasterultra228" · use rasterultra228

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra229

import "std/c/rasterultra229" · use rasterultra229

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra230

import "std/c/rasterultra230" · use rasterultra230

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra231

import "std/c/rasterultra231" · use rasterultra231

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra232

import "std/c/rasterultra232" · use rasterultra232

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra233

import "std/c/rasterultra233" · use rasterultra233

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra234

import "std/c/rasterultra234" · use rasterultra234

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra235

import "std/c/rasterultra235" · use rasterultra235

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra236

import "std/c/rasterultra236" · use rasterultra236

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra237

import "std/c/rasterultra237" · use rasterultra237

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra238

import "std/c/rasterultra238" · use rasterultra238

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra239

import "std/c/rasterultra239" · use rasterultra239

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra240

import "std/c/rasterultra240" · use rasterultra240

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra241

import "std/c/rasterultra241" · use rasterultra241

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra242

import "std/c/rasterultra242" · use rasterultra242

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra243

import "std/c/rasterultra243" · use rasterultra243

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra244

import "std/c/rasterultra244" · use rasterultra244

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra245

import "std/c/rasterultra245" · use rasterultra245

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra246

import "std/c/rasterultra246" · use rasterultra246

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra247

import "std/c/rasterultra247" · use rasterultra247

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra248

import "std/c/rasterultra248" · use rasterultra248

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra249

import "std/c/rasterultra249" · use rasterultra249

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra250

import "std/c/rasterultra250" · use rasterultra250

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra251

import "std/c/rasterultra251" · use rasterultra251

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra252

import "std/c/rasterultra252" · use rasterultra252

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra253

import "std/c/rasterultra253" · use rasterultra253

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra254

import "std/c/rasterultra254" · use rasterultra254

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra255

import "std/c/rasterultra255" · use rasterultra255

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra256

import "std/c/rasterultra256" · use rasterultra256

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra257

import "std/c/rasterultra257" · use rasterultra257

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra258

import "std/c/rasterultra258" · use rasterultra258

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra259

import "std/c/rasterultra259" · use rasterultra259

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra260

import "std/c/rasterultra260" · use rasterultra260

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra261

import "std/c/rasterultra261" · use rasterultra261

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra262

import "std/c/rasterultra262" · use rasterultra262

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra263

import "std/c/rasterultra263" · use rasterultra263

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra264

import "std/c/rasterultra264" · use rasterultra264

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra265

import "std/c/rasterultra265" · use rasterultra265

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra266

import "std/c/rasterultra266" · use rasterultra266

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra267

import "std/c/rasterultra267" · use rasterultra267

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra268

import "std/c/rasterultra268" · use rasterultra268

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra269

import "std/c/rasterultra269" · use rasterultra269

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra270

import "std/c/rasterultra270" · use rasterultra270

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra271

import "std/c/rasterultra271" · use rasterultra271

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra272

import "std/c/rasterultra272" · use rasterultra272

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra273

import "std/c/rasterultra273" · use rasterultra273

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra274

import "std/c/rasterultra274" · use rasterultra274

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra275

import "std/c/rasterultra275" · use rasterultra275

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra276

import "std/c/rasterultra276" · use rasterultra276

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra277

import "std/c/rasterultra277" · use rasterultra277

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra278

import "std/c/rasterultra278" · use rasterultra278

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra279

import "std/c/rasterultra279" · use rasterultra279

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra280

import "std/c/rasterultra280" · use rasterultra280

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra281

import "std/c/rasterultra281" · use rasterultra281

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra282

import "std/c/rasterultra282" · use rasterultra282

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra283

import "std/c/rasterultra283" · use rasterultra283

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra284

import "std/c/rasterultra284" · use rasterultra284

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra285

import "std/c/rasterultra285" · use rasterultra285

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra286

import "std/c/rasterultra286" · use rasterultra286

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra287

import "std/c/rasterultra287" · use rasterultra287

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra288

import "std/c/rasterultra288" · use rasterultra288

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra289

import "std/c/rasterultra289" · use rasterultra289

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra290

import "std/c/rasterultra290" · use rasterultra290

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra291

import "std/c/rasterultra291" · use rasterultra291

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra292

import "std/c/rasterultra292" · use rasterultra292

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra293

import "std/c/rasterultra293" · use rasterultra293

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra294

import "std/c/rasterultra294" · use rasterultra294

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra295

import "std/c/rasterultra295" · use rasterultra295

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra296

import "std/c/rasterultra296" · use rasterultra296

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra297

import "std/c/rasterultra297" · use rasterultra297

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra298

import "std/c/rasterultra298" · use rasterultra298

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra299

import "std/c/rasterultra299" · use rasterultra299

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra300

import "std/c/rasterultra300" · use rasterultra300

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra301

import "std/c/rasterultra301" · use rasterultra301

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra302

import "std/c/rasterultra302" · use rasterultra302

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra303

import "std/c/rasterultra303" · use rasterultra303

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra304

import "std/c/rasterultra304" · use rasterultra304

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra305

import "std/c/rasterultra305" · use rasterultra305

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra306

import "std/c/rasterultra306" · use rasterultra306

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra307

import "std/c/rasterultra307" · use rasterultra307

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra308

import "std/c/rasterultra308" · use rasterultra308

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra309

import "std/c/rasterultra309" · use rasterultra309

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra310

import "std/c/rasterultra310" · use rasterultra310

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra311

import "std/c/rasterultra311" · use rasterultra311

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra312

import "std/c/rasterultra312" · use rasterultra312

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra313

import "std/c/rasterultra313" · use rasterultra313

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra314

import "std/c/rasterultra314" · use rasterultra314

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra315

import "std/c/rasterultra315" · use rasterultra315

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra316

import "std/c/rasterultra316" · use rasterultra316

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra317

import "std/c/rasterultra317" · use rasterultra317

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra318

import "std/c/rasterultra318" · use rasterultra318

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra319

import "std/c/rasterultra319" · use rasterultra319

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra320

import "std/c/rasterultra320" · use rasterultra320

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra321

import "std/c/rasterultra321" · use rasterultra321

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra322

import "std/c/rasterultra322" · use rasterultra322

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra323

import "std/c/rasterultra323" · use rasterultra323

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra324

import "std/c/rasterultra324" · use rasterultra324

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra325

import "std/c/rasterultra325" · use rasterultra325

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra326

import "std/c/rasterultra326" · use rasterultra326

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra327

import "std/c/rasterultra327" · use rasterultra327

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra328

import "std/c/rasterultra328" · use rasterultra328

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra329

import "std/c/rasterultra329" · use rasterultra329

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra330

import "std/c/rasterultra330" · use rasterultra330

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra331

import "std/c/rasterultra331" · use rasterultra331

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra332

import "std/c/rasterultra332" · use rasterultra332

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra333

import "std/c/rasterultra333" · use rasterultra333

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra334

import "std/c/rasterultra334" · use rasterultra334

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra335

import "std/c/rasterultra335" · use rasterultra335

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra336

import "std/c/rasterultra336" · use rasterultra336

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra337

import "std/c/rasterultra337" · use rasterultra337

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra338

import "std/c/rasterultra338" · use rasterultra338

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra339

import "std/c/rasterultra339" · use rasterultra339

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra340

import "std/c/rasterultra340" · use rasterultra340

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra341

import "std/c/rasterultra341" · use rasterultra341

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra342

import "std/c/rasterultra342" · use rasterultra342

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra343

import "std/c/rasterultra343" · use rasterultra343

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra344

import "std/c/rasterultra344" · use rasterultra344

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra345

import "std/c/rasterultra345" · use rasterultra345

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra346

import "std/c/rasterultra346" · use rasterultra346

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra347

import "std/c/rasterultra347" · use rasterultra347

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra348

import "std/c/rasterultra348" · use rasterultra348

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra349

import "std/c/rasterultra349" · use rasterultra349

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra350

import "std/c/rasterultra350" · use rasterultra350

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra351

import "std/c/rasterultra351" · use rasterultra351

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra352

import "std/c/rasterultra352" · use rasterultra352

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra353

import "std/c/rasterultra353" · use rasterultra353

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra354

import "std/c/rasterultra354" · use rasterultra354

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra355

import "std/c/rasterultra355" · use rasterultra355

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra356

import "std/c/rasterultra356" · use rasterultra356

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra357

import "std/c/rasterultra357" · use rasterultra357

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra358

import "std/c/rasterultra358" · use rasterultra358

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra359

import "std/c/rasterultra359" · use rasterultra359

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra360

import "std/c/rasterultra360" · use rasterultra360

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra361

import "std/c/rasterultra361" · use rasterultra361

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra362

import "std/c/rasterultra362" · use rasterultra362

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra363

import "std/c/rasterultra363" · use rasterultra363

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra364

import "std/c/rasterultra364" · use rasterultra364

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra365

import "std/c/rasterultra365" · use rasterultra365

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra366

import "std/c/rasterultra366" · use rasterultra366

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra367

import "std/c/rasterultra367" · use rasterultra367

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra368

import "std/c/rasterultra368" · use rasterultra368

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra369

import "std/c/rasterultra369" · use rasterultra369

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra370

import "std/c/rasterultra370" · use rasterultra370

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra371

import "std/c/rasterultra371" · use rasterultra371

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra372

import "std/c/rasterultra372" · use rasterultra372

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra373

import "std/c/rasterultra373" · use rasterultra373

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra374

import "std/c/rasterultra374" · use rasterultra374

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra375

import "std/c/rasterultra375" · use rasterultra375

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra376

import "std/c/rasterultra376" · use rasterultra376

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra377

import "std/c/rasterultra377" · use rasterultra377

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra378

import "std/c/rasterultra378" · use rasterultra378

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra379

import "std/c/rasterultra379" · use rasterultra379

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra380

import "std/c/rasterultra380" · use rasterultra380

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra381

import "std/c/rasterultra381" · use rasterultra381

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra382

import "std/c/rasterultra382" · use rasterultra382

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra383

import "std/c/rasterultra383" · use rasterultra383

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra384

import "std/c/rasterultra384" · use rasterultra384

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra385

import "std/c/rasterultra385" · use rasterultra385

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra386

import "std/c/rasterultra386" · use rasterultra386

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra387

import "std/c/rasterultra387" · use rasterultra387

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra388

import "std/c/rasterultra388" · use rasterultra388

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra389

import "std/c/rasterultra389" · use rasterultra389

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra390

import "std/c/rasterultra390" · use rasterultra390

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra391

import "std/c/rasterultra391" · use rasterultra391

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra392

import "std/c/rasterultra392" · use rasterultra392

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra393

import "std/c/rasterultra393" · use rasterultra393

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra394

import "std/c/rasterultra394" · use rasterultra394

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra395

import "std/c/rasterultra395" · use rasterultra395

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra396

import "std/c/rasterultra396" · use rasterultra396

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra397

import "std/c/rasterultra397" · use rasterultra397

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra398

import "std/c/rasterultra398" · use rasterultra398

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra399

import "std/c/rasterultra399" · use rasterultra399

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra400

import "std/c/rasterultra400" · use rasterultra400

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra401

import "std/c/rasterultra401" · use rasterultra401

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra402

import "std/c/rasterultra402" · use rasterultra402

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra403

import "std/c/rasterultra403" · use rasterultra403

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra404

import "std/c/rasterultra404" · use rasterultra404

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra405

import "std/c/rasterultra405" · use rasterultra405

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra406

import "std/c/rasterultra406" · use rasterultra406

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra407

import "std/c/rasterultra407" · use rasterultra407

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra408

import "std/c/rasterultra408" · use rasterultra408

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra409

import "std/c/rasterultra409" · use rasterultra409

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra410

import "std/c/rasterultra410" · use rasterultra410

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra411

import "std/c/rasterultra411" · use rasterultra411

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra412

import "std/c/rasterultra412" · use rasterultra412

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra413

import "std/c/rasterultra413" · use rasterultra413

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra414

import "std/c/rasterultra414" · use rasterultra414

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra415

import "std/c/rasterultra415" · use rasterultra415

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra416

import "std/c/rasterultra416" · use rasterultra416

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra417

import "std/c/rasterultra417" · use rasterultra417

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra418

import "std/c/rasterultra418" · use rasterultra418

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra419

import "std/c/rasterultra419" · use rasterultra419

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra420

import "std/c/rasterultra420" · use rasterultra420

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra421

import "std/c/rasterultra421" · use rasterultra421

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra422

import "std/c/rasterultra422" · use rasterultra422

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra423

import "std/c/rasterultra423" · use rasterultra423

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra424

import "std/c/rasterultra424" · use rasterultra424

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra425

import "std/c/rasterultra425" · use rasterultra425

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra426

import "std/c/rasterultra426" · use rasterultra426

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra427

import "std/c/rasterultra427" · use rasterultra427

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra428

import "std/c/rasterultra428" · use rasterultra428

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra429

import "std/c/rasterultra429" · use rasterultra429

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra430

import "std/c/rasterultra430" · use rasterultra430

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra431

import "std/c/rasterultra431" · use rasterultra431

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra432

import "std/c/rasterultra432" · use rasterultra432

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra433

import "std/c/rasterultra433" · use rasterultra433

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra434

import "std/c/rasterultra434" · use rasterultra434

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra435

import "std/c/rasterultra435" · use rasterultra435

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra436

import "std/c/rasterultra436" · use rasterultra436

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra437

import "std/c/rasterultra437" · use rasterultra437

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra438

import "std/c/rasterultra438" · use rasterultra438

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra439

import "std/c/rasterultra439" · use rasterultra439

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra440

import "std/c/rasterultra440" · use rasterultra440

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra441

import "std/c/rasterultra441" · use rasterultra441

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra442

import "std/c/rasterultra442" · use rasterultra442

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra443

import "std/c/rasterultra443" · use rasterultra443

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra444

import "std/c/rasterultra444" · use rasterultra444

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra445

import "std/c/rasterultra445" · use rasterultra445

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra446

import "std/c/rasterultra446" · use rasterultra446

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra447

import "std/c/rasterultra447" · use rasterultra447

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra448

import "std/c/rasterultra448" · use rasterultra448

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra449

import "std/c/rasterultra449" · use rasterultra449

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra450

import "std/c/rasterultra450" · use rasterultra450

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra451

import "std/c/rasterultra451" · use rasterultra451

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra452

import "std/c/rasterultra452" · use rasterultra452

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra453

import "std/c/rasterultra453" · use rasterultra453

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra454

import "std/c/rasterultra454" · use rasterultra454

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra455

import "std/c/rasterultra455" · use rasterultra455

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra456

import "std/c/rasterultra456" · use rasterultra456

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra457

import "std/c/rasterultra457" · use rasterultra457

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra458

import "std/c/rasterultra458" · use rasterultra458

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra459

import "std/c/rasterultra459" · use rasterultra459

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra460

import "std/c/rasterultra460" · use rasterultra460

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra461

import "std/c/rasterultra461" · use rasterultra461

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra462

import "std/c/rasterultra462" · use rasterultra462

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra463

import "std/c/rasterultra463" · use rasterultra463

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra464

import "std/c/rasterultra464" · use rasterultra464

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra465

import "std/c/rasterultra465" · use rasterultra465

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra466

import "std/c/rasterultra466" · use rasterultra466

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra467

import "std/c/rasterultra467" · use rasterultra467

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra468

import "std/c/rasterultra468" · use rasterultra468

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra469

import "std/c/rasterultra469" · use rasterultra469

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra470

import "std/c/rasterultra470" · use rasterultra470

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra471

import "std/c/rasterultra471" · use rasterultra471

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra472

import "std/c/rasterultra472" · use rasterultra472

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra473

import "std/c/rasterultra473" · use rasterultra473

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra474

import "std/c/rasterultra474" · use rasterultra474

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra475

import "std/c/rasterultra475" · use rasterultra475

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra476

import "std/c/rasterultra476" · use rasterultra476

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra477

import "std/c/rasterultra477" · use rasterultra477

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra478

import "std/c/rasterultra478" · use rasterultra478

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra479

import "std/c/rasterultra479" · use rasterultra479

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra480

import "std/c/rasterultra480" · use rasterultra480

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra481

import "std/c/rasterultra481" · use rasterultra481

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra482

import "std/c/rasterultra482" · use rasterultra482

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra483

import "std/c/rasterultra483" · use rasterultra483

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra484

import "std/c/rasterultra484" · use rasterultra484

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra485

import "std/c/rasterultra485" · use rasterultra485

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra486

import "std/c/rasterultra486" · use rasterultra486

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra487

import "std/c/rasterultra487" · use rasterultra487

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra488

import "std/c/rasterultra488" · use rasterultra488

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra489

import "std/c/rasterultra489" · use rasterultra489

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra490

import "std/c/rasterultra490" · use rasterultra490

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra491

import "std/c/rasterultra491" · use rasterultra491

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra492

import "std/c/rasterultra492" · use rasterultra492

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra493

import "std/c/rasterultra493" · use rasterultra493

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra494

import "std/c/rasterultra494" · use rasterultra494

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra495

import "std/c/rasterultra495" · use rasterultra495

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra496

import "std/c/rasterultra496" · use rasterultra496

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra497

import "std/c/rasterultra497" · use rasterultra497

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra498

import "std/c/rasterultra498" · use rasterultra498

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra499

import "std/c/rasterultra499" · use rasterultra499

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/rasterultra500

import "std/c/rasterultra500" · use rasterultra500

- `boxBlur3(grid list number, w number, h number) → list number`
- `sobelMag(grid list number, w number, h number) → list number`
- `threshold01(grid list number, w number, h number, t number) → list number`
- `ndviMask(nir list number, red list number, thr number) → list number`
- `smoothRaster(prev list number, cur list number, alpha number) → list number`

std/c/segultra001

import "std/c/segultra001" · use segultra001

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra002

import "std/c/segultra002" · use segultra002

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra003

import "std/c/segultra003" · use segultra003

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra004

import "std/c/segultra004" · use segultra004

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra005

import "std/c/segultra005" · use segultra005

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra006

import "std/c/segultra006" · use segultra006

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra007

import "std/c/segultra007" · use segultra007

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra008

import "std/c/segultra008" · use segultra008

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra009

import "std/c/segultra009" · use segultra009

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra010

import "std/c/segultra010" · use segultra010

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra011

import "std/c/segultra011" · use segultra011

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra012

import "std/c/segultra012" · use segultra012

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra013

import "std/c/segultra013" · use segultra013

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra014

import "std/c/segultra014" · use segultra014

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra015

import "std/c/segultra015" · use segultra015

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra016

import "std/c/segultra016" · use segultra016

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra017

import "std/c/segultra017" · use segultra017

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra018

import "std/c/segultra018" · use segultra018

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra019

import "std/c/segultra019" · use segultra019

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra020

import "std/c/segultra020" · use segultra020

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra021

import "std/c/segultra021" · use segultra021

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra022

import "std/c/segultra022" · use segultra022

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra023

import "std/c/segultra023" · use segultra023

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra024

import "std/c/segultra024" · use segultra024

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra025

import "std/c/segultra025" · use segultra025

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra026

import "std/c/segultra026" · use segultra026

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra027

import "std/c/segultra027" · use segultra027

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra028

import "std/c/segultra028" · use segultra028

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra029

import "std/c/segultra029" · use segultra029

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra030

import "std/c/segultra030" · use segultra030

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra031

import "std/c/segultra031" · use segultra031

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra032

import "std/c/segultra032" · use segultra032

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra033

import "std/c/segultra033" · use segultra033

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra034

import "std/c/segultra034" · use segultra034

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra035

import "std/c/segultra035" · use segultra035

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra036

import "std/c/segultra036" · use segultra036

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra037

import "std/c/segultra037" · use segultra037

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra038

import "std/c/segultra038" · use segultra038

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra039

import "std/c/segultra039" · use segultra039

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra040

import "std/c/segultra040" · use segultra040

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra041

import "std/c/segultra041" · use segultra041

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra042

import "std/c/segultra042" · use segultra042

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra043

import "std/c/segultra043" · use segultra043

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra044

import "std/c/segultra044" · use segultra044

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra045

import "std/c/segultra045" · use segultra045

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra046

import "std/c/segultra046" · use segultra046

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra047

import "std/c/segultra047" · use segultra047

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra048

import "std/c/segultra048" · use segultra048

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra049

import "std/c/segultra049" · use segultra049

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra050

import "std/c/segultra050" · use segultra050

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra051

import "std/c/segultra051" · use segultra051

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra052

import "std/c/segultra052" · use segultra052

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra053

import "std/c/segultra053" · use segultra053

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra054

import "std/c/segultra054" · use segultra054

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra055

import "std/c/segultra055" · use segultra055

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra056

import "std/c/segultra056" · use segultra056

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra057

import "std/c/segultra057" · use segultra057

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra058

import "std/c/segultra058" · use segultra058

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra059

import "std/c/segultra059" · use segultra059

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra060

import "std/c/segultra060" · use segultra060

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra061

import "std/c/segultra061" · use segultra061

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra062

import "std/c/segultra062" · use segultra062

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra063

import "std/c/segultra063" · use segultra063

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra064

import "std/c/segultra064" · use segultra064

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra065

import "std/c/segultra065" · use segultra065

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra066

import "std/c/segultra066" · use segultra066

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra067

import "std/c/segultra067" · use segultra067

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra068

import "std/c/segultra068" · use segultra068

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra069

import "std/c/segultra069" · use segultra069

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra070

import "std/c/segultra070" · use segultra070

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra071

import "std/c/segultra071" · use segultra071

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra072

import "std/c/segultra072" · use segultra072

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra073

import "std/c/segultra073" · use segultra073

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra074

import "std/c/segultra074" · use segultra074

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra075

import "std/c/segultra075" · use segultra075

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra076

import "std/c/segultra076" · use segultra076

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra077

import "std/c/segultra077" · use segultra077

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra078

import "std/c/segultra078" · use segultra078

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra079

import "std/c/segultra079" · use segultra079

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra080

import "std/c/segultra080" · use segultra080

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra081

import "std/c/segultra081" · use segultra081

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra082

import "std/c/segultra082" · use segultra082

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra083

import "std/c/segultra083" · use segultra083

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra084

import "std/c/segultra084" · use segultra084

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra085

import "std/c/segultra085" · use segultra085

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra086

import "std/c/segultra086" · use segultra086

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra087

import "std/c/segultra087" · use segultra087

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra088

import "std/c/segultra088" · use segultra088

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra089

import "std/c/segultra089" · use segultra089

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra090

import "std/c/segultra090" · use segultra090

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra091

import "std/c/segultra091" · use segultra091

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra092

import "std/c/segultra092" · use segultra092

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra093

import "std/c/segultra093" · use segultra093

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra094

import "std/c/segultra094" · use segultra094

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra095

import "std/c/segultra095" · use segultra095

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra096

import "std/c/segultra096" · use segultra096

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra097

import "std/c/segultra097" · use segultra097

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra098

import "std/c/segultra098" · use segultra098

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra099

import "std/c/segultra099" · use segultra099

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra100

import "std/c/segultra100" · use segultra100

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra101

import "std/c/segultra101" · use segultra101

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra102

import "std/c/segultra102" · use segultra102

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra103

import "std/c/segultra103" · use segultra103

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra104

import "std/c/segultra104" · use segultra104

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra105

import "std/c/segultra105" · use segultra105

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra106

import "std/c/segultra106" · use segultra106

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra107

import "std/c/segultra107" · use segultra107

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra108

import "std/c/segultra108" · use segultra108

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra109

import "std/c/segultra109" · use segultra109

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra110

import "std/c/segultra110" · use segultra110

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra111

import "std/c/segultra111" · use segultra111

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra112

import "std/c/segultra112" · use segultra112

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra113

import "std/c/segultra113" · use segultra113

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra114

import "std/c/segultra114" · use segultra114

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra115

import "std/c/segultra115" · use segultra115

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra116

import "std/c/segultra116" · use segultra116

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra117

import "std/c/segultra117" · use segultra117

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra118

import "std/c/segultra118" · use segultra118

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra119

import "std/c/segultra119" · use segultra119

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra120

import "std/c/segultra120" · use segultra120

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra121

import "std/c/segultra121" · use segultra121

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra122

import "std/c/segultra122" · use segultra122

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra123

import "std/c/segultra123" · use segultra123

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra124

import "std/c/segultra124" · use segultra124

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra125

import "std/c/segultra125" · use segultra125

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra126

import "std/c/segultra126" · use segultra126

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra127

import "std/c/segultra127" · use segultra127

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra128

import "std/c/segultra128" · use segultra128

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra129

import "std/c/segultra129" · use segultra129

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra130

import "std/c/segultra130" · use segultra130

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra131

import "std/c/segultra131" · use segultra131

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra132

import "std/c/segultra132" · use segultra132

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra133

import "std/c/segultra133" · use segultra133

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra134

import "std/c/segultra134" · use segultra134

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra135

import "std/c/segultra135" · use segultra135

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra136

import "std/c/segultra136" · use segultra136

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra137

import "std/c/segultra137" · use segultra137

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra138

import "std/c/segultra138" · use segultra138

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra139

import "std/c/segultra139" · use segultra139

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra140

import "std/c/segultra140" · use segultra140

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra141

import "std/c/segultra141" · use segultra141

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra142

import "std/c/segultra142" · use segultra142

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra143

import "std/c/segultra143" · use segultra143

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra144

import "std/c/segultra144" · use segultra144

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra145

import "std/c/segultra145" · use segultra145

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra146

import "std/c/segultra146" · use segultra146

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra147

import "std/c/segultra147" · use segultra147

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra148

import "std/c/segultra148" · use segultra148

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra149

import "std/c/segultra149" · use segultra149

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra150

import "std/c/segultra150" · use segultra150

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra151

import "std/c/segultra151" · use segultra151

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra152

import "std/c/segultra152" · use segultra152

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra153

import "std/c/segultra153" · use segultra153

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra154

import "std/c/segultra154" · use segultra154

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra155

import "std/c/segultra155" · use segultra155

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra156

import "std/c/segultra156" · use segultra156

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra157

import "std/c/segultra157" · use segultra157

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra158

import "std/c/segultra158" · use segultra158

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra159

import "std/c/segultra159" · use segultra159

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra160

import "std/c/segultra160" · use segultra160

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra161

import "std/c/segultra161" · use segultra161

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra162

import "std/c/segultra162" · use segultra162

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra163

import "std/c/segultra163" · use segultra163

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra164

import "std/c/segultra164" · use segultra164

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra165

import "std/c/segultra165" · use segultra165

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra166

import "std/c/segultra166" · use segultra166

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra167

import "std/c/segultra167" · use segultra167

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra168

import "std/c/segultra168" · use segultra168

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra169

import "std/c/segultra169" · use segultra169

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra170

import "std/c/segultra170" · use segultra170

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra171

import "std/c/segultra171" · use segultra171

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra172

import "std/c/segultra172" · use segultra172

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra173

import "std/c/segultra173" · use segultra173

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra174

import "std/c/segultra174" · use segultra174

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra175

import "std/c/segultra175" · use segultra175

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra176

import "std/c/segultra176" · use segultra176

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra177

import "std/c/segultra177" · use segultra177

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra178

import "std/c/segultra178" · use segultra178

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra179

import "std/c/segultra179" · use segultra179

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra180

import "std/c/segultra180" · use segultra180

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra181

import "std/c/segultra181" · use segultra181

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra182

import "std/c/segultra182" · use segultra182

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra183

import "std/c/segultra183" · use segultra183

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra184

import "std/c/segultra184" · use segultra184

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra185

import "std/c/segultra185" · use segultra185

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra186

import "std/c/segultra186" · use segultra186

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra187

import "std/c/segultra187" · use segultra187

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra188

import "std/c/segultra188" · use segultra188

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra189

import "std/c/segultra189" · use segultra189

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra190

import "std/c/segultra190" · use segultra190

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra191

import "std/c/segultra191" · use segultra191

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra192

import "std/c/segultra192" · use segultra192

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra193

import "std/c/segultra193" · use segultra193

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra194

import "std/c/segultra194" · use segultra194

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra195

import "std/c/segultra195" · use segultra195

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra196

import "std/c/segultra196" · use segultra196

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra197

import "std/c/segultra197" · use segultra197

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra198

import "std/c/segultra198" · use segultra198

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra199

import "std/c/segultra199" · use segultra199

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra200

import "std/c/segultra200" · use segultra200

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra201

import "std/c/segultra201" · use segultra201

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra202

import "std/c/segultra202" · use segultra202

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra203

import "std/c/segultra203" · use segultra203

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra204

import "std/c/segultra204" · use segultra204

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra205

import "std/c/segultra205" · use segultra205

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra206

import "std/c/segultra206" · use segultra206

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra207

import "std/c/segultra207" · use segultra207

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra208

import "std/c/segultra208" · use segultra208

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra209

import "std/c/segultra209" · use segultra209

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra210

import "std/c/segultra210" · use segultra210

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra211

import "std/c/segultra211" · use segultra211

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra212

import "std/c/segultra212" · use segultra212

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra213

import "std/c/segultra213" · use segultra213

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra214

import "std/c/segultra214" · use segultra214

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra215

import "std/c/segultra215" · use segultra215

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra216

import "std/c/segultra216" · use segultra216

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra217

import "std/c/segultra217" · use segultra217

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra218

import "std/c/segultra218" · use segultra218

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra219

import "std/c/segultra219" · use segultra219

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra220

import "std/c/segultra220" · use segultra220

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra221

import "std/c/segultra221" · use segultra221

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra222

import "std/c/segultra222" · use segultra222

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra223

import "std/c/segultra223" · use segultra223

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra224

import "std/c/segultra224" · use segultra224

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra225

import "std/c/segultra225" · use segultra225

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra226

import "std/c/segultra226" · use segultra226

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra227

import "std/c/segultra227" · use segultra227

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra228

import "std/c/segultra228" · use segultra228

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra229

import "std/c/segultra229" · use segultra229

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra230

import "std/c/segultra230" · use segultra230

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra231

import "std/c/segultra231" · use segultra231

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra232

import "std/c/segultra232" · use segultra232

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra233

import "std/c/segultra233" · use segultra233

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra234

import "std/c/segultra234" · use segultra234

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra235

import "std/c/segultra235" · use segultra235

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra236

import "std/c/segultra236" · use segultra236

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra237

import "std/c/segultra237" · use segultra237

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra238

import "std/c/segultra238" · use segultra238

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra239

import "std/c/segultra239" · use segultra239

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra240

import "std/c/segultra240" · use segultra240

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra241

import "std/c/segultra241" · use segultra241

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra242

import "std/c/segultra242" · use segultra242

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra243

import "std/c/segultra243" · use segultra243

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra244

import "std/c/segultra244" · use segultra244

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra245

import "std/c/segultra245" · use segultra245

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra246

import "std/c/segultra246" · use segultra246

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra247

import "std/c/segultra247" · use segultra247

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra248

import "std/c/segultra248" · use segultra248

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra249

import "std/c/segultra249" · use segultra249

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra250

import "std/c/segultra250" · use segultra250

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra251

import "std/c/segultra251" · use segultra251

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra252

import "std/c/segultra252" · use segultra252

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra253

import "std/c/segultra253" · use segultra253

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra254

import "std/c/segultra254" · use segultra254

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra255

import "std/c/segultra255" · use segultra255

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra256

import "std/c/segultra256" · use segultra256

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra257

import "std/c/segultra257" · use segultra257

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra258

import "std/c/segultra258" · use segultra258

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra259

import "std/c/segultra259" · use segultra259

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra260

import "std/c/segultra260" · use segultra260

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra261

import "std/c/segultra261" · use segultra261

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra262

import "std/c/segultra262" · use segultra262

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra263

import "std/c/segultra263" · use segultra263

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra264

import "std/c/segultra264" · use segultra264

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra265

import "std/c/segultra265" · use segultra265

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra266

import "std/c/segultra266" · use segultra266

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra267

import "std/c/segultra267" · use segultra267

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra268

import "std/c/segultra268" · use segultra268

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra269

import "std/c/segultra269" · use segultra269

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra270

import "std/c/segultra270" · use segultra270

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra271

import "std/c/segultra271" · use segultra271

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra272

import "std/c/segultra272" · use segultra272

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra273

import "std/c/segultra273" · use segultra273

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra274

import "std/c/segultra274" · use segultra274

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra275

import "std/c/segultra275" · use segultra275

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra276

import "std/c/segultra276" · use segultra276

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra277

import "std/c/segultra277" · use segultra277

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra278

import "std/c/segultra278" · use segultra278

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra279

import "std/c/segultra279" · use segultra279

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra280

import "std/c/segultra280" · use segultra280

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra281

import "std/c/segultra281" · use segultra281

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra282

import "std/c/segultra282" · use segultra282

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra283

import "std/c/segultra283" · use segultra283

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra284

import "std/c/segultra284" · use segultra284

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra285

import "std/c/segultra285" · use segultra285

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra286

import "std/c/segultra286" · use segultra286

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra287

import "std/c/segultra287" · use segultra287

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra288

import "std/c/segultra288" · use segultra288

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra289

import "std/c/segultra289" · use segultra289

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra290

import "std/c/segultra290" · use segultra290

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra291

import "std/c/segultra291" · use segultra291

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra292

import "std/c/segultra292" · use segultra292

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra293

import "std/c/segultra293" · use segultra293

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra294

import "std/c/segultra294" · use segultra294

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra295

import "std/c/segultra295" · use segultra295

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra296

import "std/c/segultra296" · use segultra296

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra297

import "std/c/segultra297" · use segultra297

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra298

import "std/c/segultra298" · use segultra298

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra299

import "std/c/segultra299" · use segultra299

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra300

import "std/c/segultra300" · use segultra300

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra301

import "std/c/segultra301" · use segultra301

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra302

import "std/c/segultra302" · use segultra302

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra303

import "std/c/segultra303" · use segultra303

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra304

import "std/c/segultra304" · use segultra304

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra305

import "std/c/segultra305" · use segultra305

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra306

import "std/c/segultra306" · use segultra306

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra307

import "std/c/segultra307" · use segultra307

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra308

import "std/c/segultra308" · use segultra308

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra309

import "std/c/segultra309" · use segultra309

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra310

import "std/c/segultra310" · use segultra310

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra311

import "std/c/segultra311" · use segultra311

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra312

import "std/c/segultra312" · use segultra312

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra313

import "std/c/segultra313" · use segultra313

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra314

import "std/c/segultra314" · use segultra314

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra315

import "std/c/segultra315" · use segultra315

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra316

import "std/c/segultra316" · use segultra316

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra317

import "std/c/segultra317" · use segultra317

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra318

import "std/c/segultra318" · use segultra318

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra319

import "std/c/segultra319" · use segultra319

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra320

import "std/c/segultra320" · use segultra320

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra321

import "std/c/segultra321" · use segultra321

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra322

import "std/c/segultra322" · use segultra322

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra323

import "std/c/segultra323" · use segultra323

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra324

import "std/c/segultra324" · use segultra324

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra325

import "std/c/segultra325" · use segultra325

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra326

import "std/c/segultra326" · use segultra326

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra327

import "std/c/segultra327" · use segultra327

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra328

import "std/c/segultra328" · use segultra328

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra329

import "std/c/segultra329" · use segultra329

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra330

import "std/c/segultra330" · use segultra330

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra331

import "std/c/segultra331" · use segultra331

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra332

import "std/c/segultra332" · use segultra332

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra333

import "std/c/segultra333" · use segultra333

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra334

import "std/c/segultra334" · use segultra334

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra335

import "std/c/segultra335" · use segultra335

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra336

import "std/c/segultra336" · use segultra336

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra337

import "std/c/segultra337" · use segultra337

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra338

import "std/c/segultra338" · use segultra338

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra339

import "std/c/segultra339" · use segultra339

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra340

import "std/c/segultra340" · use segultra340

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra341

import "std/c/segultra341" · use segultra341

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra342

import "std/c/segultra342" · use segultra342

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra343

import "std/c/segultra343" · use segultra343

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra344

import "std/c/segultra344" · use segultra344

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra345

import "std/c/segultra345" · use segultra345

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra346

import "std/c/segultra346" · use segultra346

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra347

import "std/c/segultra347" · use segultra347

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra348

import "std/c/segultra348" · use segultra348

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra349

import "std/c/segultra349" · use segultra349

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra350

import "std/c/segultra350" · use segultra350

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra351

import "std/c/segultra351" · use segultra351

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra352

import "std/c/segultra352" · use segultra352

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra353

import "std/c/segultra353" · use segultra353

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra354

import "std/c/segultra354" · use segultra354

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra355

import "std/c/segultra355" · use segultra355

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra356

import "std/c/segultra356" · use segultra356

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra357

import "std/c/segultra357" · use segultra357

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra358

import "std/c/segultra358" · use segultra358

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra359

import "std/c/segultra359" · use segultra359

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra360

import "std/c/segultra360" · use segultra360

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra361

import "std/c/segultra361" · use segultra361

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra362

import "std/c/segultra362" · use segultra362

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra363

import "std/c/segultra363" · use segultra363

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra364

import "std/c/segultra364" · use segultra364

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra365

import "std/c/segultra365" · use segultra365

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra366

import "std/c/segultra366" · use segultra366

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra367

import "std/c/segultra367" · use segultra367

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra368

import "std/c/segultra368" · use segultra368

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra369

import "std/c/segultra369" · use segultra369

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra370

import "std/c/segultra370" · use segultra370

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra371

import "std/c/segultra371" · use segultra371

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra372

import "std/c/segultra372" · use segultra372

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra373

import "std/c/segultra373" · use segultra373

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra374

import "std/c/segultra374" · use segultra374

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra375

import "std/c/segultra375" · use segultra375

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra376

import "std/c/segultra376" · use segultra376

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra377

import "std/c/segultra377" · use segultra377

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra378

import "std/c/segultra378" · use segultra378

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra379

import "std/c/segultra379" · use segultra379

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra380

import "std/c/segultra380" · use segultra380

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra381

import "std/c/segultra381" · use segultra381

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra382

import "std/c/segultra382" · use segultra382

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra383

import "std/c/segultra383" · use segultra383

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra384

import "std/c/segultra384" · use segultra384

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra385

import "std/c/segultra385" · use segultra385

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra386

import "std/c/segultra386" · use segultra386

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra387

import "std/c/segultra387" · use segultra387

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra388

import "std/c/segultra388" · use segultra388

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra389

import "std/c/segultra389" · use segultra389

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra390

import "std/c/segultra390" · use segultra390

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra391

import "std/c/segultra391" · use segultra391

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra392

import "std/c/segultra392" · use segultra392

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra393

import "std/c/segultra393" · use segultra393

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra394

import "std/c/segultra394" · use segultra394

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra395

import "std/c/segultra395" · use segultra395

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra396

import "std/c/segultra396" · use segultra396

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra397

import "std/c/segultra397" · use segultra397

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra398

import "std/c/segultra398" · use segultra398

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra399

import "std/c/segultra399" · use segultra399

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra400

import "std/c/segultra400" · use segultra400

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra401

import "std/c/segultra401" · use segultra401

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra402

import "std/c/segultra402" · use segultra402

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra403

import "std/c/segultra403" · use segultra403

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra404

import "std/c/segultra404" · use segultra404

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra405

import "std/c/segultra405" · use segultra405

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra406

import "std/c/segultra406" · use segultra406

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra407

import "std/c/segultra407" · use segultra407

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra408

import "std/c/segultra408" · use segultra408

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra409

import "std/c/segultra409" · use segultra409

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra410

import "std/c/segultra410" · use segultra410

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra411

import "std/c/segultra411" · use segultra411

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra412

import "std/c/segultra412" · use segultra412

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra413

import "std/c/segultra413" · use segultra413

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra414

import "std/c/segultra414" · use segultra414

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra415

import "std/c/segultra415" · use segultra415

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra416

import "std/c/segultra416" · use segultra416

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra417

import "std/c/segultra417" · use segultra417

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra418

import "std/c/segultra418" · use segultra418

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra419

import "std/c/segultra419" · use segultra419

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra420

import "std/c/segultra420" · use segultra420

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra421

import "std/c/segultra421" · use segultra421

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra422

import "std/c/segultra422" · use segultra422

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra423

import "std/c/segultra423" · use segultra423

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra424

import "std/c/segultra424" · use segultra424

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra425

import "std/c/segultra425" · use segultra425

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra426

import "std/c/segultra426" · use segultra426

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra427

import "std/c/segultra427" · use segultra427

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra428

import "std/c/segultra428" · use segultra428

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra429

import "std/c/segultra429" · use segultra429

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra430

import "std/c/segultra430" · use segultra430

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra431

import "std/c/segultra431" · use segultra431

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra432

import "std/c/segultra432" · use segultra432

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra433

import "std/c/segultra433" · use segultra433

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra434

import "std/c/segultra434" · use segultra434

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra435

import "std/c/segultra435" · use segultra435

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra436

import "std/c/segultra436" · use segultra436

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra437

import "std/c/segultra437" · use segultra437

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra438

import "std/c/segultra438" · use segultra438

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra439

import "std/c/segultra439" · use segultra439

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra440

import "std/c/segultra440" · use segultra440

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra441

import "std/c/segultra441" · use segultra441

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra442

import "std/c/segultra442" · use segultra442

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra443

import "std/c/segultra443" · use segultra443

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra444

import "std/c/segultra444" · use segultra444

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra445

import "std/c/segultra445" · use segultra445

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra446

import "std/c/segultra446" · use segultra446

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra447

import "std/c/segultra447" · use segultra447

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra448

import "std/c/segultra448" · use segultra448

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra449

import "std/c/segultra449" · use segultra449

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra450

import "std/c/segultra450" · use segultra450

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra451

import "std/c/segultra451" · use segultra451

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra452

import "std/c/segultra452" · use segultra452

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra453

import "std/c/segultra453" · use segultra453

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra454

import "std/c/segultra454" · use segultra454

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra455

import "std/c/segultra455" · use segultra455

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra456

import "std/c/segultra456" · use segultra456

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra457

import "std/c/segultra457" · use segultra457

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra458

import "std/c/segultra458" · use segultra458

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra459

import "std/c/segultra459" · use segultra459

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra460

import "std/c/segultra460" · use segultra460

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra461

import "std/c/segultra461" · use segultra461

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra462

import "std/c/segultra462" · use segultra462

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra463

import "std/c/segultra463" · use segultra463

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra464

import "std/c/segultra464" · use segultra464

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra465

import "std/c/segultra465" · use segultra465

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra466

import "std/c/segultra466" · use segultra466

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra467

import "std/c/segultra467" · use segultra467

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra468

import "std/c/segultra468" · use segultra468

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra469

import "std/c/segultra469" · use segultra469

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra470

import "std/c/segultra470" · use segultra470

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra471

import "std/c/segultra471" · use segultra471

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra472

import "std/c/segultra472" · use segultra472

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra473

import "std/c/segultra473" · use segultra473

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra474

import "std/c/segultra474" · use segultra474

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra475

import "std/c/segultra475" · use segultra475

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra476

import "std/c/segultra476" · use segultra476

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra477

import "std/c/segultra477" · use segultra477

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra478

import "std/c/segultra478" · use segultra478

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra479

import "std/c/segultra479" · use segultra479

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra480

import "std/c/segultra480" · use segultra480

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra481

import "std/c/segultra481" · use segultra481

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra482

import "std/c/segultra482" · use segultra482

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra483

import "std/c/segultra483" · use segultra483

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra484

import "std/c/segultra484" · use segultra484

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra485

import "std/c/segultra485" · use segultra485

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra486

import "std/c/segultra486" · use segultra486

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra487

import "std/c/segultra487" · use segultra487

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra488

import "std/c/segultra488" · use segultra488

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra489

import "std/c/segultra489" · use segultra489

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra490

import "std/c/segultra490" · use segultra490

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra491

import "std/c/segultra491" · use segultra491

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra492

import "std/c/segultra492" · use segultra492

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra493

import "std/c/segultra493" · use segultra493

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra494

import "std/c/segultra494" · use segultra494

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra495

import "std/c/segultra495" · use segultra495

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra496

import "std/c/segultra496" · use segultra496

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra497

import "std/c/segultra497" · use segultra497

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra498

import "std/c/segultra498" · use segultra498

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra499

import "std/c/segultra499" · use segultra499

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/segultra500

import "std/c/segultra500" · use segultra500

- `floodSize(mask list number, w number, h number, sx number, sy number) → number`
- `countComponents4(mask list number, w number, h number) → number`
- `countComponents8(mask list number, w number, h number) → number`
- `dilate3(mask list number, w number, h number) → list number`
- `erode3(mask list number, w number, h number) → list number`
- `thresholdMask(grid list number, w number, h number, t number) → list number`
- `thresholdDefault(grid list number, w number, h number) → list number`
- `countComponents(mask list number, w number, h number) → number`

std/c/dataultra001

import "std/c/dataultra001" · use dataultra001

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra002

import "std/c/dataultra002" · use dataultra002

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra003

import "std/c/dataultra003" · use dataultra003

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra004

import "std/c/dataultra004" · use dataultra004

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra005

import "std/c/dataultra005" · use dataultra005

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra006

import "std/c/dataultra006" · use dataultra006

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra007

import "std/c/dataultra007" · use dataultra007

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra008

import "std/c/dataultra008" · use dataultra008

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra009

import "std/c/dataultra009" · use dataultra009

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra010

import "std/c/dataultra010" · use dataultra010

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra011

import "std/c/dataultra011" · use dataultra011

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra012

import "std/c/dataultra012" · use dataultra012

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra013

import "std/c/dataultra013" · use dataultra013

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra014

import "std/c/dataultra014" · use dataultra014

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra015

import "std/c/dataultra015" · use dataultra015

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra016

import "std/c/dataultra016" · use dataultra016

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra017

import "std/c/dataultra017" · use dataultra017

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra018

import "std/c/dataultra018" · use dataultra018

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra019

import "std/c/dataultra019" · use dataultra019

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra020

import "std/c/dataultra020" · use dataultra020

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra021

import "std/c/dataultra021" · use dataultra021

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra022

import "std/c/dataultra022" · use dataultra022

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra023

import "std/c/dataultra023" · use dataultra023

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra024

import "std/c/dataultra024" · use dataultra024

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra025

import "std/c/dataultra025" · use dataultra025

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra026

import "std/c/dataultra026" · use dataultra026

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra027

import "std/c/dataultra027" · use dataultra027

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra028

import "std/c/dataultra028" · use dataultra028

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra029

import "std/c/dataultra029" · use dataultra029

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra030

import "std/c/dataultra030" · use dataultra030

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra031

import "std/c/dataultra031" · use dataultra031

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra032

import "std/c/dataultra032" · use dataultra032

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra033

import "std/c/dataultra033" · use dataultra033

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra034

import "std/c/dataultra034" · use dataultra034

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra035

import "std/c/dataultra035" · use dataultra035

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra036

import "std/c/dataultra036" · use dataultra036

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra037

import "std/c/dataultra037" · use dataultra037

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra038

import "std/c/dataultra038" · use dataultra038

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra039

import "std/c/dataultra039" · use dataultra039

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra040

import "std/c/dataultra040" · use dataultra040

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra041

import "std/c/dataultra041" · use dataultra041

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra042

import "std/c/dataultra042" · use dataultra042

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra043

import "std/c/dataultra043" · use dataultra043

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra044

import "std/c/dataultra044" · use dataultra044

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra045

import "std/c/dataultra045" · use dataultra045

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra046

import "std/c/dataultra046" · use dataultra046

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra047

import "std/c/dataultra047" · use dataultra047

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra048

import "std/c/dataultra048" · use dataultra048

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra049

import "std/c/dataultra049" · use dataultra049

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra050

import "std/c/dataultra050" · use dataultra050

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra051

import "std/c/dataultra051" · use dataultra051

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra052

import "std/c/dataultra052" · use dataultra052

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra053

import "std/c/dataultra053" · use dataultra053

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra054

import "std/c/dataultra054" · use dataultra054

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra055

import "std/c/dataultra055" · use dataultra055

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra056

import "std/c/dataultra056" · use dataultra056

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra057

import "std/c/dataultra057" · use dataultra057

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra058

import "std/c/dataultra058" · use dataultra058

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra059

import "std/c/dataultra059" · use dataultra059

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra060

import "std/c/dataultra060" · use dataultra060

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra061

import "std/c/dataultra061" · use dataultra061

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra062

import "std/c/dataultra062" · use dataultra062

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra063

import "std/c/dataultra063" · use dataultra063

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra064

import "std/c/dataultra064" · use dataultra064

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra065

import "std/c/dataultra065" · use dataultra065

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra066

import "std/c/dataultra066" · use dataultra066

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra067

import "std/c/dataultra067" · use dataultra067

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra068

import "std/c/dataultra068" · use dataultra068

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra069

import "std/c/dataultra069" · use dataultra069

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra070

import "std/c/dataultra070" · use dataultra070

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra071

import "std/c/dataultra071" · use dataultra071

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra072

import "std/c/dataultra072" · use dataultra072

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra073

import "std/c/dataultra073" · use dataultra073

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra074

import "std/c/dataultra074" · use dataultra074

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra075

import "std/c/dataultra075" · use dataultra075

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra076

import "std/c/dataultra076" · use dataultra076

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra077

import "std/c/dataultra077" · use dataultra077

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra078

import "std/c/dataultra078" · use dataultra078

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra079

import "std/c/dataultra079" · use dataultra079

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra080

import "std/c/dataultra080" · use dataultra080

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra081

import "std/c/dataultra081" · use dataultra081

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra082

import "std/c/dataultra082" · use dataultra082

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra083

import "std/c/dataultra083" · use dataultra083

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra084

import "std/c/dataultra084" · use dataultra084

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra085

import "std/c/dataultra085" · use dataultra085

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra086

import "std/c/dataultra086" · use dataultra086

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra087

import "std/c/dataultra087" · use dataultra087

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra088

import "std/c/dataultra088" · use dataultra088

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra089

import "std/c/dataultra089" · use dataultra089

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra090

import "std/c/dataultra090" · use dataultra090

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra091

import "std/c/dataultra091" · use dataultra091

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra092

import "std/c/dataultra092" · use dataultra092

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra093

import "std/c/dataultra093" · use dataultra093

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra094

import "std/c/dataultra094" · use dataultra094

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra095

import "std/c/dataultra095" · use dataultra095

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra096

import "std/c/dataultra096" · use dataultra096

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra097

import "std/c/dataultra097" · use dataultra097

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra098

import "std/c/dataultra098" · use dataultra098

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra099

import "std/c/dataultra099" · use dataultra099

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra100

import "std/c/dataultra100" · use dataultra100

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra101

import "std/c/dataultra101" · use dataultra101

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra102

import "std/c/dataultra102" · use dataultra102

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra103

import "std/c/dataultra103" · use dataultra103

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra104

import "std/c/dataultra104" · use dataultra104

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra105

import "std/c/dataultra105" · use dataultra105

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra106

import "std/c/dataultra106" · use dataultra106

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra107

import "std/c/dataultra107" · use dataultra107

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra108

import "std/c/dataultra108" · use dataultra108

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra109

import "std/c/dataultra109" · use dataultra109

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra110

import "std/c/dataultra110" · use dataultra110

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra111

import "std/c/dataultra111" · use dataultra111

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra112

import "std/c/dataultra112" · use dataultra112

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra113

import "std/c/dataultra113" · use dataultra113

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra114

import "std/c/dataultra114" · use dataultra114

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra115

import "std/c/dataultra115" · use dataultra115

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra116

import "std/c/dataultra116" · use dataultra116

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra117

import "std/c/dataultra117" · use dataultra117

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra118

import "std/c/dataultra118" · use dataultra118

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra119

import "std/c/dataultra119" · use dataultra119

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra120

import "std/c/dataultra120" · use dataultra120

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra121

import "std/c/dataultra121" · use dataultra121

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra122

import "std/c/dataultra122" · use dataultra122

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra123

import "std/c/dataultra123" · use dataultra123

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra124

import "std/c/dataultra124" · use dataultra124

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra125

import "std/c/dataultra125" · use dataultra125

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra126

import "std/c/dataultra126" · use dataultra126

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra127

import "std/c/dataultra127" · use dataultra127

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra128

import "std/c/dataultra128" · use dataultra128

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra129

import "std/c/dataultra129" · use dataultra129

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra130

import "std/c/dataultra130" · use dataultra130

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra131

import "std/c/dataultra131" · use dataultra131

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra132

import "std/c/dataultra132" · use dataultra132

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra133

import "std/c/dataultra133" · use dataultra133

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra134

import "std/c/dataultra134" · use dataultra134

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra135

import "std/c/dataultra135" · use dataultra135

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra136

import "std/c/dataultra136" · use dataultra136

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra137

import "std/c/dataultra137" · use dataultra137

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra138

import "std/c/dataultra138" · use dataultra138

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra139

import "std/c/dataultra139" · use dataultra139

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra140

import "std/c/dataultra140" · use dataultra140

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra141

import "std/c/dataultra141" · use dataultra141

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra142

import "std/c/dataultra142" · use dataultra142

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra143

import "std/c/dataultra143" · use dataultra143

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra144

import "std/c/dataultra144" · use dataultra144

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra145

import "std/c/dataultra145" · use dataultra145

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra146

import "std/c/dataultra146" · use dataultra146

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra147

import "std/c/dataultra147" · use dataultra147

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra148

import "std/c/dataultra148" · use dataultra148

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra149

import "std/c/dataultra149" · use dataultra149

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra150

import "std/c/dataultra150" · use dataultra150

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra151

import "std/c/dataultra151" · use dataultra151

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra152

import "std/c/dataultra152" · use dataultra152

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra153

import "std/c/dataultra153" · use dataultra153

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra154

import "std/c/dataultra154" · use dataultra154

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra155

import "std/c/dataultra155" · use dataultra155

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra156

import "std/c/dataultra156" · use dataultra156

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra157

import "std/c/dataultra157" · use dataultra157

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra158

import "std/c/dataultra158" · use dataultra158

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra159

import "std/c/dataultra159" · use dataultra159

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra160

import "std/c/dataultra160" · use dataultra160

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra161

import "std/c/dataultra161" · use dataultra161

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra162

import "std/c/dataultra162" · use dataultra162

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra163

import "std/c/dataultra163" · use dataultra163

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra164

import "std/c/dataultra164" · use dataultra164

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra165

import "std/c/dataultra165" · use dataultra165

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra166

import "std/c/dataultra166" · use dataultra166

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra167

import "std/c/dataultra167" · use dataultra167

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra168

import "std/c/dataultra168" · use dataultra168

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra169

import "std/c/dataultra169" · use dataultra169

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra170

import "std/c/dataultra170" · use dataultra170

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra171

import "std/c/dataultra171" · use dataultra171

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra172

import "std/c/dataultra172" · use dataultra172

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra173

import "std/c/dataultra173" · use dataultra173

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra174

import "std/c/dataultra174" · use dataultra174

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra175

import "std/c/dataultra175" · use dataultra175

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra176

import "std/c/dataultra176" · use dataultra176

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra177

import "std/c/dataultra177" · use dataultra177

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra178

import "std/c/dataultra178" · use dataultra178

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra179

import "std/c/dataultra179" · use dataultra179

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra180

import "std/c/dataultra180" · use dataultra180

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra181

import "std/c/dataultra181" · use dataultra181

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra182

import "std/c/dataultra182" · use dataultra182

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra183

import "std/c/dataultra183" · use dataultra183

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra184

import "std/c/dataultra184" · use dataultra184

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra185

import "std/c/dataultra185" · use dataultra185

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra186

import "std/c/dataultra186" · use dataultra186

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra187

import "std/c/dataultra187" · use dataultra187

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra188

import "std/c/dataultra188" · use dataultra188

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra189

import "std/c/dataultra189" · use dataultra189

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra190

import "std/c/dataultra190" · use dataultra190

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra191

import "std/c/dataultra191" · use dataultra191

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra192

import "std/c/dataultra192" · use dataultra192

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra193

import "std/c/dataultra193" · use dataultra193

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra194

import "std/c/dataultra194" · use dataultra194

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra195

import "std/c/dataultra195" · use dataultra195

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra196

import "std/c/dataultra196" · use dataultra196

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra197

import "std/c/dataultra197" · use dataultra197

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra198

import "std/c/dataultra198" · use dataultra198

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra199

import "std/c/dataultra199" · use dataultra199

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra200

import "std/c/dataultra200" · use dataultra200

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra201

import "std/c/dataultra201" · use dataultra201

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra202

import "std/c/dataultra202" · use dataultra202

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra203

import "std/c/dataultra203" · use dataultra203

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra204

import "std/c/dataultra204" · use dataultra204

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra205

import "std/c/dataultra205" · use dataultra205

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra206

import "std/c/dataultra206" · use dataultra206

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra207

import "std/c/dataultra207" · use dataultra207

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra208

import "std/c/dataultra208" · use dataultra208

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra209

import "std/c/dataultra209" · use dataultra209

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra210

import "std/c/dataultra210" · use dataultra210

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra211

import "std/c/dataultra211" · use dataultra211

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra212

import "std/c/dataultra212" · use dataultra212

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra213

import "std/c/dataultra213" · use dataultra213

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra214

import "std/c/dataultra214" · use dataultra214

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra215

import "std/c/dataultra215" · use dataultra215

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra216

import "std/c/dataultra216" · use dataultra216

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra217

import "std/c/dataultra217" · use dataultra217

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra218

import "std/c/dataultra218" · use dataultra218

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra219

import "std/c/dataultra219" · use dataultra219

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra220

import "std/c/dataultra220" · use dataultra220

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra221

import "std/c/dataultra221" · use dataultra221

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra222

import "std/c/dataultra222" · use dataultra222

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra223

import "std/c/dataultra223" · use dataultra223

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra224

import "std/c/dataultra224" · use dataultra224

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra225

import "std/c/dataultra225" · use dataultra225

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra226

import "std/c/dataultra226" · use dataultra226

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra227

import "std/c/dataultra227" · use dataultra227

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra228

import "std/c/dataultra228" · use dataultra228

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra229

import "std/c/dataultra229" · use dataultra229

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra230

import "std/c/dataultra230" · use dataultra230

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra231

import "std/c/dataultra231" · use dataultra231

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra232

import "std/c/dataultra232" · use dataultra232

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra233

import "std/c/dataultra233" · use dataultra233

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra234

import "std/c/dataultra234" · use dataultra234

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra235

import "std/c/dataultra235" · use dataultra235

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra236

import "std/c/dataultra236" · use dataultra236

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra237

import "std/c/dataultra237" · use dataultra237

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra238

import "std/c/dataultra238" · use dataultra238

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra239

import "std/c/dataultra239" · use dataultra239

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra240

import "std/c/dataultra240" · use dataultra240

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra241

import "std/c/dataultra241" · use dataultra241

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra242

import "std/c/dataultra242" · use dataultra242

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra243

import "std/c/dataultra243" · use dataultra243

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra244

import "std/c/dataultra244" · use dataultra244

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra245

import "std/c/dataultra245" · use dataultra245

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra246

import "std/c/dataultra246" · use dataultra246

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra247

import "std/c/dataultra247" · use dataultra247

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra248

import "std/c/dataultra248" · use dataultra248

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra249

import "std/c/dataultra249" · use dataultra249

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra250

import "std/c/dataultra250" · use dataultra250

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra251

import "std/c/dataultra251" · use dataultra251

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra252

import "std/c/dataultra252" · use dataultra252

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra253

import "std/c/dataultra253" · use dataultra253

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra254

import "std/c/dataultra254" · use dataultra254

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra255

import "std/c/dataultra255" · use dataultra255

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra256

import "std/c/dataultra256" · use dataultra256

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra257

import "std/c/dataultra257" · use dataultra257

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra258

import "std/c/dataultra258" · use dataultra258

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra259

import "std/c/dataultra259" · use dataultra259

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra260

import "std/c/dataultra260" · use dataultra260

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra261

import "std/c/dataultra261" · use dataultra261

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra262

import "std/c/dataultra262" · use dataultra262

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra263

import "std/c/dataultra263" · use dataultra263

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra264

import "std/c/dataultra264" · use dataultra264

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra265

import "std/c/dataultra265" · use dataultra265

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra266

import "std/c/dataultra266" · use dataultra266

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra267

import "std/c/dataultra267" · use dataultra267

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra268

import "std/c/dataultra268" · use dataultra268

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra269

import "std/c/dataultra269" · use dataultra269

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra270

import "std/c/dataultra270" · use dataultra270

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra271

import "std/c/dataultra271" · use dataultra271

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra272

import "std/c/dataultra272" · use dataultra272

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra273

import "std/c/dataultra273" · use dataultra273

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra274

import "std/c/dataultra274" · use dataultra274

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra275

import "std/c/dataultra275" · use dataultra275

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra276

import "std/c/dataultra276" · use dataultra276

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra277

import "std/c/dataultra277" · use dataultra277

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra278

import "std/c/dataultra278" · use dataultra278

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra279

import "std/c/dataultra279" · use dataultra279

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra280

import "std/c/dataultra280" · use dataultra280

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra281

import "std/c/dataultra281" · use dataultra281

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra282

import "std/c/dataultra282" · use dataultra282

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra283

import "std/c/dataultra283" · use dataultra283

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra284

import "std/c/dataultra284" · use dataultra284

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra285

import "std/c/dataultra285" · use dataultra285

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra286

import "std/c/dataultra286" · use dataultra286

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra287

import "std/c/dataultra287" · use dataultra287

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra288

import "std/c/dataultra288" · use dataultra288

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra289

import "std/c/dataultra289" · use dataultra289

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra290

import "std/c/dataultra290" · use dataultra290

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra291

import "std/c/dataultra291" · use dataultra291

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra292

import "std/c/dataultra292" · use dataultra292

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra293

import "std/c/dataultra293" · use dataultra293

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra294

import "std/c/dataultra294" · use dataultra294

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra295

import "std/c/dataultra295" · use dataultra295

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra296

import "std/c/dataultra296" · use dataultra296

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra297

import "std/c/dataultra297" · use dataultra297

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra298

import "std/c/dataultra298" · use dataultra298

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra299

import "std/c/dataultra299" · use dataultra299

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra300

import "std/c/dataultra300" · use dataultra300

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra301

import "std/c/dataultra301" · use dataultra301

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra302

import "std/c/dataultra302" · use dataultra302

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra303

import "std/c/dataultra303" · use dataultra303

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra304

import "std/c/dataultra304" · use dataultra304

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra305

import "std/c/dataultra305" · use dataultra305

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra306

import "std/c/dataultra306" · use dataultra306

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra307

import "std/c/dataultra307" · use dataultra307

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra308

import "std/c/dataultra308" · use dataultra308

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra309

import "std/c/dataultra309" · use dataultra309

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra310

import "std/c/dataultra310" · use dataultra310

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra311

import "std/c/dataultra311" · use dataultra311

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra312

import "std/c/dataultra312" · use dataultra312

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra313

import "std/c/dataultra313" · use dataultra313

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra314

import "std/c/dataultra314" · use dataultra314

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra315

import "std/c/dataultra315" · use dataultra315

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra316

import "std/c/dataultra316" · use dataultra316

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra317

import "std/c/dataultra317" · use dataultra317

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra318

import "std/c/dataultra318" · use dataultra318

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra319

import "std/c/dataultra319" · use dataultra319

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra320

import "std/c/dataultra320" · use dataultra320

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra321

import "std/c/dataultra321" · use dataultra321

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra322

import "std/c/dataultra322" · use dataultra322

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra323

import "std/c/dataultra323" · use dataultra323

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra324

import "std/c/dataultra324" · use dataultra324

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra325

import "std/c/dataultra325" · use dataultra325

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra326

import "std/c/dataultra326" · use dataultra326

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra327

import "std/c/dataultra327" · use dataultra327

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra328

import "std/c/dataultra328" · use dataultra328

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra329

import "std/c/dataultra329" · use dataultra329

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra330

import "std/c/dataultra330" · use dataultra330

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra331

import "std/c/dataultra331" · use dataultra331

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra332

import "std/c/dataultra332" · use dataultra332

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra333

import "std/c/dataultra333" · use dataultra333

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra334

import "std/c/dataultra334" · use dataultra334

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra335

import "std/c/dataultra335" · use dataultra335

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra336

import "std/c/dataultra336" · use dataultra336

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra337

import "std/c/dataultra337" · use dataultra337

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra338

import "std/c/dataultra338" · use dataultra338

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra339

import "std/c/dataultra339" · use dataultra339

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra340

import "std/c/dataultra340" · use dataultra340

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra341

import "std/c/dataultra341" · use dataultra341

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra342

import "std/c/dataultra342" · use dataultra342

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra343

import "std/c/dataultra343" · use dataultra343

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra344

import "std/c/dataultra344" · use dataultra344

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra345

import "std/c/dataultra345" · use dataultra345

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra346

import "std/c/dataultra346" · use dataultra346

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra347

import "std/c/dataultra347" · use dataultra347

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra348

import "std/c/dataultra348" · use dataultra348

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra349

import "std/c/dataultra349" · use dataultra349

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra350

import "std/c/dataultra350" · use dataultra350

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra351

import "std/c/dataultra351" · use dataultra351

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra352

import "std/c/dataultra352" · use dataultra352

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra353

import "std/c/dataultra353" · use dataultra353

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra354

import "std/c/dataultra354" · use dataultra354

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra355

import "std/c/dataultra355" · use dataultra355

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra356

import "std/c/dataultra356" · use dataultra356

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra357

import "std/c/dataultra357" · use dataultra357

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra358

import "std/c/dataultra358" · use dataultra358

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra359

import "std/c/dataultra359" · use dataultra359

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra360

import "std/c/dataultra360" · use dataultra360

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra361

import "std/c/dataultra361" · use dataultra361

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra362

import "std/c/dataultra362" · use dataultra362

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra363

import "std/c/dataultra363" · use dataultra363

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra364

import "std/c/dataultra364" · use dataultra364

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra365

import "std/c/dataultra365" · use dataultra365

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra366

import "std/c/dataultra366" · use dataultra366

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra367

import "std/c/dataultra367" · use dataultra367

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra368

import "std/c/dataultra368" · use dataultra368

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra369

import "std/c/dataultra369" · use dataultra369

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra370

import "std/c/dataultra370" · use dataultra370

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra371

import "std/c/dataultra371" · use dataultra371

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra372

import "std/c/dataultra372" · use dataultra372

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra373

import "std/c/dataultra373" · use dataultra373

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra374

import "std/c/dataultra374" · use dataultra374

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra375

import "std/c/dataultra375" · use dataultra375

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra376

import "std/c/dataultra376" · use dataultra376

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra377

import "std/c/dataultra377" · use dataultra377

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra378

import "std/c/dataultra378" · use dataultra378

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra379

import "std/c/dataultra379" · use dataultra379

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra380

import "std/c/dataultra380" · use dataultra380

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra381

import "std/c/dataultra381" · use dataultra381

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra382

import "std/c/dataultra382" · use dataultra382

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra383

import "std/c/dataultra383" · use dataultra383

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra384

import "std/c/dataultra384" · use dataultra384

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra385

import "std/c/dataultra385" · use dataultra385

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra386

import "std/c/dataultra386" · use dataultra386

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra387

import "std/c/dataultra387" · use dataultra387

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra388

import "std/c/dataultra388" · use dataultra388

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra389

import "std/c/dataultra389" · use dataultra389

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra390

import "std/c/dataultra390" · use dataultra390

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra391

import "std/c/dataultra391" · use dataultra391

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra392

import "std/c/dataultra392" · use dataultra392

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra393

import "std/c/dataultra393" · use dataultra393

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra394

import "std/c/dataultra394" · use dataultra394

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra395

import "std/c/dataultra395" · use dataultra395

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra396

import "std/c/dataultra396" · use dataultra396

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra397

import "std/c/dataultra397" · use dataultra397

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra398

import "std/c/dataultra398" · use dataultra398

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra399

import "std/c/dataultra399" · use dataultra399

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra400

import "std/c/dataultra400" · use dataultra400

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra401

import "std/c/dataultra401" · use dataultra401

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra402

import "std/c/dataultra402" · use dataultra402

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra403

import "std/c/dataultra403" · use dataultra403

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra404

import "std/c/dataultra404" · use dataultra404

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra405

import "std/c/dataultra405" · use dataultra405

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra406

import "std/c/dataultra406" · use dataultra406

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra407

import "std/c/dataultra407" · use dataultra407

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra408

import "std/c/dataultra408" · use dataultra408

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra409

import "std/c/dataultra409" · use dataultra409

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra410

import "std/c/dataultra410" · use dataultra410

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra411

import "std/c/dataultra411" · use dataultra411

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra412

import "std/c/dataultra412" · use dataultra412

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra413

import "std/c/dataultra413" · use dataultra413

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra414

import "std/c/dataultra414" · use dataultra414

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra415

import "std/c/dataultra415" · use dataultra415

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra416

import "std/c/dataultra416" · use dataultra416

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra417

import "std/c/dataultra417" · use dataultra417

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra418

import "std/c/dataultra418" · use dataultra418

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra419

import "std/c/dataultra419" · use dataultra419

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra420

import "std/c/dataultra420" · use dataultra420

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra421

import "std/c/dataultra421" · use dataultra421

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra422

import "std/c/dataultra422" · use dataultra422

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra423

import "std/c/dataultra423" · use dataultra423

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra424

import "std/c/dataultra424" · use dataultra424

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra425

import "std/c/dataultra425" · use dataultra425

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra426

import "std/c/dataultra426" · use dataultra426

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra427

import "std/c/dataultra427" · use dataultra427

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra428

import "std/c/dataultra428" · use dataultra428

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra429

import "std/c/dataultra429" · use dataultra429

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra430

import "std/c/dataultra430" · use dataultra430

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra431

import "std/c/dataultra431" · use dataultra431

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra432

import "std/c/dataultra432" · use dataultra432

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra433

import "std/c/dataultra433" · use dataultra433

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra434

import "std/c/dataultra434" · use dataultra434

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra435

import "std/c/dataultra435" · use dataultra435

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra436

import "std/c/dataultra436" · use dataultra436

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra437

import "std/c/dataultra437" · use dataultra437

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra438

import "std/c/dataultra438" · use dataultra438

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra439

import "std/c/dataultra439" · use dataultra439

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra440

import "std/c/dataultra440" · use dataultra440

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra441

import "std/c/dataultra441" · use dataultra441

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra442

import "std/c/dataultra442" · use dataultra442

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra443

import "std/c/dataultra443" · use dataultra443

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra444

import "std/c/dataultra444" · use dataultra444

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra445

import "std/c/dataultra445" · use dataultra445

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra446

import "std/c/dataultra446" · use dataultra446

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra447

import "std/c/dataultra447" · use dataultra447

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra448

import "std/c/dataultra448" · use dataultra448

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra449

import "std/c/dataultra449" · use dataultra449

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra450

import "std/c/dataultra450" · use dataultra450

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra451

import "std/c/dataultra451" · use dataultra451

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra452

import "std/c/dataultra452" · use dataultra452

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra453

import "std/c/dataultra453" · use dataultra453

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra454

import "std/c/dataultra454" · use dataultra454

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra455

import "std/c/dataultra455" · use dataultra455

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra456

import "std/c/dataultra456" · use dataultra456

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra457

import "std/c/dataultra457" · use dataultra457

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra458

import "std/c/dataultra458" · use dataultra458

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra459

import "std/c/dataultra459" · use dataultra459

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra460

import "std/c/dataultra460" · use dataultra460

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra461

import "std/c/dataultra461" · use dataultra461

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra462

import "std/c/dataultra462" · use dataultra462

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra463

import "std/c/dataultra463" · use dataultra463

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra464

import "std/c/dataultra464" · use dataultra464

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra465

import "std/c/dataultra465" · use dataultra465

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra466

import "std/c/dataultra466" · use dataultra466

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra467

import "std/c/dataultra467" · use dataultra467

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra468

import "std/c/dataultra468" · use dataultra468

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra469

import "std/c/dataultra469" · use dataultra469

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra470

import "std/c/dataultra470" · use dataultra470

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra471

import "std/c/dataultra471" · use dataultra471

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra472

import "std/c/dataultra472" · use dataultra472

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra473

import "std/c/dataultra473" · use dataultra473

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra474

import "std/c/dataultra474" · use dataultra474

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra475

import "std/c/dataultra475" · use dataultra475

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra476

import "std/c/dataultra476" · use dataultra476

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra477

import "std/c/dataultra477" · use dataultra477

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra478

import "std/c/dataultra478" · use dataultra478

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra479

import "std/c/dataultra479" · use dataultra479

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra480

import "std/c/dataultra480" · use dataultra480

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra481

import "std/c/dataultra481" · use dataultra481

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra482

import "std/c/dataultra482" · use dataultra482

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra483

import "std/c/dataultra483" · use dataultra483

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra484

import "std/c/dataultra484" · use dataultra484

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra485

import "std/c/dataultra485" · use dataultra485

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra486

import "std/c/dataultra486" · use dataultra486

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra487

import "std/c/dataultra487" · use dataultra487

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra488

import "std/c/dataultra488" · use dataultra488

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra489

import "std/c/dataultra489" · use dataultra489

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra490

import "std/c/dataultra490" · use dataultra490

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra491

import "std/c/dataultra491" · use dataultra491

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra492

import "std/c/dataultra492" · use dataultra492

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra493

import "std/c/dataultra493" · use dataultra493

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra494

import "std/c/dataultra494" · use dataultra494

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`

- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra495

import "std/c/dataultra495" · use dataultra495

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra496

import "std/c/dataultra496" · use dataultra496

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra497

import "std/c/dataultra497" · use dataultra497

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra498

import "std/c/dataultra498" · use dataultra498

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra499

import "std/c/dataultra499" · use dataultra499

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/dataultra500

import "std/c/dataultra500" · use dataultra500

- `mean(v list number) → number`
- `stddev(v list number) → number`
- `minVal(v list number) → number`
- `maxVal(v list number) → number`
- `normalize(v list number) → list number`
- `diff1(v list number) → list number`
- `rollingMean(v list number, window number) → list number`
- `percentile(v list number, p number) → number`
- `correlation(a list number, b list number) → number`
- `linearSlope(xs list number, ys list number) → number`

std/c/vizultra001

import "std/c/vizultra001" · use vizultra001

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`

- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra002

import "std/c/vizultra002" · use vizultra002

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra003

import "std/c/vizultra003" · use vizultra003

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra004

import "std/c/vizultra004" · use vizultra004

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra005

import "std/c/vizultra005" · use vizultra005

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra006

import "std/c/vizultra006" · use vizultra006

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra007

import "std/c/vizultra007" · use vizultra007

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra008

import "std/c/vizultra008" · use vizultra008

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra009

import "std/c/vizultra009" · use vizultra009

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra010

import "std/c/vizultra010" · use vizultra010

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra011

import "std/c/vizultra011" · use vizultra011

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra012

import "std/c/vizultra012" · use vizultra012

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra013

import "std/c/vizultra013" · use vizultra013

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra014

import "std/c/vizultra014" · use vizultra014

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra015

import "std/c/vizultra015" · use vizultra015

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra016

import "std/c/vizultra016" · use vizultra016

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra017

import "std/c/vizultra017" · use vizultra017

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra018

import "std/c/vizultra018" · use vizultra018

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra019

import "std/c/vizultra019" · use vizultra019

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra020

import "std/c/vizultra020" · use vizultra020

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra021

import "std/c/vizultra021" · use vizultra021

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra022

import "std/c/vizultra022" · use vizultra022

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra023

import "std/c/vizultra023" · use vizultra023

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra024

import "std/c/vizultra024" · use vizultra024

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra025

import "std/c/vizultra025" · use vizultra025

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra026

import "std/c/vizultra026" · use vizultra026

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra027

import "std/c/vizultra027" · use vizultra027

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra028

import "std/c/vizultra028" · use vizultra028

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra029

import "std/c/vizultra029" · use vizultra029

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra030

import "std/c/vizultra030" · use vizultra030

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra031

import "std/c/vizultra031" · use vizultra031

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra032

import "std/c/vizultra032" · use vizultra032

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra033

import "std/c/vizultra033" · use vizultra033

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra034

import "std/c/vizultra034" · use vizultra034

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra035

import "std/c/vizultra035" · use vizultra035

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra036

import "std/c/vizultra036" · use vizultra036

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra037

import "std/c/vizultra037" · use vizultra037

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra038

import "std/c/vizultra038" · use vizultra038

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra039

import "std/c/vizultra039" · use vizultra039

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra040

import "std/c/vizultra040" · use vizultra040

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra041

import "std/c/vizultra041" · use vizultra041

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra042

import "std/c/vizultra042" · use vizultra042

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra043

import "std/c/vizultra043" · use vizultra043

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra044

import "std/c/vizultra044" · use vizultra044

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra045

import "std/c/vizultra045" · use vizultra045

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra046

import "std/c/vizultra046" · use vizultra046

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra047

import "std/c/vizultra047" · use vizultra047

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra048

import "std/c/vizultra048" · use vizultra048

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra049

import "std/c/vizultra049" · use vizultra049

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra050

import "std/c/vizultra050" · use vizultra050

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra051

import "std/c/vizultra051" · use vizultra051

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra052

import "std/c/vizultra052" · use vizultra052

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra053

import "std/c/vizultra053" · use vizultra053

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra054

import "std/c/vizultra054" · use vizultra054

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra055

import "std/c/vizultra055" · use vizultra055

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra056

import "std/c/vizultra056" · use vizultra056

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra057

import "std/c/vizultra057" · use vizultra057

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra058

import "std/c/vizultra058" · use vizultra058

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra059

import "std/c/vizultra059" · use vizultra059

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra060

import "std/c/vizultra060" · use vizultra060

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra061

import "std/c/vizultra061" · use vizultra061

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra062

import "std/c/vizultra062" · use vizultra062

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra063

import "std/c/vizultra063" · use vizultra063

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra064

import "std/c/vizultra064" · use vizultra064

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra065

import "std/c/vizultra065" · use vizultra065

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra066

import "std/c/vizultra066" · use vizultra066

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra067

import "std/c/vizultra067" · use vizultra067

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra068

import "std/c/vizultra068" · use vizultra068

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra069

import "std/c/vizultra069" · use vizultra069

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra070

import "std/c/vizultra070" · use vizultra070

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra071

import "std/c/vizultra071" · use vizultra071

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra072

import "std/c/vizultra072" · use vizultra072

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra073

import "std/c/vizultra073" · use vizultra073

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra074

import "std/c/vizultra074" · use vizultra074

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra075

import "std/c/vizultra075" · use vizultra075

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra076

import "std/c/vizultra076" · use vizultra076

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra077

import "std/c/vizultra077" · use vizultra077

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra078

import "std/c/vizultra078" · use vizultra078

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra079

import "std/c/vizultra079" · use vizultra079

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra080

import "std/c/vizultra080" · use vizultra080

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra081

import "std/c/vizultra081" · use vizultra081

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra082

import "std/c/vizultra082" · use vizultra082

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra083

import "std/c/vizultra083" · use vizultra083

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra084

import "std/c/vizultra084" · use vizultra084

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra085

import "std/c/vizultra085" · use vizultra085

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra086

import "std/c/vizultra086" · use vizultra086

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra087

import "std/c/vizultra087" · use vizultra087

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra088

import "std/c/vizultra088" · use vizultra088

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra089

import "std/c/vizultra089" · use vizultra089

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra090

import "std/c/vizultra090" · use vizultra090

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra091

import "std/c/vizultra091" · use vizultra091

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra092

import "std/c/vizultra092" · use vizultra092

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra093

import "std/c/vizultra093" · use vizultra093

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra094

import "std/c/vizultra094" · use vizultra094

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra095

import "std/c/vizultra095" · use vizultra095

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra096

import "std/c/vizultra096" · use vizultra096

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra097

import "std/c/vizultra097" · use vizultra097

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra098

import "std/c/vizultra098" · use vizultra098

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra099

import "std/c/vizultra099" · use vizultra099

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra100

import "std/c/vizultra100" · use vizultra100

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra101

import "std/c/vizultra101" · use vizultra101

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra102

import "std/c/vizultra102" · use vizultra102

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra103

import "std/c/vizultra103" · use vizultra103

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra104

import "std/c/vizultra104" · use vizultra104

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra105

import "std/c/vizultra105" · use vizultra105

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra106

import "std/c/vizultra106" · use vizultra106

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra107

import "std/c/vizultra107" · use vizultra107

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra108

import "std/c/vizultra108" · use vizultra108

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra109

import "std/c/vizultra109" · use vizultra109

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra110

import "std/c/vizultra110" · use vizultra110

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra111

import "std/c/vizultra111" · use vizultra111

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra112

import "std/c/vizultra112" · use vizultra112

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra113

import "std/c/vizultra113" · use vizultra113

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra114

import "std/c/vizultra114" · use vizultra114

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra115

import "std/c/vizultra115" · use vizultra115

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra116

import "std/c/vizultra116" · use vizultra116

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra117

import "std/c/vizultra117" · use vizultra117

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra118

import "std/c/vizultra118" · use vizultra118

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra119

import "std/c/vizultra119" · use vizultra119

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra120

import "std/c/vizultra120" · use vizultra120

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra121

import "std/c/vizultra121" · use vizultra121

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra122

import "std/c/vizultra122" · use vizultra122

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra123

import "std/c/vizultra123" · use vizultra123

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra124

import "std/c/vizultra124" · use vizultra124

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra125

import "std/c/vizultra125" · use vizultra125

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra126

import "std/c/vizultra126" · use vizultra126

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra127

import "std/c/vizultra127" · use vizultra127

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra128

import "std/c/vizultra128" · use vizultra128

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra129

import "std/c/vizultra129" · use vizultra129

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra130

import "std/c/vizultra130" · use vizultra130

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra131

import "std/c/vizultra131" · use vizultra131

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra132

import "std/c/vizultra132" · use vizultra132

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra133

import "std/c/vizultra133" · use vizultra133

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra134

import "std/c/vizultra134" · use vizultra134

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra135

import "std/c/vizultra135" · use vizultra135

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra136

import "std/c/vizultra136" · use vizultra136

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra137

import "std/c/vizultra137" · use vizultra137

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra138

import "std/c/vizultra138" · use vizultra138

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra139

import "std/c/vizultra139" · use vizultra139

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra140

import "std/c/vizultra140" · use vizultra140

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra141

import "std/c/vizultra141" · use vizultra141

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra142

import "std/c/vizultra142" · use vizultra142

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra143

import "std/c/vizultra143" · use vizultra143

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra144

import "std/c/vizultra144" · use vizultra144

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra145

import "std/c/vizultra145" · use vizultra145

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra146

import "std/c/vizultra146" · use vizultra146

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra147

import "std/c/vizultra147" · use vizultra147

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra148

import "std/c/vizultra148" · use vizultra148

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra149

import "std/c/vizultra149" · use vizultra149

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra150

import "std/c/vizultra150" · use vizultra150

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra151

import "std/c/vizultra151" · use vizultra151

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra152

import "std/c/vizultra152" · use vizultra152

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra153

import "std/c/vizultra153" · use vizultra153

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra154

import "std/c/vizultra154" · use vizultra154

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra155

import "std/c/vizultra155" · use vizultra155

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra156

import "std/c/vizultra156" · use vizultra156

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra157

import "std/c/vizultra157" · use vizultra157

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra158

import "std/c/vizultra158" · use vizultra158

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra159

import "std/c/vizultra159" · use vizultra159

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra160

import "std/c/vizultra160" · use vizultra160

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra161

import "std/c/vizultra161" · use vizultra161

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra162

import "std/c/vizultra162" · use vizultra162

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra163

import "std/c/vizultra163" · use vizultra163

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra164

import "std/c/vizultra164" · use vizultra164

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra165

import "std/c/vizultra165" · use vizultra165

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra166

import "std/c/vizultra166" · use vizultra166

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra167

import "std/c/vizultra167" · use vizultra167

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra168

import "std/c/vizultra168" · use vizultra168

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra169

import "std/c/vizultra169" · use vizultra169

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra170

import "std/c/vizultra170" · use vizultra170

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra171

import "std/c/vizultra171" · use vizultra171

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra172

import "std/c/vizultra172" · use vizultra172

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra173

import "std/c/vizultra173" · use vizultra173

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra174

import "std/c/vizultra174" · use vizultra174

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra175

import "std/c/vizultra175" · use vizultra175

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra176

import "std/c/vizultra176" · use vizultra176

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra177

import "std/c/vizultra177" · use vizultra177

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra178

import "std/c/vizultra178" · use vizultra178

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra179

import "std/c/vizultra179" · use vizultra179

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra180

import "std/c/vizultra180" · use vizultra180

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra181

import "std/c/vizultra181" · use vizultra181

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra182

import "std/c/vizultra182" · use vizultra182

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra183

import "std/c/vizultra183" · use vizultra183

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra184

import "std/c/vizultra184" · use vizultra184

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra185

import "std/c/vizultra185" · use vizultra185

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra186

import "std/c/vizultra186" · use vizultra186

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra187

import "std/c/vizultra187" · use vizultra187

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra188

import "std/c/vizultra188" · use vizultra188

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra189

import "std/c/vizultra189" · use vizultra189

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra190

import "std/c/vizultra190" · use vizultra190

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra191

import "std/c/vizultra191" · use vizultra191

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra192

import "std/c/vizultra192" · use vizultra192

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra193

import "std/c/vizultra193" · use vizultra193

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra194

import "std/c/vizultra194" · use vizultra194

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra195

import "std/c/vizultra195" · use vizultra195

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra196

import "std/c/vizultra196" · use vizultra196

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra197

import "std/c/vizultra197" · use vizultra197

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra198

import "std/c/vizultra198" · use vizultra198

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra199

import "std/c/vizultra199" · use vizultra199

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra200

import "std/c/vizultra200" · use vizultra200

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra201

import "std/c/vizultra201" · use vizultra201

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra202

import "std/c/vizultra202" · use vizultra202

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra203

import "std/c/vizultra203" · use vizultra203

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra204

import "std/c/vizultra204" · use vizultra204

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra205

import "std/c/vizultra205" · use vizultra205

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra206

import "std/c/vizultra206" · use vizultra206

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra207

import "std/c/vizultra207" · use vizultra207

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra208

import "std/c/vizultra208" · use vizultra208

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra209

import "std/c/vizultra209" · use vizultra209

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra210

import "std/c/vizultra210" · use vizultra210

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra211

import "std/c/vizultra211" · use vizultra211

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra212

import "std/c/vizultra212" · use vizultra212

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra213

import "std/c/vizultra213" · use vizultra213

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra214

import "std/c/vizultra214" · use vizultra214

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra215

import "std/c/vizultra215" · use vizultra215

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra216

import "std/c/vizultra216" · use vizultra216

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra217

import "std/c/vizultra217" · use vizultra217

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra218

import "std/c/vizultra218" · use vizultra218

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra219

import "std/c/vizultra219" · use vizultra219

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra220

import "std/c/vizultra220" · use vizultra220

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra221

import "std/c/vizultra221" · use vizultra221

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra222

import "std/c/vizultra222" · use vizultra222

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra223

import "std/c/vizultra223" · use vizultra223

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra224

import "std/c/vizultra224" · use vizultra224

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra225

import "std/c/vizultra225" · use vizultra225

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra226

import "std/c/vizultra226" · use vizultra226

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra227

import "std/c/vizultra227" · use vizultra227

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra228

import "std/c/vizultra228" · use vizultra228

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra229

import "std/c/vizultra229" · use vizultra229

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra230

import "std/c/vizultra230" · use vizultra230

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra231

import "std/c/vizultra231" · use vizultra231

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra232

import "std/c/vizultra232" · use vizultra232

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra233

import "std/c/vizultra233" · use vizultra233

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra234

import "std/c/vizultra234" · use vizultra234

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra235

import "std/c/vizultra235" · use vizultra235

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra236

import "std/c/vizultra236" · use vizultra236

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra237

import "std/c/vizultra237" · use vizultra237

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra238

import "std/c/vizultra238" · use vizultra238

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra239

import "std/c/vizultra239" · use vizultra239

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra240

import "std/c/vizultra240" · use vizultra240

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra241

import "std/c/vizultra241" · use vizultra241

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra242

import "std/c/vizultra242" · use vizultra242

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra243

import "std/c/vizultra243" · use vizultra243

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra244

import "std/c/vizultra244" · use vizultra244

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra245

import "std/c/vizultra245" · use vizultra245

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra246

import "std/c/vizultra246" · use vizultra246

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra247

import "std/c/vizultra247" · use vizultra247

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra248

import "std/c/vizultra248" · use vizultra248

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra249

import "std/c/vizultra249" · use vizultra249

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra250

import "std/c/vizultra250" · use vizultra250

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra251

import "std/c/vizultra251" · use vizultra251

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra252

import "std/c/vizultra252" · use vizultra252

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra253

import "std/c/vizultra253" · use vizultra253

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra254

import "std/c/vizultra254" · use vizultra254

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra255

import "std/c/vizultra255" · use vizultra255

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra256

import "std/c/vizultra256" · use vizultra256

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra257

import "std/c/vizultra257" · use vizultra257

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra258

import "std/c/vizultra258" · use vizultra258

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra259

import "std/c/vizultra259" · use vizultra259

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra260

import "std/c/vizultra260" · use vizultra260

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra261

import "std/c/vizultra261" · use vizultra261

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra262

import "std/c/vizultra262" · use vizultra262

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra263

import "std/c/vizultra263" · use vizultra263

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra264

import "std/c/vizultra264" · use vizultra264

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra265

import "std/c/vizultra265" · use vizultra265

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra266

import "std/c/vizultra266" · use vizultra266

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra267

import "std/c/vizultra267" · use vizultra267

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra268

import "std/c/vizultra268" · use vizultra268

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra269

import "std/c/vizultra269" · use vizultra269

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra270

import "std/c/vizultra270" · use vizultra270

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra271

import "std/c/vizultra271" · use vizultra271

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra272

import "std/c/vizultra272" · use vizultra272

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra273

import "std/c/vizultra273" · use vizultra273

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra274

import "std/c/vizultra274" · use vizultra274

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra275

import "std/c/vizultra275" · use vizultra275

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra276

import "std/c/vizultra276" · use vizultra276

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra277

import "std/c/vizultra277" · use vizultra277

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra278

import "std/c/vizultra278" · use vizultra278

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra279

import "std/c/vizultra279" · use vizultra279

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra280

import "std/c/vizultra280" · use vizultra280

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra281

import "std/c/vizultra281" · use vizultra281

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra282

import "std/c/vizultra282" · use vizultra282

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra283

import "std/c/vizultra283" · use vizultra283

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra284

import "std/c/vizultra284" · use vizultra284

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra285

import "std/c/vizultra285" · use vizultra285

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra286

import "std/c/vizultra286" · use vizultra286

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra287

import "std/c/vizultra287" · use vizultra287

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra288

import "std/c/vizultra288" · use vizultra288

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra289

import "std/c/vizultra289" · use vizultra289

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra290

import "std/c/vizultra290" · use vizultra290

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra291

import "std/c/vizultra291" · use vizultra291

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra292

import "std/c/vizultra292" · use vizultra292

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra293

import "std/c/vizultra293" · use vizultra293

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra294

import "std/c/vizultra294" · use vizultra294

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra295

import "std/c/vizultra295" · use vizultra295

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra296

import "std/c/vizultra296" · use vizultra296

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra297

import "std/c/vizultra297" · use vizultra297

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra298

import "std/c/vizultra298" · use vizultra298

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra299

import "std/c/vizultra299" · use vizultra299

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra300

import "std/c/vizultra300" · use vizultra300

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra301

import "std/c/vizultra301" · use vizultra301

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra302

import "std/c/vizultra302" · use vizultra302

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra303

import "std/c/vizultra303" · use vizultra303

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra304

import "std/c/vizultra304" · use vizultra304

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra305

import "std/c/vizultra305" · use vizultra305

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra306

import "std/c/vizultra306" · use vizultra306

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra307

import "std/c/vizultra307" · use vizultra307

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra308

import "std/c/vizultra308" · use vizultra308

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra309

import "std/c/vizultra309" · use vizultra309

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra310

import "std/c/vizultra310" · use vizultra310

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra311

import "std/c/vizultra311" · use vizultra311

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra312

import "std/c/vizultra312" · use vizultra312

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra313

import "std/c/vizultra313" · use vizultra313

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra314

import "std/c/vizultra314" · use vizultra314

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra315

import "std/c/vizultra315" · use vizultra315

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra316

import "std/c/vizultra316" · use vizultra316

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra317

import "std/c/vizultra317" · use vizultra317

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra318

import "std/c/vizultra318" · use vizultra318

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra319

import "std/c/vizultra319" · use vizultra319

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra320

import "std/c/vizultra320" · use vizultra320

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra321

import "std/c/vizultra321" · use vizultra321

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra322

import "std/c/vizultra322" · use vizultra322

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra323

import "std/c/vizultra323" · use vizultra323

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra324

import "std/c/vizultra324" · use vizultra324

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra325

import "std/c/vizultra325" · use vizultra325

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra326

import "std/c/vizultra326" · use vizultra326

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra327

import "std/c/vizultra327" · use vizultra327

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra328

import "std/c/vizultra328" · use vizultra328

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra329

import "std/c/vizultra329" · use vizultra329

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra330

import "std/c/vizultra330" · use vizultra330

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra331

import "std/c/vizultra331" · use vizultra331

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra332

import "std/c/vizultra332" · use vizultra332

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra333

import "std/c/vizultra333" · use vizultra333

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra334

import "std/c/vizultra334" · use vizultra334

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra335

import "std/c/vizultra335" · use vizultra335

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra336

import "std/c/vizultra336" · use vizultra336

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra337

import "std/c/vizultra337" · use vizultra337

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra338

import "std/c/vizultra338" · use vizultra338

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra339

import "std/c/vizultra339" · use vizultra339

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra340

import "std/c/vizultra340" · use vizultra340

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra341

import "std/c/vizultra341" · use vizultra341

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra342

import "std/c/vizultra342" · use vizultra342

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra343

import "std/c/vizultra343" · use vizultra343

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra344

import "std/c/vizultra344" · use vizultra344

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra345

import "std/c/vizultra345" · use vizultra345

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra346

import "std/c/vizultra346" · use vizultra346

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra347

import "std/c/vizultra347" · use vizultra347

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra348

import "std/c/vizultra348" · use vizultra348

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra349

import "std/c/vizultra349" · use vizultra349

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra350

import "std/c/vizultra350" · use vizultra350

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra351

import "std/c/vizultra351" · use vizultra351

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra352

import "std/c/vizultra352" · use vizultra352

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra353

import "std/c/vizultra353" · use vizultra353

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra354

import "std/c/vizultra354" · use vizultra354

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra355

import "std/c/vizultra355" · use vizultra355

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra356

import "std/c/vizultra356" · use vizultra356

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra357

import "std/c/vizultra357" · use vizultra357

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra358

import "std/c/vizultra358" · use vizultra358

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra359

import "std/c/vizultra359" · use vizultra359

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra360

import "std/c/vizultra360" · use vizultra360

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra361

import "std/c/vizultra361" · use vizultra361

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra362

import "std/c/vizultra362" · use vizultra362

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra363

import "std/c/vizultra363" · use vizultra363

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra364

import "std/c/vizultra364" · use vizultra364

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra365

import "std/c/vizultra365" · use vizultra365

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra366

import "std/c/vizultra366" · use vizultra366

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra367

import "std/c/vizultra367" · use vizultra367

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra368

import "std/c/vizultra368" · use vizultra368

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra369

import "std/c/vizultra369" · use vizultra369

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra370

import "std/c/vizultra370" · use vizultra370

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra371

import "std/c/vizultra371" · use vizultra371

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra372

import "std/c/vizultra372" · use vizultra372

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra373

import "std/c/vizultra373" · use vizultra373

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra374

import "std/c/vizultra374" · use vizultra374

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra375

import "std/c/vizultra375" · use vizultra375

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra376

import "std/c/vizultra376" · use vizultra376

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra377

import "std/c/vizultra377" · use vizultra377

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra378

import "std/c/vizultra378" · use vizultra378

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra379

import "std/c/vizultra379" · use vizultra379

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra380

import "std/c/vizultra380" · use vizultra380

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra381

import "std/c/vizultra381" · use vizultra381

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra382

import "std/c/vizultra382" · use vizultra382

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra383

import "std/c/vizultra383" · use vizultra383

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra384

import "std/c/vizultra384" · use vizultra384

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra385

import "std/c/vizultra385" · use vizultra385

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra386

import "std/c/vizultra386" · use vizultra386

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra387

import "std/c/vizultra387" · use vizultra387

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra388

import "std/c/vizultra388" · use vizultra388

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra389

import "std/c/vizultra389" · use vizultra389

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra390

import "std/c/vizultra390" · use vizultra390

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra391

import "std/c/vizultra391" · use vizultra391

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra392

import "std/c/vizultra392" · use vizultra392

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra393

import "std/c/vizultra393" · use vizultra393

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra394

import "std/c/vizultra394" · use vizultra394

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra395

import "std/c/vizultra395" · use vizultra395

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra396

import "std/c/vizultra396" · use vizultra396

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra397

import "std/c/vizultra397" · use vizultra397

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra398

import "std/c/vizultra398" · use vizultra398

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra399

import "std/c/vizultra399" · use vizultra399

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra400

import "std/c/vizultra400" · use vizultra400

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra401

import "std/c/vizultra401" · use vizultra401

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra402

import "std/c/vizultra402" · use vizultra402

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra403

import "std/c/vizultra403" · use vizultra403

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra404

import "std/c/vizultra404" · use vizultra404

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra405

import "std/c/vizultra405" · use vizultra405

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra406

import "std/c/vizultra406" · use vizultra406

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra407

import "std/c/vizultra407" · use vizultra407

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra408

import "std/c/vizultra408" · use vizultra408

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra409

import "std/c/vizultra409" · use vizultra409

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra410

import "std/c/vizultra410" · use vizultra410

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra411

import "std/c/vizultra411" · use vizultra411

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra412

import "std/c/vizultra412" · use vizultra412

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra413

import "std/c/vizultra413" · use vizultra413

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra414

import "std/c/vizultra414" · use vizultra414

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra415

import "std/c/vizultra415" · use vizultra415

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra416

import "std/c/vizultra416" · use vizultra416

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra417

import "std/c/vizultra417" · use vizultra417

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra418

import "std/c/vizultra418" · use vizultra418

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra419

import "std/c/vizultra419" · use vizultra419

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra420

import "std/c/vizultra420" · use vizultra420

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra421

import "std/c/vizultra421" · use vizultra421

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra422

import "std/c/vizultra422" · use vizultra422

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra423

import "std/c/vizultra423" · use vizultra423

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra424

import "std/c/vizultra424" · use vizultra424

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra425

import "std/c/vizultra425" · use vizultra425

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra426

import "std/c/vizultra426" · use vizultra426

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra427

import "std/c/vizultra427" · use vizultra427

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra428

import "std/c/vizultra428" · use vizultra428

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra429

import "std/c/vizultra429" · use vizultra429

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra430

import "std/c/vizultra430" · use vizultra430

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra431

import "std/c/vizultra431" · use vizultra431

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra432

import "std/c/vizultra432" · use vizultra432

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra433

import "std/c/vizultra433" · use vizultra433

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra434

import "std/c/vizultra434" · use vizultra434

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra435

import "std/c/vizultra435" · use vizultra435

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra436

import "std/c/vizultra436" · use vizultra436

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra437

import "std/c/vizultra437" · use vizultra437

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra438

import "std/c/vizultra438" · use vizultra438

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra439

import "std/c/vizultra439" · use vizultra439

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra440

import "std/c/vizultra440" · use vizultra440

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra441

import "std/c/vizultra441" · use vizultra441

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra442

import "std/c/vizultra442" · use vizultra442

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra443

import "std/c/vizultra443" · use vizultra443

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra444

import "std/c/vizultra444" · use vizultra444

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra445

import "std/c/vizultra445" · use vizultra445

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra446

import "std/c/vizultra446" · use vizultra446

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra447

import "std/c/vizultra447" · use vizultra447

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra448

import "std/c/vizultra448" · use vizultra448

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra449

import "std/c/vizultra449" · use vizultra449

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra450

import "std/c/vizultra450" · use vizultra450

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra451

import "std/c/vizultra451" · use vizultra451

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra452

import "std/c/vizultra452" · use vizultra452

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra453

import "std/c/vizultra453" · use vizultra453

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra454

import "std/c/vizultra454" · use vizultra454

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra455

import "std/c/vizultra455" · use vizultra455

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra456

import "std/c/vizultra456" · use vizultra456

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra457

import "std/c/vizultra457" · use vizultra457

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra458

import "std/c/vizultra458" · use vizultra458

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra459

import "std/c/vizultra459" · use vizultra459

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra460

import "std/c/vizultra460" · use vizultra460

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra461

import "std/c/vizultra461" · use vizultra461

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra462

import "std/c/vizultra462" · use vizultra462

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra463

import "std/c/vizultra463" · use vizultra463

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra464

import "std/c/vizultra464" · use vizultra464

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra465

import "std/c/vizultra465" · use vizultra465

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra466

import "std/c/vizultra466" · use vizultra466

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra467

import "std/c/vizultra467" · use vizultra467

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra468

import "std/c/vizultra468" · use vizultra468

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra469

import "std/c/vizultra469" · use vizultra469

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra470

import "std/c/vizultra470" · use vizultra470

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra471

import "std/c/vizultra471" · use vizultra471

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra472

import "std/c/vizultra472" · use vizultra472

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra473

import "std/c/vizultra473" · use vizultra473

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra474

import "std/c/vizultra474" · use vizultra474

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra475

import "std/c/vizultra475" · use vizultra475

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra476

import "std/c/vizultra476" · use vizultra476

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra477

import "std/c/vizultra477" · use vizultra477

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra478

import "std/c/vizultra478" · use vizultra478

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra479

import "std/c/vizultra479" · use vizultra479

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra480

import "std/c/vizultra480" · use vizultra480

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra481

import "std/c/vizultra481" · use vizultra481

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra482

import "std/c/vizultra482" · use vizultra482

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra483

import "std/c/vizultra483" · use vizultra483

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra484

import "std/c/vizultra484" · use vizultra484

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra485

import "std/c/vizultra485" · use vizultra485

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra486

import "std/c/vizultra486" · use vizultra486

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra487

import "std/c/vizultra487" · use vizultra487

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra488

import "std/c/vizultra488" · use vizultra488

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra489

import "std/c/vizultra489" · use vizultra489

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra490

import "std/c/vizultra490" · use vizultra490

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra491

import "std/c/vizultra491" · use vizultra491

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra492

import "std/c/vizultra492" · use vizultra492

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra493

import "std/c/vizultra493" · use vizultra493

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra494

import "std/c/vizultra494" · use vizultra494

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra495

import "std/c/vizultra495" · use vizultra495

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra496

import "std/c/vizultra496" · use vizultra496

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra497

import "std/c/vizultra497" · use vizultra497

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra498

import "std/c/vizultra498" · use vizultra498

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra499

import "std/c/vizultra499" · use vizultra499

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/vizultra500

import "std/c/vizultra500" · use vizultra500

- `writeLineChart(path text, ys list number, width number, height number) → bool`
- `writeBarChart(path text, ys list number, width number, height number) → bool`
- `writeScatter(path text, xs list number, ys list number, width number, height number) → bool`
- `writeCsv(path text, ys list number) → bool`
- `writeHeatmap(path text, grid list number, w number, h number, width number, height number) → bool`

std/c/datasci001

import "std/c/datasci001" · use datasci001

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`

- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci002

import "std/c/datasci002" · use datasci002

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci003

import "std/c/datasci003" · use datasci003

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci004

import "std/c/datasci004" · use datasci004

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci005

import "std/c/datasci005" · use datasci005

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci006

import "std/c/datasci006" · use datasci006

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci007

import "std/c/datasci007" · use datasci007

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci008

import "std/c/datasci008" · use datasci008

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci009

import "std/c/datasci009" · use datasci009

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci010

import "std/c/datasci010" · use datasci010

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci011

import "std/c/datasci011" · use datasci011

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci012

import "std/c/datasci012" · use datasci012

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci013

import "std/c/datasci013" · use datasci013

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci014

import "std/c/datasci014" · use datasci014

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci015

import "std/c/datasci015" · use datasci015

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci016

import "std/c/datasci016" · use datasci016

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci017

import "std/c/datasci017" · use datasci017

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci018

import "std/c/datasci018" · use datasci018

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci019

import "std/c/datasci019" · use datasci019

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci020

import "std/c/datasci020" · use datasci020

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci021

import "std/c/datasci021" · use datasci021

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci022

import "std/c/datasci022" · use datasci022

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci023

import "std/c/datasci023" · use datasci023

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci024

import "std/c/datasci024" · use datasci024

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci025

import "std/c/datasci025" · use datasci025

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci026

import "std/c/datasci026" · use datasci026

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci027

import "std/c/datasci027" · use datasci027

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci028

import "std/c/datasci028" · use datasci028

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci029

import "std/c/datasci029" · use datasci029

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci030

import "std/c/datasci030" · use datasci030

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci031

import "std/c/datasci031" · use datasci031

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci032

import "std/c/datasci032" · use datasci032

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci033

import "std/c/datasci033" · use datasci033

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci034

import "std/c/datasci034" · use datasci034

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci035

import "std/c/datasci035" · use datasci035

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci036

import "std/c/datasci036" · use datasci036

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci037

import "std/c/datasci037" · use datasci037

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci038

import "std/c/datasci038" · use datasci038

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci039

import "std/c/datasci039" · use datasci039

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci040

import "std/c/datasci040" · use datasci040

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci041

import "std/c/datasci041" · use datasci041

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci042

import "std/c/datasci042" · use datasci042

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci043

import "std/c/datasci043" · use datasci043

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci044

import "std/c/datasci044" · use datasci044

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci045

import "std/c/datasci045" · use datasci045

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci046

import "std/c/datasci046" · use datasci046

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci047

import "std/c/datasci047" · use datasci047

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci048

import "std/c/datasci048" · use datasci048

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci049

import "std/c/datasci049" · use datasci049

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci050

import "std/c/datasci050" · use datasci050

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci051

import "std/c/datasci051" · use datasci051

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci052

import "std/c/datasci052" · use datasci052

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci053

import "std/c/datasci053" · use datasci053

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci054

import "std/c/datasci054" · use datasci054

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci055

import "std/c/datasci055" · use datasci055

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci056

import "std/c/datasci056" · use datasci056

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci057

import "std/c/datasci057" · use datasci057

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci058

import "std/c/datasci058" · use datasci058

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci059

import "std/c/datasci059" · use datasci059

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci060

import "std/c/datasci060" · use datasci060

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci061

import "std/c/datasci061" · use datasci061

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci062

import "std/c/datasci062" · use datasci062

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci063

import "std/c/datasci063" · use datasci063

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci064

import "std/c/datasci064" · use datasci064

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci065

import "std/c/datasci065" · use datasci065

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci066

import "std/c/datasci066" · use datasci066

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci067

import "std/c/datasci067" · use datasci067

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci068

import "std/c/datasci068" · use datasci068

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci069

import "std/c/datasci069" · use datasci069

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci070

import "std/c/datasci070" · use datasci070

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci071

import "std/c/datasci071" · use datasci071

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci072

import "std/c/datasci072" · use datasci072

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci073

import "std/c/datasci073" · use datasci073

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci074

import "std/c/datasci074" · use datasci074

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci075

import "std/c/datasci075" · use datasci075

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci076

import "std/c/datasci076" · use datasci076

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci077

import "std/c/datasci077" · use datasci077

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci078

import "std/c/datasci078" · use datasci078

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci079

import "std/c/datasci079" · use datasci079

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci080

import "std/c/datasci080" · use datasci080

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci081

import "std/c/datasci081" · use datasci081

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci082

import "std/c/datasci082" · use datasci082

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci083

import "std/c/datasci083" · use datasci083

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci084

import "std/c/datasci084" · use datasci084

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci085

import "std/c/datasci085" · use datasci085

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci086

import "std/c/datasci086" · use datasci086

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci087

import "std/c/datasci087" · use datasci087

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci088

import "std/c/datasci088" · use datasci088

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci089

import "std/c/datasci089" · use datasci089

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci090

import "std/c/datasci090" · use datasci090

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci091

import "std/c/datasci091" · use datasci091

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci092

import "std/c/datasci092" · use datasci092

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci093

import "std/c/datasci093" · use datasci093

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci094

import "std/c/datasci094" · use datasci094

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci095

import "std/c/datasci095" · use datasci095

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci096

import "std/c/datasci096" · use datasci096

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci097

import "std/c/datasci097" · use datasci097

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci098

import "std/c/datasci098" · use datasci098

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci099

import "std/c/datasci099" · use datasci099

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci100

import "std/c/datasci100" · use datasci100

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci101

import "std/c/datasci101" · use datasci101

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci102

import "std/c/datasci102" · use datasci102

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci103

import "std/c/datasci103" · use datasci103

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci104

import "std/c/datasci104" · use datasci104

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci105

import "std/c/datasci105" · use datasci105

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci106

import "std/c/datasci106" · use datasci106

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci107

import "std/c/datasci107" · use datasci107

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci108

import "std/c/datasci108" · use datasci108

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci109

import "std/c/datasci109" · use datasci109

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci110

import "std/c/datasci110" · use datasci110

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci111

import "std/c/datasci111" · use datasci111

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci112

import "std/c/datasci112" · use datasci112

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci113

import "std/c/datasci113" · use datasci113

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci114

import "std/c/datasci114" · use datasci114

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci115

import "std/c/datasci115" · use datasci115

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci116

import "std/c/datasci116" · use datasci116

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci117

import "std/c/datasci117" · use datasci117

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci118

import "std/c/datasci118" · use datasci118

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci119

import "std/c/datasci119" · use datasci119

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci120

import "std/c/datasci120" · use datasci120

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci121

import "std/c/datasci121" · use datasci121

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci122

import "std/c/datasci122" · use datasci122

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci123

import "std/c/datasci123" · use datasci123

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci124

import "std/c/datasci124" · use datasci124

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci125

import "std/c/datasci125" · use datasci125

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci126

import "std/c/datasci126" · use datasci126

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci127

import "std/c/datasci127" · use datasci127

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci128

import "std/c/datasci128" · use datasci128

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci129

import "std/c/datasci129" · use datasci129

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci130

import "std/c/datasci130" · use datasci130

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci131

import "std/c/datasci131" · use datasci131

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci132

import "std/c/datasci132" · use datasci132

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci133

import "std/c/datasci133" · use datasci133

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci134

import "std/c/datasci134" · use datasci134

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci135

import "std/c/datasci135" · use datasci135

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci136

import "std/c/datasci136" · use datasci136

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci137

import "std/c/datasci137" · use datasci137

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci138

import "std/c/datasci138" · use datasci138

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci139

import "std/c/datasci139" · use datasci139

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci140

import "std/c/datasci140" · use datasci140

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci141

import "std/c/datasci141" · use datasci141

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci142

import "std/c/datasci142" · use datasci142

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci143

import "std/c/datasci143" · use datasci143

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci144

import "std/c/datasci144" · use datasci144

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci145

import "std/c/datasci145" · use datasci145

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci146

import "std/c/datasci146" · use datasci146

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci147

import "std/c/datasci147" · use datasci147

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci148

import "std/c/datasci148" · use datasci148

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci149

import "std/c/datasci149" · use datasci149

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci150

import "std/c/datasci150" · use datasci150

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci151

import "std/c/datasci151" · use datasci151

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci152

import "std/c/datasci152" · use datasci152

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci153

import "std/c/datasci153" · use datasci153

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci154

import "std/c/datasci154" · use datasci154

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci155

import "std/c/datasci155" · use datasci155

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci156

import "std/c/datasci156" · use datasci156

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci157

import "std/c/datasci157" · use datasci157

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci158

import "std/c/datasci158" · use datasci158

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci159

import "std/c/datasci159" · use datasci159

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci160

import "std/c/datasci160" · use datasci160

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci161

import "std/c/datasci161" · use datasci161

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci162

import "std/c/datasci162" · use datasci162

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci163

import "std/c/datasci163" · use datasci163

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci164

import "std/c/datasci164" · use datasci164

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci165

import "std/c/datasci165" · use datasci165

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci166

import "std/c/datasci166" · use datasci166

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci167

import "std/c/datasci167" · use datasci167

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci168

import "std/c/datasci168" · use datasci168

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci169

import "std/c/datasci169" · use datasci169

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci170

import "std/c/datasci170" · use datasci170

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci171

import "std/c/datasci171" · use datasci171

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci172

import "std/c/datasci172" · use datasci172

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci173

import "std/c/datasci173" · use datasci173

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci174

import "std/c/datasci174" · use datasci174

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci175

import "std/c/datasci175" · use datasci175

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci176

import "std/c/datasci176" · use datasci176

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci177

import "std/c/datasci177" · use datasci177

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci178

import "std/c/datasci178" · use datasci178

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci179

import "std/c/datasci179" · use datasci179

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci180

import "std/c/datasci180" · use datasci180

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci181

import "std/c/datasci181" · use datasci181

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci182

import "std/c/datasci182" · use datasci182

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci183

import "std/c/datasci183" · use datasci183

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci184

import "std/c/datasci184" · use datasci184

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci185

import "std/c/datasci185" · use datasci185

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci186

import "std/c/datasci186" · use datasci186

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci187

import "std/c/datasci187" · use datasci187

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci188

import "std/c/datasci188" · use datasci188

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci189

import "std/c/datasci189" · use datasci189

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci190

import "std/c/datasci190" · use datasci190

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci191

import "std/c/datasci191" · use datasci191

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci192

import "std/c/datasci192" · use datasci192

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci193

import "std/c/datasci193" · use datasci193

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci194

import "std/c/datasci194" · use datasci194

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci195

import "std/c/datasci195" · use datasci195

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci196

import "std/c/datasci196" · use datasci196

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci197

import "std/c/datasci197" · use datasci197

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci198

import "std/c/datasci198" · use datasci198

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci199

import "std/c/datasci199" · use datasci199

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci200

import "std/c/datasci200" · use datasci200

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci201

import "std/c/datasci201" · use datasci201

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci202

import "std/c/datasci202" · use datasci202

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci203

import "std/c/datasci203" · use datasci203

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci204

import "std/c/datasci204" · use datasci204

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci205

import "std/c/datasci205" · use datasci205

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci206

import "std/c/datasci206" · use datasci206

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci207

import "std/c/datasci207" · use datasci207

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci208

import "std/c/datasci208" · use datasci208

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci209

import "std/c/datasci209" · use datasci209

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci210

import "std/c/datasci210" · use datasci210

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci211

import "std/c/datasci211" · use datasci211

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci212

import "std/c/datasci212" · use datasci212

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci213

import "std/c/datasci213" · use datasci213

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci214

import "std/c/datasci214" · use datasci214

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci215

import "std/c/datasci215" · use datasci215

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci216

import "std/c/datasci216" · use datasci216

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci217

import "std/c/datasci217" · use datasci217

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci218

import "std/c/datasci218" · use datasci218

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci219

import "std/c/datasci219" · use datasci219

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci220

import "std/c/datasci220" · use datasci220

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci221

import "std/c/datasci221" · use datasci221

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci222

import "std/c/datasci222" · use datasci222

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci223

import "std/c/datasci223" · use datasci223

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci224

import "std/c/datasci224" · use datasci224

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci225

import "std/c/datasci225" · use datasci225

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci226

import "std/c/datasci226" · use datasci226

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci227

import "std/c/datasci227" · use datasci227

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci228

import "std/c/datasci228" · use datasci228

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci229

import "std/c/datasci229" · use datasci229

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci230

import "std/c/datasci230" · use datasci230

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci231

import "std/c/datasci231" · use datasci231

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci232

import "std/c/datasci232" · use datasci232

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci233

import "std/c/datasci233" · use datasci233

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci234

import "std/c/datasci234" · use datasci234

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci235

import "std/c/datasci235" · use datasci235

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci236

import "std/c/datasci236" · use datasci236

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci237

import "std/c/datasci237" · use datasci237

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci238

import "std/c/datasci238" · use datasci238

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci239

import "std/c/datasci239" · use datasci239

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci240

import "std/c/datasci240" · use datasci240

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci241

import "std/c/datasci241" · use datasci241

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci242

import "std/c/datasci242" · use datasci242

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci243

import "std/c/datasci243" · use datasci243

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci244

import "std/c/datasci244" · use datasci244

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci245

import "std/c/datasci245" · use datasci245

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci246

import "std/c/datasci246" · use datasci246

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci247

import "std/c/datasci247" · use datasci247

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci248

import "std/c/datasci248" · use datasci248

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci249

import "std/c/datasci249" · use datasci249

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci250

import "std/c/datasci250" · use datasci250

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci251

import "std/c/datasci251" · use datasci251

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci252

import "std/c/datasci252" · use datasci252

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci253

import "std/c/datasci253" · use datasci253

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci254

import "std/c/datasci254" · use datasci254

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci255

import "std/c/datasci255" · use datasci255

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci256

import "std/c/datasci256" · use datasci256

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci257

import "std/c/datasci257" · use datasci257

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci258

import "std/c/datasci258" · use datasci258

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci259

import "std/c/datasci259" · use datasci259

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci260

import "std/c/datasci260" · use datasci260

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci261

import "std/c/datasci261" · use datasci261

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci262

import "std/c/datasci262" · use datasci262

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci263

import "std/c/datasci263" · use datasci263

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci264

import "std/c/datasci264" · use datasci264

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci265

import "std/c/datasci265" · use datasci265

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci266

import "std/c/datasci266" · use datasci266

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci267

import "std/c/datasci267" · use datasci267

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci268

import "std/c/datasci268" · use datasci268

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci269

import "std/c/datasci269" · use datasci269

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci270

import "std/c/datasci270" · use datasci270

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci271

import "std/c/datasci271" · use datasci271

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci272

import "std/c/datasci272" · use datasci272

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci273

import "std/c/datasci273" · use datasci273

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci274

import "std/c/datasci274" · use datasci274

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci275

import "std/c/datasci275" · use datasci275

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci276

import "std/c/datasci276" · use datasci276

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci277

import "std/c/datasci277" · use datasci277

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci278

import "std/c/datasci278" · use datasci278

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci279

import "std/c/datasci279" · use datasci279

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci280

import "std/c/datasci280" · use datasci280

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci281

import "std/c/datasci281" · use datasci281

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci282

import "std/c/datasci282" · use datasci282

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci283

import "std/c/datasci283" · use datasci283

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci284

import "std/c/datasci284" · use datasci284

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci285

import "std/c/datasci285" · use datasci285

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci286

import "std/c/datasci286" · use datasci286

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci287

import "std/c/datasci287" · use datasci287

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci288

import "std/c/datasci288" · use datasci288

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci289

import "std/c/datasci289" · use datasci289

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci290

import "std/c/datasci290" · use datasci290

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci291

import "std/c/datasci291" · use datasci291

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci292

import "std/c/datasci292" · use datasci292

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci293

import "std/c/datasci293" · use datasci293

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci294

import "std/c/datasci294" · use datasci294

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci295

import "std/c/datasci295" · use datasci295

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci296

import "std/c/datasci296" · use datasci296

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci297

import "std/c/datasci297" · use datasci297

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci298

import "std/c/datasci298" · use datasci298

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci299

import "std/c/datasci299" · use datasci299

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci300

import "std/c/datasci300" · use datasci300

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci301

import "std/c/datasci301" · use datasci301

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci302

import "std/c/datasci302" · use datasci302

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci303

import "std/c/datasci303" · use datasci303

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci304

import "std/c/datasci304" · use datasci304

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci305

import "std/c/datasci305" · use datasci305

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci306

import "std/c/datasci306" · use datasci306

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci307

import "std/c/datasci307" · use datasci307

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci308

import "std/c/datasci308" · use datasci308

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci309

import "std/c/datasci309" · use datasci309

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci310

import "std/c/datasci310" · use datasci310

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci311

import "std/c/datasci311" · use datasci311

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci312

import "std/c/datasci312" · use datasci312

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci313

import "std/c/datasci313" · use datasci313

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci314

import "std/c/datasci314" · use datasci314

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci315

import "std/c/datasci315" · use datasci315

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci316

import "std/c/datasci316" · use datasci316

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci317

import "std/c/datasci317" · use datasci317

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci318

import "std/c/datasci318" · use datasci318

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci319

import "std/c/datasci319" · use datasci319

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci320

import "std/c/datasci320" · use datasci320

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci321

import "std/c/datasci321" · use datasci321

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci322

import "std/c/datasci322" · use datasci322

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci323

import "std/c/datasci323" · use datasci323

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci324

import "std/c/datasci324" · use datasci324

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci325

import "std/c/datasci325" · use datasci325

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci326

import "std/c/datasci326" · use datasci326

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci327

import "std/c/datasci327" · use datasci327

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci328

import "std/c/datasci328" · use datasci328

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci329

import "std/c/datasci329" · use datasci329

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci330

import "std/c/datasci330" · use datasci330

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci331

import "std/c/datasci331" · use datasci331

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci332

import "std/c/datasci332" · use datasci332

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci333

import "std/c/datasci333" · use datasci333

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci334

import "std/c/datasci334" · use datasci334

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci335

import "std/c/datasci335" · use datasci335

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci336

import "std/c/datasci336" · use datasci336

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci337

import "std/c/datasci337" · use datasci337

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci338

import "std/c/datasci338" · use datasci338

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci339

import "std/c/datasci339" · use datasci339

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci340

import "std/c/datasci340" · use datasci340

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci341

import "std/c/datasci341" · use datasci341

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci342

import "std/c/datasci342" · use datasci342

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci343

import "std/c/datasci343" · use datasci343

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci344

import "std/c/datasci344" · use datasci344

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci345

import "std/c/datasci345" · use datasci345

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci346

import "std/c/datasci346" · use datasci346

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci347

import "std/c/datasci347" · use datasci347

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci348

import "std/c/datasci348" · use datasci348

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci349

import "std/c/datasci349" · use datasci349

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci350

import "std/c/datasci350" · use datasci350

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci351

import "std/c/datasci351" · use datasci351

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci352

import "std/c/datasci352" · use datasci352

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci353

import "std/c/datasci353" · use datasci353

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci354

import "std/c/datasci354" · use datasci354

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci355

import "std/c/datasci355" · use datasci355

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci356

import "std/c/datasci356" · use datasci356

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci357

import "std/c/datasci357" · use datasci357

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci358

import "std/c/datasci358" · use datasci358

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci359

import "std/c/datasci359" · use datasci359

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci360

import "std/c/datasci360" · use datasci360

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci361

import "std/c/datasci361" · use datasci361

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci362

import "std/c/datasci362" · use datasci362

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci363

import "std/c/datasci363" · use datasci363

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci364

import "std/c/datasci364" · use datasci364

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci365

import "std/c/datasci365" · use datasci365

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci366

import "std/c/datasci366" · use datasci366

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci367

import "std/c/datasci367" · use datasci367

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci368

import "std/c/datasci368" · use datasci368

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci369

import "std/c/datasci369" · use datasci369

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci370

import "std/c/datasci370" · use datasci370

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci371

import "std/c/datasci371" · use datasci371

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci372

import "std/c/datasci372" · use datasci372

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci373

import "std/c/datasci373" · use datasci373

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci374

import "std/c/datasci374" · use datasci374

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci375

import "std/c/datasci375" · use datasci375

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci376

import "std/c/datasci376" · use datasci376

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci377

import "std/c/datasci377" · use datasci377

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci378

import "std/c/datasci378" · use datasci378

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci379

import "std/c/datasci379" · use datasci379

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci380

import "std/c/datasci380" · use datasci380

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci381

import "std/c/datasci381" · use datasci381

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci382

import "std/c/datasci382" · use datasci382

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci383

import "std/c/datasci383" · use datasci383

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci384

import "std/c/datasci384" · use datasci384

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci385

import "std/c/datasci385" · use datasci385

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci386

import "std/c/datasci386" · use datasci386

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci387

import "std/c/datasci387" · use datasci387

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci388

import "std/c/datasci388" · use datasci388

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci389

import "std/c/datasci389" · use datasci389

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci390

import "std/c/datasci390" · use datasci390

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci391

import "std/c/datasci391" · use datasci391

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci392

import "std/c/datasci392" · use datasci392

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci393

import "std/c/datasci393" · use datasci393

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci394

import "std/c/datasci394" · use datasci394

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci395

import "std/c/datasci395" · use datasci395

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci396

import "std/c/datasci396" · use datasci396

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci397

import "std/c/datasci397" · use datasci397

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci398

import "std/c/datasci398" · use datasci398

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci399

import "std/c/datasci399" · use datasci399

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci400

import "std/c/datasci400" · use datasci400

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci401

import "std/c/datasci401" · use datasci401

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci402

import "std/c/datasci402" · use datasci402

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci403

import "std/c/datasci403" · use datasci403

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci404

import "std/c/datasci404" · use datasci404

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci405

import "std/c/datasci405" · use datasci405

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci406

import "std/c/datasci406" · use datasci406

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci407

import "std/c/datasci407" · use datasci407

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci408

import "std/c/datasci408" · use datasci408

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci409

import "std/c/datasci409" · use datasci409

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci410

import "std/c/datasci410" · use datasci410

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci411

import "std/c/datasci411" · use datasci411

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci412

import "std/c/datasci412" · use datasci412

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci413

import "std/c/datasci413" · use datasci413

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci414

import "std/c/datasci414" · use datasci414

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci415

import "std/c/datasci415" · use datasci415

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci416

import "std/c/datasci416" · use datasci416

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci417

import "std/c/datasci417" · use datasci417

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci418

import "std/c/datasci418" · use datasci418

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci419

import "std/c/datasci419" · use datasci419

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci420

import "std/c/datasci420" · use datasci420

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci421

import "std/c/datasci421" · use datasci421

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci422

import "std/c/datasci422" · use datasci422

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci423

import "std/c/datasci423" · use datasci423

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci424

import "std/c/datasci424" · use datasci424

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci425

import "std/c/datasci425" · use datasci425

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci426

import "std/c/datasci426" · use datasci426

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci427

import "std/c/datasci427" · use datasci427

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci428

import "std/c/datasci428" · use datasci428

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci429

import "std/c/datasci429" · use datasci429

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci430

import "std/c/datasci430" · use datasci430

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci431

import "std/c/datasci431" · use datasci431

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci432

import "std/c/datasci432" · use datasci432

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci433

import "std/c/datasci433" · use datasci433

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci434

import "std/c/datasci434" · use datasci434

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci435

import "std/c/datasci435" · use datasci435

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci436

import "std/c/datasci436" · use datasci436

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci437

import "std/c/datasci437" · use datasci437

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci438

import "std/c/datasci438" · use datasci438

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci439

import "std/c/datasci439" · use datasci439

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci440

import "std/c/datasci440" · use datasci440

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci441

import "std/c/datasci441" · use datasci441

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci442

import "std/c/datasci442" · use datasci442

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci443

import "std/c/datasci443" · use datasci443

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci444

import "std/c/datasci444" · use datasci444

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci445

import "std/c/datasci445" · use datasci445

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci446

import "std/c/datasci446" · use datasci446

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci447

import "std/c/datasci447" · use datasci447

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci448

import "std/c/datasci448" · use datasci448

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci449

import "std/c/datasci449" · use datasci449

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci450

import "std/c/datasci450" · use datasci450

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci451

import "std/c/datasci451" · use datasci451

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci452

import "std/c/datasci452" · use datasci452

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci453

import "std/c/datasci453" · use datasci453

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci454

import "std/c/datasci454" · use datasci454

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci455

import "std/c/datasci455" · use datasci455

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci456

import "std/c/datasci456" · use datasci456

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci457

import "std/c/datasci457" · use datasci457

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci458

import "std/c/datasci458" · use datasci458

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci459

import "std/c/datasci459" · use datasci459

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci460

import "std/c/datasci460" · use datasci460

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci461

import "std/c/datasci461" · use datasci461

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci462

import "std/c/datasci462" · use datasci462

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci463

import "std/c/datasci463" · use datasci463

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci464

import "std/c/datasci464" · use datasci464

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci465

import "std/c/datasci465" · use datasci465

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci466

import "std/c/datasci466" · use datasci466

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci467

import "std/c/datasci467" · use datasci467

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci468

import "std/c/datasci468" · use datasci468

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci469

import "std/c/datasci469" · use datasci469

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci470

import "std/c/datasci470" · use datasci470

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci471

import "std/c/datasci471" · use datasci471

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci472

import "std/c/datasci472" · use datasci472

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci473

import "std/c/datasci473" · use datasci473

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci474

import "std/c/datasci474" · use datasci474

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci475

import "std/c/datasci475" · use datasci475

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci476

import "std/c/datasci476" · use datasci476

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci477

import "std/c/datasci477" · use datasci477

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci478

import "std/c/datasci478" · use datasci478

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci479

import "std/c/datasci479" · use datasci479

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci480

import "std/c/datasci480" · use datasci480

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci481

import "std/c/datasci481" · use datasci481

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci482

import "std/c/datasci482" · use datasci482

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci483

import "std/c/datasci483" · use datasci483

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci484

import "std/c/datasci484" · use datasci484

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci485

import "std/c/datasci485" · use datasci485

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci486

import "std/c/datasci486" · use datasci486

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci487

import "std/c/datasci487" · use datasci487

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci488

import "std/c/datasci488" · use datasci488

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci489

import "std/c/datasci489" · use datasci489

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci490

import "std/c/datasci490" · use datasci490

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci491

import "std/c/datasci491" · use datasci491

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci492

import "std/c/datasci492" · use datasci492

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci493

import "std/c/datasci493" · use datasci493

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci494

import "std/c/datasci494" · use datasci494

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci495

import "std/c/datasci495" · use datasci495

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci496

import "std/c/datasci496" · use datasci496

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`

- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci497

import "std/c/datasci497" · use datasci497

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci498

import "std/c/datasci498" · use datasci498

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`

- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci499

import "std/c/datasci499" · use datasci499

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/datasci500

import "std/c/datasci500" · use datasci500

- `median(v list number) → number`
- `variance(v list number) → number`
- `iqr(v list number) → number`
- `zscores(v list number) → list number`
- `minMaxScale(v list number) → list number`
- `covariance(a list number, b list number) → number`
- `rSquared(xs list number, ys list number) → number`
- `rollingStd(v list number, window number) → list number`
- `ema(v list number, alpha number) → list number`
- `histogram(v list number, bins number) → list number`
- `gini(v list number) → number`
- `euclidean(a list number, b list number) → number`
- `rmse(a list number, b list number) → number`
- `mae(a list number, b list number) → number`
- `outlierCount(v list number, threshold number) → number`
- `log1p(v list number) → list number`
- `trainSplitAt(n number, ratio number) → number`

std/c/iaultra001

import "std/c/iaultra001" · use iaultra001

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra002

import "std/c/iaultra002" · use iaultra002

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra003

import "std/c/iaultra003" · use iaultra003

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra004

import "std/c/iaultra004" · use iaultra004

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra005

import "std/c/iaultra005" · use iaultra005

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra006

import "std/c/iaultra006" · use iaultra006

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra007

import "std/c/iaultra007" · use iaultra007

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra008

import "std/c/iaultra008" · use iaultra008

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra009

import "std/c/iaultra009" · use iaultra009

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra010

import "std/c/iaultra010" · use iaultra010

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra011

import "std/c/iaultra011" · use iaultra011

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra012

import "std/c/iaultra012" · use iaultra012

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra013

import "std/c/iaultra013" · use iaultra013

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra014

import "std/c/iaultra014" · use iaultra014

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra015

import "std/c/iaultra015" · use iaultra015

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra016

import "std/c/iaultra016" · use iaultra016

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra017

import "std/c/iaultra017" · use iaultra017

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra018

import "std/c/iaultra018" · use iaultra018

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra019

import "std/c/iaultra019" · use iaultra019

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra020

import "std/c/iaultra020" · use iaultra020

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra021

import "std/c/iaultra021" · use iaultra021

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra022

import "std/c/iaultra022" · use iaultra022

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra023

import "std/c/iaultra023" · use iaultra023

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra024

import "std/c/iaultra024" · use iaultra024

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra025

import "std/c/iaultra025" · use iaultra025

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra026

import "std/c/iaultra026" · use iaultra026

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra027

import "std/c/iaultra027" · use iaultra027

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra028

import "std/c/iaultra028" · use iaultra028

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra029

import "std/c/iaultra029" · use iaultra029

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra030

import "std/c/iaultra030" · use iaultra030

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra031

import "std/c/iaultra031" · use iaultra031

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra032

import "std/c/iaultra032" · use iaultra032

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra033

import "std/c/iaultra033" · use iaultra033

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra034

import "std/c/iaultra034" · use iaultra034

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra035

import "std/c/iaultra035" · use iaultra035

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra036

import "std/c/iaultra036" · use iaultra036

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra037

import "std/c/iaultra037" · use iaultra037

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra038

import "std/c/iaultra038" · use iaultra038

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra039

import "std/c/iaultra039" · use iaultra039

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra040

import "std/c/iaultra040" · use iaultra040

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra041

import "std/c/iaultra041" · use iaultra041

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra042

import "std/c/iaultra042" · use iaultra042

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra043

import "std/c/iaultra043" · use iaultra043

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra044

import "std/c/iaultra044" · use iaultra044

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra045

import "std/c/iaultra045" · use iaultra045

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra046

import "std/c/iaultra046" · use iaultra046

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra047

import "std/c/iaultra047" · use iaultra047

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra048

import "std/c/iaultra048" · use iaultra048

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra049

import "std/c/iaultra049" · use iaultra049

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra050

import "std/c/iaultra050" · use iaultra050

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra051

import "std/c/iaultra051" · use iaultra051

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra052

import "std/c/iaultra052" · use iaultra052

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra053

import "std/c/iaultra053" · use iaultra053

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra054

import "std/c/iaultra054" · use iaultra054

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra055

import "std/c/iaultra055" · use iaultra055

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra056

import "std/c/iaultra056" · use iaultra056

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra057

import "std/c/iaultra057" · use iaultra057

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra058

import "std/c/iaultra058" · use iaultra058

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra059

import "std/c/iaultra059" · use iaultra059

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra060

import "std/c/iaultra060" · use iaultra060

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra061

import "std/c/iaultra061" · use iaultra061

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra062

import "std/c/iaultra062" · use iaultra062

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra063

import "std/c/iaultra063" · use iaultra063

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra064

import "std/c/iaultra064" · use iaultra064

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra065

import "std/c/iaultra065" · use iaultra065

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra066

import "std/c/iaultra066" · use iaultra066

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra067

import "std/c/iaultra067" · use iaultra067

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra068

import "std/c/iaultra068" · use iaultra068

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra069

import "std/c/iaultra069" · use iaultra069

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra070

import "std/c/iaultra070" · use iaultra070

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra071

import "std/c/iaultra071" · use iaultra071

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra072

import "std/c/iaultra072" · use iaultra072

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra073

import "std/c/iaultra073" · use iaultra073

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra074

import "std/c/iaultra074" · use iaultra074

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra075

import "std/c/iaultra075" · use iaultra075

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra076

import "std/c/iaultra076" · use iaultra076

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra077

import "std/c/iaultra077" · use iaultra077

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra078

import "std/c/iaultra078" · use iaultra078

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra079

import "std/c/iaultra079" · use iaultra079

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra080

import "std/c/iaultra080" · use iaultra080

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra081

import "std/c/iaultra081" · use iaultra081

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra082

import "std/c/iaultra082" · use iaultra082

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra083

import "std/c/iaultra083" · use iaultra083

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra084

import "std/c/iaultra084" · use iaultra084

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra085

import "std/c/iaultra085" · use iaultra085

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra086

import "std/c/iaultra086" · use iaultra086

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra087

import "std/c/iaultra087" · use iaultra087

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra088

import "std/c/iaultra088" · use iaultra088

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra089

import "std/c/iaultra089" · use iaultra089

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra090

import "std/c/iaultra090" · use iaultra090

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra091

import "std/c/iaultra091" · use iaultra091

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra092

import "std/c/iaultra092" · use iaultra092

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra093

import "std/c/iaultra093" · use iaultra093

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra094

import "std/c/iaultra094" · use iaultra094

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra095

import "std/c/iaultra095" · use iaultra095

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra096

import "std/c/iaultra096" · use iaultra096

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra097

import "std/c/iaultra097" · use iaultra097

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra098

import "std/c/iaultra098" · use iaultra098

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra099

import "std/c/iaultra099" · use iaultra099

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra100

import "std/c/iaultra100" · use iaultra100

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra101

import "std/c/iaultra101" · use iaultra101

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra102

import "std/c/iaultra102" · use iaultra102

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra103

import "std/c/iaultra103" · use iaultra103

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra104

import "std/c/iaultra104" · use iaultra104

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra105

import "std/c/iaultra105" · use iaultra105

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra106

import "std/c/iaultra106" · use iaultra106

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra107

import "std/c/iaultra107" · use iaultra107

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra108

import "std/c/iaultra108" · use iaultra108

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra109

import "std/c/iaultra109" · use iaultra109

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra110

import "std/c/iaultra110" · use iaultra110

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra111

import "std/c/iaultra111" · use iaultra111

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra112

import "std/c/iaultra112" · use iaultra112

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra113

import "std/c/iaultra113" · use iaultra113

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra114

import "std/c/iaultra114" · use iaultra114

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra115

import "std/c/iaultra115" · use iaultra115

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra116

import "std/c/iaultra116" · use iaultra116

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra117

import "std/c/iaultra117" · use iaultra117

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra118

import "std/c/iaultra118" · use iaultra118

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra119

import "std/c/iaultra119" · use iaultra119

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra120

import "std/c/iaultra120" · use iaultra120

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra121

import "std/c/iaultra121" · use iaultra121

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra122

import "std/c/iaultra122" · use iaultra122

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra123

import "std/c/iaultra123" · use iaultra123

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra124

import "std/c/iaultra124" · use iaultra124

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra125

import "std/c/iaultra125" · use iaultra125

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra126

import "std/c/iaultra126" · use iaultra126

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra127

import "std/c/iaultra127" · use iaultra127

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra128

import "std/c/iaultra128" · use iaultra128

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra129

import "std/c/iaultra129" · use iaultra129

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra130

import "std/c/iaultra130" · use iaultra130

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra131

import "std/c/iaultra131" · use iaultra131

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra132

import "std/c/iaultra132" · use iaultra132

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra133

import "std/c/iaultra133" · use iaultra133

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra134

import "std/c/iaultra134" · use iaultra134

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra135

import "std/c/iaultra135" · use iaultra135

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra136

import "std/c/iaultra136" · use iaultra136

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra137

import "std/c/iaultra137" · use iaultra137

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra138

import "std/c/iaultra138" · use iaultra138

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra139

import "std/c/iaultra139" · use iaultra139

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra140

import "std/c/iaultra140" · use iaultra140

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra141

import "std/c/iaultra141" · use iaultra141

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra142

import "std/c/iaultra142" · use iaultra142

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra143

import "std/c/iaultra143" · use iaultra143

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra144

import "std/c/iaultra144" · use iaultra144

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra145

import "std/c/iaultra145" · use iaultra145

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra146

import "std/c/iaultra146" · use iaultra146

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra147

import "std/c/iaultra147" · use iaultra147

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra148

import "std/c/iaultra148" · use iaultra148

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra149

import "std/c/iaultra149" · use iaultra149

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra150

import "std/c/iaultra150" · use iaultra150

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra151

import "std/c/iaultra151" · use iaultra151

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra152

import "std/c/iaultra152" · use iaultra152

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra153

import "std/c/iaultra153" · use iaultra153

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra154

import "std/c/iaultra154" · use iaultra154

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra155

import "std/c/iaultra155" · use iaultra155

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra156

import "std/c/iaultra156" · use iaultra156

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra157

import "std/c/iaultra157" · use iaultra157

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra158

import "std/c/iaultra158" · use iaultra158

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra159

import "std/c/iaultra159" · use iaultra159

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra160

import "std/c/iaultra160" · use iaultra160

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra161

import "std/c/iaultra161" · use iaultra161

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra162

import "std/c/iaultra162" · use iaultra162

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra163

import "std/c/iaultra163" · use iaultra163

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra164

import "std/c/iaultra164" · use iaultra164

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra165

import "std/c/iaultra165" · use iaultra165

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra166

import "std/c/iaultra166" · use iaultra166

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra167

import "std/c/iaultra167" · use iaultra167

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra168

import "std/c/iaultra168" · use iaultra168

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra169

import "std/c/iaultra169" · use iaultra169

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra170

import "std/c/iaultra170" · use iaultra170

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra171

import "std/c/iaultra171" · use iaultra171

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra172

import "std/c/iaultra172" · use iaultra172

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra173

import "std/c/iaultra173" · use iaultra173

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra174

import "std/c/iaultra174" · use iaultra174

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra175

import "std/c/iaultra175" · use iaultra175

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra176

import "std/c/iaultra176" · use iaultra176

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra177

import "std/c/iaultra177" · use iaultra177

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra178

import "std/c/iaultra178" · use iaultra178

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra179

import "std/c/iaultra179" · use iaultra179

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra180

import "std/c/iaultra180" · use iaultra180

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra181

import "std/c/iaultra181" · use iaultra181

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra182

import "std/c/iaultra182" · use iaultra182

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra183

import "std/c/iaultra183" · use iaultra183

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra184

import "std/c/iaultra184" · use iaultra184

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra185

import "std/c/iaultra185" · use iaultra185

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra186

import "std/c/iaultra186" · use iaultra186

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra187

import "std/c/iaultra187" · use iaultra187

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra188

import "std/c/iaultra188" · use iaultra188

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra189

import "std/c/iaultra189" · use iaultra189

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra190

import "std/c/iaultra190" · use iaultra190

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra191

import "std/c/iaultra191" · use iaultra191

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra192

import "std/c/iaultra192" · use iaultra192

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra193

import "std/c/iaultra193" · use iaultra193

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra194

import "std/c/iaultra194" · use iaultra194

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra195

import "std/c/iaultra195" · use iaultra195

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra196

import "std/c/iaultra196" · use iaultra196

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra197

import "std/c/iaultra197" · use iaultra197

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra198

import "std/c/iaultra198" · use iaultra198

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra199

import "std/c/iaultra199" · use iaultra199

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra200

import "std/c/iaultra200" · use iaultra200

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra201

import "std/c/iaultra201" · use iaultra201

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra202

import "std/c/iaultra202" · use iaultra202

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra203

import "std/c/iaultra203" · use iaultra203

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra204

import "std/c/iaultra204" · use iaultra204

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra205

import "std/c/iaultra205" · use iaultra205

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra206

import "std/c/iaultra206" · use iaultra206

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra207

import "std/c/iaultra207" · use iaultra207

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra208

import "std/c/iaultra208" · use iaultra208

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra209

import "std/c/iaultra209" · use iaultra209

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra210

import "std/c/iaultra210" · use iaultra210

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra211

import "std/c/iaultra211" · use iaultra211

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra212

import "std/c/iaultra212" · use iaultra212

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra213

import "std/c/iaultra213" · use iaultra213

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra214

import "std/c/iaultra214" · use iaultra214

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra215

import "std/c/iaultra215" · use iaultra215

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra216

import "std/c/iaultra216" · use iaultra216

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra217

import "std/c/iaultra217" · use iaultra217

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra218

import "std/c/iaultra218" · use iaultra218

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra219

import "std/c/iaultra219" · use iaultra219

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra220

import "std/c/iaultra220" · use iaultra220

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra221

import "std/c/iaultra221" · use iaultra221

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra222

import "std/c/iaultra222" · use iaultra222

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra223

import "std/c/iaultra223" · use iaultra223

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra224

import "std/c/iaultra224" · use iaultra224

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra225

import "std/c/iaultra225" · use iaultra225

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra226

import "std/c/iaultra226" · use iaultra226

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra227

import "std/c/iaultra227" · use iaultra227

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra228

import "std/c/iaultra228" · use iaultra228

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra229

import "std/c/iaultra229" · use iaultra229

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra230

import "std/c/iaultra230" · use iaultra230

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra231

import "std/c/iaultra231" · use iaultra231

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra232

import "std/c/iaultra232" · use iaultra232

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra233

import "std/c/iaultra233" · use iaultra233

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra234

import "std/c/iaultra234" · use iaultra234

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra235

import "std/c/iaultra235" · use iaultra235

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra236

import "std/c/iaultra236" · use iaultra236

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra237

import "std/c/iaultra237" · use iaultra237

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra238

import "std/c/iaultra238" · use iaultra238

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra239

import "std/c/iaultra239" · use iaultra239

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra240

import "std/c/iaultra240" · use iaultra240

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra241

import "std/c/iaultra241" · use iaultra241

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra242

import "std/c/iaultra242" · use iaultra242

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra243

import "std/c/iaultra243" · use iaultra243

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra244

import "std/c/iaultra244" · use iaultra244

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra245

import "std/c/iaultra245" · use iaultra245

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra246

import "std/c/iaultra246" · use iaultra246

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra247

import "std/c/iaultra247" · use iaultra247

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra248

import "std/c/iaultra248" · use iaultra248

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra249

import "std/c/iaultra249" · use iaultra249

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra250

import "std/c/iaultra250" · use iaultra250

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra251

import "std/c/iaultra251" · use iaultra251

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra252

import "std/c/iaultra252" · use iaultra252

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra253

import "std/c/iaultra253" · use iaultra253

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra254

import "std/c/iaultra254" · use iaultra254

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra255

import "std/c/iaultra255" · use iaultra255

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra256

import "std/c/iaultra256" · use iaultra256

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra257

import "std/c/iaultra257" · use iaultra257

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra258

import "std/c/iaultra258" · use iaultra258

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra259

import "std/c/iaultra259" · use iaultra259

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra260

import "std/c/iaultra260" · use iaultra260

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra261

import "std/c/iaultra261" · use iaultra261

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra262

import "std/c/iaultra262" · use iaultra262

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra263

import "std/c/iaultra263" · use iaultra263

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra264

import "std/c/iaultra264" · use iaultra264

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra265

import "std/c/iaultra265" · use iaultra265

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra266

import "std/c/iaultra266" · use iaultra266

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra267

import "std/c/iaultra267" · use iaultra267

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra268

import "std/c/iaultra268" · use iaultra268

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra269

import "std/c/iaultra269" · use iaultra269

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra270

import "std/c/iaultra270" · use iaultra270

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra271

import "std/c/iaultra271" · use iaultra271

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra272

import "std/c/iaultra272" · use iaultra272

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra273

import "std/c/iaultra273" · use iaultra273

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra274

import "std/c/iaultra274" · use iaultra274

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra275

import "std/c/iaultra275" · use iaultra275

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra276

import "std/c/iaultra276" · use iaultra276

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra277

import "std/c/iaultra277" · use iaultra277

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra278

import "std/c/iaultra278" · use iaultra278

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra279

import "std/c/iaultra279" · use iaultra279

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra280

import "std/c/iaultra280" · use iaultra280

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra281

import "std/c/iaultra281" · use iaultra281

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra282

import "std/c/iaultra282" · use iaultra282

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra283

import "std/c/iaultra283" · use iaultra283

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra284

import "std/c/iaultra284" · use iaultra284

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra285

import "std/c/iaultra285" · use iaultra285

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra286

import "std/c/iaultra286" · use iaultra286

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra287

import "std/c/iaultra287" · use iaultra287

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra288

import "std/c/iaultra288" · use iaultra288

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra289

import "std/c/iaultra289" · use iaultra289

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra290

import "std/c/iaultra290" · use iaultra290

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra291

import "std/c/iaultra291" · use iaultra291

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra292

import "std/c/iaultra292" · use iaultra292

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra293

import "std/c/iaultra293" · use iaultra293

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra294

import "std/c/iaultra294" · use iaultra294

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra295

import "std/c/iaultra295" · use iaultra295

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra296

import "std/c/iaultra296" · use iaultra296

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra297

import "std/c/iaultra297" · use iaultra297

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra298

import "std/c/iaultra298" · use iaultra298

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra299

import "std/c/iaultra299" · use iaultra299

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra300

import "std/c/iaultra300" · use iaultra300

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra301

import "std/c/iaultra301" · use iaultra301

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra302

import "std/c/iaultra302" · use iaultra302

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra303

import "std/c/iaultra303" · use iaultra303

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra304

import "std/c/iaultra304" · use iaultra304

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra305

import "std/c/iaultra305" · use iaultra305

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra306

import "std/c/iaultra306" · use iaultra306

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra307

import "std/c/iaultra307" · use iaultra307

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra308

import "std/c/iaultra308" · use iaultra308

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra309

import "std/c/iaultra309" · use iaultra309

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra310

import "std/c/iaultra310" · use iaultra310

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra311

import "std/c/iaultra311" · use iaultra311

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra312

import "std/c/iaultra312" · use iaultra312

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra313

import "std/c/iaultra313" · use iaultra313

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra314

import "std/c/iaultra314" · use iaultra314

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra315

import "std/c/iaultra315" · use iaultra315

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra316

import "std/c/iaultra316" · use iaultra316

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra317

import "std/c/iaultra317" · use iaultra317

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra318

import "std/c/iaultra318" · use iaultra318

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra319

import "std/c/iaultra319" · use iaultra319

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra320

import "std/c/iaultra320" · use iaultra320

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra321

import "std/c/iaultra321" · use iaultra321

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra322

import "std/c/iaultra322" · use iaultra322

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra323

import "std/c/iaultra323" · use iaultra323

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra324

import "std/c/iaultra324" · use iaultra324

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra325

import "std/c/iaultra325" · use iaultra325

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra326

import "std/c/iaultra326" · use iaultra326

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra327

import "std/c/iaultra327" · use iaultra327

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra328

import "std/c/iaultra328" · use iaultra328

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra329

import "std/c/iaultra329" · use iaultra329

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra330

import "std/c/iaultra330" · use iaultra330

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra331

import "std/c/iaultra331" · use iaultra331

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra332

import "std/c/iaultra332" · use iaultra332

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra333

import "std/c/iaultra333" · use iaultra333

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra334

import "std/c/iaultra334" · use iaultra334

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra335

import "std/c/iaultra335" · use iaultra335

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra336

import "std/c/iaultra336" · use iaultra336

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra337

import "std/c/iaultra337" · use iaultra337

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra338

import "std/c/iaultra338" · use iaultra338

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra339

import "std/c/iaultra339" · use iaultra339

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra340

import "std/c/iaultra340" · use iaultra340

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra341

import "std/c/iaultra341" · use iaultra341

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra342

import "std/c/iaultra342" · use iaultra342

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra343

import "std/c/iaultra343" · use iaultra343

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra344

import "std/c/iaultra344" · use iaultra344

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra345

import "std/c/iaultra345" · use iaultra345

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra346

import "std/c/iaultra346" · use iaultra346

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra347

import "std/c/iaultra347" · use iaultra347

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra348

import "std/c/iaultra348" · use iaultra348

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra349

import "std/c/iaultra349" · use iaultra349

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra350

import "std/c/iaultra350" · use iaultra350

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra351

import "std/c/iaultra351" · use iaultra351

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra352

import "std/c/iaultra352" · use iaultra352

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra353

import "std/c/iaultra353" · use iaultra353

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra354

import "std/c/iaultra354" · use iaultra354

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra355

import "std/c/iaultra355" · use iaultra355

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra356

import "std/c/iaultra356" · use iaultra356

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra357

import "std/c/iaultra357" · use iaultra357

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra358

import "std/c/iaultra358" · use iaultra358

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra359

import "std/c/iaultra359" · use iaultra359

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra360

import "std/c/iaultra360" · use iaultra360

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra361

import "std/c/iaultra361" · use iaultra361

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra362

import "std/c/iaultra362" · use iaultra362

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra363

import "std/c/iaultra363" · use iaultra363

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra364

import "std/c/iaultra364" · use iaultra364

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra365

import "std/c/iaultra365" · use iaultra365

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra366

import "std/c/iaultra366" · use iaultra366

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra367

import "std/c/iaultra367" · use iaultra367

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra368

import "std/c/iaultra368" · use iaultra368

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra369

import "std/c/iaultra369" · use iaultra369

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra370

import "std/c/iaultra370" · use iaultra370

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra371

import "std/c/iaultra371" · use iaultra371

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra372

import "std/c/iaultra372" · use iaultra372

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra373

import "std/c/iaultra373" · use iaultra373

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra374

import "std/c/iaultra374" · use iaultra374

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra375

import "std/c/iaultra375" · use iaultra375

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra376

import "std/c/iaultra376" · use iaultra376

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra377

import "std/c/iaultra377" · use iaultra377

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra378

import "std/c/iaultra378" · use iaultra378

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra379

import "std/c/iaultra379" · use iaultra379

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra380

import "std/c/iaultra380" · use iaultra380

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra381

import "std/c/iaultra381" · use iaultra381

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra382

import "std/c/iaultra382" · use iaultra382

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra383

import "std/c/iaultra383" · use iaultra383

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra384

import "std/c/iaultra384" · use iaultra384

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra385

import "std/c/iaultra385" · use iaultra385

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra386

import "std/c/iaultra386" · use iaultra386

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra387

import "std/c/iaultra387" · use iaultra387

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra388

import "std/c/iaultra388" · use iaultra388

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra389

import "std/c/iaultra389" · use iaultra389

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra390

import "std/c/iaultra390" · use iaultra390

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra391

import "std/c/iaultra391" · use iaultra391

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra392

import "std/c/iaultra392" · use iaultra392

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra393

import "std/c/iaultra393" · use iaultra393

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra394

import "std/c/iaultra394" · use iaultra394

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra395

import "std/c/iaultra395" · use iaultra395

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra396

import "std/c/iaultra396" · use iaultra396

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra397

import "std/c/iaultra397" · use iaultra397

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra398

import "std/c/iaultra398" · use iaultra398

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra399

import "std/c/iaultra399" · use iaultra399

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra400

import "std/c/iaultra400" · use iaultra400

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra401

import "std/c/iaultra401" · use iaultra401

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra402

import "std/c/iaultra402" · use iaultra402

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra403

import "std/c/iaultra403" · use iaultra403

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra404

import "std/c/iaultra404" · use iaultra404

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra405

import "std/c/iaultra405" · use iaultra405

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra406

import "std/c/iaultra406" · use iaultra406

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra407

import "std/c/iaultra407" · use iaultra407

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra408

import "std/c/iaultra408" · use iaultra408

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra409

import "std/c/iaultra409" · use iaultra409

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra410

import "std/c/iaultra410" · use iaultra410

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra411

import "std/c/iaultra411" · use iaultra411

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra412

import "std/c/iaultra412" · use iaultra412

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra413

import "std/c/iaultra413" · use iaultra413

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra414

import "std/c/iaultra414" · use iaultra414

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra415

import "std/c/iaultra415" · use iaultra415

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra416

import "std/c/iaultra416" · use iaultra416

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra417

import "std/c/iaultra417" · use iaultra417

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra418

import "std/c/iaultra418" · use iaultra418

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra419

import "std/c/iaultra419" · use iaultra419

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra420

import "std/c/iaultra420" · use iaultra420

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra421

import "std/c/iaultra421" · use iaultra421

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra422

import "std/c/iaultra422" · use iaultra422

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra423

import "std/c/iaultra423" · use iaultra423

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra424

import "std/c/iaultra424" · use iaultra424

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra425

import "std/c/iaultra425" · use iaultra425

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra426

import "std/c/iaultra426" · use iaultra426

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra427

import "std/c/iaultra427" · use iaultra427

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra428

import "std/c/iaultra428" · use iaultra428

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra429

import "std/c/iaultra429" · use iaultra429

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra430

import "std/c/iaultra430" · use iaultra430

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra431

import "std/c/iaultra431" · use iaultra431

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra432

import "std/c/iaultra432" · use iaultra432

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra433

import "std/c/iaultra433" · use iaultra433

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra434

import "std/c/iaultra434" · use iaultra434

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra435

import "std/c/iaultra435" · use iaultra435

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra436

import "std/c/iaultra436" · use iaultra436

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra437

import "std/c/iaultra437" · use iaultra437

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra438

import "std/c/iaultra438" · use iaultra438

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra439

import "std/c/iaultra439" · use iaultra439

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra440

import "std/c/iaultra440" · use iaultra440

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra441

import "std/c/iaultra441" · use iaultra441

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra442

import "std/c/iaultra442" · use iaultra442

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra443

import "std/c/iaultra443" · use iaultra443

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra444

import "std/c/iaultra444" · use iaultra444

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra445

import "std/c/iaultra445" · use iaultra445

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra446

import "std/c/iaultra446" · use iaultra446

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra447

import "std/c/iaultra447" · use iaultra447

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra448

import "std/c/iaultra448" · use iaultra448

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra449

import "std/c/iaultra449" · use iaultra449

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra450

import "std/c/iaultra450" · use iaultra450

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra451

import "std/c/iaultra451" · use iaultra451

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra452

import "std/c/iaultra452" · use iaultra452

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra453

import "std/c/iaultra453" · use iaultra453

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra454

import "std/c/iaultra454" · use iaultra454

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra455

import "std/c/iaultra455" · use iaultra455

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra456

import "std/c/iaultra456" · use iaultra456

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra457

import "std/c/iaultra457" · use iaultra457

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra458

import "std/c/iaultra458" · use iaultra458

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra459

import "std/c/iaultra459" · use iaultra459

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra460

import "std/c/iaultra460" · use iaultra460

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra461

import "std/c/iaultra461" · use iaultra461

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra462

import "std/c/iaultra462" · use iaultra462

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra463

import "std/c/iaultra463" · use iaultra463

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra464

import "std/c/iaultra464" · use iaultra464

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra465

import "std/c/iaultra465" · use iaultra465

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra466

import "std/c/iaultra466" · use iaultra466

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra467

import "std/c/iaultra467" · use iaultra467

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra468

import "std/c/iaultra468" · use iaultra468

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra469

import "std/c/iaultra469" · use iaultra469

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra470

import "std/c/iaultra470" · use iaultra470

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra471

import "std/c/iaultra471" · use iaultra471

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra472

import "std/c/iaultra472" · use iaultra472

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra473

import "std/c/iaultra473" · use iaultra473

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra474

import "std/c/iaultra474" · use iaultra474

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra475

import "std/c/iaultra475" · use iaultra475

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra476

import "std/c/iaultra476" · use iaultra476

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra477

import "std/c/iaultra477" · use iaultra477

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra478

import "std/c/iaultra478" · use iaultra478

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra479

import "std/c/iaultra479" · use iaultra479

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra480

import "std/c/iaultra480" · use iaultra480

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra481

import "std/c/iaultra481" · use iaultra481

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra482

import "std/c/iaultra482" · use iaultra482

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra483

import "std/c/iaultra483" · use iaultra483

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra484

import "std/c/iaultra484" · use iaultra484

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra485

import "std/c/iaultra485" · use iaultra485

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra486

import "std/c/iaultra486" · use iaultra486

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra487

import "std/c/iaultra487" · use iaultra487

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra488

import "std/c/iaultra488" · use iaultra488

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra489

import "std/c/iaultra489" · use iaultra489

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra490

import "std/c/iaultra490" · use iaultra490

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra491

import "std/c/iaultra491" · use iaultra491

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra492

import "std/c/iaultra492" · use iaultra492

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra493

import "std/c/iaultra493" · use iaultra493

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra494

import "std/c/iaultra494" · use iaultra494

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra495

import "std/c/iaultra495" · use iaultra495

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra496

import "std/c/iaultra496" · use iaultra496

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra497

import "std/c/iaultra497" · use iaultra497

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`

- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra498

import "std/c/iaultra498" · use iaultra498

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra499

import "std/c/iaultra499" · use iaultra499

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`

- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/iaultra500

import "std/c/iaultra500" · use iaultra500

- `sigmoid(x number) → number`
- `relu(x number) → number`
- `leakyRelu(x number) → number`
- `tanh(x number) → number`
- `softmax(logits list number) → list number`
- `argmax(v list number) → number`
- `dot(a list number, b list number) → number`
- `linearPredict(weights list number, features list number, bias number) → number`
- `cosineSim(a list number, b list number) → number`
- `entropy(probs list number) → number`
- `crossEntropy(probs list number, target number) → number`
- `oneHot(idx number, size number) → list number`
- `l2norm(v list number) → number`
- `normalizeVec(v list number) → list number`
- `mseLists(pred list number, actual list number) → number`
- `accuracy(ytrue list number, ypred list number) → number`
- `precisionScore(ytrue list number, ypred list number) → number`
- `recallScore(ytrue list number, ypred list number) → number`
- `f1Score(ytrue list number, ypred list number) → number`
- `sgdStep(weights list number, features list number, target number, lr number) → list number`
- `nearestIdx(query list number, matrix list number, dims number) → number`
- `assignClusters(points list number, centroids list number, dims number) → list number`
- `clipVec(v list number, limit number) → list number`
- `perceptronStep(weights list number, bias number, features list number, label number, lr number) → list number`
- `defaultK(k number) → number`

std/c/dbultra001

import "std/c/dbultra001" · use dbultra001

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra002

import "std/c/dbultra002" · use dbultra002

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra003

import "std/c/dbultra003" · use dbultra003

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra004

import "std/c/dbultra004" · use dbultra004

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra005

import "std/c/dbultra005" · use dbultra005

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra006

import "std/c/dbultra006" · use dbultra006

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra007

import "std/c/dbultra007" · use dbultra007

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra008

import "std/c/dbultra008" · use dbultra008

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra009

import "std/c/dbultra009" · use dbultra009

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra010

import "std/c/dbultra010" · use dbultra010

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra011

import "std/c/dbultra011" · use dbultra011

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra012

import "std/c/dbultra012" · use dbultra012

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra013

import "std/c/dbultra013" · use dbultra013

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra014

import "std/c/dbultra014" · use dbultra014

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra015

import "std/c/dbultra015" · use dbultra015

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra016

import "std/c/dbultra016" · use dbultra016

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra017

import "std/c/dbultra017" · use dbultra017

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra018

import "std/c/dbultra018" · use dbultra018

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra019

import "std/c/dbultra019" · use dbultra019

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra020

import "std/c/dbultra020" · use dbultra020

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra021

import "std/c/dbultra021" · use dbultra021

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra022

import "std/c/dbultra022" · use dbultra022

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra023

import "std/c/dbultra023" · use dbultra023

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra024

import "std/c/dbultra024" · use dbultra024

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra025

import "std/c/dbultra025" · use dbultra025

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra026

import "std/c/dbultra026" · use dbultra026

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra027

import "std/c/dbultra027" · use dbultra027

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra028

import "std/c/dbultra028" · use dbultra028

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra029

import "std/c/dbultra029" · use dbultra029

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra030

import "std/c/dbultra030" · use dbultra030

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra031

import "std/c/dbultra031" · use dbultra031

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra032

import "std/c/dbultra032" · use dbultra032

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra033

import "std/c/dbultra033" · use dbultra033

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra034

import "std/c/dbultra034" · use dbultra034

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra035

import "std/c/dbultra035" · use dbultra035

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra036

import "std/c/dbultra036" · use dbultra036

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra037

import "std/c/dbultra037" · use dbultra037

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra038

import "std/c/dbultra038" · use dbultra038

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra039

import "std/c/dbultra039" · use dbultra039

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra040

import "std/c/dbultra040" · use dbultra040

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra041

import "std/c/dbultra041" · use dbultra041

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra042

import "std/c/dbultra042" · use dbultra042

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra043

import "std/c/dbultra043" · use dbultra043

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra044

import "std/c/dbultra044" · use dbultra044

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra045

import "std/c/dbultra045" · use dbultra045

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra046

import "std/c/dbultra046" · use dbultra046

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra047

import "std/c/dbultra047" · use dbultra047

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra048

import "std/c/dbultra048" · use dbultra048

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra049

import "std/c/dbultra049" · use dbultra049

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra050

import "std/c/dbultra050" · use dbultra050

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra051

import "std/c/dbultra051" · use dbultra051

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra052

import "std/c/dbultra052" · use dbultra052

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra053

import "std/c/dbultra053" · use dbultra053

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra054

import "std/c/dbultra054" · use dbultra054

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra055

import "std/c/dbultra055" · use dbultra055

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra056

import "std/c/dbultra056" · use dbultra056

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra057

import "std/c/dbultra057" · use dbultra057

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra058

import "std/c/dbultra058" · use dbultra058

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra059

import "std/c/dbultra059" · use dbultra059

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra060

import "std/c/dbultra060" · use dbultra060

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra061

import "std/c/dbultra061" · use dbultra061

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra062

import "std/c/dbultra062" · use dbultra062

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra063

import "std/c/dbultra063" · use dbultra063

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra064

import "std/c/dbultra064" · use dbultra064

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra065

import "std/c/dbultra065" · use dbultra065

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra066

import "std/c/dbultra066" · use dbultra066

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra067

import "std/c/dbultra067" · use dbultra067

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra068

import "std/c/dbultra068" · use dbultra068

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra069

import "std/c/dbultra069" · use dbultra069

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra070

import "std/c/dbultra070" · use dbultra070

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra071

import "std/c/dbultra071" · use dbultra071

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra072

import "std/c/dbultra072" · use dbultra072

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra073

import "std/c/dbultra073" · use dbultra073

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra074

import "std/c/dbultra074" · use dbultra074

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra075

import "std/c/dbultra075" · use dbultra075

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra076

import "std/c/dbultra076" · use dbultra076

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra077

import "std/c/dbultra077" · use dbultra077

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra078

import "std/c/dbultra078" · use dbultra078

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra079

import "std/c/dbultra079" · use dbultra079

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra080

import "std/c/dbultra080" · use dbultra080

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra081

import "std/c/dbultra081" · use dbultra081

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra082

import "std/c/dbultra082" · use dbultra082

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra083

import "std/c/dbultra083" · use dbultra083

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra084

import "std/c/dbultra084" · use dbultra084

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra085

import "std/c/dbultra085" · use dbultra085

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra086

import "std/c/dbultra086" · use dbultra086

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra087

import "std/c/dbultra087" · use dbultra087

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra088

import "std/c/dbultra088" · use dbultra088

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra089

import "std/c/dbultra089" · use dbultra089

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra090

import "std/c/dbultra090" · use dbultra090

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra091

import "std/c/dbultra091" · use dbultra091

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra092

import "std/c/dbultra092" · use dbultra092

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra093

import "std/c/dbultra093" · use dbultra093

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra094

import "std/c/dbultra094" · use dbultra094

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra095

import "std/c/dbultra095" · use dbultra095

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra096

import "std/c/dbultra096" · use dbultra096

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra097

import "std/c/dbultra097" · use dbultra097

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra098

import "std/c/dbultra098" · use dbultra098

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra099

import "std/c/dbultra099" · use dbultra099

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra100

import "std/c/dbultra100" · use dbultra100

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra101

import "std/c/dbultra101" · use dbultra101

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra102

import "std/c/dbultra102" · use dbultra102

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra103

import "std/c/dbultra103" · use dbultra103

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra104

import "std/c/dbultra104" · use dbultra104

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra105

import "std/c/dbultra105" · use dbultra105

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra106

import "std/c/dbultra106" · use dbultra106

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra107

import "std/c/dbultra107" · use dbultra107

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra108

import "std/c/dbultra108" · use dbultra108

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra109

import "std/c/dbultra109" · use dbultra109

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra110

import "std/c/dbultra110" · use dbultra110

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra111

import "std/c/dbultra111" · use dbultra111

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra112

import "std/c/dbultra112" · use dbultra112

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra113

import "std/c/dbultra113" · use dbultra113

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra114

import "std/c/dbultra114" · use dbultra114

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra115

import "std/c/dbultra115" · use dbultra115

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra116

import "std/c/dbultra116" · use dbultra116

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra117

import "std/c/dbultra117" · use dbultra117

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra118

import "std/c/dbultra118" · use dbultra118

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra119

import "std/c/dbultra119" · use dbultra119

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra120

import "std/c/dbultra120" · use dbultra120

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra121

import "std/c/dbultra121" · use dbultra121

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra122

import "std/c/dbultra122" · use dbultra122

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra123

import "std/c/dbultra123" · use dbultra123

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra124

import "std/c/dbultra124" · use dbultra124

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra125

import "std/c/dbultra125" · use dbultra125

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra126

import "std/c/dbultra126" · use dbultra126

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra127

import "std/c/dbultra127" · use dbultra127

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra128

import "std/c/dbultra128" · use dbultra128

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra129

import "std/c/dbultra129" · use dbultra129

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra130

import "std/c/dbultra130" · use dbultra130

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra131

import "std/c/dbultra131" · use dbultra131

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra132

import "std/c/dbultra132" · use dbultra132

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra133

import "std/c/dbultra133" · use dbultra133

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra134

import "std/c/dbultra134" · use dbultra134

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra135

import "std/c/dbultra135" · use dbultra135

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra136

import "std/c/dbultra136" · use dbultra136

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra137

import "std/c/dbultra137" · use dbultra137

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra138

import "std/c/dbultra138" · use dbultra138

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra139

import "std/c/dbultra139" · use dbultra139

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra140

import "std/c/dbultra140" · use dbultra140

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra141

import "std/c/dbultra141" · use dbultra141

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra142

import "std/c/dbultra142" · use dbultra142

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra143

import "std/c/dbultra143" · use dbultra143

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra144

import "std/c/dbultra144" · use dbultra144

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra145

import "std/c/dbultra145" · use dbultra145

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra146

import "std/c/dbultra146" · use dbultra146

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra147

import "std/c/dbultra147" · use dbultra147

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra148

import "std/c/dbultra148" · use dbultra148

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra149

import "std/c/dbultra149" · use dbultra149

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra150

import "std/c/dbultra150" · use dbultra150

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra151

import "std/c/dbultra151" · use dbultra151

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra152

import "std/c/dbultra152" · use dbultra152

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra153

import "std/c/dbultra153" · use dbultra153

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra154

import "std/c/dbultra154" · use dbultra154

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra155

import "std/c/dbultra155" · use dbultra155

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra156

import "std/c/dbultra156" · use dbultra156

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra157

import "std/c/dbultra157" · use dbultra157

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra158

import "std/c/dbultra158" · use dbultra158

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra159

import "std/c/dbultra159" · use dbultra159

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra160

import "std/c/dbultra160" · use dbultra160

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra161

import "std/c/dbultra161" · use dbultra161

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra162

import "std/c/dbultra162" · use dbultra162

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra163

import "std/c/dbultra163" · use dbultra163

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra164

import "std/c/dbultra164" · use dbultra164

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra165

import "std/c/dbultra165" · use dbultra165

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra166

import "std/c/dbultra166" · use dbultra166

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra167

import "std/c/dbultra167" · use dbultra167

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra168

import "std/c/dbultra168" · use dbultra168

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra169

import "std/c/dbultra169" · use dbultra169

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra170

import "std/c/dbultra170" · use dbultra170

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra171

import "std/c/dbultra171" · use dbultra171

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra172

import "std/c/dbultra172" · use dbultra172

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra173

import "std/c/dbultra173" · use dbultra173

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra174

import "std/c/dbultra174" · use dbultra174

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra175

import "std/c/dbultra175" · use dbultra175

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra176

import "std/c/dbultra176" · use dbultra176

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra177

import "std/c/dbultra177" · use dbultra177

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra178

import "std/c/dbultra178" · use dbultra178

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra179

import "std/c/dbultra179" · use dbultra179

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra180

import "std/c/dbultra180" · use dbultra180

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra181

import "std/c/dbultra181" · use dbultra181

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra182

import "std/c/dbultra182" · use dbultra182

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra183

import "std/c/dbultra183" · use dbultra183

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra184

import "std/c/dbultra184" · use dbultra184

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra185

import "std/c/dbultra185" · use dbultra185

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra186

import "std/c/dbultra186" · use dbultra186

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra187

import "std/c/dbultra187" · use dbultra187

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra188

import "std/c/dbultra188" · use dbultra188

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra189

import "std/c/dbultra189" · use dbultra189

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra190

import "std/c/dbultra190" · use dbultra190

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra191

import "std/c/dbultra191" · use dbultra191

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra192

import "std/c/dbultra192" · use dbultra192

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra193

import "std/c/dbultra193" · use dbultra193

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra194

import "std/c/dbultra194" · use dbultra194

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra195

import "std/c/dbultra195" · use dbultra195

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra196

import "std/c/dbultra196" · use dbultra196

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra197

import "std/c/dbultra197" · use dbultra197

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra198

import "std/c/dbultra198" · use dbultra198

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra199

import "std/c/dbultra199" · use dbultra199

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra200

import "std/c/dbultra200" · use dbultra200

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra201

import "std/c/dbultra201" · use dbultra201

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra202

import "std/c/dbultra202" · use dbultra202

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra203

import "std/c/dbultra203" · use dbultra203

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra204

import "std/c/dbultra204" · use dbultra204

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra205

import "std/c/dbultra205" · use dbultra205

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra206

import "std/c/dbultra206" · use dbultra206

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra207

import "std/c/dbultra207" · use dbultra207

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra208

import "std/c/dbultra208" · use dbultra208

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra209

import "std/c/dbultra209" · use dbultra209

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra210

import "std/c/dbultra210" · use dbultra210

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra211

import "std/c/dbultra211" · use dbultra211

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra212

import "std/c/dbultra212" · use dbultra212

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra213

import "std/c/dbultra213" · use dbultra213

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra214

import "std/c/dbultra214" · use dbultra214

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra215

import "std/c/dbultra215" · use dbultra215

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra216

import "std/c/dbultra216" · use dbultra216

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra217

import "std/c/dbultra217" · use dbultra217

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra218

import "std/c/dbultra218" · use dbultra218

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra219

import "std/c/dbultra219" · use dbultra219

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra220

import "std/c/dbultra220" · use dbultra220

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra221

import "std/c/dbultra221" · use dbultra221

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra222

import "std/c/dbultra222" · use dbultra222

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra223

import "std/c/dbultra223" · use dbultra223

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra224

import "std/c/dbultra224" · use dbultra224

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra225

import "std/c/dbultra225" · use dbultra225

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra226

import "std/c/dbultra226" · use dbultra226

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra227

import "std/c/dbultra227" · use dbultra227

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra228

import "std/c/dbultra228" · use dbultra228

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra229

import "std/c/dbultra229" · use dbultra229

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra230

import "std/c/dbultra230" · use dbultra230

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra231

import "std/c/dbultra231" · use dbultra231

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra232

import "std/c/dbultra232" · use dbultra232

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra233

import "std/c/dbultra233" · use dbultra233

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra234

import "std/c/dbultra234" · use dbultra234

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra235

import "std/c/dbultra235" · use dbultra235

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra236

import "std/c/dbultra236" · use dbultra236

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra237

import "std/c/dbultra237" · use dbultra237

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra238

import "std/c/dbultra238" · use dbultra238

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra239

import "std/c/dbultra239" · use dbultra239

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra240

import "std/c/dbultra240" · use dbultra240

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra241

import "std/c/dbultra241" · use dbultra241

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra242

import "std/c/dbultra242" · use dbultra242

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra243

import "std/c/dbultra243" · use dbultra243

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra244

import "std/c/dbultra244" · use dbultra244

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra245

import "std/c/dbultra245" · use dbultra245

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra246

import "std/c/dbultra246" · use dbultra246

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra247

import "std/c/dbultra247" · use dbultra247

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra248

import "std/c/dbultra248" · use dbultra248

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra249

import "std/c/dbultra249" · use dbultra249

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra250

import "std/c/dbultra250" · use dbultra250

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra251

import "std/c/dbultra251" · use dbultra251

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra252

import "std/c/dbultra252" · use dbultra252

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra253

import "std/c/dbultra253" · use dbultra253

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra254

import "std/c/dbultra254" · use dbultra254

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra255

import "std/c/dbultra255" · use dbultra255

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra256

import "std/c/dbultra256" · use dbultra256

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra257

import "std/c/dbultra257" · use dbultra257

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra258

import "std/c/dbultra258" · use dbultra258

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra259

import "std/c/dbultra259" · use dbultra259

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra260

import "std/c/dbultra260" · use dbultra260

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra261

import "std/c/dbultra261" · use dbultra261

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra262

import "std/c/dbultra262" · use dbultra262

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra263

import "std/c/dbultra263" · use dbultra263

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra264

import "std/c/dbultra264" · use dbultra264

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra265

import "std/c/dbultra265" · use dbultra265

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra266

import "std/c/dbultra266" · use dbultra266

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra267

import "std/c/dbultra267" · use dbultra267

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra268

import "std/c/dbultra268" · use dbultra268

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra269

import "std/c/dbultra269" · use dbultra269

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra270

import "std/c/dbultra270" · use dbultra270

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra271

import "std/c/dbultra271" · use dbultra271

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra272

import "std/c/dbultra272" · use dbultra272

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra273

import "std/c/dbultra273" · use dbultra273

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra274

import "std/c/dbultra274" · use dbultra274

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra275

import "std/c/dbultra275" · use dbultra275

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra276

import "std/c/dbultra276" · use dbultra276

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra277

import "std/c/dbultra277" · use dbultra277

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra278

import "std/c/dbultra278" · use dbultra278

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra279

import "std/c/dbultra279" · use dbultra279

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra280

import "std/c/dbultra280" · use dbultra280

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra281

import "std/c/dbultra281" · use dbultra281

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra282

import "std/c/dbultra282" · use dbultra282

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra283

import "std/c/dbultra283" · use dbultra283

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra284

import "std/c/dbultra284" · use dbultra284

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra285

import "std/c/dbultra285" · use dbultra285

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra286

import "std/c/dbultra286" · use dbultra286

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra287

import "std/c/dbultra287" · use dbultra287

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra288

import "std/c/dbultra288" · use dbultra288

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra289

import "std/c/dbultra289" · use dbultra289

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra290

import "std/c/dbultra290" · use dbultra290

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra291

import "std/c/dbultra291" · use dbultra291

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra292

import "std/c/dbultra292" · use dbultra292

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra293

import "std/c/dbultra293" · use dbultra293

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra294

import "std/c/dbultra294" · use dbultra294

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra295

import "std/c/dbultra295" · use dbultra295

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra296

import "std/c/dbultra296" · use dbultra296

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra297

import "std/c/dbultra297" · use dbultra297

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra298

import "std/c/dbultra298" · use dbultra298

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra299

import "std/c/dbultra299" · use dbultra299

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra300

import "std/c/dbultra300" · use dbultra300

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra301

import "std/c/dbultra301" · use dbultra301

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra302

import "std/c/dbultra302" · use dbultra302

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra303

import "std/c/dbultra303" · use dbultra303

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra304

import "std/c/dbultra304" · use dbultra304

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra305

import "std/c/dbultra305" · use dbultra305

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra306

import "std/c/dbultra306" · use dbultra306

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra307

import "std/c/dbultra307" · use dbultra307

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra308

import "std/c/dbultra308" · use dbultra308

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra309

import "std/c/dbultra309" · use dbultra309

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra310

import "std/c/dbultra310" · use dbultra310

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra311

import "std/c/dbultra311" · use dbultra311

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra312

import "std/c/dbultra312" · use dbultra312

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra313

import "std/c/dbultra313" · use dbultra313

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra314

import "std/c/dbultra314" · use dbultra314

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra315

import "std/c/dbultra315" · use dbultra315

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra316

import "std/c/dbultra316" · use dbultra316

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra317

import "std/c/dbultra317" · use dbultra317

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra318

import "std/c/dbultra318" · use dbultra318

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra319

import "std/c/dbultra319" · use dbultra319

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra320

import "std/c/dbultra320" · use dbultra320

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra321

import "std/c/dbultra321" · use dbultra321

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra322

import "std/c/dbultra322" · use dbultra322

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra323

import "std/c/dbultra323" · use dbultra323

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra324

import "std/c/dbultra324" · use dbultra324

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra325

import "std/c/dbultra325" · use dbultra325

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra326

import "std/c/dbultra326" · use dbultra326

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra327

import "std/c/dbultra327" · use dbultra327

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra328

import "std/c/dbultra328" · use dbultra328

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra329

import "std/c/dbultra329" · use dbultra329

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra330

import "std/c/dbultra330" · use dbultra330

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra331

import "std/c/dbultra331" · use dbultra331

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra332

import "std/c/dbultra332" · use dbultra332

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra333

import "std/c/dbultra333" · use dbultra333

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra334

import "std/c/dbultra334" · use dbultra334

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra335

import "std/c/dbultra335" · use dbultra335

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra336

import "std/c/dbultra336" · use dbultra336

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra337

import "std/c/dbultra337" · use dbultra337

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra338

import "std/c/dbultra338" · use dbultra338

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra339

import "std/c/dbultra339" · use dbultra339

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra340

import "std/c/dbultra340" · use dbultra340

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra341

import "std/c/dbultra341" · use dbultra341

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra342

import "std/c/dbultra342" · use dbultra342

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra343

import "std/c/dbultra343" · use dbultra343

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra344

import "std/c/dbultra344" · use dbultra344

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra345

import "std/c/dbultra345" · use dbultra345

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra346

import "std/c/dbultra346" · use dbultra346

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra347

import "std/c/dbultra347" · use dbultra347

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra348

import "std/c/dbultra348" · use dbultra348

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra349

import "std/c/dbultra349" · use dbultra349

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra350

import "std/c/dbultra350" · use dbultra350

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra351

import "std/c/dbultra351" · use dbultra351

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra352

import "std/c/dbultra352" · use dbultra352

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra353

import "std/c/dbultra353" · use dbultra353

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra354

import "std/c/dbultra354" · use dbultra354

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra355

import "std/c/dbultra355" · use dbultra355

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra356

import "std/c/dbultra356" · use dbultra356

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra357

import "std/c/dbultra357" · use dbultra357

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra358

import "std/c/dbultra358" · use dbultra358

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra359

import "std/c/dbultra359" · use dbultra359

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra360

import "std/c/dbultra360" · use dbultra360

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra361

import "std/c/dbultra361" · use dbultra361

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra362

import "std/c/dbultra362" · use dbultra362

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra363

import "std/c/dbultra363" · use dbultra363

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra364

import "std/c/dbultra364" · use dbultra364

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra365

import "std/c/dbultra365" · use dbultra365

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra366

import "std/c/dbultra366" · use dbultra366

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra367

import "std/c/dbultra367" · use dbultra367

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra368

import "std/c/dbultra368" · use dbultra368

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra369

import "std/c/dbultra369" · use dbultra369

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra370

import "std/c/dbultra370" · use dbultra370

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra371

import "std/c/dbultra371" · use dbultra371

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra372

import "std/c/dbultra372" · use dbultra372

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra373

import "std/c/dbultra373" · use dbultra373

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra374

import "std/c/dbultra374" · use dbultra374

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra375

import "std/c/dbultra375" · use dbultra375

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra376

import "std/c/dbultra376" · use dbultra376

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra377

import "std/c/dbultra377" · use dbultra377

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra378

import "std/c/dbultra378" · use dbultra378

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra379

import "std/c/dbultra379" · use dbultra379

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra380

import "std/c/dbultra380" · use dbultra380

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra381

import "std/c/dbultra381" · use dbultra381

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra382

import "std/c/dbultra382" · use dbultra382

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra383

import "std/c/dbultra383" · use dbultra383

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra384

import "std/c/dbultra384" · use dbultra384

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra385

import "std/c/dbultra385" · use dbultra385

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra386

import "std/c/dbultra386" · use dbultra386

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra387

import "std/c/dbultra387" · use dbultra387

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra388

import "std/c/dbultra388" · use dbultra388

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra389

import "std/c/dbultra389" · use dbultra389

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra390

import "std/c/dbultra390" · use dbultra390

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra391

import "std/c/dbultra391" · use dbultra391

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra392

import "std/c/dbultra392" · use dbultra392

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra393

import "std/c/dbultra393" · use dbultra393

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra394

import "std/c/dbultra394" · use dbultra394

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra395

import "std/c/dbultra395" · use dbultra395

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra396

import "std/c/dbultra396" · use dbultra396

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra397

import "std/c/dbultra397" · use dbultra397

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra398

import "std/c/dbultra398" · use dbultra398

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra399

import "std/c/dbultra399" · use dbultra399

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra400

import "std/c/dbultra400" · use dbultra400

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra401

import "std/c/dbultra401" · use dbultra401

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra402

import "std/c/dbultra402" · use dbultra402

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra403

import "std/c/dbultra403" · use dbultra403

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra404

import "std/c/dbultra404" · use dbultra404

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra405

import "std/c/dbultra405" · use dbultra405

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra406

import "std/c/dbultra406" · use dbultra406

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra407

import "std/c/dbultra407" · use dbultra407

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra408

import "std/c/dbultra408" · use dbultra408

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra409

import "std/c/dbultra409" · use dbultra409

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra410

import "std/c/dbultra410" · use dbultra410

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra411

import "std/c/dbultra411" · use dbultra411

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra412

import "std/c/dbultra412" · use dbultra412

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra413

import "std/c/dbultra413" · use dbultra413

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra414

import "std/c/dbultra414" · use dbultra414

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra415

import "std/c/dbultra415" · use dbultra415

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra416

import "std/c/dbultra416" · use dbultra416

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra417

import "std/c/dbultra417" · use dbultra417

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra418

import "std/c/dbultra418" · use dbultra418

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra419

import "std/c/dbultra419" · use dbultra419

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra420

import "std/c/dbultra420" · use dbultra420

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra421

import "std/c/dbultra421" · use dbultra421

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra422

import "std/c/dbultra422" · use dbultra422

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra423

import "std/c/dbultra423" · use dbultra423

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra424

import "std/c/dbultra424" · use dbultra424

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra425

import "std/c/dbultra425" · use dbultra425

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra426

import "std/c/dbultra426" · use dbultra426

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra427

import "std/c/dbultra427" · use dbultra427

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra428

import "std/c/dbultra428" · use dbultra428

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra429

import "std/c/dbultra429" · use dbultra429

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra430

import "std/c/dbultra430" · use dbultra430

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra431

import "std/c/dbultra431" · use dbultra431

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra432

import "std/c/dbultra432" · use dbultra432

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra433

import "std/c/dbultra433" · use dbultra433

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra434

import "std/c/dbultra434" · use dbultra434

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra435

import "std/c/dbultra435" · use dbultra435

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra436

import "std/c/dbultra436" · use dbultra436

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra437

import "std/c/dbultra437" · use dbultra437

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra438

import "std/c/dbultra438" · use dbultra438

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra439

import "std/c/dbultra439" · use dbultra439

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra440

import "std/c/dbultra440" · use dbultra440

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra441

import "std/c/dbultra441" · use dbultra441

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra442

import "std/c/dbultra442" · use dbultra442

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra443

import "std/c/dbultra443" · use dbultra443

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra444

import "std/c/dbultra444" · use dbultra444

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra445

import "std/c/dbultra445" · use dbultra445

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra446

import "std/c/dbultra446" · use dbultra446

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra447

import "std/c/dbultra447" · use dbultra447

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra448

import "std/c/dbultra448" · use dbultra448

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra449

import "std/c/dbultra449" · use dbultra449

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra450

import "std/c/dbultra450" · use dbultra450

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra451

import "std/c/dbultra451" · use dbultra451

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra452

import "std/c/dbultra452" · use dbultra452

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra453

import "std/c/dbultra453" · use dbultra453

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra454

import "std/c/dbultra454" · use dbultra454

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra455

import "std/c/dbultra455" · use dbultra455

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra456

import "std/c/dbultra456" · use dbultra456

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra457

import "std/c/dbultra457" · use dbultra457

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra458

import "std/c/dbultra458" · use dbultra458

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra459

import "std/c/dbultra459" · use dbultra459

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra460

import "std/c/dbultra460" · use dbultra460

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra461

import "std/c/dbultra461" · use dbultra461

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra462

import "std/c/dbultra462" · use dbultra462

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra463

import "std/c/dbultra463" · use dbultra463

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra464

import "std/c/dbultra464" · use dbultra464

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra465

import "std/c/dbultra465" · use dbultra465

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra466

import "std/c/dbultra466" · use dbultra466

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra467

import "std/c/dbultra467" · use dbultra467

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra468

import "std/c/dbultra468" · use dbultra468

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra469

import "std/c/dbultra469" · use dbultra469

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra470

import "std/c/dbultra470" · use dbultra470

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra471

import "std/c/dbultra471" · use dbultra471

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra472

import "std/c/dbultra472" · use dbultra472

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra473

import "std/c/dbultra473" · use dbultra473

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra474

import "std/c/dbultra474" · use dbultra474

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra475

import "std/c/dbultra475" · use dbultra475

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra476

import "std/c/dbultra476" · use dbultra476

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra477

import "std/c/dbultra477" · use dbultra477

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra478

import "std/c/dbultra478" · use dbultra478

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra479

import "std/c/dbultra479" · use dbultra479

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra480

import "std/c/dbultra480" · use dbultra480

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra481

import "std/c/dbultra481" · use dbultra481

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra482

import "std/c/dbultra482" · use dbultra482

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra483

import "std/c/dbultra483" · use dbultra483

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra484

import "std/c/dbultra484" · use dbultra484

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra485

import "std/c/dbultra485" · use dbultra485

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra486

import "std/c/dbultra486" · use dbultra486

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra487

import "std/c/dbultra487" · use dbultra487

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra488

import "std/c/dbultra488" · use dbultra488

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra489

import "std/c/dbultra489" · use dbultra489

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra490

import "std/c/dbultra490" · use dbultra490

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra491

import "std/c/dbultra491" · use dbultra491

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra492

import "std/c/dbultra492" · use dbultra492

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra493

import "std/c/dbultra493" · use dbultra493

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra494

import "std/c/dbultra494" · use dbultra494

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra495

import "std/c/dbultra495" · use dbultra495

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra496

import "std/c/dbultra496" · use dbultra496

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra497

import "std/c/dbultra497" · use dbultra497

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra498

import "std/c/dbultra498" · use dbultra498

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra499

import "std/c/dbultra499" · use dbultra499

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

std/c/dbultra500

import "std/c/dbultra500" · use dbultra500

- `execSql(path text, sql text) → bool`
- `queryRows(path text, sql text) → text`
- `selectAll(path text, table text) → text`
- `insertRow(path text, table text, columns text, values text) → number`
- `deleteWhere(path text, table text, whereClause text) → bool`
- `rowCount(path text, table text) → number`
- `tableExists(path text, table text) → bool`
- `dropTable(path text, table text) → bool`
- `createTable(path text, ddl text) → bool`
- `listTables(path text) → text`
- `escapeLiteral(value text) → text`
- `scalarQuery(path text, sql text) → number`
- `writeQueryCsv(outPath text, dbPath text, sql text) → bool`
- `vacuumDb(path text) → bool`
- `maxColumn(path text, table text, col text) → number`
- `updateWhere(path text, table text, setClause text, whereClause text) → bool`
- `clearTable(path text, table text) → bool`
- `selectTop(path text, table text, limit number) → text`
- `countLines(text text) → number`
- `sumColumn(path text, table text, col text) → number`

Génération

Génération de la bibliothèque standard

La stdlib AFRILANG (~7971 modules) est générée à partir de scripts Python et de catalogues C++.

Tiers

```
| Tier | Import | Script | Catalogue C++ |
|------|--------|--------|--------------|
| Simple | `import "std/nom"` | `scripts/gen_simple_stdlib.py` | `src/utils/stdlib_catalog.cpp` |
| Medium | `import "std/m/nom"` | `scripts/gen_medium_stdlib.py` | `src/utils/medium_catalog.cpp` |
| Complex | `import "std/c/nom"` | `scripts/gen_complex_stdlib.py` | `src/utils/complex_catalog.cpp` |
```

Packs numérotés (500 modules)

- **giskit** — SIG simple (`gen_simple_stdlib.py`)
- **game2dkit**, **game3dkit**, **gamekit** — kits jeu
- **gisultra**, **rasterultra**, **segultra**, **dataultra**, **datasci**, **vizultra**, **iaultra**, **dbultra**, **gameultra**, **game3dultra** — `gen_complex_stdlib.py`

Workflow après modification

```
# Régénérer un tier (exemple : simple)
python3 scripts/gen_simple_stdlib.py

# Recompiler le compilateur
cd build && cmake .. && make -j$(nproc)

# Synchroniser le site Django
cd web && .venv/bin/python manage.py sync_site
```

Architecture

1. Stub .afr dans stdlib/ — signature des fonctions (IDE, docs). 2. Runtime C++ — runtime/simple_libs.hpp, runtime/medium_libs.hpp, runtime/complex_libs.hpp + headers dédiés (runtime/hex.hpp, runtime/game2d.hpp, ...). 3. Injection — src/utils/stdlib_registry.cpp résout import "std/..." et injecte les modules dans l'AST.

Les modules legacy (io, json, hex, html, email, game2d, ...) ont un header dédié dans runtime/ et ne doivent pas être dupliqués dans simple_libs.hpp.

Tests

afrilang test compile les exemples de référence + un échantillon par famille ultra (segultra, dataultra, giskit, ...).

Partie IX — Gestionnaire de paquets

Guide des paquets AFRILANG

Équivalent pratique de Nimble / Shards pour AFRILANG.

Méthode en 6 étapes

1. Créer — afrilang init / pkg init 2. Déclarer — [dependencies] dans afrilang.toml (version, git, path) 3. Synchroniser — pkg sync (index distant) 4. Installer — pkg install (respecte afrilang.lock s'il existe) 5. Tester — pkg test / afrilang test 6. Publier — pkg publish (+ --remote si registre)

Guide web détaillé (bilingue) : /docs/package-manager/.

Fichiers projet

```
| Fichier | Rôle |
|---------|------|
| `afrilang.toml` | Métadonnées + `[dependencies]` |
| `afrilang.lock` | Versions exactes installées (source de vérité pour `install`) |
| `vendor/` | Copies locales des paquets |
| `manifest.toml` | Métadonnées d'un **paquet** (bibliothèque) |
| `packages/index.json` | Index local |
| `packages/remote-index.json` | Index distant (`pkg sync`) |
```

Créer un projet ou un paquet

```
afrilang init mon_app              # application
afrilang init mon_lib --lib        # alias de pkg init
afrilang pkg init mon_lib          # paquet bibliothèque
cd mon_lib && afrilang pkg test    # self-vendor + tests/
afrilang doc .                     # docs/api/
```

Dépendances

Registre (local ou distant)

```
[dependencies]
math = "0.1.0"
strings = "^0.1.0"
```

```
afrilang pkg sync                  # télécharge remote-index.json
afrilang pkg add math              # local packages/ ou clone via url/method
afrilang pkg install
afrilang pkg update
```

Si le paquet n'est pas dans packages/<name> mais apparaît dans l'index avec "method":"git" + "url", AFRILANG clone comme Nimble (cache sous packages/.cache/).

Voir le modèle : packages/remote-index.example.json.

Semver

```
| Range | Signification |
|-------|---------------|
| `1.2.3` | `actual >= 1.2.3` (compat historique) |
| `^1.2.3` | `>= 1.2.3` et `< 2.0.0` |
| `~1.2.3` | `>= 1.2.3` et `< 1.3.0` |
```

Git

```
[dependencies]
mylib = { git = "https://github.com/org/mylib.git", tag = "v0.1.0" }
```

```
afrilang pkg add mylib --git https://github.com/org/mylib.git --tag v0.1.0
```

Path (développement local)

```
[dependencies]
mylib = { path = "../mylib" }
```

```
afrilang pkg add mylib --path ../mylib
```

Transitivité & conflits

install / update résolvent les deps des manifest.toml récursivement. Si deux contraintes sur le même nom sont incompatibles → erreur de conflit.

Install vs update

- `pkg install` — si `afrilang.lock` existe, installe **exactement** le lock ; sinon résout depuis le toml et écrit le lock.
- `pkg update` — ignore les pins du lock, re-résout depuis les ranges du toml, réécrit le lock.

Publier

Local

```
afrilang pkg publish ./mon_lib
afrilang pkg list
```

Distant (style Nimble)

```
export AFRILANG_PACKAGE_GIT_URL=https://github.com/vous/mon_lib
export AFRILANG_REGISTRY_TOKEN=...          # optionnel
export AFRILANG_REGISTRY_PUBLISH_URL=...    # optionnel
afrilang pkg publish ./mon_lib --remote
```

Écrit registry-entry.json, affiche la recette d'ajout au registre, et POST si un token est défini.

Variables utiles :

```
| Variable | Rôle |
|----------|------|
| `AFRILANG_REGISTRY_URL` | URL de l'index JSON (`pkg sync`) |
| `AFRILANG_REGISTRY_PUBLISH_URL` | Endpoint POST publish |
| `AFRILANG_REGISTRY_TOKEN` | Bearer token |
| `AFRILANG_PACKAGE_GIT_URL` | URL git du paquet pour la recette |
```

Imports

```
import pkg/math/math
```

Partie X — Outils (LSP, VS Code, WASM)

afrilang lsp — diagnostics, hover, définition, complétion EN/FR.

Extension VS Code

AFRILANG — extension VS Code / Cursor / VSCodium

Coloration syntaxique, diagnostics, complétion LSP, formatage et exécution de fichiers .afr.

Installation (utilisateurs)

Depuis l'éditeur : Extensions → rechercher AFRILANG → Install

```
code --install-extension afrilang.afrilang      # VS Code
codium --install-extension afrilang.afrilang    # VSCodium
# Cursor : même marketplace que VS Code
```

Compilateur requis

L'extension appelle afrilang pour le LSP et l'exécution. Compilez AFRILANG puis configurez :

```
{
  "afrilang.serverPath": "/chemin/vers/AFRILANG/build/afrilang"
}
```

Installation locale (développeurs)

```
cd vscode-afrilang
./install.sh
```

Puis Developer: Reload Window.

Publication (mainteneurs)

Voir PUBLISHING.md.

```
cd vscode-afrilang
npm install
./publish.sh          # crée afrilang-1.4.0.vsix (+ publish si VSCE_PAT/OVSX_PAT)
```

Tag Git optionnel : git tag vscode-v1.4.0 && git push origin vscode-v1.4.0 déclenche le workflow CI.

Open VSX (VSCodium / Cursor)

Après publication, l'extension est disponible sur Open VSX :

```
codium --install-extension afrilang.afrilang
```

Configuration

```
| Paramètre | Défaut | Description |
|-----------|--------|-------------|
| `afrilang.serverPath` | `afrilang` | Chemin vers l'exécutable (absolu si hors PATH) |
| `afrilang.trace.server` | `off` | Trace LSP dans le panneau « AFRILANG LSP » |
```

Exemple settings.json :

```
{
  "afrilang.serverPath": "/home/user/AFRILANG/build/afrilang",
  "editor.formatOnSave": true,
  "[afrilang]": { "editor.tabSize": 4 }
}
```

Fonctionnalités

- **Diagnostics** — erreurs de compilation en temps réel (soulignés rouges)
- **Complétion** — mots-clés FR/EN + Phase 6 (`enum`, `match`, `nothing`, ...)
- **Formatage** — `Shift+Alt+F` ou format on save via LSP
- **Exécuter** — bouton ▶ dans l'éditeur ou commande *AFRILANG: Exécuter le fichier*
- **Vérifier** — *AFRILANG: Vérifier le fichier* (`afrilang check`)
- **Débugger** — *AFRILANG: Débugger le fichier* (adaptateur DAP + GDB, types Afrilang dans Variables)
- **Config debug** — *AFRILANG: Initialiser la config debug* → `launch.json` pour F5
- **Icône** — carte d'Afrique pour les fichiers `.afr`

Icône + coloration des fichiers `.afr`

Le workspace active le thème d'icônes AFRILANG (afrilang-icons) : les .afr affichent la carte d'Afrique. La grammaire TextMate colore mots-clés, types, chaînes, etc.

Important — Cursor Agents Window (Glass) La coloration et les icônes d'extensions ne s'appliquent pas dans la vue Agents (bug Cursor connu). Ce n'est pas un défaut de l'extension AFRILANG.

Pour voir couleurs + logo :

1. Ctrl+Shift+P → Cursor: Open or Focus Editor Window (cursor.openOrFocusEditorWindow) — quitte Glass / Agents 2. Ouvrir examples/hello.afr dans cet éditeur classique 3. Barre d'état (bas à droite) = AFRILANG 4. File Icon Theme = AFRILANG

Réinstallation si besoin :

```
cd vscode-afrilang && ./install.sh
# ou :
cursor --install-extension ./afrilang-1.4.0.vsix
```

Puis Developer: Reload Window.

Commandes

- `AFRILANG: Exécuter le fichier`
- `AFRILANG: Vérifier le fichier`
- `AFRILANG: Redémarrer le serveur LSP`

WASM

Compatibilité WASM (wasm32)

AFRILANG peut cibler wasm32 via Emscripten (afrilang run fichier.afr --target wasm32).

Fonctionne en WASM

```
| Domaine | Modules / features |
|---------|-------------------|
| Langage de base | variables, fonctions, boucles, `if`, listes, maps |
| POO | classes, héritage, interfaces |
| Stdlib core | `json`, `str`, `math`, `collections`, `args`, `path`, `log` |
| Async | `std/async` — `sleep`, coroutines (Emscripten) |
| Démos CI | `examples/tier8_demo.afr`, `examples/tier8_stdlib.afr` |
```

Natif uniquement (pas WASM)

```
| Domaine | Raison |
|---------|--------|
| `std/ui`, SDL2 | pas de fenêtre native en navigateur |
| `std/game2d`, `std/game3d` | OpenGL / SDL |
| `std/http` (sync) | sockets système |
| `std/fs`, `std/io` fichiers | accès disque limité en WASM |
| `std/thread` | threads POSIX |
| `std/gamenet` UDP | sockets réseau |
| Packs ultra lourds | compilation/link très longue ; tester en natif |
```

Playground JS (`compile-js`)

Sous-ensemble élargi (Vague 3) : say, assign/set, if/while/repeat/for, fonctions + appels + return. Toujours exclus : imports, classes, async, UI, tests.

Recommandation

- **Playground / CI WASM** : petits programmes sans SDL/OpenGL ; stdlib `str`/`math`/`json` OK.
- **Jeux, SIG, data science** : cible `native` ou `linux-arm64`.

- fmt
- lint
- debug (GDB)
- explain

Partie XI — Site web, playground, mobile

```
afrilang serve  # http://127.0.0.1:8080
```

AFRILANG Web — Django + PostgreSQL + Bootstrap + Tailwind

Site officiel dynamique pour l'écosystème AFRILANG.

Stack

```
| Couche | Technologie |
|--------|-------------|
| Backend | **Django 5** |
| Base de données | **PostgreSQL 16** |
| UI | **Bootstrap 5** + **Tailwind CSS** (préfixe `tw-`) |
| Compilateur | binaire `afrilang` (subprocess) |
```

Démarrage rapide

```
# 1. PostgreSQL
cd web
docker compose up -d

# 2. Python
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env

# 3. Tailwind (optionnel — CSS minimal inclus)
npm install && npm run build:css

# 4. Django
python manage.py migrate
python manage.py seed_data
python manage.py sync_packages
python manage.py createsuperuser   # optionnel
python manage.py runserver
```

Ouvrir http://127.0.0.1:8000

Pages

- `/` — Accueil (stats, paquets DB, exemples)
- `/explore/` — Capacités (modèle `Capability`)
- `/docs/` — Documentation
- `/packages/` — Registre PostgreSQL + recherche
- `/playground/` — Éditeur + API run/fmt
- `/benchmarks/` — Graphiques compile/exec (Chart.js)
- `/showcase/` — Galerie projets GitHub
- `/admin/` — Administration Django

API REST

```
| Endpoint | Méthode | Description |
|----------|---------|-------------|
| `/api/run/` | POST | `{ "source": "..." }` → exécution native |
| `/api/compile/js/` | POST | Transpile vers JavaScript (exécution navigateur) |
| `/api/build/wasm/` | POST | Module WASM pour le navigateur (Emscripten serveur) |
| `/api/wasm/<id>/module.js` | GET | Artefact WASM de session |
| `/api/run/wasm/` | POST | Exécution WASM serveur (emscripten requis) |
| `/api/fmt/` | POST | Formatage |
| `/api/packages/` | GET | Liste JSON des paquets |
```

Les runs playground sont enregistrés dans PlaygroundRun (PostgreSQL).

Variables d'environnement

Voir .env.example :

- `DATABASE_URL` — PostgreSQL
- `AFRILANG_BIN` — chemin vers le compilateur
- `AFRILANG_ROOT` — racine du repo
- `PLAYGROUND_WASM_DIR` — sessions WASM temporaires (défaut: `web/tmp/wasm_sessions`)

Compilateur WASM client (optionnel)

Pour exécuter la compilation dans le navigateur sans appeler /api/compile/js :

```
# Nécessite Emscripten (emsdk)
./scripts/build_wasm_compiler.sh
# → web/static/compiler/afrilang-compiler.js
```

Commandes utiles

```
python manage.py sync_packages   # packages/index.json → PostgreSQL
python manage.py sync_benchmarks # scripts/benchmark.sh → SiteMetric
python manage.py seed_data       # exemples + capacités
```

Ancien site statique

Le dossier site/ (serveur C++ afrilang serve) reste disponible. Ce projet Django le remplace pour la production avec base de données.

Mobile

AFRILANG Mobile (Flutter)

Application mobile qui reproduit le frontend du site web et consomme le même backend Django (web/).

Prérequis

- Flutter 3.22+ (`flutter doctor`)
- Backend web avec le **venv** (Django n’est pas installé globalement) :

```
cd web
source .venv/bin/activate
python manage.py runserver 0.0.0.0:8000
```

Si .venv n’existe pas encore :

```
cd web
python3 -m venv .venv   # ou: uv venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Lancer l'app

```
cd mobile/afrilang
flutter pub get
flutter run
```

URL du backend

```
| Cible | Base URL |
|--------|----------|
| Émulateur Android | `http://10.0.2.2:8000` (défaut) |
| iOS Simulator / desktop / Chrome | `http://127.0.0.1:8000` |
| Téléphone physique | IP LAN, ex. `http://192.168.1.10:8000` |
```

Override :

```
flutter run --dart-define=AFRILANG_API_BASE=http://192.168.1.10:8000
```

Icône d'application

L'icône launcher utilise le logo Afrique AFRILANG (plus le logo Flutter).

```
cd mobile/afrilang
python3 tool/generate_app_icons.py
```

Sur Android, le fond de l'icône adaptive est transparent pour laisser apparaître la silhouette Afrique (le launcher impose toujours un masque cercle/squircle). Sur iOS, Apple impose un carré arrondi opaque — logo Afrique sur fond navy.

- Accueil (hero, stats, features, paquets certifiés, CTA)
- Documentation (pages FR/EN + blocs code / callouts)
- Paquets + détail
- Exemples + Run
- Stdlib (cœur / expérimental) + détail API
- Playground (Run / Format / Check)
- Tutoriel interactif, releases, FR/EN, mode sombre

API Django utilisée

Préfixe : /api/mobile/

- `GET bootstrap/`, `home/`, `docs/`, `docs/<slug>/`
- `GET packages/`, `packages/<name>/`, `examples/`, `stdlib/`, `stdlib/<name>/`
- `GET tutorial/`, `tutorial/<step>/`, `releases/`
- `POST run/`, `fmt/`, `check/` (CSRF exempt, rate-limité comme le playground web)

Les exemples détaillés utilisent aussi GET /api/examples/<slug>/.

Structure

```
lib/
  config/api_config.dart
  services/api_client.dart
  state/app_state.dart
  theme/afrilang_theme.dart   # Plus Jakarta Sans + JetBrains Mono, #1d4ed8
  widgets/common.dart         # code panel, hero mesh, fade-ins
  screens/…
```

Partie XII — Catalogue d'exemples

```
afrilang run examples/NOM.afr
```

88 fichiers

1. advanced.afr

```
enum Status
    case Ok
    case Error with message text
end

create s = Status.Ok
create e = Status.Error with "failed"

match s
    case Ok then
        say "ok"
    end
    default
        say "other"
    end
end

match e
    case Error then
        say e.message
    end
    default
        say "unexpected"
    end
end

create nickname text? = nothing
if nickname is defined then
    say nickname
else
    say "no nickname"
end
```

2. args_path_demo.afr

```
import "std/args"
import "std/path"

use args
use path

say count()
say join("home", "user")

say "stdlib args path complete"
```

3. async_demo.afr

```
import "std/async"

use async

async function waitAndReply(ms number) returns text
    await sleep(ms)
    return "done"
end

async function runDemo()
    say "starting async demo"
    create msg = await waitAndReply(10)
    say msg
    await sleep(5)
    say "async demo complete"
end

await runDemo()
```

4. async_full.afr

```
import "std/async"

use async

async function delayedValue(ms number) returns number or error
    if ms is less than 0 then
        return error "negative delay"
    end
    await sleep(ms)
    return ms * 2
end

async function runAll()
    create r = await delayedValue(5)
    if r is error then
        say r.message
    else
        say r.value
    end
end

test "async await in test"
    create r = await delayedValue(1)
    assert r.value is equal to 2
end

await runAll()
```

5. complex_libs_demo.afr

```
import "std/c/nlplevenshtein"
import "std/c/statcorrel"
import "std/c/mlcosine"
import "std/format"

use nlplevenshtein
use statcorrel
use mlcosine
use format

create dist = distance("kitten", "sitting")
say formatNumber(dist, 0)

create sim = similarity("hello", "hallo")
say formatNumber(sim, 3)

create numsA = list of 1, 2, 3, 4, 5
create numsB = list of 2, 4, 6, 8, 10
create corr = pearson(numsA, numsB)
say formatNumber(corr, 3)

create vecA = list of 1, 0, 1
create vecB = list of 1, 1, 0
create cos = cosineSim(vecA, vecB)
say formatNumber(cos, 3)
```

6. conditions.afr

```
create age = 25

if age is greater than 18 then
    say "Adulte"
end

if age is less than 30 then
    say "Jeune adulte"
end

create score = 100

if score is equal to 100 then
    say "Score parfait!"
end

create x = 10
create y = 3

say x + y
say x - y
say x * y
say x / y

repeat 2 times
```

```
    say "Boucle conditionnelle"
end
```

7. datasci_demo.afr

```
import "std/c/datasci001"
use datasci001

create values list number = list of 4, 8, 6, 12, 10, 16, 14, 20
create xs list number = list of 0, 1, 2, 3, 4, 5, 6, 7
create ys list number = list of 2, 4, 5, 8, 9, 12, 13, 18

say "median={median(values)} variance={variance(values)} iqr={iqr(values)}"
say "gini={gini(values)} cov={covariance(xs, ys)} r2={rSquared(xs, ys)}"
say "rmse={rmse(xs, ys)} mae={mae(xs, ys)} outliers={outlierCount(values, 0)}"

create scaled = minMaxScale(values)
create zs = zscores(values)
say "scaled[0]={scaled at 0} z[0]={zs at 0}"

create hist = histogram(values, 0)
say "hist bins={hist at 0} {hist at 1} {hist at 2}"

create smooth = ema(values, 0)
say "ema[7]={smooth at 7} split={trainSplitAt(8, 0.75)}"
```

8. dataultra_demo.afr

```
import "std/c/dataultra001"
use dataultra001

create values list number = list of 2, 4, 6, 8, 10, 12
create xs list number = list of 0, 1, 2, 3, 4, 5

say "mean={mean(values)} stddev={stddev(values)}"
say "min={minVal(values)} max={maxVal(values)}"
say "slope={linearSlope(xs, values)}"
say "corr={correlation(xs, values)}"

create norm = normalize(values)
say "norm[0]={norm at 0} norm[5]={norm at 5}"

create dif = diff1(values)
say "diff1[0]={dif at 0} diff1[4]={dif at 4}"
```

9. dbultra_demo.afr

```
import "std/c/dbultra001"
use dbultra001

import "std/sql"
use sql

create db text = "dbultra_demo.db"
create tbl text = "products"
create ddl text = "create table if not exists products (id integer primary key, name text, price real)"

create ok bool = createTable(db, ddl)
say "create={ok} exists={tableExists(db, tbl)}"

create id number = insertRow(db, tbl, "name|price", "mango|3.5")
create id2 number = insertRow(db, tbl, "name|price", "banana|1.2")
say "ids={id} {id2} rows={rowCount(db, tbl)}"

create rows text = selectTop(db, tbl, 0)
create col text = "price"
say "top={rows}"
say "sum={sumColumn(db, tbl, col)} max={maxColumn(db, tbl, col)}"

create csvOk bool = writeQueryCsv("dbultra_export.csv", db, "select name,price from products;")
say "csv={csvOk} lines={countLines(rows)}"
```

10. defaults.afr

```
import "std/math"
import "std/str"

use math
use str

function power(base number, exp number = 2) returns number
    return pow(base, exp)
end

say power(3)
```

```
say power(2, 10)

function label(message text, tag text = "info") returns text
    return "{tag} {message}"
end

say label("System ready")
say label("Warning", "warn")
```

11. educational.afr

```
explain say "This message is explained then printed"

explain create x = 10
say x

say "Program finished"
```

12. exceptions.afr

```
function divide(a number, b number) returns number
    if b is equal to 0 then
        raise "division by zero"
    end
    return a / b
end

try
    say divide(10, 2)
    say divide(10, 0)
catch error e
    say "caught:"
    say e
end

say "program continues"
```

13. ffi.afr

```
extern from "m" function sin(x number) returns number
extern from "m" function sqrt(x number) returns number

say sin(3.14159265359 / 2)
say sqrt(16)
```

14. fields.afr

```
class Person
    public field name text
    private field age number

    function init(aName text, aAge number)
        set this.name = aName
        set this.age = aAge
    end

    function greet()
        say name
        say age
    end
end

create bob = new Person("Bob", 25)
bob.greet()
say bob.name
```

15. fs_demo.afr

```
import "std/fs"
import "std/io"

use fs
use io

create dir = "/tmp/afrilang_fs_demo"

if fileExists(dir) is equal to true then
    removeFile(dir + "/sample.txt")
end

say makeDir(dir)
writeFile(dir + "/sample.txt", "filesystem demo")

say fileSize(dir + "/sample.txt")
```

```
for each name in listDir(dir) do
    say name
end

say readFile(dir + "/sample.txt")
```

16. full_demo.afr

```
import "helpers.afr"

module Utils
    function twice(x number) returns number
        return x * 2
    end
end

use Utils

create active bool = true

if active is equal to true then
    say "System active"
else
    say "System inactive"
end

create counter number = 0
while counter is less than 3 do
    say twice(counter)
    set counter = counter + 1
end

create items = list of 1, 2, 3
for each item in items do
    if item is equal to 2 then
        stop
    end
    say item
end

say sum2(10, 32)
say "Full demo complete"
```

17. functions.afr

```
function add(a number, b number) returns number
    return a + b
end

function greet(name text) returns text
    return "Bonjour, " + name
end

function square(n number) returns number
    return n * n
end

class Calculator
    function compute(x number, y number) returns number
        return x + y * 2
    end
end

create result = add(10, 32)
say result

create message = greet("AFRILANG")
say message

create squared = square(7)
say squared

create calc = new Calculator
say calc.compute(5, 3)
```

18. game3d_advanced.afr

```
import "std/game3d"
use game3d

openWindow("AFRILANG 3D Avance", 1024, 640)
setCamera(0, 8, 22, 0, -12)

loadSkyboxFace("px", "examples/assets/sky/px.png")
loadSkyboxFace("nx", "examples/assets/sky/nx.png")
loadSkyboxFace("py", "examples/assets/sky/py.png")
loadSkyboxFace("ny", "examples/assets/sky/ny.png")
loadSkyboxFace("pz", "examples/assets/sky/pz.png")
loadSkyboxFace("nz", "examples/assets/sky/nz.png")

loadShader("lit", "examples/assets/shaders/lit.vert", "examples/assets/shaders/lit.frag")
loadGlb("pyramid", "examples/assets/pyramid_anim.glb")
loadTexture3d("grass", "examples/assets/grass.png")
loadModel("crate", "examples/assets/crate.obj")

setFog(yes, 120, 140, 165, 30, 120)
setWind(0.4, 0, 0.2)

createBody("ball", -2, 12, 0, 0.5)
createBoxBody("crateA", 2, 10, 1, 0.9, 0.9, 0.9)
createBoxBody("crateB", 0, 14, -2, 0.7, 0.7, 0.7)
setBodyVelocity("ball", 1.5, 0, 0.8)
setBodyRestitution("crateA", 0.65)
setBodyRestitution("crateB", 0.7)

create rot number = 0
create picked text = ""
playGltfAnim("pyramid", 0, true)

while isOpen() do
    beginFrame()

    if wasKeyPressed("Escape") then
        shutdown()
    end

    create dt number = deltaMs()
    set rot = rot + dt * 0.02
    stepPhysicsEx(dt, -18)
    updateGltfAnims(dt)

    if wasMouseClicked() then
        set picked = pickBodyName(mouseX(), mouseY())
    end

    clearScreen(10, 12, 18)
    applyFog()
    applyCamera()

    if hasSkybox() then
        drawSkybox(180)
    end

    setSceneRotation(rot)
    applySceneRotation()

    drawGrid(0, 22, 22)
    drawPlaneTextured(0, 22, "grass")

    setShaderVec3("lit", "uSunDir", 0.4, 1, 0.3)
    setShaderVec3("lit", "uSunColor", 1, 0.95, 0.85)
    setShaderVec3("lit", "uAmbient", 0.2, 0.22, 0.28)
    drawModelShader("lit", "pyramid", -5, 0, 4, 2.2, rot * 0.6 + gltfAnimRotY("pyramid"))

    drawModelLit("crate", 6, 0.5, -4, 1.0, rot * 0.4, "grass")
    drawBody("crateA")
    drawBody("crateB")
    drawSphere(bodyX("ball"), bodyY("ball"), bodyZ("ball"), 0.5, 255, 200, 100)

    showFrame()
end
```

19. game3d_complex.afr

```
import "std/game3d"
use game3d

openWindow("AFRILANG 3D Complexe", 1024, 640)
setCamera(0, 10, 28, 0, -16)

loadTexture3d("grass", "examples/assets/grass.png")
loadTexture3d("crate", "examples/assets/crate.png")
loadModel("crate", "examples/assets/crate.obj")
loadModel("crystal", "examples/assets/crystal.obj")
```

```
enableLighting(yes)
setAmbientLight(55, 60, 75)
setSunLight(-0.4, -1, -0.3, 255, 245, 220)
setFog(yes, 140, 155, 175, 22, 85)
setWind(0.8, 0, 0.3)

createBody("hero", 0, 6, 0, 0.55)
setBodyVelocity("hero", 2, 0, 1)
setBodyRestitution("hero", 0.72)

createBody("orb1", -3, 10, 2, 0.45)
createBody("orb2", 2, 12, -2, 0.5)
createBody("orb3", -1, 14, -3, 0.4)

createBoxBody("box1", 4, 8, 1, 0.8, 0.8, 0.8)
createBoxBody("box2", -5, 9, -1, 1, 0.6, 1.2)
setBodyFriction("box1", 0.88)
setBodyFriction("box2", 0.9)

create rot number = 0
create crystalRot number = 0
create followCam bool = yes
create spawnTick number = 0

while isOpen() do
    beginFrame()

    if wasKeyPressed("Escape") then
        shutdown()
    end

    if wasKeyPressed("C") then
        if followCam is equal to true then
            set followCam = false
        else
            set followCam = true
        end
    end

    create dt number = deltaMs()
    set rot = rot + dt * 0.015
    set crystalRot = crystalRot + dt * 0.04
    set spawnTick = spawnTick + dt
    setSceneRotation(rot)

    stepPhysicsEx(dt, -20)
    updateParticles(dt)

    if spawnTick is greater than 350 then
        set spawnTick = 0
        emitBurst(0, 2.5, 0, 12, 4)
    end

    if followCam then
        followBody("hero", 5, 14, 0.1)
    else
        create move number = dt * 0.012
        create turn number = dt * 0.035
        updateFlyCamera(move, turn)
    end

    clearScreen(20, 24, 32)
    applyFog()
    applyLighting()
    applyCamera()
    applySceneRotation()

    drawGrid(0, 25, 25)
    drawPlaneTextured(0, 25, "grass")

    drawModelLit("crate", -6, 0.6, 4, 1.1, rot * 0.5, "crate")
    drawModelLit("crystal", 6, 1.5, -5, 1.8, crystalRot, "")

    drawBody("box1")
    drawBody("box2")
    drawSphere(bodyX("hero"), bodyY("hero"), bodyZ("hero"), 0.55, 120, 255, 160)
    drawSphere(bodyX("orb1"), bodyY("orb1"), bodyZ("orb1"), 0.45, 255, 120, 90)
    drawSphere(bodyX("orb2"), bodyY("orb2"), bodyZ("orb2"), 0.5, 100, 200, 255)
    drawSphere(bodyX("orb3"), bodyY("orb3"), bodyZ("orb3"), 0.4, 220, 100, 220)
    drawParticles()

    showFrame()
end
```

20. game3d_demo.afr

```
import "std/game3d"
use game3d

openWindow("AFRILANG 3D", 960, 600)
setCamera(0, 6, 18, 0, -14)

create rot number = 0

while isOpen() do
    beginFrame()

    if wasKeyPressed("Escape") then
        shutdown()
    end

    create dt number = deltaMs()
    create move number = dt * 0.012
    create turn number = dt * 0.04
    updateFlyCamera(move, turn)

    set rot = rot + dt * 0.05
    setSceneRotation(rot)

    clearScreen(14, 16, 24)
    applyCamera()
    applySceneRotation()

    drawGrid(0, 24, 24)
    drawPlane(0, 24, 34, 38, 48)
    drawAxis(3)

    drawCube(0, 1, 0, 2, 100, 210, 255)
    drawCube(5, 0.75, -4, 1.5, 255, 120, 80)
    drawCube(-4, 0.5, 3, 1.2, 120, 230, 130)
    drawCube(3, 2.5, 5, 1, 220, 100, 220)

    drawSphere(-6, 1.2, -2, 1.2, 255, 220, 90)
    drawSphere(7, 1.5, 2, 1.5, 90, 180, 255)

    showFrame()
end
```

21. game3d_editor.afr

```
import "std/game3d"
use game3d

openWindow("AFRILANG Editeur 3D", 1024, 640)
setCamera(0, 12, 24, 0, -14)

loadGlb("pyramid", "examples/assets/pyramid_anim.glb")
loadModel("crate", "examples/assets/crate.obj")
loadTexture3d("grass", "examples/assets/grass.png")
loadLevel("examples/assets/demo_level.txt")

create brush number = 0
create rot number = 0
playGltfAnim("pyramid", 0, true)

while isOpen() do
    beginFrame()

    if wasKeyPressed("Escape") then
        shutdown()
    end

    if wasKeyPressed("E") then
        if isEditMode() then
            setEditMode(false)
        else
            setEditMode(true)
        end
    end

    if wasKeyPressed("1") then
        set brush = 0
    end
    if wasKeyPressed("2") then
        set brush = 1
    end

    if wasKeyPressed("S") then
        saveLevel("examples/assets/demo_level.txt")
    end
```

```
    if wasKeyPressed("L") then
        loadLevel("examples/assets/demo_level.txt")
    end

    create dt number = deltaMs()
    set rot = rot + dt * 0.01
    updateGltfAnims(dt)

    create move number = dt * 0.014
    create turn number = dt * 0.04
    updateFlyCamera(move, turn)

    if isEditMode() then
        if wasMouseClicked() then
            if pickGround(mouseX(), mouseY()) then
                if brush is equal to 0 then
                    addLevelGltf("pyramid", pickGroundX(), 0, pickGroundZ(), 2, 0)
                else
                    addLevelModel("crate", pickGroundX(), 0.5, pickGroundZ(), 1, 0)
                end
            end
        end
    end

    clearScreen(14, 18, 26)
    applyCamera()
    drawGrid(0, 24, 24)
    drawPlaneTextured(0, 24, "grass")
    drawLevel()

    showFrame()
end
```

22. game3d_net.afr

```
import "std/game3d"
use game3d

import "std/gamenet"
use gamenet

import "std/stopwatch"
use stopwatch

openWindow("AFRILANG 3D Net", 960, 600)
setCamera(0, 8, 18, 0, -10)

loadTexture3d("grass", "examples/assets/grass.png")

create isHost bool = yes
create px number = -2
create py number = 1
create pz number = 0
create rot number = 0

if isHost then
    hostGame(4789)
else
    joinGame("127.0.0.1", 4789)
end

while isOpen() do
    beginFrame()

    if wasKeyPressed("Escape") then
        shutdownNet()
        shutdown()
    end

    create dt number = deltaMs()
    create move number = dt * 0.012
    if isKeyDown("W") then
        set pz = pz - move * 8
    end
    if isKeyDown("S") then
        set pz = pz + move * 8
    end
    if isKeyDown("A") then
        set px = px - move * 8
    end
    if isKeyDown("D") then
        set px = px + move * 8
    end
    if isKeyDown("Q") then
        set rot = rot - move * 80
    end
    if isKeyDown("E") then
```

```
        set rot = rot + move * 80
    end

    sendPose(px, py, pz, rot)
    pollNet(nowMs(0))

    clearScreen(16, 20, 28)
    applyCamera()
    drawGrid(0, 16, 16)
    drawPlaneTextured(0, 16, "grass")

    drawSphere(px, py, pz, 0.45, 100, 220, 255)

    if hasRemotePlayer() then
        drawSphere(remoteX(), remoteY(), remoteZ(), 0.45, 255, 140, 100)
    end

    showFrame()
end
```

23. game3d_physics.afr

```
import "std/game3d"
use game3d

openWindow("AFRILANG 3D — physique", 960, 600)
setCamera(0, 8, 22, 0, -18)

loadTexture3d("grass", "examples/assets/grass.png")
loadTexture3d("crate", "examples/assets/crate.png")
loadModel("crate", "examples/assets/crate.obj")

createBody("ball1", -2, 8, 0, 0.6)
createBody("ball2", 0, 11, 1, 0.5)
createBody("ball3", 2, 9, -1, 0.7)
createBody("ball4", -1, 13, 2, 0.45)
createBody("ball5", 1.5, 15, -2, 0.55)

setBodyVelocity("ball1", 1, 0, 0.5)
setBodyVelocity("ball2", -0.5, 0, 0)
setBodyVelocity("ball3", 0.3, 0, -0.8)

create rot number = 0
create crateRot number = 0

while isOpen() do
    beginFrame()

    if wasKeyPressed("Escape") then
        shutdown()
    end

    if wasKeyPressed("Space") then
        createBody("spawn", 0, 12, 0, 0.5)
        setBodyVelocity("spawn", 0, 0, 0)
    end

    create dt number = deltaMs()
    set rot = rot + dt * 0.02
    set crateRot = crateRot + dt * 0.03
    setSceneRotation(rot)

    stepPhysics(dt, -18)

    create move number = dt * 0.01
    create turn number = dt * 0.035
    updateFlyCamera(move, turn)

    clearScreen(12, 14, 22)
    applyCamera()
    applySceneRotation()

    drawGrid(0, 20, 20)
    drawPlaneTextured(0, 20, "grass")
    drawAxis(2)

    drawModel("crate", 0, 0.5, 0, 1.2, crateRot, "crate")

    drawSphere(bodyX("ball1"), bodyY("ball1"), bodyZ("ball1"), 0.6, 100, 200, 255)
    drawSphere(bodyX("ball2"), bodyY("ball2"), bodyZ("ball2"), 0.5, 255, 120, 90)
    drawSphere(bodyX("ball3"), bodyY("ball3"), bodyZ("ball3"), 0.7, 120, 230, 130)
    drawSphere(bodyX("ball4"), bodyY("ball4"), bodyZ("ball4"), 0.45, 220, 100, 220)
    drawSphere(bodyX("ball5"), bodyY("ball5"), bodyZ("ball5"), 0.55, 255, 220, 90)

    if hasBody("spawn") then
        drawSphere(bodyX("spawn"), bodyY("spawn"), bodyZ("spawn"), 0.5, 240, 240, 255)
    end
```

```
    showFrame()
end
```

24. game_scroll.afr

```
import "std/game2d"
use game2d

import "std/ui"
use ui

configureGrid(50, 40, 24, 20, 60)
configureViewport(26, 18)

openWindow("Carte defilante", gridWindowWidth(), gridWindowHeight())

create px number = 25
create py number = 20

while isOpen() do
    beginFrame()

    if wasKeyPressed("Escape") then
        shutdown()
        closeWindow()
    end

    if isKeyDown("Right") then set px = px + 1 end
    if isKeyDown("Left") then set px = px - 1 end
    if isKeyDown("Down") then set py = py + 1 end
    if isKeyDown("Up") then set py = py - 1 end

    if px is less than 1 then set px = 1 end
    if py is less than 1 then set py = 1 end
    if px is greater than 48 then set px = 48 end
    if py is greater than 38 then set py = 38 end

    followCamera(cellWorldX(px) + 12, cellWorldY(py) + 12, 0.2)

    clearBackground(15, 18, 28)
    drawHud("Deplacez-vous avec les fleches", 20, 20, 20, 230, 230, 240)
    fillBoard(28, 32, 46)
    drawGridLines(42, 46, 58)
    drawWalls(70, 74, 92)
    fillCell(px, py, 120, 220, 255)
    showFrame()
end
```

25. game_triggers.afr

```
import "std/game2d"
use game2d

import "std/ui"
use ui

import "std/gamestate"
use gamestate

create W = 18
create H = 14
create CELL = 32

configureGrid(W, H, CELL, 24, 72)
configureViewport(W, H)

create px number = 2
create py number = 2
create goalX number = W - 3
create goalY number = H - 3

create gx number = cellWorldX(goalX)
create gy number = cellWorldY(goalY)
defineTrigger("goal", gx, gy, CELL, CELL)

setState("play")

openWindow("AFRILANG Triggers", gridWindowWidth(), gridWindowHeight())

while isOpen() do
    beginFrame()
    tickState(deltaMs())

    if wasKeyPressed("Escape") then
        shutdown()
```

```
        closeWindow()
    end

    if isState("play") then
        create spd number = 140
        create step number = spd * deltaMs() / 1000
        if isKeyDown("Left") then
            set px = px - step
        end
        if isKeyDown("Right") then
            set px = px + step
        end
        if isKeyDown("Up") then
            set py = py - step
        end
        if isKeyDown("Down") then
            set py = py + step
        end

        if px is less than 1 then
            set px = 1
        end
        if py is less than 1 then
            set py = 1
        end
        if px is greater than W - 2 then
            set px = W - 2
        end
        if py is greater than H - 2 then
            set py = H - 2
        end

        create pwx number = cellWorldX(px) + CELL / 2
        create pwy number = cellWorldY(py) + CELL / 2
        if pointInTrigger("goal", pwx, pwy) then
            setState("win")
        end
    end

    if isState("win") then
        if wasKeyPressed("R") then
            set px = 2
            set py = 2
            setState("play")
        end
    end

    clearBackground(18, 22, 32)
    drawHud("Triggers demo", 24, 20, 26, 240, 240, 250)
    drawHud("Fleches : joueur bleu  ·  Zone verte : sortie", 24, 52, 16, 150, 170, 200)

    fillBoard(30, 34, 48)
    drawWalls(70, 75, 95)
    fillCell(goalX, goalY, 60, 200, 120)

    create cx number = cellWorldX(px) + 6
    create cy number = cellWorldY(py) + 6
    fillCircleSolid(cx, cy, 10, 80, 160, 255)

    if isState("win") then
        drawCenteredText("VICTOIRE !", gridWindowHeight() / 2 - 20, 38, 100, 255, 160)
        drawCenteredText("R pour rejouer", gridWindowHeight() / 2 + 24, 22, 210, 210, 230)
    end

    drawFps(gridWindowWidth() - 72, gridWindowHeight() - 28)
    showFrame()
end
```

26. generic_class.afr

```
class Box<T>
    public field data T

    function init(v T)
        set this.data = v
    end

    function get() returns T
        return this.data
    end
end

create numbers = new Box<number>(42)
say numbers.get()

create label = new Box<text>("hello")
say label.get()
```

```
say "generic class complete"
```

27. generics.afr

```
function identity<T>(x T) returns T
    return x
end

function first<T>(items list of T) returns T
    return items at 0
end

say identity(42)
say identity("hello")

create nums = list of 10, 20, 30
say first(nums)

create words = list of "a", "b", "c"
say first(words)
```

28. giskit_demo.afr

```
import "std/giskit001"
use giskit001

import "std/giskit025"
use giskit025

import "std/giskit042"
use giskit042

import "std/geo"
use geo

create lat1 number = 48.8566
create lon1 number = 2.3522
create lat2 number = 45.7640
create lon2 number = 4.8357

create bearing number = bearingDeg(lat1, lon1, lat2, lon2)
create distKm number = haversineKm(lat1, lon1, lat2, lon2)
create ndviVal number = ndvi(0.82, 0.21)
create gsd number = gsdFromAltitude(1200, 35, 4.5)

say "Paris -> Lyon"
say "Distance km: {distKm}"
say "Cap: {bearing} deg"
say "NDVI demo: {ndviVal}"
say "GSD m: {gsd}"
```

29. gisultra_demo.afr

```
import "std/c/gisultra001"
use gisultra001

create nir list number = list of 0.8, 0.7, 0.1, 0.2
create red list number = list of 0.2, 0.3, 0.1, 0.4

create nd = ndviRaster(nir, red)
say "NDVI[0]={nd at 0} NDVI[1]={nd at 1} NDVI[2]={nd at 2} NDVI[3]={nd at 3}"

create w number = 3
create h number = 3
create grid list number = list of 1, 2, 3, 4, 5, 6, 7, 8, 9
create m number = rasterWindowMean(grid, w, h, 1, 1, 1)
say "Mean window center r=1: {m}"

create slope list number = list of 10, 15, 20, 30
create aspect list number = list of 0, 90, 180, 270
create hs = hillshadeRaster(slope, aspect)
say "Hillshade[0]={hs at 0} Hillshade[1]={hs at 1} Hillshade[2]={hs at 2} Hillshade[3]={hs at 3}"
```

30. gui_demo.afr

```
open window titled "AFRILANG GUI" with width 640, height 480

while window is open do
    clear background color 28, 28, 36
    draw text "Bonjour depuis AFRILANG!" at 60, 80 size 32
    draw text "Cliquez sur Quitter pour fermer." at 60, 130 size 20
    if button "Quitter" at 220, 350 width 200 height 50 is clicked then
        close window
    end
    show frame
end
```

31. hello.afr

```
say "Bonjour depuis AFRILANG!"
say "Le compilateur fonctionne."

repeat 3 times
    say "Hello"
end
```

32. helpers.afr

```
function sum2(a number, b number) returns number
    return a + b
end
```

33. iaultra_demo.afr

```
import "std/c/iaultra001"
use iaultra001

create logits list number = list of 1, 3, 2
create probs = softmax(logits)
create features list number = list of 1, 2, 3
create weights list number = list of 0.2, -0.1, 0.5
create ytrue list number = list of 1, 0, 1, 1, 0
create ypred list number = list of 1, 0, 0, 1, 0

say "softmax={probs at 0} {probs at 1} {probs at 2} argmax={argmax(probs)}"
say "predict={linearPredict(weights, features, 0.1)} sigmoid={sigmoid(0.5)}"
say "cosine={cosineSim(features, weights)} entropy={entropy(probs)}"
say "acc={accuracy(ytrue, ypred)} prec={precisionScore(ytrue, ypred)} f1={f1Score(ytrue, ypred)}"

create step = sgdStep(weights, features, 1, 0)
say "sgd w0={step at 0} w1={step at 1}"

create matrix list number = list of 0, 0, 1, 0, 0, 1, 1, 1
say "nearest={nearestIdx(features, matrix, 3)} defaultK={defaultK(0)}"
```

34. inheritance.afr

```
class Animal
    function speak()
        say "..."
    end
end

class Dog extends Animal
    function speak()
        say "Woof!"
    end

    function breed() returns text
        return "Labrador"
    end
end

class Cat extends Animal
    function speak()
        say "Meow!"
    end
end

create animals = new Dog
animals.speak()
say animals.breed()

create cat = new Cat
cat.speak()

create animal = new Animal
animal.speak()
```

35. interfaces.afr

```
interface Speakable
    function speak()
end

interface Named
    function getName() returns text
end

class Animal
    function speak()
        say "..."
    end
end

class Dog extends Animal implements Speakable, Named
    function speak()
        say "Woof!"
    end

    function getName() returns text
        return "Rex"
    end
end

create dog = new Dog
dog.speak()
say dog.getName()
```

36. lambdas.afr

```
create doubleIt = function(x number) returns number
    return x * 2
end

create addFive = function(x number) returns number
    return x + 5
end

say doubleIt(21)
say addFive(10)

function apply(fn function number to number, value number) returns number
    return fn(value)
end

say apply(doubleIt, 10)
say apply(addFive, 100)

create pipeline = function(x number) returns number
    return apply(addFive, apply(doubleIt, x))
end

say pipeline(3)

create greet = function(name text) returns text
    return "Hello, " + name
end

say greet("AFRILANG")

create runTwice = function(fn function number to number, x number) returns number
    return fn(fn(x))
end

say runTwice(doubleIt, 2)
```

37. language_demo.afr

```
module Calcul
    export function add(a number, b number) returns number
        return a + b
    end

    private function scale(x number) returns number
        return x * 2
    end

    export function twice(x number) returns number
        return scale(x)
    end
end

function identity<T>(x T) returns T
```

```
    return x
end

enum Status
    case Ok
    case Error with message text
end

use Calcul

dire Calcul.add(3, 4)
dire Calcul.twice(10)
dire identity<number>(42)
dire identity<text>("bonjour")

create e = Status.Error with "echec"

match e
    cas Error avec msg alors
        dire msg
    fin
    defaut
        dire "autre"
    fin
fin
```

38. list_ops.afr

```
import "std/collections"
import "std/str"

use collections
use str

create nums = list of 1, 2, 3, 4, 5

create doubled = mapNumbers(nums, function(x number) returns number
    return x * 2
end)

say doubled at 0
say doubled at 4

create bigOnes = filterNumbers(nums, function(x number) returns bool
    if x is greater than 3 then
        return yes
    else
        return no
    end
end)

say length of bigOnes
say bigOnes at 0
say bigOnes at 1

create total = reduceNumbers(nums, function(acc number, x number) returns number
    return acc + x
end, 0)

say total

create words = list of "hi", "hello", "hey"

create upper = mapText(words, function(w text) returns text
    return w + "!"
end)

say upper at 0
say upper at 2

create longWords = filterText(words, function(w text) returns bool
    if contains(w, "el") then
        return yes
    else
        return no
    end
end)

say length of longWords
say longWords at 0

say "list ops complete"
```

39. lists.afr

```
create nums = list of 1, 2, 3
say nums at 0
say nums at 1
say length of nums

set nums at 1 = 99
say nums at 1

add 4 to nums
say length of nums

create bracket = [10, 20, 30]
say bracket at 0
say bracket[2]

create nums2 = empty list number
add 42 to nums2
say length of nums2

create names = list of "Alice", "Bob"
for each name in names do
    say name
end

create flags = list of true, false, yes, no
say flags at 0
say flags at 2
```

40. maps.afr

```
create scores = map of "alice" to 95, "bob" to 88, "carol" to 91

say scores at "alice"
say length of scores

set scores at "dave" = 76
say scores at "dave"

for each name, score in scores do
    say name
    say score
end

create emptyMap = empty map text to number
say length of emptyMap
```

41. match_expr_demo.afr

```
enum Status
    case Ok
    case Error with message text
end

create s = Status.Ok
create e = Status.Error with "failed"

create okLabel = match s
    case Ok then "success"
    end
    default "unknown"
    end
end

create errLabel = match e
    case Error with msg then msg
    end
    default "no error"
    end
end

create code = match s
    case Ok then 0
    end
    case Error with msg then 1
    end
end

say okLabel
say errLabel
say code
```

42. medium_libs_demo.afr

```
import "std/m/textstats"
import "std/m/numstats"
import "std/m/uriparse"
import "std/m/datecalc"
import "std/format"

use textstats
use numstats
use uriparse
use datecalc
use format

create words = wordCount("bonjour le monde afrilang")
say formatNumber(words, 0)

create nums = list of 3, 7, 2, 9, 4
create avg = mean(nums)
say formatNumber(avg, 2)

create host = getHost("https://example.com/path?q=1")
say host

create days = daysBetween("2024-01-01", "2024-12-31")
say formatNumber(days, 0)
```

43. modules.afr

```
module Math
    function add(a number, b number) returns number
        return a + b
    end

    function multiply(a number, b number) returns number
        return a * b
    end
end

use Math

say add(3, 4)
say multiply(5, 6)
```

44. natural.afr

```
say "Hello from AFRILANG natural syntax!"

if 42 is greater than 40 then
    say "The condition is true"
else
    say "Impossible"
end

repeat 3 times
    say "Hello"
end
```

45. natural_list_ops.afr

```
create nums = list of 1, 2, 3, 4, 5

create doubled = map each x in nums do
    return x * 2
end

say doubled at 0
say doubled at 4

create bigOnes = filter each x in nums where x is greater than 3

say length of bigOnes
say bigOnes at 0
say bigOnes at 1

create total = reduce nums from 0 with each acc, x do
    return acc + x
end

say total

create words = list of "hi", "hello", "hey"

create upper = map each w in words do
    return w + "!"
```

```
end

say upper at 0
say upper at 2

create longWords = filter each w in words where w is equal to "hello"

say length of longWords
say longWords at 0

create joined = reduce words from "" with each acc, w do
    return acc + w + ","
end

say joined

create expanded = flatMap each n in nums do
    return list of n, n * 100
end

say length of expanded
say expanded at 3

say "natural list ops complete"
```

46. oop.afr

```
class Person
    function speak()
        say "Bonjour, je suis une Personne"
    end

    function greet()
        say "Bienvenue en AFRILANG!"
    end
end

class Dog
    function speak()
        say "Woof!"
    end
end

create person = new Person
person.speak()
person.greet()

create dog = new Dog
dog.speak()
```

47. oop_full.afr

```
abstract class Shape
    abstract function area() returns number
end

class Circle extends Shape
    public field radius number

    function init(r number)
        set this.radius = r
    end

    function area() returns number
        return 3.14159 * radius * radius
    end
end

class Counter
    static field count number

    static function nextId() returns number
        set Counter.count = Counter.count + 1
        return Counter.count
    end
end

class Animal
    protected field species text

    function init(aSpecies text)
        set this.species = aSpecies
    end

    function describe()
        say species
```

```
    end
end

class Dog extends Animal
    public field name text

    function init(aSpecies text, aName text)
        super(aSpecies)
        set this.name = aName
    end

    function describe()
        super.describe()
        say name + " (dog)"
    end
end

create circle = new Circle(5)
say circle.area()

say Counter.nextId()
say Counter.nextId()

create rex = new Dog("Canis", "Rex")
rex.describe()

create pet Animal = new Dog("Canis", "Buddy")
pet.describe()

say "OOP full demo complete"
```

48. operators_demo.afr

```
class Vector
    public field x number
    public field y number

    function init(ax number, ay number)
        set this.x = ax
        set this.y = ay
    end

    operator + (other Vector) returns Vector
        return new Vector(x + other.x, y + other.y)
    end
end

create a = new Vector(1, 2)
create b = new Vector(3, 4)
create c = a + b

say c.x
say c.y
```

49. paht.afr

```
say "=== PAHT calculator ==="
say ""

ask "Last name: " into lastName
ask "First name: " into firstName
ask "Unit price (TTC): " into unitPrice as number
ask "VAT rate (%): " into vat as number

create paht = unitPrice / (1 + vat / 100)

say ""
say "Client: " + firstName + " " + lastName
say "Unit price TTC: " + unitPrice
say "VAT: " + vat + " %"
say "PAHT (purchase price excl. tax): " + paht
```

50. phase10_demo.afr

```
import "std/collections"
import "std/str"
import "std/math"

use collections
use str
use math

function greet(name text, prefix text = "Hello") returns text
    return "{prefix} {name}!"
end
```

```
say greet("World")
say greet("Africa", "Bonjour")

create const PI = 3.14159
say PI

create nums = list of 3, 1, 4, 1, 5
create sorted = sortNumbers(nums)
say sorted at 0
say sorted at 4
say sumNumbers(nums)
say containsNumber(nums, 4)
say indexOfNumber(nums, 5)

create words = list of "zebra", "apple", "mango"
create ordered = sortText(words)
say ordered at 0
say containsText(words, "apple")

create total = 0
for i from 1 to 5 do
    set total = total + i
end
say total

for n from 0 to 10 step 2 do
    say n
end

say "Phase 10 demo complete"
```

51. phase11_demo.afr

```
class Animal
    function speak()
        say "..."
    end
end

class Dog extends Animal
    function speak()
        say "Woof!"
    end
end

class Box<T>
    public field data T

    function init(v T)
        set this.data = v
    end

    function get() returns T
        return this.data
    end
end

final class AppConfig
    property text title
    end

    function init(t text)
        set this.title = t
    end

    destroy
        say "config cleanup"
    end
end

create nums = list of 1, 2, 3

create expanded = flatMap each n in nums do
    return list of n, n * 10
end

say length of expanded
say expanded at 0
say expanded at 1
say expanded at 2

create words = list of "a", "bb", "ccc"

create joined = reduce words from "" with each acc, w do
    return acc + w + "-"
end
```

```
say joined

create pets list of Animal = list of new Dog(), new Dog()

for each pet in pets do
    pet.speak()
end

create box = new Box<number>(99)
say box.get()

create cfg = new AppConfig("AFRILANG")
say cfg.title

import "std/args"
import "std/path"
use args
use path

say count()
say join("examples", "phase11_demo.afr")

say "phase 11 complete"
```

52. phase9_demo.afr

```
import "std/str"
import "std/log"
import "std/math"
import "std/time"

use str
use logging
use math
use chrono

create name = "AFRILANG"
create version = 1
say "Welcome to {name} v{version}!"

info("starting demo")
warn("this is a warning")

create value = pow(2, 10)
say abs(-42)
say floor(3.9)
say value

create nowTs = now()
say formatTimestamp(nowTs)

say trim("  hello  ")
say contains("hello world", "world")

create parts = split("a,b,c", ",")
say join(parts, "-")
```

53. pkg_demo.afr

```
import "pkg/math/math.afr"

use Math

say square(5)
say twice(21)
say pi()
```

54. polymorphic_list.afr

```
class Animal
    function speak()
        say "..."
    end
end

class Dog extends Animal
    function speak()
        say "Woof!"
    end
end

class Cat extends Animal
    function speak()
        say "Meow!"
```

```
    end
end

create pets list of Animal = list of new Dog(), new Cat()

say length of pets

for each pet in pets do
    pet.speak()
end

add new Dog() to pets

say length of pets

say "polymorphic lists complete"
```

55. poo_advanced.afr

```
final class Config
    property text name
    end

    function init(n text)
        set this.name = n
    end

    destroy
        say "config destroyed"
    end
end

create cfg = new Config("app")
say cfg.name
set cfg.name = "server"
say cfg.name

say "poo advanced complete"
```

56. poo_demo.afr

```
class Animal
    function speak()
        say "..."
    end
end

class Dog extends Animal
    public field name text

    function init(aName text)
        set this.name = aName
    end

    function speak()
        say name + " says Woof!"
    end
end

class Cat extends Animal
    function speak()
        say "Meow!"
    end
end

create rex = new Dog("Rex")
rex.speak()

create whiskers = new Cat
whiskers.speak()

say "POO demo complete"
```

57. rasterultra_demo.afr

```
import "std/c/rasterultra001"
use rasterultra001

create w number = 4
create h number = 3
create grid list number = list of 0, 0, 0, 0,
                           0, 1, 1, 0,
                           0, 0, 0, 0

create blurred = boxBlur3(grid, w, h)
```

```
create edges = sobelMag(blurred, w, h)
create mask = threshold01(edges, w, h, 0.2)

say "mask[0]={mask at 0} mask[1]={mask at 1} mask[2]={mask at 2} mask[3]={mask at 3}"
say "mask[4]={mask at 4} mask[5]={mask at 5} mask[6]={mask at 6} mask[7]={mask at 7}"

create nir list number = list of 0.8, 0.7, 0.1, 0.2
create red list number = list of 0.2, 0.3, 0.1, 0.4
create ndm = ndviMask(nir, red, 0.2)
say "ndviMask: {ndm at 0} {ndm at 1} {ndm at 2} {ndm at 3}"
```

58. records.afr

```
record Point
    field x number
    field y number
end

create px number = 10
create py number = 20

say px
say py
say px + py

create dist number = px * px + py * py
say dist
```

59. result.afr

```
function divide(a number, b number) returns number or error
    if b is equal to 0 then
        return error "Division par zero"
    end
    return a / b
end

create ok = divide(10, 2)
if ok is error then
    say ok.message
else
    say ok.value
end

create bad = divide(10, 0)
if bad is error then
    say bad.message
else
    say bad.value
end
```

60. segultra_demo.afr

```
import "std/c/segultra001"
use segultra001

create w number = 6
create h number = 4

create grid list number = list of 0, 0, 0, 0, 0, 0,
                           0, 1, 1, 0, 1, 0,
                           0, 1, 0, 0, 1, 0,
                           0, 0, 0, 0, 0, 0

create mask = thresholdMask(grid, w, h, 0.5)
say "components4={countComponents4(mask, w, h)} components8={countComponents8(mask, w, h)} components={count
Components(mask, w, h)}"

create d = dilate3(mask, w, h)
create e = erode3(d, w, h)
say "after dilate+erode components={countComponents(e, w, h)}"

say "floodSize at (1,1) = {floodSize(mask, w, h, 1, 1)}"
```

61. simple_libs_demo.afr

```
import "std/celsius"
import "std/upper"
import "std/trig"
import "std/hex"
import "std/i18n"
import "std/quiz"
import "std/format"

use celsius
use upper
use trig
use hex
use i18n
use quiz
use format

create tempF = cToF(20)
say formatNumber(tempF, 1)

create greeting = greetFr("AFRILANG")
say greeting

create label = toUpper("bonjour")
say label

create angle = sinDeg(90)
say formatNumber(angle, 4)

create code = toHex(255)
say code

create score = scorePercent(8, 10)
say formatNumber(score, 1)
```

62. snake.afr

```
import "snake_logic.afr"

import "std/game2d"
use game2d

import "std/ui"
use ui

import "std/game"
use game

import "std/gamestate"
use gamestate

import "std/m/random2"
use random2

import "std/stopwatch"
use stopwatch

create W = 36
create H = 28
create VIEW_W = 22
create VIEW_H = 16

create SAVE_PATH = "snake_save.txt"

create difficulty number = 1
create volume number = 110
create showDebug bool = no
create streak number = 0
create flashRecord number = 0
create justStartedPlay bool = no
create wantFreshPlay bool = no
create warmupMoves number = 0

create xs list number = list of 0, 0, 0
create ys list number = list of 0, 0, 0
create snakeLen number = 3
create dirX number = 1
create dirY number = 0
create score number = 0
create fx number = 1
create fy number = 1

function drawWallsLayer(w number, h number)
    create y number = 0
    while y is less than h do
        create x number = 0
        while x is less than w do
            if isBorderCell(x, y) then
```

```
                if hasSprite("wall") then
                    drawSpriteCell("wall", x, y)
                else
                    create edge number = 62 + ((x + y) / 3) * 2
                    fillCell(x, y, edge - 8, edge, edge + 18)
                end
            end
            set x = x + 1
        end
        set y = y + 1
    end
end

function drawSnake(xs list number, ys list number, snakeLen number)
    create i number = 0
    while i is less than snakeLen do
        if i is equal to 0 then
            if hasSprite("head") then
                drawSpriteCell("head", xs at i, ys at i)
            else
                create pulse number = pulse01(350)
                create radius number = 10 + pulse * 5
                create cx number = cellWorldX(xs at i) + 14
                create cy number = cellWorldY(ys at i) + 14
                fillCircleSolid(cx, cy, radius + 6, 20, 80, 50)
                fillCircleSolid(cx, cy, radius, 80, 255, 150)
            end
        else
            if hasSprite("body") then
                drawSpriteCell("body", xs at i, ys at i)
            else
                create shade number = 100 - i * 4
                if shade is less than 50 then
                    set shade = 50
                end
                fillCellRgb(xs at i, ys at i, rgb(35, shade, shade + 25))
            end
        end
        set i = i + 1
    end
end

function drawFood(fx number, fy number)
    if hasSprite("food") then
        drawSpriteCell("food", fx, fy)
    else
        create pulse number = pulse01(260)
        create cx number = cellWorldX(fx) + 14
        create cy number = cellWorldY(fy) + 14
        create radius number = 9 + pulse * 5
        fillCircleSolid(cx, cy, radius + 8, 255, 60, 90)
        fillCircleSolid(cx, cy, radius + 3, 180, 40, 70)
        fillCircleSolid(cx, cy, radius, 255, 120, 140)
    end
end

function drawMoveBar(moveMs number, x number, y number, w number)
    create fill number = pulse01(moveMs)
    create filled number = w * fill
    fillRect(x, y, w, 6, 30, 34, 52)
    fillRect(x, y, filled, 6, 90, 220, 160)
end

function drawMenuOverlay()
    create panelX number = gridWindowWidth() / 2 - 150
    create panelY number = gridWindowHeight() / 2 - 160
    fillRect(panelX, panelY, 300, 320, 16, 20, 36)
    drawRect(panelX, panelY, 300, 320, 70, 110, 200)
end

seedRandom(nowMs(0))

configureGrid(W, H, 28, 32, 88)
configureViewport(VIEW_W, VIEW_H)

create initPack = resetGame(W, H)
set snakeLen = initPack at 0
set dirX = initPack at 1
set dirY = initPack at 2
set score = initPack at 3
set fx = initPack at 4
set fy = initPack at 5
set xs at 0 = initPack at 6
set xs at 1 = initPack at 7
set xs at 2 = initPack at 8
set ys at 0 = initPack at 9
set ys at 1 = initPack at 10
set ys at 2 = initPack at 11
```

```
syncDirection(dirX, dirY)

loadHighScore(SAVE_PATH)
setState("menu")

openWindow("AFRILANG Snake", gridWindowWidth(), gridWindowHeight())

loadSprite("head", "examples/assets/snake_head.png")
loadSprite("body", "examples/assets/snake_body.png")
loadSprite("food", "examples/assets/food.png")
loadSprite("wall", "examples/assets/wall.png")
loadSound("eat", "examples/assets/eat.wav")

while isOpen() do
    beginFrame()
    tickState(deltaMs())

    if flashRecord is greater than 0 then
        set flashRecord = flashRecord - deltaMs()
    end

    if wasKeyPressed("Escape") then
        shutdown()
        closeWindow()
    end

    if wasKeyPressed("F1") then
        if showDebug then
            set showDebug = no
        else
            set showDebug = yes
        end
    end

    if isState("play") then
        if wasKeyPressed("P") then
            setState("pause")
        end
    end
    if isState("pause") then
        if wasKeyPressed("P") then
            setState("play")
        end
    end

    if isState("menu") then
        if wasKeyPressed("Space") then
            set wantFreshPlay = yes
            setState("play")
        end
        if wasKeyPressed("Enter") then
            set wantFreshPlay = yes
            setState("play")
        end
    end

    if isState("play") then
        if wasKeyPressed("Space") then
            if shouldPauseOnSpace(wasStateChanged()) then
                setState("pause")
            end
        end
    end

    if isState("pause") then
        if wasKeyPressed("Space") then setState("play") end
    end

    if isState("gameover") then
        set streak = 0
        if wasKeyPressed("R") then
            set wantFreshPlay = yes
            setState("play")
        end
    end

    if isState("play") then
        if wasStateChanged() then
            resetTimer("move")
            if wantFreshPlay then
                create pack = resetGame(W, H)
                set snakeLen = pack at 0
                set dirX = pack at 1
                set dirY = pack at 2
                set score = pack at 3
                set fx = pack at 4
                set fy = pack at 5
                set xs at 0 = pack at 6
```

```
                set xs at 1 = pack at 7
                set xs at 2 = pack at 8
                set ys at 0 = pack at 9
                set ys at 1 = pack at 10
                set ys at 2 = pack at 11
                set justStartedPlay = yes
                set warmupMoves = 2
                syncDirection(dirX, dirY)
                set wantFreshPlay = no
            end
        end
        if isPlayStateValid(xs, ys, snakeLen, W, H) is equal to false then
            create repair = resetGame(W, H)
            set snakeLen = repair at 0
            set dirX = repair at 1
            set dirY = repair at 2
            set score = repair at 3
            set fx = repair at 4
            set fy = repair at 5
            set xs at 0 = repair at 6
            set xs at 1 = repair at 7
            set xs at 2 = repair at 8
            set ys at 0 = repair at 9
... [172 lignes]
```

63. snake_logic.afr

```
import "std/game"
use game

import "std/game2d"
use game2d

import "std/m/random2"
use random2

function snakeOccupies(xs list number, ys list number, count number, gx number, gy number) returns bool
    create i number = 0
    while i is less than count do
        if xs at i is equal to gx then
            if ys at i is equal to gy then
                return yes
            end
        end
        set i = i + 1
    end
    return no
end

function isWallCell(col number, row number, w number, h number) returns bool
    if col is less than 1 then return yes end
    if row is less than 1 then return yes end
    if col is greater than w - 2 then return yes end
    if row is greater than h - 2 then return yes end
    return no
end

function foodNotOnSnake(xs list number, ys list number, snakeLen number, fx number, fy number) returns bool
    if snakeOccupies(xs, ys, snakeLen, fx, fy) then
        return no
    end
    return yes
end

function placeFood(xs list number, ys list number, snakeLen number, w number, h number) returns list number
    create fx number = 1
    create fy number = 1
    repeat 400 times
        set fx = randomRange(1, w - 2)
        set fy = randomRange(1, h - 2)
        if snakeOccupies(xs, ys, snakeLen, fx, fy) is equal to false then
            return list of fx, fy
        end
    end
    return list of fx, fy
end

function snakeStartCoords(w number, h number) returns list number
    create startX number = w / 2
    create startY number = h / 2
    return list of startX, startX - 1, startX - 2, startY, startY, startY
end

function resetGame(w number, h number) returns list number
    create snakeLen number = 3
    create dirX number = 1
    create dirY number = 0
```

```
    create score number = 0

    create coords = snakeStartCoords(w, h)
    create x0 number = coords at 0
    create x1 number = coords at 1
    create x2 number = coords at 2
    create y0 number = coords at 3
    create y1 number = coords at 4
    create y2 number = coords at 5
    create tx list number = list of x0, x1, x2
    create ty list number = list of y0, y1, y2

    resetTimer("move")
    create food0 = placeFood(tx, ty, snakeLen, w, h)
    create fx number = food0 at 0
    create fy number = food0 at 1
    return list of snakeLen, dirX, dirY, score, fx, fy, x0, x1, x2, y0, y1, y2
end

function isMoveSafe(xs list number, ys list number, snakeLen number, w number, h number, dx number, dy numbe
r) returns bool
    if dx is equal to 0 then
        if dy is equal to 0 then
            return no
        end
    end
    create headX number = xs at 0
    create headY number = ys at 0
    create newX number = headX + dx
    create newY number = headY + dy
    if isWallCell(newX, newY, w, h) then
        return no
    end
    create bodyLen number = snakeLen - 1
    if snakeOccupies(xs, ys, bodyLen, newX, newY) then
        return no
    end
    return yes
end

function pickSafeDirection(xs list number, ys list number, snakeLen number, w number, h number, curDx number
, curDy number, wantDx number, wantDy number) returns list number
    if isMoveSafe(xs, ys, snakeLen, w, h, wantDx, wantDy) then
        return list of wantDx, wantDy
    end
    if isMoveSafe(xs, ys, snakeLen, w, h, curDx, curDy) then
        return list of curDx, curDy
    end
    return list of 1, 0
end

function moveMsForDifficulty(score number, difficulty number) returns number
    create level number = levelFromXp(xpForScore(score))
    create moveMs number = speedMsForLevel(level)
    if difficulty is equal to 2 then set moveMs = moveMs * 0.82 end
    if difficulty is equal to 3 then set moveMs = moveMs * 0.68 end
    return moveMs
end

function isPlayStateValid(xs list number, ys list number, snakeLen number, w number, h number) returns bool
    if snakeLen is less than 1 then
        return no
    end
    if isWallCell(xs at 0, ys at 0, w, h) then
        return no
    end
    return yes
end

function firstMoveIsSafe(pack list number, w number, h number) returns bool
    create hx number = pack at 6
    create hy number = pack at 9
    create dirX number = pack at 1
    create dirY number = pack at 2
    create newX number = hx + dirX
    create newY number = hy + dirY
    if isWallCell(newX, newY, w, h) then
        return no
    end
    create tx list number = list of pack at 6, pack at 7, pack at 8
    create ty list number = list of pack at 9, pack at 10, pack at 11
    if snakeOccupies(tx, ty, 2, newX, newY) then
        return no
    end
    return yes
end

function shouldPauseOnSpace(stateChanged bool) returns bool
```

```
    if stateChanged then
        return no
    end
    return yes
end

function canMoveAfterCountdown(stateMs number) returns bool
    if stateMs is less than 2500 then
        return no
    end
    return yes
end

function countdownLabel(stateMs number) returns text
    if stateMs is less than 500 then
        return "3"
    end
    if stateMs is less than 1000 then
        return "2"
    end
    if stateMs is less than 1500 then
        return "1"
    end
    if stateMs is less than 2500 then
        return "GO!"
    end
    return ""
end
```

64. snake_test.afr

```
import "snake_logic.afr"

import "std/m/random2"
use random2

import "std/stopwatch"
use stopwatch

test "snakeOccupies detects head"
    create xs list number = list of 5, 4, 3
    create ys list number = list of 5, 5, 5
    assert snakeOccupies(xs, ys, 3, 5, 5) is equal to true
end

test "snakeOccupies ignores empty cell"
    create xs list number = list of 5
    create ys list number = list of 5
    assert snakeOccupies(xs, ys, 1, 9, 9) is equal to false
end

test "isWallCell detects borders"
    create w number = 12
    create h number = 10
    assert isWallCell(0, 3, w, h) is equal to true
    assert isWallCell(5, 5, w, h) is equal to false
end

test "resetGame pack shape"
    seedRandom(42)
    create pack = resetGame(12, 10)
    assert pack at 0 is equal to 3
    assert pack at 1 is equal to 1
    assert pack at 3 is equal to 0
    assert pack at 6 is equal to 6
    assert pack at 9 is equal to 5
end

test "first move after reset is safe"
    seedRandom(42)
    create pack = resetGame(36, 28)
    assert firstMoveIsSafe(pack, 36, 28) is equal to true
    create xs list number = list of pack at 6, pack at 7, pack at 8
    create ys list number = list of pack at 9, pack at 10, pack at 11
    assert isPlayStateValid(xs, ys, 3, 36, 28) is equal to true
end

test "placeFood avoids snake body"
    seedRandom(99)
    create xs list number = list of 4, 3, 2
    create ys list number = list of 4, 4, 4
    create food = placeFood(xs, ys, 3, 12, 10)
    assert foodNotOnSnake(xs, ys, 3, food at 0, food at 1) is equal to true
end

test "tail cell not counted in short body check"
    create xs list number = list of 4, 5, 3
```

```
    create ys list number = list of 5, 5, 5
    assert snakeOccupies(xs, ys, 2, 3, 5) is equal to false
end

test "moveMsForDifficulty scales speed"
    assert moveMsForDifficulty(0, 3) is less than moveMsForDifficulty(0, 1)
end

test "shouldPauseOnSpace blocks same frame"
    assert shouldPauseOnSpace(yes) is equal to false
    assert shouldPauseOnSpace(no) is equal to true
end

test "countdown blocks early movement"
    assert canMoveAfterCountdown(0) is equal to false
    assert canMoveAfterCountdown(2000) is equal to false
    assert canMoveAfterCountdown(2600) is equal to true
end

test "countdown labels sequence"
    assert countdownLabel(100) is equal to "3"
    assert countdownLabel(600) is equal to "2"
    assert countdownLabel(1200) is equal to "1"
    assert countdownLabel(1800) is equal to "GO!"
    assert countdownLabel(2600) is equal to ""
end

test "unsafe left turn keeps moving right"
    create xs list number = list of 20, 19, 18
    create ys list number = list of 14, 14, 14
    create pick = pickSafeDirection(xs, ys, 3, 36, 28, 1, 0, -1, 0)
    assert pick at 0 is equal to 1
    assert pick at 1 is equal to 0
end

test "safe turn up is accepted"
    create xs list number = list of 20, 19, 18
    create ys list number = list of 14, 14, 14
    create pick = pickSafeDirection(xs, ys, 3, 36, 28, 1, 0, 0, -1)
    assert pick at 0 is equal to 0
    assert pick at 1 is equal to -1
end

test "zero direction is never safe"
    create xs list number = list of 18, 17, 16
    create ys list number = list of 14, 14, 14
    assert isMoveSafe(xs, ys, 3, 36, 28, 0, 0) is equal to false
end

say "Snake logic tests OK"
```

65. stdlib_demo.afr

```
import "std/io"
import "std/json"

use io
use json

create doc json = makeObject("lang", "AFRILANG")
say getString(doc, "lang")

create roundtrip json = parse(stringify(doc))
say getString(roundtrip, "lang")

create numDoc json = parse("{\"value\": 42}")
say getNumber(numDoc, "value")

create ok = fileExists("/tmp/afrilang_test.txt")
if ok is equal to false then
    writeFile("/tmp/afrilang_test.txt", "Hello from stdlib io")
end

create fileContent = readFile("/tmp/afrilang_test.txt")
say fileContent
```

66. test_game2dkit.afr

```
import "std/game2dkit001"
use game2dkit001

create hit bool = aabbHit(0, 0, 10, 10, 5, 5, 10, 10)
create d number = dist2(0, 0, 3, 4)
```

67. test_game_module.afr

```
import "std/game"
use game

create s number = 5
create xp number = xpForScore(s)
create lvl number = levelFromXp(xp)
create ms number = speedMsForLevel(lvl)
```

68. test_game_module2.afr

```
import "std/game"
use game

create a number = game.xpForScore(3)
create b number = xpForScore(3)
```

69. test_game_module3.afr

```
import "std/game"
use game

create lvl number = levelFromXp(1000)
```

70. test_game_module4.afr

```
import "std/game"
use game

create ms number = speedMsForLevel(2)
```

71. test_gameultra.afr

```
import "std/c/gameultra001"
use gameultra001

create w number = 5
create h number = 5
create grid list number = list of 0,0,0,0,0, 0,1,1,1,0, 0,0,0,1,0, 0,1,0,0,0, 0,0,0,0,0

create path list number = astarGridPath(grid, w, h, 0, 0, 4, 4, 0)
create simp list number = simplifyGridPath(path, w)
create cnt number = floodCount(grid, w, h, 0, 0, 0)
```

72. test_gameultra3d.afr

```
import "std/c/gameultra3d001"
use gameultra3d001

create pos list number = list of 0,0,0, 2,0,0, 1,0,2
create vel list number = list of 0,0,0, 0,0,0, 0,0,0
create nv list number = boidsVelStep(pos, vel, 3, 0.016, 3, 1, 4)
create np list number = integratePos(pos, nv, 3, 0.016)
create hit list number = raySphereHit(0, 2, 5, 0, -0.3, -1, 1, 0, 0, 1)
```

73. tests.afr

```
function add(a number, b number) returns number
    return a + b
end

test "addition works"
    create r = add(2, 3)
    assert r is equal to 5
end

test "boolean logic"
    create flag bool = true
    assert flag is equal to true
end
```

```
say "All tests defined"
```

74. tier10_demo.afr

```
say "Phase I — LSP & benchmark OK"

function double(x number) returns number
    return x * 2
end

create message text = "outline"
say double(21)
say message
```

75. tier1_demo.afr

```
import "std/json"

use json

create age int = 42
create pi number = 3.14
say age
say pi

create doc json = parse("{\"name\": \"AFRILANG\", \"version\": 1}")
say getString(doc, "name")
say getInt(doc, "version")

create built json = makeObject("status", "ok")
say stringify(built)
```

76. tier2_demo.afr

```
create nums = list of 1, 2, 3, 4, 5
create part = nums from 1 to 4
say part at 0
say part at 2

create status = 200
match status
    case 200 then
        say "OK"
    end
    case 404 then
        say "Not Found"
    end
    default
        say "Other"
    end
end

create msg text = "hello"
match msg
    case "hello" then
        say "greetings"
    end
    default
        say "unknown"
    end
end

import "std/sql"
use sql

create dbPath text = "/tmp/afrilang_tier2.db"
create tableOk = exec(dbPath, "create table if not exists kv (k text, v text)")
say tableOk

create inserted = exec(dbPath, "insert into kv values ('lang', 'AFRILANG')")
say inserted

create rows = query(dbPath, "select v from kv where k='lang'")
say rows
```

77. tier3_demo.afr

```
create nums = list of 1, 2, 3, 4, 5
create bigNums = list each x in nums where x is greater than 2 return x
say length of bigNums
say bigNums at 0
say bigNums at 1

generator function range(n int) returns list int
    create i int = 0
    while i is less than n do
        yield i
        set i = i + 1
    end
end

create seq = range(4)
say seq at 0
say seq at 3

import "std/bigint"
use bigint

create huge bigint = fromInt(1000000000000)
say toText(mul(huge, fromInt(2)))

import "std/web"
use web

create router = createRouter()
addRoute(router, "GET", "/hello", "Hello Tier3")
say dispatch(router, "GET", "/hello")

import "std/thread"
use thread

sleepMs(5)
say hardwareConcurrency()

import "std/sql"
use sql
import "std/orm"
use orm

create dbPath text = "/tmp/afrilang_tier3.db"
create tableOk = exec(dbPath, "create table if not exists users (id integer primary key, name text)")
say tableOk

create rowId = insert(dbPath, "users", "name", "Alice")
say rowId

create rows = findAll(dbPath, "users")
say rows

test "tier3 fixtures"
    setup
        create ready = true
    end
    assert ready
    teardown
        say "teardown ok"
    end
end

say "tier3 complete"
```

78. tier4_demo.afr

```
import "pkg/cli/cli.afr"
use Cli

say banner("Tier 4 — supply chain & tooling")
say ok("SHA256 lockfile")
say ok("test --coverage")
say ok("run --profile / --watch")

import "pkg/datetime/datetime.afr"
use DateTime

say formatDuration(42)
```

79. tier5_demo.afr

```
record Point
    field x number
    field y number
end

record Vec2
    field x number
    field y number
end

interface Named
    function label() returns text
end

interface Greetable extends Named
    function greet() returns text
end

class Greeter implements Greetable
    public field title text
    function label() returns text
        return title
    end
    function greet() returns text
        return "hello " + title
    end
end

generator function range(n int) returns list int
    create i int = 0
    while i is less than n do
        yield i
        set i = i + 1
    end
end

create seq = range(5)
say seq at 0
say seq at 4

create score number = 85
match score
    case 100 if score is equal to 100 then
        say "perfect"
    end
    case 85 if score is greater than 80 then
        say "great"
    end
    default
        say "ok"
    end
end

create g = new Greeter()
set g.title = "Tier5"
say g.greet()

@deprecated
function legacyPing() returns text
    return "pong"
end

say legacyPing()
```

80. tier6_demo.afr

```
import "std/crypto"
use crypto

import "std/yaml"
use yaml

import "std/datetime"
use datetime

import "std/env"
use env

import "std/base64"
use base64

import "std/url"
use url

import "std/random"
use random
```

```
import "std/hex"
use hex

import "std/csv"
use csv

import "std/html"
use html

import "std/cli"
use cli

import "std/email"
use email

import "std/uuid"
use uuid

import "std/tempfile"
use tempfile

import "std/json"
use json

say sha256("tier6")
say base64.encode("hello")
say base64.decode("aGVsbG8=")
say url.encode("hello world")
say url.decode(url.encode("a+b"))
seed(42)
say randomInt(1, 10)
say hex.encode("AB")
say escape("<tag>")

create yamlText text = "name: AFRILANG\nversion: 1\nactive: true\n"
create doc = parse(yamlText)
say getString(doc, "name")
say getInt(doc, "version")

say formatIso(nowSeconds())
say isValid("dev@afrilang.org")
say domain("dev@afrilang.org")
say v4()

create fields = splitLine("a,b,c")
say fields at 0
say fields at 2

create tmp = createPath("afr")
say write(tmp, "phase-c")
say remove(tmp)

say green("Phase C OK")
```

81. tier7_demo.afr

```
import "pkg/math/math.afr"
import "pkg/strings/strings.afr"
import "pkg/fmt/fmt.afr"
import "pkg/validate/validate.afr"
import "pkg/config/config.afr"
import "pkg/testing/testing.afr"
import "pkg/colors/colors.afr"
import "pkg/uuid/uuid.afr"

use Math
use Strings
use Fmt
use Validate
use Config
use Testing
use Colors
use Uuid

say square(7)
say replicate("ab", 3)
say label("phase", "D")
say check("prefix", hasPrefix("hello", "hel"))
say getOr("", "default")
say success("Phase D OK")
say newId()
```

82. tier8_demo.afr

```
say "Phase E WASM OK"

create i int = 0
while i is less than 3 do
    say "hello wasm"
    set i = i + 1
end

create total int = 0
for n from 1 to 3 do
    set total = total + n
end
say total
```

83. tier8_stdlib.afr

```
// WASM stdlib smoke (str + math — documented WASM-ok)

import "std/str"
import "std/math"

use str
use math

say "Phase E WASM stdlib OK"
say trim("  wasm  ")
say abs(0 - 4)
say floor(3.9)
say pow(2, 3)
```

84. tier9_demo.afr

```
say "Phase H — instant JS OK"

create n int = 0
while n is less than 2 do
    say "client compile"
    set n = n + 1
end
```

85. traits_demo.afr

```
interface Speakable
    function speak()
end

interface Named
    function getName() returns text
end

class Dog implements Speakable, Named
    function speak()
        say "Woof!"
    end

    function getName() returns text
        return "Rex"
    end
end

create pet Speakable = new Dog()
pet.speak()

create speakers list of Speakable = list of new Dog()
for each s in speakers do
    s.speak()
end

module Shapes
    enum Kind
        case Circle
        case Square
    end
end

create k = Kind.Circle
match k
    case Circle then
        say "circle"
    end
    case Square then
        say "square"
    end
```

```
end
```

86. unions_demo.afr

```
union Shape
    case Circle with radius number
    case Rect with w number, h number
end

create circle = Shape.Circle with 5
create rect = Shape.Rect with 10, 3

match circle
    case Circle with r then
        say "circle radius {r}"
    end
    case Rect with w, h then
        say "rect {w}x{h}"
    end
end

create area = match rect
    case Circle with r then 3.14 * r * r
    end
    case Rect with w, h then w * h
    end
end

say area
```

87. vizultra_demo.afr

```
import "std/c/vizultra001"
use vizultra001

create ys list number = list of 2, 5, 3, 9, 7, 11, 8
create xs list number = list of 0, 1, 2, 3, 4, 5, 6

create ok1 bool = writeLineChart("chart_line.svg", ys, 640, 360)
create ok2 bool = writeBarChart("chart_bar.svg", ys, 640, 360)
create ok3 bool = writeScatter("chart_scatter.svg", xs, ys, 640, 360)
create ok4 bool = writeCsv("chart_data.csv", ys)

create w number = 6
create h number = 4
create grid list number = list of 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23
create ok5 bool = writeHeatmap("chart_heatmap.svg", grid, w, h, 480, 320)

say "line={ok1} bar={ok2} scatter={ok3} csv={ok4} heatmap={ok5}"
```

88. while.afr

```
create count number = 0

while count is less than 5 do
    say count
    set count = count + 1
end

say "Done counting"

create score number = 75

if score is greater than 90 then
    say "Excellent"
else
    if score is greater than 60 then
        say "Passed"
    else
        say "Failed"
    end
end
```

Partie XIII — Architecture interne

- Lexer → tokens
- Parser → AST
- SemanticAnalyzer → types / usesAsk
- CodeGenerator → C++ / g++
- Sandbox + cache

```
list→vector  text→string  number→double
ask as number → getline + stod
say/+ → str::toString/concat
```

Partie XIV — Roadmap, contribution, licence

Roadmap AFRILANG

Objectif : rapprocher AFRILANG d'un produit langage de type Nim / Crystal (haut niveau → natif + écosystème paquets), sans prétendre cloner 15 ans d'adoption d'un coup.

Vague 1 — Parité écosystème (livrée)

Inspiré de Nimble et Shards :

• [x] `pkg init` — scaffold paquet (`manifest.toml`, module, tests, README)
• [x] Deps **git** / **path** dans `afrilang.toml`
• [x] `afrilang.lock` comme source de vérité pour `pkg install`
• [x] `pkg update` — re-résolution + régénération du lock
• [x] Semver `^` / `~`
• [x] `afrilang init` enrichi (`.gitignore`, README, `tests` avec `assert`) + `--lib`
• [x] `afrilang test` découvre `tests/**/*.afr` ; `--examples` pour la suite demos
• [x] Docs : ce roadmap, `PKG.md`, `CORE_STDLIB.md`
• [x] `afrilang doc <dir>` → `docs/api/`

Vague 2 — Registre distant (livrée)

Modèle nim-lang/packages (url + method: git) :

• [x] Téléchargement de paquets depuis l'index distant si absent du registre local
• [x] `pkg sync` enrichit `list` / `search` / `lookup` via `remote-index.json`
• [x] `pkg publish --remote` — recette `registry-entry.json` + POST si `AFRILANG_REGISTRY_TOKEN`
• [x] Résolution transitive multi-niveaux + détection de conflits de versions
• [x] Index enrichi (`url`, `method`, `tags`, `license`, `web`)
• [x] `pkg test` — boucle auteur (self-vendor + tests/)

Vague 3 — Profondeur & cibles (livrée)

• [x] Qualité du **core stdlib** : `docs/STDLIB_API.md` + `tests/stdlib/*.afr`
• [x] UX stubs : hover LSP core vs experimental ; site stdlib **core** par défaut (`?experimental=1`)
• [x] WASM / JS : sous-ensemble documenté (`WASM_COMPAT.md`) + `examples/tier8_stdlib.afr` ; playground JS avec fonctions/`for`
• [x] Batterie specs : `afrilang test --specs` (`tests/specs` + `tests/stdlib`)
• [x] Site packages : tags / repo / README / recherche (`sync_packages`, détail paquet)

Hors roadmap (non codable ici)

Communauté, libs tierces, adoption — progressent avec le registre distant + usage réel.

Contribution

PR bienvenues sur https://github.com/MaximeKELI/AFRILANG. Tests dans tests/test_compiler.cpp.

Licence

Contact

- Auteur : Maxime Dzidula KELI
- Tél. : 98 60 00 18
- WhatsApp : +33 754830039
- Email : MaximeKELI25@gmail.com
- GitHub : https://github.com/MaximeKELI/AFRILANG

Annexe A — Glossaire

AST
Arbre syntaxique abstrait.

CLI
Interface ligne de commande.

Codegen
Génération C++.

FFI
Appels bibliothèques C.

LSP
Language Server Protocol.

PAHT
Prix d'Achat Hors Taxes.

REPL
Boucle interactive.

Result
Type T or error.

Sandbox
Exécution isolée.

Stdlib
Bibliothèque standard.

WASM
WebAssembly.

Annexe B — Index des chapitres (avec pages)

Colophon et contacts

Ouvrage généré depuis https://github.com/MaximeKELI/AFRILANG.

- Maxime Dzidula KELI
- Tél. : 98 60 00 18
- WhatsApp : +33 754830039
- Email : MaximeKELI25@gmail.com
- GitHub : https://github.com/MaximeKELI/AFRILANG